

Git & GitHub Mastery

From Zero → Internals → Production → Senior Engineer

Volume 3 of 4: GitHub, GitHub Actions and security

Baseline: Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Every Git transcript in this book is real output from the lab scripts under `labs/`, reproducible with `labs/verify-all.sh`. GitHub-side behavior is described from GitHub's documentation and is marked as such.

Contents of this volume

- Chapter 15: GitHub
- Chapter 16: Authentication
- Chapter 17: Pull Requests
- Chapter 18: Branch Protection and Rulesets
- Chapter 19: CODEOWNERS
- Chapter 20A: GitHub Actions Fundamentals
- Chapter 20B: GitHub Actions: delivery, runners, cost and debugging
- Chapter 21A: GitHub Actions security
- Chapter 21B: Repository security, identity, the Git client, and secret-leak response

Chapter 15: GitHub

Baseline. Git 2.55.0 on macOS; GitHub CLI 2.88.1; GitHub facts as of 1 October 2026, re-read on docs.github.com on 2 October 2026 where a link is given. Transcripts are real output from labs/ch15/. Nothing in this chapter was run against GitHub: where the text says what GitHub shows or does, it is described from the linked documentation, and the interface it describes changes on a scale of months.

15.1 Why this matters

Four questions a CTO can ask, none of which is about a Git command:

1. "The intern pushed a staging key to her fork. We deleted the fork within the hour. Is the key gone?"
2. "The release page says v0.3.0. The build machine runs `git describe` and prints v0.2.0-1. Which one is the version?"
3. "A new engineer got Write on one repository. Why can she clone all forty private ones?"
4. "We may move to another host next year. What does `git clone --mirror` take with it, and what stays behind?"

The answers: no, and the commit is reachable through your own repository's URL (section 15.7). Both are, because a release and a tag are different objects made by different programs (section 15.12). Because access is the highest of several grants, and one of them applies to every member (section 15.4). And the mirror takes refs and objects; issues, reviews, releases, rules, roles and secrets stay behind, because none of them is Git data (section 15.2).

All four answers come from one habit: for every thing you touch, know whether it is **Git data**, which lives in refs and objects and travels with `clone`, `fetch` and `push`, or a **GitHub object**, which lives in GitHub's database and is reached through the web interface, the GitHub CLI or the API. This chapter builds that map, then walks the platform with it. Authentication (Chapter 16), pull requests (Chapter 17), rulesets (Chapter 18), CODEOWNERS (Chapter 19), Actions (Chapter 20A) and security (Chapter 21B) have their own chapters and are only pointed to here.

The demos use `prompt-registry`, a small Python package that stores versioned prompt templates for an LLM application. The same files become your practice repository in Lab 19.1.

15.2 Git data and GitHub objects

In one sentence. A repository on GitHub is a Git repository, which you can copy completely, surrounded by records in GitHub's database, which you cannot copy with Git at all.

Analogy. A manuscript in a publisher's office. You can carry a perfect copy of the manuscript out of the building. The reviewers' notes, the list of who may edit it and the approval stamps belong to the office's filing system. The analogy breaks in two places. The office edits your manuscript when asked: a merge button creates commits. And the office reads instructions written inside the manuscript: files under `.github/` and words such as `Fixes #12` are Git data that GitHub interprets.

Precisely. Chapter 1 gave a first version of this table. This one adds the command that reads each thing. The GitHub rows are from the Phase 0 report, sections 2 and 12.

Thing	Layer	Where it lives	Read it with	Arrives with git clone ?
Commits, trees, file contents, annotated tag objects	Git	objects	<code>git cat-file -p</code>	yes
Branches and tags	Git	refs	<code>git for-each-ref</code> , <code>git ls-remote</code>	yes; branches as remote-tracking branches
Author, committer, message, trailers, signature	Git	inside the commit object	<code>git log --format=fuller</code> , <code>git cat-file -p</code>	yes
README, LICENSE, .github/ (issue forms, templates, CODEOWNERS, workflow files)	Git data that GitHub interprets	tracked files	<code>git ls-files</code>	yes
<code>Fixes #12</code> in a message or description	text that GitHub interprets	commit object, or the pull request description	<code>git log --grep</code>	the commit yes
Pull request head refs	Git refs that GitHub creates	<code>refs/pull/N/head</code> on the server, read-only	<code>git ls-remote origin 'refs/pull/*'</code>	no: outside the default refspec (Chapter 17)
Pull request: title, description, reviews, comments, checks, merge state	GitHub	database	<code>gh pr view</code> , <code>gh api</code>	no
Issue, sub-issue, label, milestone, project, discussion	GitHub	database	<code>gh issue</code> , <code>gh label</code> , <code>gh project</code> , <code>gh api</code>	no
Release: title, notes, assets, draft, pre-release and latest flags	GitHub	database; points at a tag	<code>gh release view</code>	the tag yes, the release no
The "Verified" badge; which account a commit is attributed to	GitHub	database	<code>web</code> , <code>gh api</code>	no (Chapter 21B)
Fork relationship, stars, watchers	GitHub	database	<code>gh repo view --json parent, stargazerCount</code>	no
Roles, teams, rulesets, classic branch protection, secrets, variables, webhooks, deploy keys, settings	GitHub	repository and organization settings	<code>gh api</code> , <code>gh ruleset</code> , <code>gh secret list</code>	no
Wiki	Git, in a second repository	<code>OWNER/REPO.wiki.git</code>	<code>git clone</code> of that URL (wikis)	no: a separate clone
Packages and images; workflow runs, logs, artifacts, caches	GitHub Packages; GitHub Actions	GitHub	the package manager; <code>gh run</code> , <code>gh cache</code>	no
Large files tracked with Git LFS	pointer files are Git; contents are in LFS storage	Chapter 22	<code>git lfs</code>	pointers yes

Inside .git. A clone holds objects, refs, HEAD, the index, reflogs and config (Chapter 3). There is no file for an issue or a review. The only traces of the platform are the URL in `remote.origin.url` and what a platform tool wrote into your configuration, such as a credential helper line (Chapter 16).

See it. A bare repository on disk plays the server. It has a default branch, a feature branch whose commit message says `Fixes #12`, and an annotated tag.

```
$ git clone server/practice-repo.git you/practice-repo
Cloning into 'you/practice-repo'...
done.
$ cd you/practice-repo
$ git for-each-ref --format="%(objecttype) %(refname)"
commit refs/heads/main
commit refs/remotes/origin/HEAD
commit refs/remotes/origin/feature/list-names
commit refs/remotes/origin/main
tag refs/tags/v0.1.0
```

Five refs, each naming an object: with the objects reachable from them, that is everything the clone received. The files that configure the platform came along because they are tracked files:

```
# Files that GitHub reads are ordinary tracked files:
$ git ls-files .github README.md CONTRIBUTING.md SECURITY.md
.github/ISSUE_TEMPLATE/1-bug.yml
.github/ISSUE_TEMPLATE/2-feature.yml
.github/ISSUE_TEMPLATE/config.yml
.github/pull_request_template.md
CONTRIBUTING.md
README.md
SECURITY.md
```

```
$ git log -1 --format=%B origin/feature/list-names
Add names() to list registered prompts

Fixes #12

$ git log --all --oneline --grep="#12"
c130f61 Add names() to list registered prompts
```

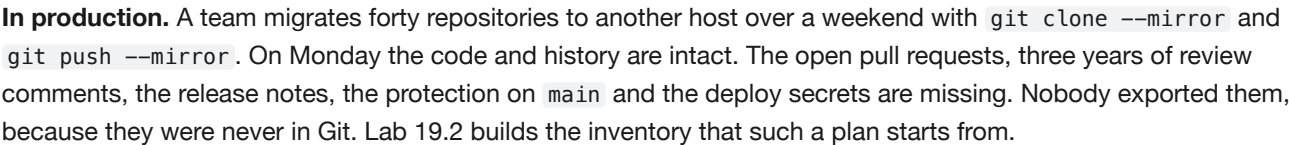
To Git, `Fixes #12` is two words in a message; no issue 12 exists in this sandbox. On GitHub the same text links the commit to an issue and can close it (section 15.8).

```
$ cd ../../
$ git clone --mirror server/practice-repo.git backup/practice-repo.git
Cloning into bare repository 'backup/practice-repo.git'...
done.
$ git -C backup/practice-repo.git for-each-ref --format="%(objecttype) %(refname)"
commit refs/heads/feature/list-names
commit refs/heads/main
tag refs/tags/v0.1.0
$ git -C backup/practice-repo.git rev-list --all --objects | wc -l | tr -d " "
33
```

A mirror clone asks for every ref the server offers (Chapter 12, section 12.3) and still receives only refs and objects: three refs, 33 objects. Against GitHub it also receives the `refs/pull/*` refs, and no issue, review, release or setting.

Picture.

```
+----- GitHub -----+
| database: issues, pull requests, reviews, releases, labels, stars, |
| roles, teams, rulesets, secrets, webhooks, Actions runs |
```



15.3 Accounts: personal, organization, enterprise

Precisely. GitHub has three account types (types of accounts).

A machine user is a user account that automation signs in with. A managed user account is created by an enterprise through its identity provider and cannot contribute outside that enterprise. Converting a personal account into an organization was deprecated on 12 January 2026 in favor of moving work to an organization ([changelog](#)).

In production. A startup's code lives under the founder's personal account, with ten collaborators. Every collaborator can push to everything they were added to, there are no teams, and the company's access model is one person's account. Transferring a repository to an organization keeps its Git data, issues, pull requests, wiki, stars and watchers, and its webhooks, secrets and deploy keys stay attached ([transferring a repository](#)); what you gain is the access model of the next section.

15.4 Who can do what: members, teams, roles and effective access

In one sentence. In an organization, what a person can do in a repository is the highest of every grant that reaches them: the organization-wide base permission, the roles of their teams, a direct grant, and ownership of the organization.

Analogy. Key cards in an office building. One rule opens some doors for every employee, your department's card opens more, and a card issued to you personally may open one extra room. A door opens if any card you hold opens it. The analogy breaks because buildings have blacklists and GitHub has none: no grant can reduce what another grant gives.

Precisely. The people:

- A **member** belongs to the organization. An **owner** is a member with complete administrative access; GitHub advises at least two ([organization roles](#)). Owners have admin access to every repository of the organization.
- A **team** is a group of members. Teams can be nested, and a child team inherits the parent's access. Outside collaborators cannot be on a team ([teams](#)).
- An **outside collaborator** is not a member and has access to one or more repositories through direct grants only. Base permissions do not apply to them ([base permissions](#)).

The five repository roles, from least to most access, with the rows of GitHub's permission matrix that decide most arguments ([repository roles](#)):

Action	Read	Triage	Write	Maintain	Admin
Clone, fork, open issues, comment, submit a review	yes	yes	yes	yes	yes
Apply labels, close and assign any issue or pull request, request reviews	no	yes	yes	yes	yes
Push, merge a pull request, create labels and milestones	no	no	yes	yes	yes
Give an approval that counts toward required reviews; act as a code owner	no	no	yes	yes	yes
Create and edit releases; create and edit Actions secrets and variables	no	no	yes	yes	yes
Configure pull request merges, edit the description, manage topics	no	no	no	yes	yes
Push to protected branches (classic protection only)	no	no	no	yes	yes
Manage rulesets and branch protection; change visibility; manage access, webhooks and deploy keys; rename the default branch; archive, transfer, delete	no	no	no	no	yes

Three rows surprise people. Write can create Actions secrets and edit workflow files, so Write is enough to run code with the repository's secrets (Chapter 21A). The Maintain role's right to push to protected branches does not apply to rulesets, which "have a different bypass model" in the matrix's words (Chapter 18). And Read includes submitting a review, although only a review from someone with Write counts toward a requirement.

The **base permission** is a role that every member has on every repository of the organization; a higher grant on a repository overrides it. GitHub's page states the default as Read on the organization's public repositories. Custom repository roles, up to 20, exist only on Enterprise Cloud ([custom roles](#)).

Effective access is the highest of the base permission, team grants (including those inherited from parent teams), direct grants, and admin through organization ownership; an outside collaborator has direct grants only. The report marks this combined rule as an inference from the pages above, not a sentence GitHub publishes.

Picture. Meera joins the organization and the team `eval-platform`, which has Write on one repository.

base permission	repo: evalkit-service Read	repo: billing-export (private) Read
-----------------	-------------------------------	--

team eval-platform	Write	—	
direct grant	—	—	
organization owner	no	no	
	-----	-----	
effective access	Write (highest wins)	Read	<-- "why can she clone it?"

Observed behavior	: A new engineer with Write on one repository can clone every private repository.
Git state	: Not involved. Her clone works because the server authorized it.
Mechanism	: The organization's base permission is Read. It applies to every member on every repository. Her team's Write grant adds to it on one repository.
Root cause	: Access was designed as "add her to one repository". The base permission that already covered all repositories was never part of the design.
Why GitHub does it	: Grants only add. The highest grant wins, and nothing subtracts.
Correct fix	: Decide the base permission on purpose (it can be set to none), and grant repositories through teams. Or make short-term staff outside collaborators, whom base permissions do not reach.
Prevention	: Review the base permission when the organization is created and when people join.

In production. Deploy keys are outside this model. GitHub's roles page warns that anyone holding a deploy key's private half can use it "even if they're later removed from the organization". Offboarding a person does not offboard the credentials they created (Chapter 16).

15.5 Visibility, and the repository settings that matter

In one sentence. Visibility decides who can read a repository at all; the settings decide which platform features exist in it and how pull requests land.

Precisely. A repository is **public** (readable by everyone on the internet) or **private** (readable by the owner, people granted access and, in an organization, members according to their grants). Organizations owned by an enterprise can also create **internal** repositories, readable by all enterprise members ([about repositories](#)). Organization owners can read every repository of the organization, whatever its visibility.

Changing visibility is an Admin action with documented side effects ([setting repository visibility](#)):

Change	What also happens
public to private	Stars and watchers are erased. Public forks stay public, detached into a network of their own. On a Free plan some features stop working, and code scanning stops unless licensed for private repositories.
private to public	Everyone can read the code, the complete history, and the Actions history and logs. Anyone can fork. Push rulesets are disabled. Private forks stay private, detached. Stars and watchers are erased.

Neither direction changes a Git object. Making a repository private does not take back what was cloned or forked while it was public, and making it public publishes every commit that any ref reaches, including the old one that contained a password (Chapter 21B).

The settings a professional sets on purpose. Each is a GitHub object, so none of them is in a clone, and none of them is restored by pushing a mirror:

Setting	What it does	Why it matters	Check or set
Default branch	what clones check out, pull requests target and closing keywords act on	clones keep the old name after a rename (Chapter 12, section 12.3)	<code>gh repo view --json defaultBranchRef; gh repo edit --default-branch</code>
Features: issues, wiki, discussions, projects	shows or hides each tab	an unused feature is a place where questions go unanswered	<code>gh repo edit --enable-wiki=false</code> and siblings
Pull request access	since 13 February 2026 pull requests can be disabled, or	mirrors and read-only code	web interface

Setting	What it does	Why it matters	Check or set
	creation limited to collaborators (changelog)		
Allowed merge methods	which of merge commit, squash and rebase the merge button offers	decides the shape of history on the default branch (Chapter 17)	<code>gh repo edit --enable-squash-merge</code> and <code>siblings</code>
Automatically delete head branches	deletes a pull request's branch after merging (docs)	your clone still needs <code>git fetch --prune</code>	<code>gh repo edit --delete-branch-on-merge</code>
Forking policy	whether a private organization repository may be forked	a fork of a private repository is a second copy under someone else's control	<code>gh repo edit --allow-forking</code>
Archive	makes issues, pull requests, code, releases and settings read-only (docs)	the honest state for code nobody maintains	<code>gh repo archive</code>

The flags were checked against `gh repo edit --help` of gh 2.88.1. `gh repo edit --visibility` also needs `--accept-visibility-change-consequences`.

In production. With automatic deletion enabled, an author's `git branch -vv` shows twenty branches marked `gone` after a month. The setting deleted refs on the server. Remote-tracking refs go when you prune, and local branches when you delete them (Chapter 12, section 12.11): one setting, three kinds of ref, three owners.

15.6 Stars and watchers

A **star** is a bookmark and a public signal. It makes a repository appear on your stars page and feeds rankings: "many of GitHub's repository rankings depend on the number of stars", says GitHub's page, and anyone can list the stargazers of a repository they can read by appending `/stargazers` to its URL ([stars](#)).

Watching is a notification subscription. When you watch a repository you are subscribed to its activity; you can narrow that to chosen event types (issues, pull requests, releases, security alerts, discussions) or ignore the repository. By default you automatically watch repositories you create, and repositories you are given push access to, except forks. An account can watch at most 10,000 repositories ([configuring notifications](#), [about notifications](#)).

Neither is evidence of quality. Stars can be manufactured: the Phase 0 report cites a network of more than 3,000 accounts that distributed malware through repositories made to look popular ([Check Point](#)), and a star count says nothing about who controls a repository today. Chapter 21B covers the attack. A useful subscription is narrow: releases and security alerts of the dependencies you ship.

15.7 Forks and the fork network

In one sentence. A fork is a second repository on GitHub with its own refs, settings and permissions, connected to the repository it was made from, and storing its Git data together with it.

Analogy. A branch office that keeps its own ledger and uses head office's warehouse. Each office has its own list of goods (its refs). The goods (the objects) are in one building, and anyone who knows a crate's serial number can ask either office for it. The analogy breaks when an office closes: its list is gone and the crates remain.

Precisely. From the forks reference ([forks](#)):

- A fork has its own branches, tags, issues, pull requests, Actions, labels and wiki, and its own permissions. Forks of public repositories are public and forks of private repositories are private; a fork's visibility cannot

be changed by itself.

- A repository network is the upstream repository, its forks, and forks of those forks. "Repositories in the network share Git data." Commits pushed to any repository in a network "can be accessible from other repositories in that network, including the upstream repository", and this holds "even after a fork is deleted". Owners of an upstream repository can read all forks in the network.
- Deleting a private repository deletes its private forks. Deleting a public repository makes an active public fork the new upstream. Removing a person's access to a private repository deletes their forks of it.
- Branch and tag rulesets are not inherited by forks; push rulesets apply to the whole network (Chapter 18).
- A deleted repository can be restored within 90 days, unless its fork network is not empty ([restoring a repository](#)).

Inside .git. To your clone a fork is one more remote (Chapter 12, section 12.10). The parent-and-fork relationship is a GitHub object that no ref or configuration key records. The `upstream` remote that `gh repo fork` and `gh repo clone` add is a convenience in `.git/config`, not the relationship.

See it. GitHub documents the behavior and not the mechanism. Git has a documented mechanism of its own with the same property: **namespaces**, which divide the refs of one repository into several sets that are served as separate repositories "while sharing the object store" ([gitnamespaces](#)). The demo uses it as a model. It is not a claim about how GitHub is built.

```
# The platform: one object database. The upstream repository acme-ml/prompt-registry is a namespace in it.
$ git init -q --bare platform/network.git
$ GIT_NAMESPACE=acme-ml git -C seed push -q ../platform/network.git main
$ git -C platform/network.git for-each-ref --format="%(refname)"
refs/namespaces/acme-ml/refs/heads/main
$ find platform/network.git/objects -type f | wc -l | tr -d ' '
27
```

```
# Forking: the platform writes a ref for the fork. No object is copied.
$ git -C platform/network.git update-ref refs/namespaces/you/refs/heads/main refs/namespaces/acme-ml/refs/heads/main
$ git -C platform/network.git for-each-ref --format="%(refname)"
refs/namespaces/acme-ml/refs/heads/main
refs/namespaces/you/refs/heads/main
$ find platform/network.git/objects -type f | wc -l | tr -d ' '
27
```

Forking wrote one ref. The object count did not move: 27 before, 27 after.

```
$ GIT_NAMESPACE=you git clone -q platform/network.git you/prompt-registry
$ cd you/prompt-registry
$ git branch -a
* main
remotes/origin/main
```

```
$ git switch -q -c debug/staging-config
$ printf 'STAGING_API_KEY=FAKE-KEY-for-the-lab\n' > staging.env
$ git add -f staging.env && git commit -q -m "Add staging config for debugging"
$ GIT_NAMESPACE=you git push -q origin debug/staging-config
$ git rev-parse HEAD
915a81e1e3ff7856fb06c7b1863ec0c4baf6d2b5
$ cd ../../
$ find platform/network.git/objects -type f | wc -l | tr -d ' '
30
```

You pushed a commit with a staging key to a branch of your fork. Three new objects went into the one object database. Each repository still lists only its own refs:

```
# What each repository lists:
$ GIT_NAMESPACE=acme-ml git ls-remote platform/network.git
bf7889c1f62437e0b230c6660eafdf164e53abb3      refs/heads/main
$ GIT_NAMESPACE=you git ls-remote platform/network.git
915a81e1e3ff7856fb06c7b1863ec0c4baf6d2b5      refs/heads/debug/staging-config
bf7889c1f62437e0b230c6660eafdf164e53abb3      refs/heads/main
```

The upstream does not list the branch. Now a visitor who knows nothing but the upstream repository and the commit ID 915a81e:

```
# A visitor who only knows the upstream repository, and the commit ID:
$ GIT_NAMESPACE=acme-ml git clone -q platform/network.git visitor/prompt-registry
$ cd visitor/prompt-registry
$ GIT_NAMESPACE=acme-ml git fetch origin 915a81e1e3ff7856fb06c7b1863ec0c4baf6d2b5
From $LAB/ch15/fork-network/platform/network
 * branch          915a81e1e3ff7856fb06c7b1863ec0c4baf6d2b5 -> FETCH_HEAD
$ git show --stat --format="%h %s" FETCH_HEAD
915a81e Add staging config for debugging

staging.env | 1 +
1 file changed, 1 insertion(+)
$ cd ../../
```

The upstream served a commit that none of its refs reaches, because the object is in the store it shares with the fork. Deleting the fork removes refs and nothing else:

```
# The fork is deleted: its refs go. The object database is not touched.
$ git -C platform/network.git for-each-ref --format='delete %(refname)' refs/namespaces/you | git -C
platform/network.git update-ref --stdin
$ git -C platform/network.git for-each-ref --format="%(refname)"
refs/namespaces/acme-ml/refs/heads/main
$ git -C platform/network.git cat-file -t 915a81e1e3ff7856fb06c7b1863ec0c4baf6d2b5
commit
```

```
# Only when the server prunes unreachable objects does the commit cease to exist there:
$ git -C platform/network.git gc --quiet --prune=now
$ git -C platform/network.git cat-file -t 915a81e1e3ff7856fb06c7b1863ec0c4baf6d2b5
fatal: git cat-file: could not get object info
[exit status: 128]
```

In the model the commit dies when the server prunes unreachable objects. For GitHub, the report found no documented retention period for unreachable commits. The manual page of Git namespaces carries its own warning: namespaces "are not effective for read access control".

Picture.

```
platform: one object database for the whole network
+-----+
| objects:  ... bf7889c (main)  915a81e (the commit with the key) |
|                                                                    |
| refs of acme-ml/prompt-registry      refs of you/prompt-registry |
|   refs/heads/main -> bf7889c          refs/heads/main -> bf7889c  |
|                                       refs/heads/debug/staging-config |
|                                       -> 915a81e                      |
+-----+
^ a request by ID through either name finds the object
```

State table. Operations of this section, by what they change:

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
Fork (web interface, or <code>gh repo fork</code>)	unchanged	unchanged	unchanged	unchanged	unchanged; with <code>--clone</code> , a new clone whose remotes are the fork and its parent	a new repository exists	the fork relationship is recorded
<code>gh repo sync OWNER/FORK</code>	unchanged	unchanged	unchanged	unchanged	unchanged	the fork's branch is fast-forwarded to its parent's; with <code>--force</code> , hard reset to it	unchanged
Delete a fork	unchanged	unchanged	unchanged	unchanged	your remote for it now fails	its refs are gone	objects can remain reachable through the network

According to `gh repo sync --help`, a sync is a fast-forward "except when the `--force` flag is specified, then the two branches will be synced using a hard reset". 🚫 `gh repo sync --force` discards commits on the destination branch that the source does not have. Preview with `git log upstream/main..origin/main` after a fetch; recovery needs a clone that still has those commits; it is appropriate only when the fork's branch is meant to be an exact copy of its parent. The Git commands for keeping a fork current are in Chapter 12, section 12.10, and the full fork workflow is in Chapter 17 and Chapter 27.

In production. The CTO's first question. The key was published the moment the push to the public fork finished. Deleting the fork changed which names list the commit, not whether the object can be fetched. Revoke the key first, then decide whether removal is worth requesting; a secret store and push protection are the prevention (Chapter 21B).

15.8 Issues: structure, and the keywords that close them

In one sentence. An issue is a numbered record in GitHub's database for a unit of work or a report, which since 2025 can have a type, a parent, children and dependencies.

Precisely. To the REST API "every pull request is an issue, but not every issue is a pull request" ([REST reference](#)), which is why `#12` can be either and why issue endpoints return both. The structure added in 2025 and 2026 (report, section 2):

Feature	Facts	Since
Sub-issues	up to 100 per parent, eight levels deep; a sub-issue may live in another repository (docs)	generally available 9 April 2025 (changelog)
Issue types	defined per organization, up to 25; the defaults are task, bug and feature (docs)	9 April 2025
Dependencies	"blocked by" and "blocking", up to 50 links per relationship type (docs)	21 August 2025 (changelog)
Issue fields	typed metadata defined per organization; four default fields (Priority, Effort, Start date, Target date)	2 July 2026 (changelog)

Types and fields belong to an organization, one more thing a personal account does not have.

Closing keywords. Nine words link a pull request or a commit to an issue: `close`, `closes`, `closed`, `fix`, `fixes`, `fixed`, `resolve`, `resolves`, `resolved`, followed by `#N` for the same repository or `OWNER/REPO#N` for

another, in any letter case, with an optional colon ([linking a pull request to an issue](#)). The rules that explain every "why did it not close" question are on the same page:

- In a pull request description the keywords are interpreted **only when the pull request targets the default branch**. Against any other branch they are ignored: no link is made and nothing closes.
- In a commit message the issue closes when the commit reaches the default branch, and the pull request that carried the commit is not listed as linked.
- The issue closes at merge time, not when the pull request is opened.

Git, not GitHub. The keyword is text in a commit object or in a database field. Git does nothing with it (section 15.2). It also travels with the message: a cherry-picked or rebased copy of the commit carries the same words, and they act when that copy reaches a default branch.

`gh issue create`, `list`, `view`, `edit`, `close` and `develop` cover daily work; `gh issue develop N --checkout` creates a branch linked to issue N and switches to it (`gh issue develop --help`). The installed 2.88.1 has no flags for types, parents or dependencies; `gh issue create --type` and `--parent` arrived in 2.94.0 ([release notes](#)).

In production. A fix is merged into `release/2.4` with "Fixes #812" in the description. The issue stays open and the customer is told it is unresolved. Nothing failed: the pull request did not target the default branch. Teams that ship from release branches close issues by hand or by automation, and say so in `CONTRIBUTING.md`.

15.9 Labels and milestones

A **label** classifies issues, pull requests and discussions within one repository. Every new repository receives default labels, among them `bug`, `documentation`, `duplicate`, `enhancement`, `good first issue`, `help wanted`, `invalid`, `question` and `wontfix`; organization owners can change the defaults for new repositories. Creating a label needs Write, applying one needs Triage ([managing labels](#)). Labels are more than decoration when something reads them: an issue form can apply labels (section 15.13), generated release notes group pull requests by label (section 15.12), and workflows can react to them.

A **milestone** groups issues and pull requests toward a date and shows a completion percentage ([about milestones](#)). It is not a Git tag and not a release: a milestone called `v0.3.0` promises nothing about which commit ships.

`gh label create`, `edit`, `list`, `delete` and `clone` manage labels; `gh label clone` copies a label set from one repository to another, which is how a team keeps forty repositories consistent. The CLI has no milestone command in 2.88.1; milestones are reached through `gh api on repos/{owner}/{repo}/milestones` ([REST reference](#)).

15.10 Projects, Discussions and wikis

Projects is the planning layer: "an adaptable table, board, and roadmap" over issues and pull requests, owned by a user or an organization, not by a repository, with up to 50 fields and, since April 2025, up to 50,000 items ([about Projects](#), [changelog](#)).

Outdated advice. "Projects (classic)", a board attached to one repository, was sunset on GitHub.com on 23 August 2024, and GitHub CLI versions older than 2.82.1 fail to fetch projects ([sunset notice](#)). `gh project` needs the `project` scope: `gh auth refresh -s project` (`gh pr create --help`).

Discussions is a forum beside the issue tracker: announcements, questions with a marked answer, polls. Someone with the Maintain or Admin role must enable it for a repository; organization-level discussions span repositories ([about discussions](#)). The working rule: an issue can be done and closed; a question or an open-ended idea is a discussion.

A **wiki** is documentation in a second Git repository, cloned as `OWNER/REP0.wiki.git`; only changes pushed to its default branch are shown. By default people with Write can edit it. Wikis in private repositories need a paid plan, and a wiki has a soft limit of 5,000 files ([about wikis](#)). Because it is a separate repository, a wiki is not reviewed through pull requests, not covered by the rulesets of the main repository, and not included when you clone or mirror the code. Documentation that must be reviewed with the code belongs in `docs/`.

15.11 Packages and the container registry

GitHub Packages hosts registries for npm, RubyGems, Maven, Gradle, Docker and NuGet; the Container registry at `ghcr.io` is the home for Docker and OCI images ([introduction to Packages](#)). Four facts shape how you use it:

- **A package is not Git data and not part of a repository.** An image at `ghcr.io/OWNER/IMAGE` belongs to a user or an organization and is "not linked to a repository by default"; it can be linked, and its access can be inherited from a repository or set separately ([container registry](#)).
- **Authentication is an exception to the token advice.** "GitHub Packages only supports authentication using a personal access token (classic)"; inside a workflow the job's `GITHUB_TOKEN` is used instead. A fine-grained token cannot log in to `ghcr.io` (Chapter 16).
- **Public packages are free;** private packages get an allowance according to the plan.
- **Provenance is separate from storage.** An artifact attestation is a signed claim about which workflow run built an artifact from which commit, checked with `gh attestation verify`; attestations work for public repositories on every plan and for private repositories only on Enterprise Cloud ([artifact attestations](#)).

Outdated advice. `docker.pkg.github.com`, the old Docker registry of GitHub Packages, was closed on 24 February 2025 ([changelog](#)). Use `ghcr.io`.

Publishing an image from a workflow is workflow 6 in Chapter 20A.

15.12 Releases: a Git tag versus a GitHub Release

In one sentence. A tag is a Git ref that names a commit; a release is a GitHub record that points at a tag name and adds a title, notes, files and flags.

Analogy. A tag is the mark stamped on a casting; a release is the catalogue page for it, with a description and a download. The page can be rewritten without touching the metal. The analogy breaks in one dangerous place: asking for a page for a mark that does not exist makes the platform stamp the mark for you, on whatever is at the front of the line.

Precisely. Tags are Chapter 14B: lightweight or annotated, under `refs/tags/`, moved between repositories by `git push` and `git fetch`. On the GitHub side ([about releases](#)):

- "Releases are based on Git tags." A release adds a title, notes, uploaded assets and three flags: draft, pre-release, latest. Managing releases needs Write.
- **Creating a release can create the tag.** The REST reference describes `target_commitish` as the value "that determines where the Git tag is created from", unused if the tag already exists, defaulting to the default branch ([create a release](#)). The CLI says the same about itself:

```
$ gh release create --help | sed -n '/^If a matching git tag/,/^If the tag is not annotated/p'
If a matching git tag does not yet exist, one will automatically get created
from the latest state of the default branch.
Use '--target' to point to a different branch or commit for the automatic tag creation.
Use '--verify-tag' to abort the release if the tag doesn't already exist.
To fetch the new tag locally after the release, do `git fetch --tags origin`.

To create a release from an annotated git tag, first create one locally with
git, push the tag to GitHub, then run this command.
Use '--notes-from-tag' to get the release notes from the annotated git tag.
If the tag is not annotated, the commit message will be used instead.
$ gh release create --help | grep -E -- '--(verify-tag|target|notes-from-tag|generate-
notes|draft|prerelease) '
  -d, --draft                Save the release as a draft instead of publishing it
  --generate-notes           Automatically generate title and notes for the release via
GitHub Release Notes API
  --notes-from-tag          Fetch notes from the tag annotation or message of commit
associated with tag
  -p, --prerelease          Mark the release as a prerelease
  --target branch           Target branch or full commit SHA (default [main branch])
  --verify-tag              Abort in case the git tag doesn't already exist in the remote
repository
$ gh release delete --help | grep -- '--cleanup-tag'
  --cleanup-tag             Delete the specified tag in addition to its release
```

This transcript is marked volatile: it is the help text of gh 2.88.1 and may read differently in your version.

- **Generated notes** list merged pull requests and contributors, and are configured in `.github/release.yml`, where labels and authors can be excluded or grouped ([generated release notes](#)). Their quality is the quality of your pull request titles and labels.
- **Immutable releases**, generally available since 28 October 2025: once published, the tag is locked to its commit and cannot be deleted while the release exists; assets cannot be changed; the tag name can never be reused; and a signed release attestation is generated, checked with `gh release verify` ([immutable releases](#), [changelog](#)). The documented order is: create a draft, attach the assets, publish.

Inside `.git`. A release leaves no trace in any clone. The tag does, and only after a fetch.

See it. You create an annotated tag and push it, the way a release is cut on purpose:

```
$ cd you/prompt-registry
$ git tag -a v0.2.0 -m "prompt-registry 0.2.0: first tagged version"
$ git push origin v0.2.0
To ../server/prompt-registry.git
 * [new tag]          v0.2.0 -> v0.2.0
$ git ls-remote --tags origin
db476fed4f3e711eb2a4ef4c4fe223c0844878dc      refs/tags/v0.2.0
bf7889c1f62437e0b230c6660eafdf164e53abb3      refs/tags/v0.2.0^{} }
```

Two lines for one tag: the tag object, and the commit it points at (`^{}`). Then Asha merges one more commit, and someone creates a release v0.3.0 on the platform without creating a tag first. A plain `git tag` on the bare repository stands in for what the platform does then:

```
# Asha has merged one more commit. Then a release "v0.3.0" is created on the platform for a tag
# that does not exist. Stand-in for what the platform does: a tag at the tip of the default branch.
$ git -C ../server/prompt-registry.git tag v0.3.0 main
$ git tag --list
v0.2.0
$ git ls-remote --tags origin
db476fed4f3e711eb2a4ef4c4fe223c0844878dc      refs/tags/v0.2.0
bf7889c1f62437e0b230c6660eafdf164e53abb3      refs/tags/v0.2.0^{} }
89837fa8e400e276bac7ab81de1460e9df49316a      refs/tags/v0.3.0
```

The server has a tag that your clone has never seen. One line this time: no tag object, only a ref that names a commit.

```
$ git fetch origin
From ../.. /server/prompt-registry
  bf7889c..89837fa  main      -> origin/main
  * [new tag]       v0.3.0    -> v0.3.0
$ git tag --list
v0.2.0
v0.3.0
```

```
$ git for-each-ref --format="% (refname:short) %(objecttype)  tagger=%(taggername)  %(subject)"
refs/tags
v0.2.0 tag  tagger=Lab User  prompt-registry 0.2.0: first tagged version
v0.3.0 commit tagger=  Document that versions are immutable
```

```
$ git describe origin/main
v0.2.0-1-g89837fa
$ git describe --tags origin/main
v0.3.0
```

Observed behavior : The release page says v0.3.0. "git describe" on the build machine prints v0.2.0-1-g89837fa.

Git state : refs/tags/v0.3.0 exists and names the commit 89837fa directly. It is a lightweight tag: no tag object, no tagger, no message.

Mechanism : "git describe" uses annotated tags only, unless --tags is given (Chapter 14B).

Root cause : The release was created for a tag that did not exist, so the tag was created by the platform at the tip of the default branch, not by a person with "git tag -a".

Why Git does this : Annotated tags are meant for releases, lightweight tags for private labels.

Correct fix : Decide which commit is the version. Then either describe with --tags, or replace the tag properly: a new annotated tag under a new name is safer than moving one.

Prevention : Create and push the annotated tag first; create the release with --verify-tag. Protect tags with a tag ruleset; use immutable releases for anything you ship.

Unverified. That a tag created by the release API is lightweight is an inference: the CLI help tells you to create annotated tags yourself. Lab 25.1 has you check with `git cat-file -t`. Also not stated in the documentation read: what happens to a release when its tag is deleted, and whether immutable releases are limited to certain plans (report, section 2, flags).

State table.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
git tag -a v0.2.0 then git push origin v0.2.0	unchanged	unchanged	unchanged	unchanged	refs/tags/v0.2.0 and a tag object	the tag ref and tag object are created	the tag is listed; no release exists
gh release create v0.2.0 --verify-tag --generate-notes	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged	a release pointing at the existing tag

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
gh release create v0.3.0 when no such tag exists	unchanged	unchanged	unchanged	unchanged	unchanged, until you fetch	a tag is created from the default branch, or from --target	a release, and a tag you did not make
gh release delete v0.3.0	unchanged	unchanged	unchanged	unchanged	unchanged	the tag stays, unless --cleanup-tag is given	the release is gone

In production. A workflow builds a container image when a tag `v*` is pushed and derives the version with `git describe`. A product manager creates "v1.9.0" in the web interface on Friday evening. The tag lands on whatever `main` was at that second, the workflow fires, and the version string in the artifact is wrong, because the tag is lightweight. Three controls close the gap: a tag ruleset that restricts who may create `v*` tags (Chapter 18), `--verify-tag` in every script, and immutable releases.

15.13 Templates, issue forms and community health files

In one sentence. Community health files are tracked files with well-known names that GitHub finds and uses to guide the people who interact with a repository.

Precisely. GitHub looks for each file in `.github/`, then the repository root, then `docs/` ([default community health files](#)).

File	What GitHub does with it	Source
README.md	renders it on the repository's front page	about READMEs
LICENSE	detects the license and shows it; without one, default copyright applies and nobody is permitted to reuse the code	licensing a repository
CONTRIBUTING.md	links it when someone opens an issue or pull request; since August 2025 also shows a Contributing tab	contributor guidelines , changelog
SECURITY.md	shows it as the security policy: how to report a vulnerability	security policy
CODE_OF_CONDUCT.md	counts it in the community profile	code of conduct
<code>.github/ISSUE_TEMPLATE/*.md</code> , <code>*.yaml</code> , <code>config.yaml</code>	offers them in the "new issue" chooser; read from the default branch	issue templates
<code>pull_request_template.md</code> in the root, <code>docs/</code> or <code>.github/</code>	pre-fills the description of a new pull request; read from the default branch	pull request template
<code>.github/CODEOWNERS</code>	requests reviews from owners of changed paths	Chapter 19
<code>.github/workflows/*.yaml</code>	defines GitHub Actions workflows	Chapter 20A

An **issue form** is a YAML file that turns the free-text issue into a form with required fields. This one is from the practice repository:

```
$ cat .github/ISSUE_TEMPLATE/1-bug.yaml
name: Bug report
description: Something does not work as documented.
```

```

title: "[Bug]: "
labels: ["bug"]
body:
  - type: markdown
    attributes:
      value: |
        Thank you for reporting. Do not report vulnerabilities here: see SECURITY.md.
  - type: textarea
    id: what-happened
    attributes:
      label: What happened?
      description: What did you do, what did you expect, and what happened instead?
    validations:
      required: true
  - type: input
    id: commit
    attributes:
      label: Commit
      description: The output of `git rev-parse --short HEAD` in your clone.
    validations:
      required: true
  - type: textarea
    id: logs
    attributes:
      label: Relevant output
      description: Paste the command and its output. It is formatted as code automatically.
      render: shell

```

Every key and element type in it is from the syntax reference, which also has `dropdown`, `checkboxes` and `upload` elements and the top-level keys `assignees`, `projects` and `type` ([syntax for issue forms](#)). The file was parsed with a YAML parser [here](#); only GitHub validates a form, which Lab 19.1 checks. The label it names, `bug`, is a default label.

```

$ cat .github/ISSUE_TEMPLATE/config.yml
blank_issues_enabled: false
$ cat .github/pull_request_template.md
## What changed

<!-- One or two sentences. Link the issue: "Fixes #123" closes it when this merges into the default
branch. -->

## Why

## How it was tested

- [ ] `python3 -m unittest discover -s tests` passes
- [ ] New behavior has a test

## Checklist

- [ ] The pull request does one thing
- [ ] No credentials, tokens, datasets or model files are in the diff

```

`blank_issues_enabled: false` removes the blank issue from the chooser for people with the Read or Triage role; people with Write still see it, according to the same page. A pull request template is plain Markdown; content inside an HTML comment is hidden in the rendered description ([writing syntax](#)), so the comment guides the author and does not clutter the result. Issue forms cannot be used for pull requests.

Unlike settings, these files are versioned and reviewed like code. An organization can also supply defaults: a public repository named `.github` holds health files for every repository of the account that lacks its own, except a license (same page as above).

Unverified. GitHub's syntax page still carries the note that issue forms are in public preview. The report could

not determine whether that note is current or stale.

The **community profile** of a public repository is a checklist of these files; an issue form counts only when it has valid `name` and `description` keys ([community profiles](#)).

15.14 A professional repository layout, file by file

The layout below is a convention, not a GitHub requirement. GitHub reads the files of section 15.13 and nothing else in it.

```
project/
├── .github/
│   ├── workflows/
│   ├── CODEOWNERS
│   └── pull_request_template.md
├── src/
├── tests/
├── docs/
├── scripts/
├── configs/
├── .gitignore
├── README.md
├── LICENSE
├── CONTRIBUTING.md
├── SECURITY.md
├── pyproject.toml
└── Dockerfile
```

The same layout as a real repository, listed by Git:

```
$ git ls-files
.github/CODEOWNERS
.github/ISSUE_TEMPLATE/1-bug.yml
.github/ISSUE_TEMPLATE/2-feature.yml
.github/ISSUE_TEMPLATE/config.yml
.github/pull_request_template.md
.gitignore
CONTRIBUTING.md
Dockerfile
LICENSE
README.md
SECURITY.md
configs/default.yaml
docs/architecture.md
pyproject.toml
scripts/release.sh
src/prompt_registry/__init__.py
src/prompt_registry/registry.py
tests/test_registry.py
```

There is no `.github/workflows/` in the listing. Git tracks files, not directories (Chapter 4), so the directory appears with the first workflow file in Chapter 20A.

Path	Read by	Purpose, and the mistake it prevents
<code>.github/workflows/</code>	GitHub Actions	CI and delivery, reviewed like code. Write can change what runs with the repository's secrets, so this directory deserves a code owner.
<code>.github/CODEOWNERS</code>	GitHub	Who is asked to review which paths. Only a request until a rule requires it (Chapter 19).

Path	Read by	Purpose, and the mistake it prevents
<code>.github/pull_request_template.md</code>	GitHub	Makes every description answer the same questions: what, why, how tested.
<code>src/</code>	your build tool	The package lives under <code>src/</code> , so that it is importable only on purpose (installed, or put on the path by the tests) and never by accident from the current directory.
<code>tests/</code>	your test runner	Kept apart from the package so that the tests are not shipped with it.
<code>docs/</code>	people; GitHub for health files	Documentation that changes in the same pull request as the code it describes.
<code>scripts/</code>	people, CI	Release and deploy commands as files with history.
<code>configs/</code>	the application	Configuration without secrets.
<code>.gitignore</code>	Git	Keeps build output, virtual environments and local secret files out of the index (Chapter 4). It does not untrack what is already tracked.
<code>README.md</code>	GitHub, people	What this is, how to run it, where to go next.
<code>LICENSE</code>	GitHub, lawyers	The terms of reuse. Without it nobody outside may use the code, whatever the visibility.
<code>CONTRIBUTING.md</code>	GitHub, contributors	How a change gets in: branch names, tests, review.
<code>SECURITY.md</code>	GitHub, reporters	Where to report a vulnerability privately, so that the first report is not a public issue.
<code>pyproject.toml</code>	Python tools	Project metadata and tool configuration in one file.
<code>Dockerfile</code>	Docker	How the service image is built. The one in the demo was not built here, because nothing may be downloaded while authoring.

Two checks that belong to the layout. A local secret file must be ignored before the first commit, and you can ask Git which rule ignores it:

```
$ printf 'LLM_API_KEY=FAKE-KEY-for-the-lab\n' > .env
$ git status --short
$ git check-ignore -v .env
.gitignore:10:.env      .env
```

And the tests must run from a fresh clone with one documented command, leaving the working tree clean:

```
$ python3 -m unittest discover -s tests 2>&1 | tail -1
OK
$ git status --short
```

Chapter 28 grows this skeleton into an AI/ML project layout.

15.15 Repository limits

GitHub's limits come in three kinds: enforced, recommended, and display limits. Mixing them up produces both needless worry and real outages.

Limit	Value	Kind	Source
One file in a push	warning above 50 MiB; blocked above 100 MiB	enforced	large files

Limit	Value	Kind	Source
One file added in the browser	25 MiB	enforced	same page
One push	2 GB	enforced	repository limits
Repository size	"ideally less than 1 GB, and less than 5 GB is strongly recommended"	recommended	large files page
Repository size on disk	10 GB recommended maximum	recommended	repository limits page
Entries in one directory; directory depth; branches	3,000; 50; 5,000	recommended	repository limits page
Pull request diff	20,000 lines or 1 MB that can be loaded; 300 files	display	repository limits page
Commits listed in a pull request or comparison	250	display	repository limits page
"Rebase and merge"	100 commits	enforced	repository limits page
Repositories per organization or account	100,000	enforced	repository limits page
Release assets	1,000 per release, each under 2 GiB	enforced	about releases

The two size recommendations sit on two pages and differ; both were read on 2 October 2026. Treat 1 GB as the target and 10 GB as the point where GitHub expects problems.

The file limit applies to every object a push sends, not to the files in your working tree. A file that you deleted in a later commit is still in history, and a first push sends all of history:

```
$ git log --oneline -3
39c4105 Move the classifier out of Git
c7182c9 Add trained intent classifier
bf7889c Add contribution guide, security policy and templates
$ git ls-files | grep -c classifier
0
```

```
$ git rev-list --objects --all | git cat-file --batch-check='%<objecttype> %<objectsize> %<rest>' |
awk '$1 == "blob"' | sort -k2,2nr | head -3
blob 3145728 models/intent-classifier.bin
blob 1324 tests/test_registry.py
blob 996 src/prompt_registry/registry.py
```

```
$ git log --oneline --diff-filter=A -- models/intent-classifier.bin
c7182c9 Add trained intent classifier
```

`git rev-list --objects --all` lists every reachable object with its path, `git cat-file --batch-check` adds type and size, and the sort puts the largest blob first. The model file is gone from the working tree and still 3 MiB of the repository. Had it been 300 MiB, the push would be rejected for commit `c7182c9`. The remedies are rewriting unpushed history (Chapter 9) or Git LFS (Chapter 22), and for a model file the better answer is usually a registry (Chapter 28).

15.16 The GitHub CLI

In one sentence. `gh` is a client for GitHub's API that knows which repository you are in, so the GitHub objects of section 15.2 become reachable from the directory where your Git data is.

Precisely. `gh` is not Git and does not replace it. It calls the REST and GraphQL APIs with a stored token, runs `git` for you where a task needs both (`gh repo clone`, `gh pr checkout`), and can act as Git's credential helper (Chapter 16). The command families of the installed version:

```
$ gh --help | sed -n '/^CORE COMMANDS/,/^HELP TOPICS/p' | sed '$d'
CORE COMMANDS
  auth:      Authenticate gh and git with GitHub
  browse:    Open repositories, issues, pull requests, and more in the browser
  codespace: Connect to and manage codespaces
  gist:      Manage gists
  issue:     Manage issues
  org:       Manage organizations
  pr:        Manage pull requests
  project:   Work with GitHub Projects.
  release:   Manage releases
  repo:      Manage repositories

GITHUB ACTIONS COMMANDS
  cache:     Manage GitHub Actions caches
  run:       View details about workflow runs
  workflow:  View details about GitHub Actions workflows

ALIAS COMMANDS
  co:        Alias for "pr checkout"

ADDITIONAL COMMANDS
  agent-task: Work with agent tasks (preview)
  alias:      Create command shortcuts
  api:        Make an authenticated GitHub API request
  attestation: Work with artifact attestations
  completion: Generate shell completion scripts
  config:     Manage configuration for gh
  copilot:    Run the GitHub Copilot CLI (preview)
  extension:  Manage gh extensions
  gpg-key:    Manage GPG keys
  label:      Manage labels
  licenses:   View third-party license information
  preview:    Execute previews for gh features
  ruleset:    View info about repo rulesets
  search:     Search for repositories, issues, and pull requests
  secret:     Manage GitHub secrets
  ssh-key:    Manage SSH keys
  status:     Print information about relevant issues, pull requests, and notifications across
repositories
  variable:   Manage GitHub Actions variables
```

(Volatile: this is `gh` 2.88.1.) The families you will use, with flags that were each checked against `--help` of that version:

Family	Subcommands you need	Reaches	Verified flags worth knowing
<code>gh auth</code>	<code>login</code> , <code>status</code> , <code>setup-git</code> , <code>refresh</code> , <code>switch</code> , <code>token</code> , <code>logout</code>	the stored token	<code>--hostname</code> , <code>--git-protocol</code> , <code>--scopes</code> , <code>--with-token</code> (reads standard input)
<code>gh repo</code>	<code>create</code> , <code>clone</code> , <code>fork</code> , <code>view</code> , <code>edit</code> , <code>sync</code> , <code>set-default</code> , <code>list</code> , <code>archive</code> , <code>rename</code> , <code>delete</code> , <code>deploy-key</code> , <code>license</code> , <code>gitignore</code>	repositories and their settings	<code>create --public --source=. --remote=origin --push</code> ; <code>view --json</code> ; <code>edit</code> as in section 15.5
<code>gh issue</code>	<code>create</code> , <code>list</code> , <code>view</code> , <code>edit</code> , <code>close</code> , <code>comment</code> , <code>develop</code>	issues	<code>create --title --body --label</code> ; <code>develop --checkout</code>
<code>gh pr</code>	<code>create</code> , <code>list</code> , <code>status</code> , <code>view</code> , <code>checkout</code> , <code>checks</code> , <code>diff</code> ,	pull requests (Chapter 17)	<code>create --fill --draft --base</code> ; <code>checks --watch --required</code> ; <code>merge --</code>

Family	Subcommands you need	Reaches	Verified flags worth knowing
	review, ready, merge, update-branch, revert		--squash --merge --rebase --auto --delete-branch --match-head-commit
gh release	create, list, view, edit, upload, download, delete, verify, verify-asset	releases	section 15.12
gh run, gh workflow, gh cache	run list, view, watch, rerun, download; workflow list, run, view	Actions (Chapter 20B)	covered there
gh secret, gh variable	set, list, delete; variable get	Actions, Dependabot and Codespaces secrets	secret set NAME reads the value from a prompt or standard input; values "are locally encrypted before being sent"
gh ruleset	list, view, check	rulesets, read-only	check --default; list --org
gh api	one command	everything else	section 15.17

```
$ gh ruleset --help | sed -n '/^AVAILABLE COMMANDS/,/^FLAGS/p' | sed '$d'
AVAILABLE COMMANDS
  check:      View rules that would apply to a given branch
  list:       List rulesets for a repository or organization
  view:       View information about a ruleset
```

gh ruleset can list, view and check. It cannot create or change a ruleset, so rulesets are automated through gh api (Chapter 18).

Four behaviors explain most surprises:

- **Which repository?** Inside a clone, gh picks the repository from your remotes. With a fork there are two candidates, and gh repo set-default records which one "to use when querying the GitHub API" for pull requests, issues, releases and Actions. Outside a clone, or to override, pass -R OWNER/REPO or set GH_REPO.
- **Machine-readable output.** Most listing commands take --json FIELDS, then --jq EXPRESSION or --template; the filter language is built in (gh help formatting). --json without a field list prints the available fields.
- **Exit codes.** 0 for success, 1 for failure, 2 when cancelled, 4 when authentication is required; gh pr checks adds 8 for pending checks (gh help exit-codes, gh pr checks --help). Scripts should test them and not parse human-readable output.
- **Its own token.** GH_TOKEN in the environment takes precedence over the stored login (gh help environment): a stale one in a shell profile explains "gh works in one terminal and not in another".

Version note. Older behavior: gh 2.88.1, installed here and dated 12 March 2026, has no gh discussion, no gh skill, no gh repo read-file, and no flags for issue types, sub-issues or dependencies. Current behavior: gh 2.102.0 of 30 September 2026 has them (discussion and skill in preview), adds gh pr checkout --worktree, and offers to install official extensions such as gh stack for stacked pull requests. Since: 2.89.0 to 2.102.0 (releases, manual). Recommended: upgrade, because six of those releases contain security fixes (research notes, section 9). From 2.91.0 the CLI collects pseudonymous telemetry unless you opt out with gh config set telemetry disabled (changelog). The two versions were not compared flag by flag, so check --help after upgrading.

Outdated advice. The gh-copilot extension stopped functioning on 25 October 2025 (changelog). Extensions in general "are not verified, signed, or endorsed by GitHub" (manual): installing one is running

someone's code with your token.

In production. A release script runs `gh release create "$VERSION" --generate-notes`. On the day `VERSION` holds a typo, a tag that nobody intended exists on the default branch. `--verify-tag` turns that into an error before anything is created.

15.17 `gh api`, the REST API, GraphQL and rate limits

In one sentence. Everything GitHub shows you comes from an API that you can call yourself; `gh api` adds your token, the host and the placeholders, and prints the JSON.

Precisely. The **REST API** is a set of URLs under `https://api.github.com`, one per resource, such as `GET /repos/{owner}/{repo}/pulls`. `gh api` takes the path and fills in `{owner}`, `{repo}` and `{branch}` from the current repository:

```
gh api repos/{owner}/{repo} # one object
gh api repos/{owner}/{repo} --jq '{full_name, default_branch, visibility, fork}'
gh api repos/{owner}/{repo}/pulls --jq '.[] | "#\(.number) \(.title)"'
gh api repos/{owner}/{repo}/rulesets --jq '.[] | "\(.id) \(.name) \(.enforcement)"'
gh api rate_limit --jq '.resources.core'
gh api -H 'X-GitHub-API-Version: 2026-03-10' repos/{owner}/{repo}/pulls --paginate --jq '.[].number'
```

The paths and field names were checked against the REST reference on 2 October 2026 ([repositories](#), [pulls](#), [rules](#), [rate limit](#)). The commands were not run. The flags are those of the installed CLI:

```
$ gh api --help | sed -n '/^FLAGS/,/^INHERITED FLAGS/p' | sed '$d'
FLAGS
  --cache duration      Cache the response, e.g. "3600s", "60m", "1h"
  -F, --field key=value Add a typed parameter in key=value format (use "@<path>" or "@-" to
read value from file or stdin)
  -H, --header key:value Add a HTTP request header in key:value format
  --hostname string      The GitHub hostname for the request (default "github.com")
  -i, --include          Include HTTP response status line and headers in the output
  --input file           The file to use as body for the HTTP request (use "-" to read from
standard input)
  -q, --jq string        Query to select values from the response using jq syntax
  -X, --method string     The HTTP method for the request (default "GET")
  --paginate            Make additional HTTP requests to fetch all pages of results
  -p, --preview strings  Opt into GitHub API previews (names should omit '-preview')
  -f, --raw-field key=value Add a string parameter in key=value format
  --silent              Do not print the response body
  --slurp               Use with "--paginate" to return an array of all pages of either JSON
arrays or objects
  -t, --template string  Format JSON output using a Go template; see "gh help formatting"
  --verbose             Include full HTTP request and response in the output
```

Rules of the road, each from the help text or the linked page:

- **Method.** `GET` by default, `POST` as soon as you add a field with `-f` or `-F`. To send parameters with a `GET`, say `-X GET`.
- **Fields.** `-f key=value` sends a string. `-F key=value` converts `true`, `false`, `null` and integers to JSON types and reads `@file`.
- **Pagination.** Lists return 30 items per page unless you ask otherwise ([pagination](#)). `--paginate` follows the pages; `--slurp` wraps them into one array.
- **Versioning.** The REST API is versioned by date through the `X-GitHub-API-Version` header. Without the header a request gets version `2022-11-28`, which is supported until 10 March 2028; `2026-03-10` is the first version with breaking changes; an unsupported version answers `410 Gone` ([API versions](#), [changelog](#)).

The CLI pins its own REST calls to 2022-11-28 since 2.87.0 ([release](#)). In a script that must keep working, send the header explicitly.

- **Not found means "not for you".** For a private resource and a token without access, the API answers 404 Not Found, not 403, "to avoid confirming the existence of private repositories" ([troubleshooting](#)). Chapter 16 builds its diagnosis on this.

See it. `--jq` takes the filter language of the `jq` program. The filters can be rehearsed offline with `jq` itself. The input here is `labs/ch15/api-examples/pulls-sample.json`, a practice document written for this course with the field names of the pull request list; it is not output from GitHub.

```
$ jq -r '.[ ] | "#\(.number) \(.user.login) \(.head.ref) -> \(.base.ref)'" pulls-sample.json
#14 asha-rao feature/list-names -> main
#15 ravi-menon feature/sqlite-store -> main
#16 asha-rao backport/empty-names -> release/0.1
```

```
$ jq -r '.[ ] | select(.draft | not) | select(.base.ref == "main") | .number' pulls-sample.json
14
$ jq '[. [ ] | select(.user.login == "asha-rao")] | length' pulls-sample.json
2
```

`. [ ]` iterates over the array, `\( )` interpolates a field into a string, `select` keeps the elements for which the condition holds, and `-r` prints strings without quotes. With `gh api`, the same filter goes after `--jq` and no `-r` is needed.

GraphQL is the second API: one endpoint, a typed schema, and a query that names exactly the fields wanted, so one call can replace several REST requests ([about the GraphQL API](#)). `gh api graphql -f query='...'` calls it. Some features exist in only one of the two APIs.

Rate limits ([rate limits](#)):

Caller	Primary limit
No authentication	60 requests per hour, per IP address
An authenticated user (your token, <code>gh</code>)	5,000 requests per hour
A GitHub App installation	5,000 per hour at minimum; 15,000 on Enterprise Cloud organizations
<code>GITHUB_TOKEN</code> in a workflow	1,000 requests per hour per repository

Secondary limits apply on top: at most 100 concurrent requests, and a points budget per minute. Exceeding a limit returns 403 or 429, so a 403 is not always a permission problem; the `x-ratelimit-*` headers and `gh api rate_limit` say where you stand.

In production. A nightly job lists pull requests of sixty repositories with unauthenticated `curl`. It worked from a laptop and fails on the shared CI runner, where the sixty requests per hour of that IP address are spent before it starts. Unauthenticated limits were lowered on 8 May 2025, anonymous HTTPS clones included ([changelog](#)). The fix is an identity for the job, not a retry loop.

15.18 Webhooks and GitHub Apps: the integration model

Polling asks GitHub repeatedly whether something happened. A **webhook** reverses the direction: you register a URL and the events you care about on a repository, an organization or a GitHub App, and GitHub sends an HTTP request with a JSON payload when one occurs ([about webhooks](#)). A `push` webhook carries the ref and the commit IDs before and after the push ([webhook events](#)), which is the information of a reflog line, delivered to your server.

A **GitHub App** is the identity an integration should have. It is installed on chosen repositories, holds fine-grained permissions, receives webhooks, and acts with installation tokens that expire after one hour. It is not tied to a person, so it does not stop working when someone leaves. GitHub's own guidance is that "GitHub Apps are preferred over OAuth apps" ([deciding when to build a GitHub App](#)). The alternatives are a personal token, which acts as you everywhere you have access, and a machine user. Chapter 16 compares the credentials and their lifetimes.

One fact for anyone who writes tooling: the API for check runs and check suites is available only to GitHub Apps (same page); the older commit statuses can be created with Write access (section 15.4).

Unverified. Webhook delivery retries and signature validation were not researched for the Phase 0 report, and are not described here.

15.19 Organization governance in brief

What an organization gives you beyond a container, each item a GitHub object that no clone carries (report, sections 2 and 13; notes, section 13):

Control	What it does	Plan
Base permission, teams, roles	section 15.4	all
Required two-factor authentication	members without it lose access to the organization's resources until they enable it (docs)	all
Personal access token policy	allow or restrict each token type, set a maximum lifetime, require approval of fine-grained tokens (docs)	all
Repository rulesets; organization rulesets	rules on branches, tags and pushes (Chapter 18)	repository rulesets: public repositories on Free, all on paid plans; organization rulesets: Team
Custom properties	typed metadata on repositories that rulesets can target (docs)	availability on Free organizations not confirmed by the report
Audit log	who did what in the organization, 180 days, searchable (docs)	all; streaming and Git events on Enterprise Cloud, where Git events are retained for seven days and are not shown in the web interface (Chapter 13, section 13.15; Chapter 21B, section 21B.8)
SAML single sign-on, IP allow lists, custom roles, enterprise policies	identity and network controls	Enterprise Cloud

Unverified. GitHub's documentation contradicts itself on whether organization rulesets need Enterprise or Team; the report prefers the later sources, which say Team ([changelog](#)).

15.20 What can go wrong

Symptom	Diagnosis	Fix	Prevention
A member can read repositories nobody	the base permission, then the person's teams (section 15.4)	set the base permission on	review both whenever people

Symptom	Diagnosis	Fix	Prevention
granted		purpose; grant through teams	join
A person who left still has access	deploy keys and tokens are not membership: <code>gh repo deploy-key list</code>	remove the key; rotate what it could read	GitHub Apps and short-lived tokens (Chapter 16)
A commit from a deleted fork is still reachable	the network's shared object store (section 15.7)	revoke what it exposed (Chapter 21B)	push protection; no secrets in Git
The issue did not close after the merge	<code>gh pr view N --json baseRefName, closingIssuesReferences</code> : wrong base, or no keyword	close by hand	document how issues close on release branches
<code>git describe</code> disagrees with the release page	<code>git cat-file -t TAG</code> prints <code>commit</code> (section 15.12)	describe with <code>--tags</code> , or tag properly under a new name	annotated tag first; <code>--verify-tag</code>
A push is rejected for a file that is not in the working tree	the pipeline of section 15.15	rewrite unpushed history; LFS or a registry	size check before the first push
The issue form is missing from the chooser	invalid for GitHub, or not on the default branch (validation errors)	fix on the default branch	review forms like code
<code>gh</code> acts on the wrong one of fork and upstream	<code>gh repo set-default --view</code>	<code>gh repo set-default OWNER/REPO</code> , or <code>-R</code>	set it once per clone
A script reports exactly 30 items, or created something by accident	no pagination; a field flag switched the method to <code>POST</code>	<code>--paginate</code> ; <code>-X GET</code>	both in every script
<code>404</code> from the API for a repository you can open in the browser	the token lacks access: <code>gh auth status</code> (Chapter 16)	a credential with access	record each token's owner and resources

15.21 When not to use it, and dangerous edge cases

- **A fork, a private repository or a deleted branch does not make a secret safe or gone.** Fork networks share Git data, upstream owners can read all forks, and visibility changes rewrite nothing.
- **Do not create releases for tags that do not exist.** Push the tag you mean, then `--verify-tag`.
- **Do not grant Write for convenience.** Write can merge, release, define code owners, and edit workflows and Actions secrets.
- **Do not build tooling on `gh`'s human-readable output or on a personal token.** Use `--json`, the version header, and a GitHub App for anything that must outlive you.
- **Visibility changes, transfers, renames and deletion act on GitHub objects that no clone can restore:** stars, watchers, forks' attachment, push rulesets, and the 90-day restore window that a non-empty fork network removes.
- **A public practice repository is public.** Never push real secrets, customer names or work code to it.

15.22 Command safety

Command	Label	What it changes	Preview	Recovery
<code>gh repo view</code> , <code>gh issue list</code> , <code>gh pr view</code> , <code>gh release list</code> , <code>gh ruleset list</code> , <code>gh auth status</code> , <code>gh api</code> with GET, <code>gh api graphql</code> with a query	● SAFE	nothing (a GraphQL query only reads, although it is sent as a POST)	not needed	not needed
<code>gh repo clone</code> , <code>gh pr checkout</code> , <code>gh release download</code>	● SAFE	local files and refs	not needed	delete them
<code>gh repo create --source=.</code> <code>--push</code>	● CAUTION	a repository on GitHub, a remote, a push	<code>git log</code> , <code>git status</code> ; check <code>--public</code> or <code>--private</code>	<code>gh repo delete</code> ; what reached a public repository is published
<code>gh issue create</code> , <code>gh pr create</code> , <code>gh label create</code> , <code>gh release create --verify-tag --draft</code>	● CAUTION	GitHub objects that notify people	<code>gh pr create --dry-run</code> ; <code>--draft</code>	close or delete; notifications are not recalled
<code>gh repo edit</code> (features, merge methods), <code>gh repo fork</code> , <code>gh repo sync</code>	● CAUTION	settings; a new repository; a fast-forward	<code>gh repo view --json</code> ; <code>git log HEAD..upstream/main</code>	set the previous value; delete the fork
<code>gh api</code> with POST, PATCH, PUT, DELETE; <code>gh api graphql</code> with a mutation	● DANGEROUS	whatever the endpoint says, with all your permissions	the GET first	depends on the endpoint; often none
<code>gh release create TAG</code> without <code>--verify-tag</code>	● DANGEROUS	may create a tag on the default branch and start tag workflows	<code>git ls-remote --tags origin TAG</code>	delete release and tag; consumers may have fetched it
<code>gh release delete --cleanup-tag</code> , <code>git push origin --delete TAG</code>	● DANGEROUS	a published version's name	<code>gh release view TAG</code>	re-create from the recorded commit ID
<code>gh repo sync --force</code>	● DANGEROUS	hard reset of the destination branch	<code>git log upstream/main..origin/main</code> after a fetch	a clone that still has the commits
<code>gh repo edit --visibility</code> , <code>gh repo delete</code> , <code>gh repo rename</code> , <code>gh repo archive</code>	● DANGEROUS	visibility with its side effects; the repository, its URL, its writability	section 15.5; ask who depends on the URL	visibility can be changed back, its side effects cannot; restore within 90 days unless the fork network is not empty

Every ● command here changes GitHub objects that no clone contains. Run the preview, which is always a read with the same tool, and name who is affected.

15.23 Version notes

The dated changes are given where they are used: accounts (15.3), pull request access (15.5), issues (15.8), Projects (15.10), the Docker registry (15.11), immutable releases (15.12), the CLI (15.16) and the REST API (15.17).

Version note. Older behavior: tag protection rules guarded release tags. Current behavior: they were retired and migrated to tag rulesets. Since: 30 August 2024 ([sunset notice](#)). Recommended: a tag ruleset on your version pattern, plus immutable releases.

15.24 Practice

- **Labs 19.1 and 19.2** in the Module 19 lab manual: create the practice organization and repository, then inventory what in it is Git data and what is a GitHub object. Every later GitHub lab uses that repository.
- **Labs 25.1 and 25.2** in the Module 25 lab manual: a feature cycle with `gh`, and queries with `gh api`. Do them after Chapter 16.
- Answers: Module 19, Module 25.
- Replay any transcript with `labs/run ch15/<demo>`. Two drills; predict, then run:
 1. `ch15/fork-network`: between steps 7 and 8, clone the upstream namespace again and fetch the commit by ID. Does it work, and why?
 2. `ch15/large-objects`: push only a branch that never contained the model file to a new bare repository. Is the 3 MiB blob sent?

15.25 Interview questions

1. Take ten things on a repository's page. For each, is it Git data or a GitHub object, and how do you prove it from a terminal?
2. A commit with a credential was pushed to a fork of your public repository, and the fork was deleted. What is still true, what does GitHub document, and what do you do first?
3. A member has Read through the base permission, Triage through one team and Write through another. What can she do? Which setting do you check when she can do more than intended?
4. Explain a Git tag versus a GitHub Release. How can a release point at a commit nobody chose, and what prevents it?
5. `git describe` prints a different version than the release page. Root cause and evidence?
6. A pull request said "Fixes #812" and the issue is still open after the merge. Give three causes and how to tell them apart.
7. You must move a repository to another host without loss. What does a mirror clone carry, and how do you inventory the rest?
8. What changes and what does not when a public repository is made private? Answer for Git data, forks, stars, and someone who cloned it yesterday.
9. A script using `gh api` returns 30 results everywhere, and once created an issue by accident. Explain both.
10. When do you build a GitHub App instead of using a personal token or a machine user?

15.26 Sources

Primary sources

- GitHub Docs, read on 2 October 2026: [types of accounts](#), [repository roles](#), [base permissions](#), [repository visibility](#), [forks](#), [linking a pull request to an issue](#), [about releases](#), [immutable releases](#), [issue forms](#), [community health files](#), [repository limits](#), [large files](#), [Packages](#), [API versions](#), [rate limits](#), [webhooks](#).
- The GitHub CLI: `gh <command> --help` of the installed 2.88.1 for every command and flag in this chapter; the [manual](#) and [release notes](#) for newer versions.
- The GitHub Changelog entries linked in the sections.

- Git: [gitnamespaces](#) for the model in section 15.7; [git-describe](#), [git-rev-list](#), [git-cat-file](#), as installed with Git 2.55.0.

Secondary sources

- The Phase 0 report of this course, sections 2, 3, 4 and 12, and its research notes on the GitHub platform: the dated changes, the plan gates and the flags marked unverified here.
- Check Point Research, [Stargazers Ghost Network](#), cited by the report.

Videos (optional; the report's assessments rest on captions and chapter lists, not on full viewing)

- [The ultimate beginner's guide to GitHub in 2026](#), GitHub, 51 minutes, 22 September 2025. Caveats: mechanics only; compiled from 2024 episodes.
- [Git and GitHub - Full Course](#), ThePrimeagen for Boot.dev, 12 November 2024: remotes and GitHub from 1:54:18, then forks.
- The report found no verified video on roles, fork networks, releases or the API at this depth.

Further reading

- [GitHub flow](#) and [best practices for organizations](#).
- Chapter 18 for governance, Chapter 21B for fork networks during a secret leak, Chapter 27 for open-source practice.

Chapter 16: Authentication

Baseline. Git 2.55.0, OpenSSH 10.2 and GitHub CLI 2.88.1 on macOS; GitHub facts as of 1 October 2026, re-read on docs.github.com on 2 October 2026 where a link is given. Transcripts are real output from `labs/ch16/`. Nothing in this chapter connected to GitHub, read the keychain, an SSH agent, `~/.ssh` or the real Git configuration: every demo runs in a sandbox with a toy credential helper, throwaway keys, and `ssh -G`, which prints configuration without connecting. What GitHub's servers answer is described from the linked documentation.

16.1 Why this matters

Four questions a CTO can ask the morning after:

1. "The deploy job says `Repository not found`. I am looking at the repository in my browser. Which of the two is wrong?"
2. "The new laptop asked for a password on `git push`. She typed the right one. Why was it refused?"
3. "We gave Ravi write access on Monday. On Wednesday his push still fails with 403, and Git never asks him for anything. Why?"
4. "The engineer who wrote the nightly sync left in June. What does the job authenticate as today?"

Neither is wrong: the job's credential cannot see the repository, and GitHub answers "not found" on purpose (section 16.19). GitHub has not accepted account passwords for Git since 13 August 2021 (section 16.3). His keychain holds a credential for another account; a 403 does not make Git discard it, so Git keeps sending it (section 16.5). And the job runs as whoever created its token or key, which is why the answer should be "as an app that nobody can leave" (section 16.14).

Every one of these is diagnosed with the same three questions, asked in this order:

Question	Decided by	Evidence
Which transport and which server?	the remote URL, after rewrites	<code>git remote -v</code> , <code>git remote get-url origin</code>
Which credential does the client present?	Git's credential helpers for HTTPS; <code>ssh</code> , its configuration and the agent for SSH	<code>GIT_TRACE=1, git config get --show-origin --all credential.helper, ssh -G, ssh-add -l</code>
Who does the server say you are, and what may that identity do?	GitHub: the account behind the credential, its roles, token permissions, SSO authorization	<code>ssh -T git@github.com, gh auth status</code> , the repository's access settings

A failure is always in exactly one of the three. Commands copied from a search result usually change a different one.

16.2 Authentication versus authorization

In one sentence. Authentication establishes which account a connection belongs to; authorization decides what that account may do to this repository.

Analogy. A badge reader and a door list. The reader checks that the badge is real and whose it is. The list on each door says which badge holders may enter. The analogy breaks because GitHub often gives the same

answer to both failures: an unknown badge and a known badge without access can both get "there is no such door", so the error text alone does not tell you which check failed.

Precisely. Git has no accounts and no passwords. It delegates the connection to a transport (Chapter 12, section 12.13):

- Over **HTTPS**, Git speaks HTTP and attaches a username and a secret obtained from a credential helper or a prompt. For GitHub the secret is a token.
- Over **SSH**, Git starts the `ssh` program, which proves possession of a private key. Git never sees the key.

The server maps the token or key to an account (authentication) and then evaluates roles, token permissions and organization rules for the repository (authorization, Chapter 15, section 15.4).

Three identities are involved in everyday work, and they are independent of each other:

Identity	What it is	Set by	Checked by
Commit identity	the author and committer name and email written into commits	<code>user.name</code> , <code>user.email</code>	nobody: it is an assertion (Chapter 14B, section 14B.18)
Authentication identity	the GitHub account that a token or SSH key belongs to	the credential that the transport presents	GitHub, on every connection
Signing identity	the key that signed a commit or tag	<code>user.signingKey</code>	whoever verifies the signature (Chapter 21B)

You can push commits written by anyone, as any account that has Write, signed by any key or none. "It says Asha in the log" is not evidence of who pushed.

In production. A push to the release branch is traced. `git log` names the author, which proves nothing. The push itself was authenticated: the organization's audit log and the repository's activity view record the account, and for a token the audit log records which token (Chapter 21B). Keep the two questions apart: who wrote the commit, and who moved the ref.

16.3 HTTPS: a token, never a password

In one sentence. Over HTTPS, GitHub accepts a token in the place where HTTP expects a password, and nothing else.

Precisely. "Beginning August 13, 2021, we will no longer accept account passwords when authenticating Git operations", GitHub announced ([GitHub Blog](#)). Git still calls the secret a password, because to Git it is one: the prompt reads `Password for 'https://USER@github.com':` (section 16.4 shows it). GitHub's documentation says what to do at that prompt: "enter your personal access token", or better, let a credential helper supply one ([about remote repositories](#)). In practice nobody should type a token: the GitHub CLI or Git Credential Manager obtains one through the browser and hands it to Git ([caching credentials](#)).

Outdated advice. Any tutorial in which `git push` asks for the GitHub account password and succeeds was recorded before 13 August 2021, or is wrong: the Phase 0 report lists 2023 videos that still state it. The same goes for `git://` URLs and DSA keys, removed on 15 March 2022 ([GitHub Blog](#)).

Unverified. The exact text that GitHub's server sends when a password is used is not quoted in any GitHub documentation the report could find. User reports show `remote: Invalid username or token. Password authentication is not supported for Git operations.`, and older reports a sentence naming the date 13

August 2021. Recognize the situation by Git's own last line, fatal: Authentication failed for '<URL>', which is in Git's source ([remote-curl.c](https://github.com/git/git/blob/master/remote-curl.c)).

Two more facts about the HTTPS path. Anonymous HTTPS works for public repositories, with rate limits that were lowered on 8 May 2025 ([changelog](#)). And since 15 September 2026 GitHub refuses SHA-1 in TLS, which only very old clients notice ([changelog](#)).

16.4 How Git asks for a credential

In one sentence. When a server demands authentication, Git asks each configured credential helper in turn, then falls back to prompting, and afterwards tells the helpers whether the credential worked.

Analogy. A receptionist with a key cabinet. Asked for a key, the receptionist looks in the cabinet (`get`). If the visitor had to bring their own key and it opened the door, it is hung in the cabinet (`store`). If a key from the cabinet did not open the door, it is thrown away (`erase`). The analogy breaks at the last step: the receptionist throws a key away only when the door says "wrong key" (401), not when it says "you may not enter" (403).

Precisely. A helper is a program that Git runs with one argument, `get`, `store` or `erase`, and a description of the request on standard input as `key=value` lines: `protocol`, `host`, and optionally `path` and `username` ([gitcredentials](#)). `git credential fill`, `approve` and `reject` are the documented way to drive the same machinery by hand ([git-credential](#)), so the whole exchange can be watched without a server.

See it. No helper is configured in the sandbox, and the lab forbids terminal prompts, as a CI job does:

```
$ git config get --show-origin --all credential.helper
[exit status: 1]
$ printf 'protocol=https\nhost=github.com\n\n' | git credential fill
fatal: could not read Username for 'https://github.com': terminal prompts disabled
[exit status: 128]
```

This is the error of every CI job that reaches a private repository without a credential. In a terminal Git would prompt. A stand-in for the person shows what it would ask, through `GIT_ASKPASS`:

```
# ~/lab-bin/askpass stands in for you at the keyboard: it prints the prompt and answers.
$ printf 'protocol=https\nhost=github.com\n\n' | GIT_ASKPASS=~/lab-bin/askpass git credential fill
Git asked: Username for 'https://github.com':
Git asked: Password for 'https://lab-user@github.com':
protocol=https
host=github.com
username=lab-user
password=taped-at-the-prompt
```

Two prompts, a username and a "password". Now a helper. The lab's helper is a short shell script, named `git-credential-labstore` so that the configuration value `labstore` finds it; it stores one credential in a plain file and logs every call (Lab 20.2 prints it):

```
$ git config set --global credential.helper labstore
$ git config get --show-scope --all credential.helper
global labstore
$ printf 'protocol=https\nhost=github.com\n\n' | git credential fill
fatal: could not read Username for 'https://github.com': terminal prompts disabled
[exit status: 128]
$ cat ~/helper.log
labstore get <- protocol=https host=github.com
```

Git asked the helper (`get`), the helper had nothing, and Git fell through to the prompt that the lab forbids. After a successful request Git calls `approve`, here by hand:

```
# What Git does after a server accepted a credential that you typed:
$ printf 'protocol=https\nhost=github.com\nusername=lab-user\npassword=FAKE-TOKEN-not-a-real-credential\n\n' | git credential approve
$ cat ~/.labstore-credentials
username=lab-user
password=FAKE-TOKEN-not-a-real-credential
```

```
$ printf 'protocol=https\nhost=github.com\n\n' | git credential fill
protocol=https
host=github.com
username=lab-user
password=FAKE-TOKEN-not-a-real-credential
$ printf 'url=https://github.com/acme-pay/billing-api.git\n\n' | git credential fill
protocol=https
host=github.com
username=lab-user
password=FAKE-TOKEN-not-a-real-credential
```

The second form gives Git a URL and lets it derive protocol and host. The path was dropped: by default a credential is per host, not per repository (section 16.5). `GIT_TRACE=1` names the program that answered:

```
$ printf 'protocol=https\nhost=github.com\n\n' | GIT_TRACE=1 git credential fill 2>&1 >/dev/null |
grep -o 'trace: run_command.*'
trace: run_command: 'git credential-labstore get'
trace: run_command: git-credential-labstore get
```

When a server answers 401 to a credential that Git sent, Git calls `reject`:

```
# What Git does after a server answered 401 to a credential it had sent:
$ printf 'protocol=https\nhost=github.com\nusername=lab-user\npassword=FAKE-TOKEN-not-a-real-credential\n\n' | git credential reject
$ cat ~/.labstore-credentials
cat: $LAB/ch16/credential-protocol/home/.labstore-credentials: No such file or directory
[exit status: 1]
$ printf 'protocol=https\nhost=github.com\n\n' | git credential fill
fatal: could not read Username for 'https://github.com': terminal prompts disabled
[exit status: 128]
```

```
$ cat ~/helper.log
labstore get  <- protocol=https host=github.com
labstore store <- protocol=https host=github.com username=lab-user password=<hidden>
labstore get  <- protocol=https host=github.com
labstore get  <- protocol=https host=github.com
labstore get  <- protocol=https host=github.com
labstore erase <- protocol=https host=github.com username=lab-user password=<hidden>
labstore get  <- protocol=https host=github.com
```

The log is the protocol: `get` before a request, `store` after success, `erase` after a rejected credential.

Inside `.git`. Nothing. A credential is never stored in the repository unless you put it there (section 16.7). The helper list is configuration, usually at system or global scope, and the secret is wherever the helper keeps it.

State table.

Event	Working tree, index, HEAD, refs	Git configuration	Helper's store	Remote	GitHub
Request needs a credential (<code>fill</code>)	unchanged	unchanged	read (<code>get</code>)	unchanged	unchanged

Event	Working tree, index, HEAD, refs	Git configuration	Helper's store	Remote	GitHub
Server accepted it (approve)	unchanged	unchanged	written (store)	unchanged	unchanged
Server answered 401 (reject)	unchanged	unchanged	entry removed (erase)	unchanged	unchanged
Server answered 403 or 404	unchanged	unchanged	unchanged	unchanged	unchanged

The last row is from Git's source: `credential_reject` is called for a 401 with a credential that was sent, and for nothing else on this path ([http.c](http://c)).

In production. The CTO's third question. Ravi once cloned a personal project over HTTPS, and the keychain stored that account. For the work repository GitHub answers 403: a valid account without permission. Git reports the failure and erases nothing, so every later push sends the same credential. Granting access to his work account changed nothing, because that account never arrives at the server. The evidence is `GIT_TRACE=1` (which helper) and `gh auth status` (which account); the fix is to make the right account answer for that host (section 16.5).

16.5 Which helper answers, and where the token lives

Precisely. `credential.helper` is a list. Git asks each helper in order and stops at the first that returns a username and a password. An empty value resets the list, which is how a later configuration file discards helpers set by an earlier one. `credential.<URL>.helper` and `credential.<URL>.username` apply only to matching URLs, and `credential.useHttpPath` makes the repository path part of the lookup ([gitcredentials](#)).

See it. Two helpers stand for two stored accounts:

```
$ git config set --global credential.helper labstore
$ git config set --global --append credential.helper workstore
$ git config get --all credential.helper
labstore
workstore
$ printf 'protocol=https\nhost=github.com\n\n' | git credential fill
protocol=https
host=github.com
username=personal-user
password=FAKE-TOKEN-not-a-real-credential
$ cat ~/helper.log
labstore get <- protocol=https host=github.com
```

The first helper answered and the second was never asked. Order is the whole rule. To make a different helper answer for one host, reset the list for that host:

```
# An empty value resets the list; what follows it replaces every helper configured before.
$ git config set --global --append credential.https://github.com.helper ''
$ git config set --global --append credential.https://github.com.helper workstore
$ cat ~/.gitconfig
[user]
    name = Lab User
    email = you@example.com
[init]
    defaultBranch = main
[gc]
    reflogExpire = never
    reflogExpireUnreachable = never
[credential]
    helper = labstore
    helper = workstore
```

```
[credential "https://github.com"]
    helper =
    helper = workstore
$ printf 'protocol=https\nhost=github.com\n\n' | git credential fill
protocol=https
host=github.com
username=work-user
password=FAKE-TOKEN-not-a-real-credential
$ printf 'protocol=https\nhost=gitlab.example\n\n' | git credential fill
protocol=https
host=gitlab.example
username=personal-user
password=FAKE-TOKEN-not-a-real-credential
$ cat ~/helper.log
workstore get    <- protocol=https host=github.com
labstore get    <- protocol=https host=gitlab.example
```

```
$ git config set --global credential.https://github.com.username work-user
$ git config set --global credential.https://github.com.useHttpPath true
$ printf 'url=https://github.com/acme-pay/billing-api.git\n\n' | git credential fill
protocol=https
host=github.com
path=acme-pay/billing-api.git
username=work-user
password=FAKE-TOKEN-not-a-real-credential
$ cat ~/helper.log
workstore get    <- protocol=https host=github.com path=acme-pay/billing-api.git username=work-user
```

With `useHttpPath`, the helper receives `path=`, so it can keep one credential per repository; with a fixed `username`, it is asked for that account. GitHub's guide for several accounts over HTTPS uses exactly this setting ([managing multiple accounts](#)).

The helpers you will meet on a Mac:

Helper	Configuration value	Where the secret lives	Notes
<code>git-credential-osxkeychain</code>	<code>osxkeychain</code>	the macOS login keychain, as an "internet password" for <code>github.com</code>	ships with Git on macOS; stores whatever you typed, typically a token you created by hand. GitHub's page recommends SSH or Git Credential Manager instead (docs)
GitHub CLI	<code>!<path>/to/gh auth git-credential</code> , for <code>https://github.com</code>	gh's token in the system credential store; a plain text file if none is available (<code>gh auth login --help</code>)	written by <code>gh auth setup-git</code> , or by <code>gh auth login</code> when you let it authenticate Git
Git Credential Manager	<code>manager</code>	the keychain	obtains tokens through the browser, handles two-factor authentication; installed separately (docs)
<code>git-credential-store</code>	<code>store</code>	a plain text file	the manual calls it discouraged; do not use it
<code>git-credential-cache</code>	<code>cache</code>	memory of a background process, for a limited time	not used in this course: it starts a daemon

What `gh auth setup-git` writes was read from the CLI's source at 2.88.1, not run: for each authenticated host it first sets `credential.https://github.com.helper` to the empty value, to cut off helpers configured elsewhere, then adds `!<path> to gh> auth git-credential`, in the global configuration, and the same for the gist host ([helper_config.go](#)). It is the reset-then-add pattern of the transcript above. Lab 20.2 has you read the result on your own machine:

```
git config get --show-origin --all credential.helper
git config get --show-origin --all credential.https://github.com.helper
gh auth status
```

`gh auth status` prints, per host, the active account, where its token is stored and the token's scopes, with the token masked; it exits with status 1 when an account has a problem (`gh auth status --help`). `gh auth login` runs a browser flow by default and stores the resulting token "securely in the system credential store" (`gh auth login --help`).

Git, not GitHub. `labs/shell` and the replays switch off the system configuration, where macOS installations of Git usually configure `osxkeychain`. That is why the labs that contact GitHub run in your normal shell, and why a command that works there can fail inside the lab shell with "could not read Username".

For one command, the same reset keeps every stored credential out of the picture. Nothing is read, changed or erased:

```
# The same reset for a single command: no stored credential is read, changed or erased.
$ printf 'protocol=https\nhost=github.com\n\n' | git -c credential.helper= -c
credential.https://github.com.helper= -c credential.helper='!f() { test "$1" = get && printf
"username=nobody\npassword=wrong\n"; }; f' credential fill
protocol=https
host=github.com
username=nobody
password=wrong
$ cat ~/helper.log
cat: $LAB/ch16/credential-scope/home/helper.log: No such file or directory
[exit status: 1]
```

Lab 20.3 uses this form to send a wrong credential on purpose without damaging the stored one.

16.6 Tokens: fine-grained, classic, and what a prefix tells you

In one sentence. A token is a string that stands for an account with a subset of its rights; the prefix says what kind it is, and the kind decides how far a leak reaches.

Precisely. GitHub's credential types, from its consolidated reference ([credential types](#), [token formats](#)):

Credential	Prefix	Lifetime	Belongs to	Reaches
Personal access token (classic)	ghp_	long-lived; no expiry required	a user	by scope (<code>repo</code> , <code>read:org</code> , ...), across every repository and organization the user can reach
Fine-grained personal access token	github_pat_	configurable, up to one year or none	a user	one owner (a user or one organization), optionally selected repositories, with per-permission read or write
OAuth app token	gho_	long-lived; expiring tokens are the default for new apps since 14 August 2026 (changelog)	a user, through an app	by scope; <code>gh auth login</code> produces one
GitHub App user token	ghu_	8 hours, with a <code>ghr_</code> refresh token of 6 months	a user, through an app	the app's permissions, limited by the user's

Credential	Prefix	Lifetime	Belongs to	Reaches
GitHub App installation token	ghs_	1 hour	an app installation	the repositories and permissions of the installation
GITHUB_TOKEN in a workflow	(an installation token)	the job	a workflow run	one repository (Chapter 21A)
SSH key, deploy key	none	no expiry date; until deleted, and GitHub deletes a user SSH key that has not been used for a year (deleted or missing SSH keys); the documentation states no such automatic deletion for deploy keys	a user; a repository	sections 16.8 and 16.14

GitHub recommends fine-grained tokens over classic ones "whenever possible" ([managing tokens](#)). They have been generally available since 18 March 2025, and by default an organization owner must approve one before it can reach the organization's resources; until then it can read only public resources ([changelog](#)). An account can hold at most 50 of them.

The documented **gaps** decide when a classic token is still needed. A fine-grained token cannot:

- contribute to public repositories where you are not a member, or act for you as an outside collaborator;
- reach several organizations at once;
- access Packages, including `docker login ghcr.io` (Chapter 15, section 15.11);
- call the Checks API, or a few other REST endpoints.

For open-source contribution that rules fine-grained tokens out, and the browser login of `gh` or an SSH key is the answer, not a classic token typed into a prompt.

Expiry and revocation ([token expiration and revocation](#)): a token found in a public repository or gist is revoked automatically; personal and OAuth tokens unused for a year are removed; and anyone who holds a leaked token value can submit it to an unauthenticated revocation API ([changelog](#)). Organizations can restrict each token type and set a maximum lifetime ([token policy](#)).

Version note. Older behavior: installation tokens were short opaque strings, and code assumed 40 characters. Current behavior: newly minted installation tokens, including the Actions `GITHUB_TOKEN`, use a format of about 520 characters. Since: staged rollout from 27 April 2026 ([changelog](#)). Recommended: never validate a token by length or pattern in your own code.

In production. A classic token with `repo` scope sits in a CI variable "for the changelog bot". It can read and write every private repository its creator can, in every organization. When it leaks through a log, the blast radius is a person, not a bot. The same job with an installation token leaks one hour of access to one repository. Chapter 21B ranks the credentials for automation.

16.7 Why a token never goes into a URL

A URL of the form `https://USER:TOKEN@github.com/OWNER/REPO.git` works, which is the problem.

```
# What an old tutorial tells you to do. The token here is fake.
$ git remote set-url origin https://lab-user:FAKE-TOKEN-not-a-real-credential@github.com/acme-pay/billing-api.git
$ git remote -v
origin https://lab-user:FAKE-TOKEN-not-a-real-credential@github.com/acme-pay/billing-api.git (fetch)
origin https://lab-user:FAKE-TOKEN-not-a-real-credential@github.com/acme-pay/billing-api.git (push)
$ grep url .git/config
```

```
url = https://lab-user:FAKE-TOKEN-not-a-real-credential@github.com/acme-pay/billing-api.git
$ git config list --show-scope | grep url
local remote.origin.url=https://lab-user:FAKE-TOKEN-not-a-real-credential@github.com/acme-pay/billing-api.git
```

The secret is in a plain text file inside the repository directory, it is printed by `git remote -v` and `git config list`, which people paste into tickets and CI logs, and it is copied by every backup of that directory. Git itself treats the URL as a credential:

```
$ printf 'url=%s\n\n' "$(git remote get-url origin)" | git credential fill
protocol=https
host=github.com
path=acme-pay/billing-api.git
username=lab-user
password=FAKE-TOKEN-not-a-real-credential
```

Git can be told to refuse such URLs. `transfer.credentialsInUrl` takes `allow` (the default), `warn` or `die` ([git-config](#)):

```
$ git config set --global transfer.credentialsInUrl die
$ git fetch origin
fatal: URL 'https://lab-user:<redacted>@github.com/acme-pay/billing-api.git' uses plaintext
credentials
[exit status: 128]
$ git push origin main
fatal: URL 'https://lab-user:<redacted>@github.com/acme-pay/billing-api.git' uses plaintext
credentials
[exit status: 128]
```

Git redacts the secret in its own message. With `die` in force there is a surprise worth knowing before an incident:

```
# With "die" in force, even the command that would repair the URL refuses to read it:
$ git remote set-url origin https://github.com/acme-pay/billing-api.git
fatal: URL 'https://lab-user:<redacted>@github.com/acme-pay/billing-api.git' uses plaintext
credentials
[exit status: 128]
$ git config set remote.origin.url https://github.com/acme-pay/billing-api.git
$ git remote -v
origin https://github.com/acme-pay/billing-api.git (fetch)
origin https://github.com/acme-pay/billing-api.git (push)
# The URL is clean. The token is still compromised: it sat in a file and on a screen. Revoke it.
```

Observed behavior : With `transfer.credentialsInUrl=die`, `"git remote set-url"` refuses to replace the bad URL.

Git state	: remote.origin.url contains user:secret@. Nothing else is wrong.
Mechanism	: set-url reads the remote's configuration before changing it, and reading a URL with a plaintext credential is what "die" forbids.
Root cause	: The check sits where remotes are parsed, not where connections are opened.
Why Git does this	: So that no command can use such a URL by accident.
Correct fix	: Write the key directly: <code>git config set remote.origin.url <clean URL></code> .
Prevention	: Set "die" globally before the first bad URL exists, and revoke any token that was ever in a URL. Cleaning the file does not un-leak it.

Outdated advice. The Phase 0 report found a 2024 workshop video that writes a classic token into the remote URL, and a 2025 course that shows it while calling it not recommended. Both observations rest on auto-generated captions and should be re-checked before quoting. The technique is wrong in any year.

Version note. Older behavior: no check. Current behavior: `transfer.credentialsInUrl`, covering `remote.<name>.url` and not `pushurl`. Since: Git 2.37 ([release notes](#)). Recommended: `git config set --global`

```
transfer.credentialsInUrl die.
```

The same reasoning excludes every other place where a token ends up in plain sight: a command line (`ps` and shell history record it), an environment file under version control, a `.netrc` in a home directory that is backed up. Hand a token to a program on standard input (`gh auth login --with-token < file`, `gh secret set NAME` with no `--body`), and let a helper keep it.

16.8 SSH: a key pair instead of a secret that travels

In one sentence. With SSH you prove who you are by signing a challenge with a private key that never leaves your machine; GitHub holds only the public key.

Analogy. A signet ring and its wax impression. You give everyone a sample impression (the public key); only the ring (the private key) can make a new one, and anyone can compare. The analogy breaks because a wax seal can be copied from a sample, and a private key cannot be derived from the public key.

Precisely. `ssh-keygen -t ed25519 -C "your_email@example.com"` is the command GitHub documents ([generating a key](#)). It writes two files: the private key, and a `.pub` file with one line (type, key, comment) that you upload to your account. For hardware security keys the types are `ed25519-sk` and `ecdsa-sk`. When you add a key to GitHub you choose whether it is an **authentication** key or a **signing** key; to use one key for both, you upload it twice ([adding a key](#)). GitHub deletes SSH keys that have not been used for a year ([about SSH](#)).

See it. A throwaway key in the sandbox. This demo is volatile: the key and its fingerprints differ on every run. It has no passphrase at first only because a transcript cannot type one.

```
$ ssh-keygen -q -t ed25519 -N '' -C 'you@example.com laptop 2026' -f ~/.ssh/id_ed25519_personal
$ cd ~/.ssh && stat -f '%Sp %N' id_ed25519_personal id_ed25519_personal.pub && cd ~
-rw-----  id_ed25519_personal
-rw-r--r--  id_ed25519_personal.pub
```

The private file is readable by its owner only; `ssh` ignores a private key file that others can access (`man ssh`, section FILES).

```
$ cat ~/.ssh/id_ed25519_personal.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIA8GlpVWYok0FGT/hmYU56d4Hw4gnMrujJRDYhk8ZDvu you@example.com
laptop 2026
$ ssh-keygen -l -f ~/.ssh/id_ed25519_personal.pub
256 SHA256:0V5JdQEk7wx2wTQatq0VlFtaBWvv+9G9fkq1plvs5ig you@example.com laptop 2026 (ED25519)
$ ssh-keygen -l -E md5 -f ~/.ssh/id_ed25519_personal.pub
256 MD5:29:38:de:ca:c6:b1:c4:64:ba:27:29:0f:a8:35:21:38 you@example.com laptop 2026 (ED25519)
```

The **fingerprint** is a hash of the public key, short enough to compare by eye. GitHub's page for auditing your keys has you compare this SHA256 form with what your account lists ([reviewing your SSH keys](#)); the MD5 form with colons is the older notation that some tools still print.

```
$ head -1 ~/.ssh/id_ed25519_personal
-----BEGIN OPENSSH PRIVATE KEY-----
$ ssh-keygen -y -f ~/.ssh/id_ed25519_personal
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIA8GlpVWYok0FGT/hmYU56d4Hw4gnMrujJRDYhk8ZDvu you@example.com
laptop 2026
```

The public key can always be recomputed from the private one, so losing the `.pub` file loses nothing. A passphrase encrypts the private file:

```
$ ssh-keygen -p -q -P '' -N 'lab passphrase, not a real one' -f ~/.ssh/id_ed25519_personal
Key has comment 'you@example.com laptop 2026'
Your identification has been saved with the new passphrase.
$ ssh-keygen -y -P 'a wrong guess' -f ~/.ssh/id_ed25519_personal
Load key "$LAB/ch16/ssh-keys/home/.ssh/id_ed25519_personal": incorrect passphrase supplied to decrypt
private key

[exit status: 255]
$ ssh-keygen -y -P 'lab passphrase, not a real one' -f ~/.ssh/id_ed25519_personal | cut -c1-40
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIA8G
```

A copy of that file is now useless without the passphrase. On your machine `ssh-keygen` asks for the passphrase interactively; it never belongs on a command line.

Inside `.git`. Nothing. Keys live under `~/.ssh`, and which key is offered is decided by `ssh` (section 16.10), unless `core.sshCommand` or `GIT_SSH_COMMAND` says otherwise (Chapter 14B, section 14B.7).

In production. One key per machine, with a comment that names the machine and the year. When a laptop is lost you delete one key from the account and every other machine keeps working. A key copied to five machines must be revoked on all five at once, and nobody remembers the fifth.

16.9 The agent, the passphrase and the keychain

A passphrase that must be typed for every fetch gets removed within a week. The **agent** is the remedy: a background process that holds decrypted keys in memory and signs on request, so that `ssh` never needs the passphrase again during the session and no program reads the private key file.

- `ssh-add PATH` gives a key to the agent. `ssh-add -l` lists the keys it holds. Its exit status distinguishes three states: 0 with a list; 1 with "The agent has no identities."; 2 when no agent can be reached ([ssh-add](#)).
- macOS provides an agent for your login session through `launchd`. GitHub's guide adds two things: a `Host github.com` block with `AddKeysToAgent yes` and `UseKeychain yes`, and `ssh-add --apple-use-keychain ~/.ssh/id_ed25519`, which stores the passphrase in the login keychain so that the agent can load the key after a restart ([generating a key](#)). `UseKeychain` exists only in Apple's build of OpenSSH.
- The replays run without any agent, which produces the third state:

```
$ ssh-add -l
Could not open a connection to your authentication agent.
[exit status: 2]
```

You meet that message under `sudo`, in `cron` jobs and in containers: they do not inherit `SSH_AUTH_SOCK`, the variable that tells `ssh` where the agent is. GitHub's troubleshooting page says the same about `sudo`: it uses a different set of keys ([Permission denied](#)).

Agent forwarding lets a remote machine use your local agent. The manual says it "should be enabled with caution": whoever controls that machine can use your identity while you are connected (`man ssh_config`, `ForwardAgent`). Prefer a deploy key or an app on the server (section 16.14).

16.10 `~/.ssh/config`, host aliases, and `ssh -G`

In one sentence. `~/.ssh/config` maps the host name that Git hands to `ssh` onto a real host, a user, a port and a key, and `ssh -G` prints the result of that mapping without connecting.

Precisely. The file is a list of `Host` blocks. For each setting, the **first value obtained wins**, so specific blocks go first and `Host *` defaults go last ([ssh_config](#)). The settings that matter for GitHub:

Keyword	Meaning	For GitHub
Host	the name or pattern you type, or that appears in a URL	github.com, or an alias of your own such as github-work
HostName	the real host to connect to	github.com, or ssh.github.com for port 443
User	the login name	always git: "all connections, including those for remote URLs, must be made as the 'git' user" (docs)
IdentityFile	a private key to offer; several lines add up	the key registered with the account you mean
IdentitiesOnly yes	offer only the configured files, even if the agent holds more keys	keeps a key of your other account, which the agent may also offer, from being the one GitHub accepts
AddKeysToAgent, UseKeychain	section 16.9	

Your GitHub account name appears nowhere. GitHub identifies you by which key was accepted.

See it. The sandbox file has a personal identity for github.com, a work identity behind an alias, the port-443 fallback, and defaults at the end. ssh finds its files through the account's home directory and not through \$HOME, so every command names the sandbox file with -F; on your machine you omit -F. -T only keeps ssh from mentioning a terminal in a recorded transcript.

```
$ cat ~/.ssh/config
# Personal account: plain github.com URLs
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_personal
  IdentitiesOnly yes

# Work account: URLs written with the alias, git@github-work:ORG/REPO.git
Host github-work
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_work
  IdentitiesOnly yes

# Port 22 blocked: SSH over the HTTPS port
Host github-443
  HostName ssh.github.com
  Port 443
  User git
  IdentityFile ~/.ssh/id_ed25519_personal
  IdentitiesOnly yes

# Defaults for every host come last
Host *
  AddKeysToAgent yes
```

```
$ ssh -T -F ~/.ssh/config -G github.com | grep -E
'^(hostname|user|port|identityfile|identitiesonly|addkeystoagent) '
user git
hostname github.com
port 22
identitiesonly yes
identityfile ~/.ssh/id_ed25519_personal
addkeystoagent true
```

```
$ ssh -T -F ~/.ssh/config -G github-work | grep -E
'^(hostname|user|port|identityfile|identitiesonly|addkeystoagent) '
user git
hostname github.com
port 22
```

```

identitiesonly yes
identityfile ~/.ssh/id_ed25519_work
addkeystoagent true

```

The alias connects to the same host as a different key holder. Nothing was contacted: `-G` evaluates the `Host` blocks, prints about eighty settings, and exits.

A user written in the command or in the URL wins over the file, which is how a well-meant `USERNAME@github.com` breaks a working setup:

```

# A user written in the command or in the URL beats "User git" in the file:
$ ssh -T -F ~/.ssh/config -G asha-rao@github.com | grep -E '^(hostname|user) '
user asha-rao
hostname github.com

```

And the order trap. The same file with a `Host *` block moved to the top:

```

# The same file with the defaults moved to the top, and a default user added to them:
$ head -4 ~/.ssh/config-defaults-first
Host *
  User deploy
  IdentityFile ~/.ssh/id_rsa_old

$ ssh -T -F ~/.ssh/config-defaults-first -G github.com | grep -E
'^(hostname|user|port|identityfile|identitiesonly|addkeystoagent)'
user deploy
hostname github.com
port 22
identitiesonly yes
identityfile ~/.ssh/id_rsa_old
identityfile ~/.ssh/id_ed25519_personal
addkeystoagent false

```

User `deploy` won because it was obtained first, and an old RSA key is now offered before the right one. On GitHub this configuration fails with `Permission denied (publickey)`, and the only place the cause is visible is this output.

Git's part is small. It splits the URL and runs `ssh` with what it found:

```

# ~/lab-bin/fake-ssh prints the arguments Git gives to ssh and exits. No connection is made.
$ cd ~ && git init -q work/billing-api && cd work/billing-api
$ git remote add origin git@github-work:acme-pay/billing-api.git
$ GIT_SSH_COMMAND=~/.lab-bin/fake-ssh git fetch origin
ssh was asked to run: git@github-work git-upload-pack 'acme-pay/billing-api.git'
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
[exit status: 128]

```

Git passed the alias `github-work` untouched. Git does not read `~/.ssh/config` and does not know that the alias means `github.com`.

Picture.

```

remote.origin.url = git@github-work:acme-pay/billing-api.git
      |
      v
ssh git@github-work git-upload-pack 'acme-pay/billing-api.git'
      |
      v
HostName github.com  Port 22  User git  IdentityFile ~/.ssh/id_ed25519_work
      |
      v
server: host key checked against known_hosts (16.11)

```

v
GitHub: which account has this public key? may that account do this to acme-pay/billing-api?

In production. A support ticket says "SSH is broken since yesterday". The first request is one line of output: `ssh -G github.com | grep -E '^(hostname|user|port|identityfile|identitiesonly) '`. It shows a corporate tool that prepended a `Host *` block, an alias that someone shadowed, or a port override, in less time than reading `ssh -v`.

16.11 Host keys and `known_hosts`

In one sentence. Before you prove who you are, the server proves who it is with its host key, and `ssh` compares that key with the one it remembered in `~/.ssh/known_hosts`.

Precisely. Without this check, anyone on the network path could pose as GitHub, accept your connection and relay it. On the first connection `ssh` shows the server key's fingerprint and asks whether to continue; your answer is the whole security of the scheme, so compare it with the fingerprints GitHub publishes ([GitHub's SSH key fingerprints](#)):

Key type	Fingerprint
Ed25519	SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvC0qU
ECDSA	SHA256:p2QAMXNIC1TJYWeI0ttrVc98/R1BUFWu3/LiyKgUfQM
RSA	SHA256:uNiVztkCsDhcc0u9e8BujQXVUpKZIDTMczCvj3tD2s

The same page publishes the three `known_hosts` lines, so you can install them and never be asked. `labs/ch16/github-known-hosts.txt` is a copy made on 2 October 2026.

See it. A fingerprint is computed from the key, so the published lines can be checked against the published fingerprints with no connection at all:

```
$ cut -c1-78 ~/github-known-hosts.txt
github.com ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIOMqqnkVzrm0SdG6U0oqKLsabgH5C9o
github.com ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTY
github.com ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGCj7ndNxQowgcQnjsHcLrqPEiiphnt
```

```
$ ssh-keygen -l -f ~/github-known-hosts.txt
256 SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvC0qU github.com (ED25519)
256 SHA256:p2QAMXNIC1TJYWeI0ttrVc98/R1BUFWu3/LiyKgUfQM github.com (ECDSA)
3072 SHA256:uNiVztkCsDhcc0u9e8BujQXVUpKZIDTMczCvj3tD2s github.com (RSA)
```

Three fingerprints, equal to the table. `ssh-keygen -F` answers "what do I have on file for this host":

```
$ cp ~/github-known-hosts.txt ~/.ssh/known_hosts
$ ssh-keygen -l -F github.com -f ~/.ssh/known_hosts
# Host github.com found: line 1
github.com ED25519 SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvC0qU
# Host github.com found: line 2
github.com ECDSA SHA256:p2QAMXNIC1TJYWeI0ttrVc98/R1BUFWu3/LiyKgUfQM
# Host github.com found: line 3
github.com RSA SHA256:uNiVztkCsDhcc0u9e8BujQXVUpKZIDTMczCvj3tD2s
$ ssh-keygen -l -F ssh.github.com -f ~/.ssh/known_hosts
[exit status: 1]
```

`ssh.github.com`, the host for port 443, is a different name and has no entry yet, which is why the first connection through port 443 asks again. A stale or forged entry looks like any other line; only the fingerprint gives it away:

```
# A different key filed under the name github.com: what a stale or forged entry looks like.
# stale-host-key.pub is a throwaway public key made for this course.
$ printf 'github.com %s\n' "$(cut -d' ' -f1,2 ~/stale-host-key.pub)" > ~/.ssh/known_hosts
$ ssh-keygen -l -F github.com -f ~/.ssh/known_hosts
# Host github.com found: line 1
github.com ED25519 SHA256:8UpYeYa2V/LbESZNhPkXZY4A0tNVNIYX1YezLa3mHbw
$ ssh-keygen -l -f ~/github-known-hosts.txt | grep ED25519
256 SHA256:+DiY3wvV6TuJJhbpZisF/zLDA0zPMSvHdKr4UvC0qU github.com (ED25519)
```

With that file, a connection to the real GitHub fails with `Host key verification failed.`, because the server presents a key that differs from the stored one. That is the check working. The repair removes the lines for the host and installs the published ones:

```
# The repair: remove the lines for the host, then add the published ones.
$ ssh-keygen -R github.com -f ~/.ssh/known_hosts
# Host github.com found: line 1
$LAB/ch16/known-hosts/home/.ssh/known_hosts updated.
Original contents retained as $LAB/ch16/known-hosts/home/.ssh/known_hosts.old
$ ssh-keygen -l -F github.com -f ~/.ssh/known_hosts
[exit status: 1]
$ cat ~/github-known-hosts.txt >> ~/.ssh/known_hosts
$ ssh-keygen -l -F github.com -f ~/.ssh/known_hosts | grep -c SHA256
3
```

Version note. Older behavior: GitHub's RSA host key had the fingerprint that old `known_hosts` files still contain. Current behavior: GitHub replaced its RSA host key on 24 March 2023, after the private key was briefly exposed in a public repository; the ECDSA and Ed25519 keys did not change. Since: 24 March 2023 ([GitHub Blog](#)). Recommended: on a machine that reports a changed host key, find the announcement first; GitHub states that a host key change "will be announced on the GitHub Blog" ([docs](#)). Then `ssh-keygen -R github.com` and the published lines.

🔴 `StrictHostKeyChecking no`, and deleting `known_hosts` wholesale, make the error disappear by removing the check. On a CI runner, install the published lines instead.

16.12 Testing the connection, and SSH over port 443

`ssh -T git@github.com` authenticates and runs nothing. GitHub's documented answer is `Hi USERNAME! You've successfully authenticated, but GitHub does not provide shell access.`, and the command exits with status 1, which is expected ([testing your SSH connection](#)). The name in the greeting is the answer to "who does the server say I am", and it is the first thing to read when a repository is "not found".

`ssh -vT git@github.com` adds the client's reasoning: which configuration files were read, which host and port were contacted, which identity files exist and which keys were offered. GitHub's troubleshooting page walks through that output ([Permission denied](#)).

Some networks block port 22. GitHub accepts SSH on port 443 of a different host name ([SSH over the HTTPS port](#)):

```
ssh -T -p 443 git@ssh.github.com # test
```

and, to route every `github.com` connection that way, a block in `~/.ssh/config`. Its effective configuration, from the sandbox file where it sits behind the alias `github-443`:

```
$ ssh -T -F ~/.ssh/config -G github-443 | grep -E
'^ (hostname|user|port|identityfile|identitiesonly|addkeystoagent) '
```

```

user git
hostname ssh.github.com
port 443
identitiesonly yes
identityfile ~/.ssh/id_ed25519_personal
addkeystoagent true

```

GitHub's page notes that the first connection asks about the host key again, under the name `[ssh.github.com]:443`, and that it is not available on GitHub Enterprise Server. The report did not verify the host-key behavior beyond that page.

State table. None of the setup steps of sections 16.5 to 16.12 touches a repository: working tree, index, HEAD, branch refs, other refs and the remote are unchanged in every row. What changes is outside.

Operation	Working tree, index, HEAD, refs, files in .git, remote	Your machine, outside any repository	GitHub
<code>ssh-keygen -t ed25519</code>	unchanged	two files under <code>~/.ssh</code>	unchanged
<code>ssh-add --apple-use-keychain KEY</code>	unchanged	the agent holds the key; the keychain holds the passphrase	unchanged
<code>gh ssh-key add KEY.pub</code> , or the web form	unchanged	unchanged	the public key is attached to your account
first <code>ssh -T git@github.com</code>	unchanged	a line in <code>~/.ssh/known_hosts</code> , after you confirm	unchanged
<code>gh auth login</code>	unchanged	a token in the system credential store; gh's configuration	an authorization of the GitHub CLI app for your account
<code>gh auth setup-git</code> , <code>git config set --global credential...</code>	unchanged	the global Git configuration	unchanged

16.13 Two GitHub identities on one machine

In one sentence. Give each account its own key and its own `ssh` host alias, and let the directory a repository lives in choose the alias and the commit email.

Precisely. GitHub's guide for several accounts shows the two building blocks: `Host` aliases with `IdentityFile` and `IdentitiesOnly`, and `url.<alias>.insteadOf` to send an organization's URLs through an alias ([managing multiple accounts](#)). Combined with `includeIf` (Chapter 14B, section 14B.4), one file holds everything that makes a repository a work repository:

```

$ git config set --global user.name "Lab User"
$ git config set --global user.email lab-user@personal.example
$ git config set --global 'includeIf.gitdir:~/work/.path' '~/gitconfig-work'
$ printf '[user]\n\temail = lab.user@acme-pay.example\n[url "git@github-work:"]\n\tinsteadOf =
git@github.com:\n' > ~/.gitconfig-work
$ cat ~/.gitconfig-work
[user]
    email = lab.user@acme-pay.example
[url "git@github-work:"]
    insteadOf = git@github.com:

```

Both repositories keep the canonical URL `git@github.com:OWNER/REPO.git`. In a repository under `~/work/`, the include applies:

```
$ cd ~/work/billing-api
$ git config get --show-origin user.email
file:$LAB/ch16/two-identities/home/.gitconfig-work      lab.user@acme-pay.example
$ git config get remote.origin.url
git@github.com:acme-pay/billing-api.git
$ git remote get-url origin
git@github-work:acme-pay/billing-api.git
$ GIT_SSH_COMMAND=~/.lab-bin/fake-ssh git ls-remote origin 2>&1 | head -1
ssh was asked to run: git@github-work git-upload-pack 'acme-pay/billing-api.git'
```

The stored URL is unchanged, the effective URL goes through the alias, and Git would start `ssh` for `git@github-work`. The personal repository is untouched:

```
$ cd ~/personal/notes-app
$ git config get user.email
lab-user@personal.example
$ git config get remote.origin.url
git@github.com:lab-user/notes-app.git
$ git remote get-url origin
git@github.com:lab-user/notes-app.git
$ GIT_SSH_COMMAND=~/.lab-bin/fake-ssh git ls-remote origin 2>&1 | head -1
ssh was asked to run: git@github.com git-upload-pack 'lab-user/notes-app.git'
```

```
$ printf 'Runbook: rotate the payment gateway key every 90 days.\n' > RUNBOOK.md
$ git add RUNBOOK.md && git commit -q -m "Add key rotation runbook"
$ git log -1 --format='%an <%ae> %s'
Lab User <lab.user@acme-pay.example> Add key rotation runbook
$ cd ~/personal/notes-app
$ printf 'Ideas for the weekend.\n' > IDEAS.md
$ git add IDEAS.md && git commit -q -m "Add ideas file"
$ git log -1 --format='%an <%ae> %s'
Lab User <lab-user@personal.example> Add ideas file
```

Three things changed together: the commit email, the URL and, through the alias, the key. They are still three independent identities (section 16.2); the include file is what keeps them aligned.

Over HTTPS the equivalent is one credential per account: `credential.https://github.com.username` inside the include, or `credential.https://github.com.useHttpPath true` (section 16.5). `gh` holds several accounts per host and `gh auth switch` changes the active one (`gh auth switch --help`); that switch is global, not per directory, which is the reason to prefer SSH aliases when you work in both identities on the same day.

In production. The failure this prevents: a commit pushed to the employer's repository under the personal email, or a push to the personal repository authenticated as the work account, which GitHub may reject with an error naming the other user ([docs](#)). A work repository cloned outside `~/work/` escapes a rule keyed on the directory; Lab 20.4 breaks it that way and repairs it with an `includeIf` keyed on the remote URL.

16.14 Credentials for machines: deploy keys, GitHub Apps, OAuth apps

A job that runs without you needs an identity that is not you.

Credential	Scope	Lifetime	What to know
Deploy key	one repository; read-only unless "Allow write access" is chosen	no expiry	an SSH key attached to the repository, not to a person. It cannot be reused for a second repository, usually has no passphrase, and keeps working after the person who added it leaves (deploy keys)
GitHub App installation token	the repositories and permissions of the installation	1 hour	not tied to a user and consumes no seat; GitHub's stated preference for integrations (apps)

Credential	Scope	Lifetime	What to know
GitHub App user token	what both the app and the user may do	8 hours, refresh token 6 months	acts on behalf of a person
OAuth app token	scopes granted by the user, across everything that user can reach	long-lived, or expiring for apps that use the option of 14 August 2026	an OAuth app that only needs to read still asks for the broad <code>repo</code> scope
Machine user with a token or key	whatever the account can reach	as the credential	a user account for automation: it occupies a seat, and somebody has to own its password and second factor
<code>GITHUB_TOKEN</code>	the workflow's repository	the job	Chapter 21A

A server that needs several repositories with deploy keys uses the alias technique of section 16.10, one alias per repository, as GitHub's page shows. At that point an app is the better tool: one installation, short-lived tokens, and a permission list that an auditor can read.

In production. The CTO's fourth question. The nightly sync uses a personal token that the departed engineer created. If the account was removed from the organization, the job is already failing; if the account is still a member, the job works with a former employee's access, and the token appears in nobody's inventory. Replacing it with an app installation, or at least a deploy key, makes the job's identity independent of people. Chapter 21B covers the audit-log side.

16.15 SAML single sign-on and mandatory two-factor authentication

Two-factor authentication. Since March 2023 GitHub requires everyone who contributes code on GitHub.com to enable it; accounts are enrolled in groups with a 45-day window ([mandatory 2FA](#)). It protects the browser sign-in. Tokens and SSH keys are used without a second factor, which is why they must be guarded and, where possible, short-lived. An organization can additionally require two-factor authentication of its members (Chapter 15, section 15.19).

SAML single sign-on is an Enterprise Cloud feature. In an organization that uses it, a credential must be **authorized for that organization** in addition to being valid ([about SSO](#)):

- a classic token is authorized after creation, per organization; a fine-grained token during creation; an SSH key per organization. Tokens from apps that you authorize during an active SSO session are authorized automatically. Deploy keys, installation tokens and `GITHUB_TOKEN` need no authorization ([credential types](#)).
- An owner can revoke the authorization. A revoked SSH key cannot be authorized again: a new key is needed.
- Through the REST API, an unauthorized classic token gets `404` or `403`, and a `403` carries an `X-GitHub-SSO` header with the URL to authorize it ([REST authentication](#)).

Unverified. The text Git shows when an SSH key or token is not authorized for SSO is not quoted in any documentation page the report found; user reports show an `ERROR:` line naming the organization. The documented, citable behavior is the REST header above.

16.16 The SSH changes of 14 October 2026 and 13 January 2027

Announced on 22 September 2026 ([changelog](#)), and after this chapter's baseline:

Date	Change	Who is affected
14 October 2026	RSA keys uploaded after this date must be at least 3072 bits, for authentication and for signing. A post-quantum key exchange, <code>mlkem768x25519-sha256</code> , is enabled.	anyone adding a new RSA key
4 November and 9 December 2026	brownouts of the two removals below	very old SSH clients, temporarily
13 January 2027	the <code>ssh-rsa</code> signature type (RSA with SHA-1) and the key exchange <code>diffie-hellman-group-exchange-sha256</code> are removed	clients that cannot sign RSA with SHA-2: OpenSSH older than 7.2, and old embedded libraries

Three points prevent wrong conclusions. The signature type `ssh-rsa` is not the key type `ssh-rsa`: an existing RSA key keeps working as long as the client signs with `rsa-sha2-256` or `rsa-sha2-512`, which current clients choose by themselves. Ed25519 and ECDSA keys are not affected at all. And HTTPS remotes are not affected.

What to check: `ssh -V` for the client version (OpenSSH 10.2 here), and `ssh-keygen -l -f ~/.ssh/KEY.pub` for a key's type and size, in the format shown in section 16.8. The machines to worry about are not laptops but old build agents, appliances and libraries inside tools.

Unverified. The announcement does not state a minimum size for RSA keys that are already uploaded, and GitHub publishes no single list of supported key types (report, section 2, flags). The legacy command in GitHub's documentation, `ssh-keygen -t rsa -b 4096`, satisfies the new minimum.

16.17 Diagnosis: who wrote this line?

An authentication error on your screen is a short stack of lines written by different programs. Reading it starts with attributing each line.

Line begins with	Written by	Means
<code>ssh: Permission denied (publickey), Host key verification failed., WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!</code>	your <code>ssh</code> client	the SSH connection or authentication failed before Git's protocol started
<code>remote: over HTTPS; over SSH, lines such as ERROR: We're doing an SSH key audit.</code>	GitHub's server, passed through	the server knows something and tells you: repository not found, permission denied to a named user, a rule
<code>fatal: Could not read from remote repository. and the two lines after it</code>	Git	the other side closed the connection; this line never contains the cause
<code>fatal: Authentication failed for, fatal: repository '...' not found, fatal: unable to access '...': The requested URL returned error: NNN</code>	Git's HTTPS code	HTTP status 401, 404, or another status, in that order (remote-curl.c)
<code>fatal: could not read Username for</code>	Git	no helper answered and prompting was impossible

See it. Four failures with four different causes. Nothing listens on port 1 of this machine, so the first one is a real `ssh` error without any network:

```
# Nothing listens on port 1 of this machine, so the connection is refused at once.
$ ssh -T -F ~/.ssh/config-offline -p 1 git@127.0.0.1
ssh: connect to host 127.0.0.1 port 1: Connection refused

[exit status: 255]
$ GIT_SSH_COMMAND='ssh -F ~/.ssh/config-offline' git ls-remote ssh://git@127.0.0.1:1/acme-pay/billing-api.git
```

```
ssh: connect to host 127.0.0.1 port 1: Connection refused

fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
[exit status: 128]
```

Alone, `ssh` reports the cause. Under Git, the same line is followed by Git's three-line trailer. A stand-in for `ssh` that prints its arguments and exits, a second one that exits silently, and a local path that does not exist all end the same way:

```
$ GIT_SSH_COMMAND=~/.lab-bin/fake-ssh git ls-remote git@github.com:acme-pay/billing-api.git
ssh was asked to run: git@github.com git-upload-pack 'acme-pay/billing-api.git'
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
[exit status: 128]
```

```
# ~/.lab-bin/silent-ssh exits with status 255 and prints nothing.
$ GIT_SSH_COMMAND=~/.lab-bin/silent-ssh git ls-remote git@github.com:acme-pay/billing-api.git
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
[exit status: 128]
```

```
$ git ls-remote ~/no-such-repository.git
fatal: '$LAB/ch16/failure-anatomy/home/no-such-repository.git' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
[exit status: 128]
```

```
Observed behavior : "fatal: Could not read from remote repository. Please make sure you have the
                    correct access rights and the repository exists."
Git state          : Unchanged. No ref moved and no object was transferred.
Mechanism          : Git started a program for the other side (ssh, or git-upload-pack for a path)
                    and it exited before the conversation began.
Root cause         : Not in this message. It is in the line above it, written by ssh, by the server
                    or by Git's own check of a local path. When ssh is silent, there is no line.
Why Git does this  : Git cannot know why another program gave up. It offers the two most common
                    reasons as a hint, and people read the hint as a diagnosis.
Correct fix        : Read the line above. If there is none, run the transport alone:
                    ssh -vT git@github.com.
Prevention         : Quote the whole error in a ticket, never only the last line.
```

16.18 SSH failures, one by one

Error	Who writes it, and the mechanism	Diagnose	Fix
Permission denied (publickey)	the server rejected the connection: none of the keys offered belongs to an account, or no key was offered, or the user is not <code>git</code> (docs)	<code>git remote -v</code> (is the user <code>git</code> ?); <code>ssh -G github.com</code> (which user, which identity files, <code>identitiesonly</code>); <code>ssh-add -l</code> (is the key loaded, is an agent reachable?); <code>ssh -vT git@github.com</code> (which keys were offered)	correct the URL user; point <code>IdentityFile</code> at the key that is on your account; load the key into the agent; upload the public key; do not use <code>sudo</code>

Error	Who writes it, and the mechanism	Diagnose	Fix
Permission to OWNER/REPO denied to USER	the server: the key authenticated as an account, or as a deploy key of another repository, that has no access here (docs)	the name in the message; <code>ssh -T git@github.com</code> ; <code>ssh -G</code> for the alias you meant	use the alias or key of the right account (section 16.13); get the role you need
Repository not found over SSH	the server: section 16.19, the same deliberate answer as over HTTPS	<code>ssh -T git@github.com</code> names the account; spelling of OWNER/REPO	the right account, access, or URL
Host key verification failed.	your client: the server's key differs from <code>known_hosts</code> (docs)	<code>ssh-keygen -l -F github.com</code> ; compare with the published fingerprints; look for an announcement	section 16.11. If no announcement explains it, do not connect
<code>ssh: connect to host github.com port 22: ...</code> (refused, timed out)	your client: the network	<code>ssh -vT</code> ; try <code>ssh -T -p 443 git@ssh.github.com</code>	SSH over port 443 (section 16.12), or HTTPS
ERROR: We're doing an SSH key audit.	the server: the key is unverified (docs)	the message carries the reason and the address of your key settings	approve or remove the key there
fatal: Could not read from remote repository.	Git: section 16.17	the line above it	the cause named there

The first row carries the most confusion. `Permission denied (publickey)` does not mean "you lack permission on the repository". The server has not looked at the repository yet. It means the server could not attach your connection to any account.

16.19 HTTPS failures, one by one

Error	Who writes it, and the mechanism	Diagnose	Fix
Repository not found in a remote: line, then fatal: repository '...' not found	the server answers 404 when the repository does not exist or the credential has no access to it (docs)	<code>git remote -v</code> (spelling); <code>GIT_TRACE=1 git ls-remote origin 2>&1 grep run_command</code> (which helper); <code>gh auth status</code> (which account); can that account open the repository in a browser?	the account that has access; for a fine-grained token, the repository in its selection and organization approval; SSO authorization
fatal: Authentication failed for '...'	401: the credential was rejected outright: revoked or expired token, a password, a typo. Git then tells the helper to erase it (section 16.4)	<code>gh auth status</code> ; token expiry in your account settings	<code>gh auth login</code> , or a new token through the helper; never in the URL
the password-removal message (unverified wording, section 16.3)	the same 401, with a remote: line saying that passwords are not supported	you typed, or a helper stored, an account password	a token through a helper, or SSH
The requested URL returned error: 403	the server knows who you are and refuses: no write permission, a classic token	<code>gh auth status</code> (account and scopes); your role on the repository;	get the permission or authorization; if it is the wrong account,

Error	Who writes it, and the mechanism	Diagnose	Fix
	without the needed scope or without SSO authorization, an organization policy, or a rate limit. Git does not erase the credential	for the API, the X-GitHub-SSO and rate-limit headers	remove or override the stored credential (section 16.5)
fatal: could not read Username for 'https://github.com': terminal prompts disabled	Git: no helper answered, and there is no terminal (CI, cron, the lab shell)	<code>git config get --show-origin --all credential.helper</code>	configure a helper or provide a token through the CI system's mechanism
fatal: URL '...' uses plaintext credentials	Git: section 16.7	<code>git config get remote.origin.url</code>	clean the URL; revoke the token

Observed behavior : "Repository not found" for a repository that exists.
Git state : The remote URL is spelled correctly. Nothing local is wrong.
Mechanism : The server authenticated the request as an account or token that cannot see the repository, and answered as if it did not exist.
Root cause : An identity problem: the wrong account's credential answered, a fine-grained token does not include this repository, a credential is not authorized for the organization's SSO, or access was never granted.
Why GitHub does it: "To avoid confirming the existence of private repositories" (REST troubleshooting guide). A 403 would tell a stranger that the name is taken.
Correct fix : Find out who the server thinks you are (gh auth status, ssh -T), then make the right identity answer, or grant it access.
Prevention : One identity per host or alias, chosen by configuration and not by habit.

Unverified. No GitHub documentation page explains HTTP 403 on `git push` specifically; the cloning-errors page says only that 401 and 403 "usually indicate you have an old version of Git, or you don't have access to the repository" ([docs](#)). The list of causes above combines that page, the token documentation and the REST guide; the statement that Git keeps the credential after a 403 is from Git's source and was not observed against GitHub here.

16.20 A decision tree

The command failed. Read ALL the lines, then:

```

1. git remote get-url origin          Which transport?  ssh (git@... or ssh://)  https://
                                     |                      |
SSH -----+                      |
2s. Is there an "ssh:" line (connect, resolve, timeout)?  -> network: try port 443 (16.12)
3s. "Host key verification failed"?                        -> compare fingerprints (16.11)
4s. "Permission denied (publickey)"?                      -> no account matched:
    ssh -G HOST | grep -E '(user|hostname|identityfile|identitiesonly) '
    ssh-add -l      (exit 2: no agent, exit 1: no keys)
    ssh -vT git@HOST (which keys were offered?)
5s. "Permission ... denied to USER" or "Repository not found"? -> authenticated as someone:
    ssh -T git@HOST   -> is that the account you meant?  no -> alias or key (16.13)
                                     yes -> access, SSO (16.15)
HTTPS -----+
2h. "could not read Username"?                -> no helper, no terminal (16.5)
3h. "Authentication failed" (401)?            -> credential rejected: gh auth status, re-login (16.3)
4h. "not found" (404) or 403?                -> authenticated, or anonymous, and not allowed:
    GIT_TRACE=1 git ls-remote origin 2>&1 | grep run_command (which helper answered?)
    gh auth status (which account, which scopes?)
wrong account -> fix which helper or username answers for this host (16.5)
right account -> role on the repository, token permissions, SSO (16.6, 16.15)

```

Last, for both: does the same identity work in the browser or with "gh repo view OWNER/REPO"?

`GIT_TRACE=1` shows which programs Git runs. `GIT_TRACE_CURL=1` (or the older `GIT_CURL_VERBOSE=1`) shows the HTTP exchange, including the status code and the `WWW-Authenticate` header; Git redacts the `Authorization` header by default, and you should still read such a trace before pasting it anywhere (Chapter 14B, section 14B.7).

16.21 What can go wrong

Sections 16.17 to 16.20 cover the errors. These are the failures without a clear error:

Symptom	Diagnosis	Fix	Prevention
Git never asks for a credential and keeps failing with 403	a stored credential for another account answers; 403 does not erase it (section 16.4)	reset the helper list for the host, or delete the keychain entry (docs)	one helper per host, set on purpose
gh works, git push over HTTPS does not	gh has a token; Git is not using it: <code>git config get --show-origin --all credential.https://github.com.helper</code>	<code>gh auth setup-git</code>	let <code>gh auth login</code> configure Git
gh works in one terminal and not in another	<code>GH_TOKEN</code> or <code>GITHUB_TOKEN</code> is exported in one of them and takes precedence	unset it	no tokens in shell profiles
SSH works in the terminal and fails under <code>sudo</code> , <code>cron</code> or in a container	no <code>SSH_AUTH_SOCK</code> there: <code>ssh-add -l</code> exits 2	a deploy key or app for the job; never <code>sudo git</code>	machine credentials for machines (section 16.14)
The wrong account's key is accepted	the agent offers its other keys too: <code>ssh -vT</code> ; <code>ssh -G</code> shows <code>identitiesonly no</code>	<code>IdentitiesOnly yes</code> with the right <code>IdentityFile</code>	host aliases (section 16.13)
A key that worked last year is refused	GitHub deletes keys unused for a year; or an owner revoked its SSO authorization	upload a new key; authorize it	one key per machine, reviewed yearly
A token stopped working	it expired, was unused for a year, was found in a public repository and revoked, or the organization changed its token policy	create a new one; find where the old one leaked	expiry dates in a calendar; no tokens in repositories
Commits appear under the wrong name on GitHub	commit identity, not authentication (section 16.2)	<code>git config get --show-origin user.email</code>	<code>includeIf</code> per directory
Everything works, and a token is in <code>.git/config</code>	section 16.7	clean the URL and revoke the token	<code>transfer.credentialsInUrl=die</code>

16.22 When not to use it, and dangerous edge cases

- **Never paste a token into a command line, a URL, a file under version control, a chat or a ticket.** Standard input and helpers exist for this. The labs never ask you to.
- **Do not fix a host key error by switching the check off.** Section 16.11.
- **Do not copy one private key to several machines, and do not use a key without a passphrase on a laptop.** Passphrase-less keys are for machine identities with a narrow scope, such as a read-only deploy key.
- **Do not use a classic token where a narrower credential works,** and do not use a personal credential for a job that must survive your departure.
- **Do not use** `credential-store`. It writes tokens to a plain file.
- **Do not run** `git` **with** `sudo`. It changes the key set and the configuration, and leaves root-owned files in `.git`.
- **Erasing is not revoking.** Removing a token from the keychain, from a URL or from a file leaves the token valid. Only GitHub can revoke it: in your settings, by expiry, or through the revocation API.
- `gh auth logout` **is not revoking either.** Its help text says the token is removed locally and "does not revoke authentication tokens".
- **A deploy key with write access can do what an admin collaborator can do in that repository,** according to GitHub's page, and it has no expiry. Treat it like a password to the repository.

16.23 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git remote -v</code> , <code>git config get --show-origin ...</code> , <code>ssh -G</code> , <code>ssh-add -l</code> , <code>ssh-keygen -l</code> , <code>ssh-keygen -F</code> , <code>gh auth status</code> , <code>git credential fill</code>	● SAFE	nothing (<code>fill</code> may prompt, and prints a secret on your screen)	not needed	not needed
<code>ssh -T git@github.com</code> , <code>ssh -vT ...</code>	● SAFE	on first contact, may add a line to <code>known_hosts</code> after asking you	compare the fingerprint	<code>ssh-keygen -R github.com</code>
<code>ssh-keygen -t ed25519 -f NEW</code>	● CAUTION	creates two files; overwrites an existing key only after asking	<code>ls ~/.ssh</code>	none for an overwritten private key
<code>ssh-add</code> , <code>ssh-add --apple-use-keychain</code>	● CAUTION	the agent's key list; a keychain entry	<code>ssh-add -l</code>	<code>ssh-add -d</code>
<code>ssh-keygen -R HOST</code>	● CAUTION	removes a host's lines from <code>known_hosts</code> , keeping a <code>.old</code> copy	<code>ssh-keygen -F HOST</code>	the <code>.old</code> file
<code>git config set --global credential...</code> , <code>gh auth setup-git</code> , <code>gh auth login</code> , <code>gh auth switch</code>	● CAUTION	which credential answers, for every repository	<code>git config get --show-origin --all credential.helper</code>	set the previous value; <code>git config unset</code>
<code>git credential reject</code> , <code>git credential-osxkeychain erase</code>	● CAUTION	deletes a stored credential	<code>git credential fill</code> shows what is stored	log in again
<code>gh auth token</code> , <code>gh auth status --show-token</code>	● DANGEROUS	prints a live token to the terminal and its scrollbar	ask whether you need the value at all	revoke the token

Command	Label	What it changes	Preview	Recovery
<code>gh ssh-key delete</code> , deleting a token or a deploy key in the web interface	 DANGEROUS	every machine and job that used it stops	list where it is used	create and distribute a new one
Writing a token into a URL, a file or a command line	 DANGEROUS	publishes the secret to logs, history and backups	none	revoke it; cleaning is not enough

16.24 Version notes

Placed where they are needed: the installation token format (16.6), `transfer.credentialsInUrl` (16.7), GitHub's RSA host key (16.11), and the SSH changes of late 2026 (16.16).

Version note. Older behavior: account passwords over HTTPS; DSA keys; the `git://` protocol. Current behavior: tokens or SSH keys only; Ed25519 recommended. Since: 13 August 2021 for passwords; 15 March 2022 for DSA and `git://` ([GitHub Blog](#)). Recommended: `gh auth login` or SSH with an Ed25519 key.

Version note. Older behavior: classic tokens were the only personal tokens. Current behavior: fine-grained tokens are generally available and recommended, with the documented gaps of section 16.6. Since: 18 March 2025. Recommended: fine-grained first; classic only where a gap forces it; an app for automation.

Version note. Older behavior: tutorials debug HTTPS with `GIT_CURL_VERBOSE`. Current behavior: `GIT_TRACE_CURL` is the documented variable in Git 2.55 (Chapter 14B, section 14B.7). Recommended: `GIT_TRACE_CURL=1`, with redaction left on.

16.25 Practice

- **Labs 20.1 to 20.4** in the Module 20 lab manual: SSH setup and verification; HTTPS through the GitHub CLI and the helper behind it; three failures reproduced and diagnosed; two identities (optional). Each has a local rehearsal in the lab shell and a part against GitHub in your normal shell.
- Answers: Module 20.
- Replay any transcript with `labs/run ch16/<demo>`. Three drills; predict, then run:
 1. `ch16/credential-scope`: configure `credential.https://github.com.helper` with only the empty value. What does `git credential fill` do for `github.com`, and for another host?
 2. `ch16/ssh-config`: add `Port 2222` to the `Host *` block of the sandbox file. Which of the three hosts change port, and why not all?
 3. `ch16/failure-anatomy`: run `git ls-remote` with `GIT_SSH_COMMAND` set to a script of your own that prints one line to standard error and exits 1. Which lines appear, and who wrote each?

16.26 Interview questions

1. A deploy job reports "Repository not found" for a repository that exists. Walk through your diagnosis and name the possible root causes.
2. What is the difference between `Permission denied (publickey)` and `Permission to OWNER/REPO denied to USER`? What has the server established in each case?

3. Describe what Git does, step by step, from "the server answers 401" to "the token is stored", naming the helper operations. What happens differently on a 403, and what is the consequence?
4. Why must a token never be written into a remote URL? Name four places where it then appears, and the Git setting that refuses such URLs.
5. Compare a classic token, a fine-grained token, a deploy key and a GitHub App installation token by scope, lifetime, and what happens when the person who created it leaves.
6. A developer has a personal and a work GitHub account on one laptop. Design the setup, and say how you would prove from the terminal which account a given repository will use.
7. `ssh` reports that GitHub's host key has changed. What are the two explanations, how do you decide between them, and what do you never do?
8. Commit identity, authentication identity, signing identity: which of them does GitHub verify on a push, and what does each one prove?
9. What changes for SSH users of GitHub on 14 October 2026 and 13 January 2027? Which clients and keys are affected, and which are not?
10. An engineer pasted a token into a CI log an hour ago and has already deleted the log line. What is the state of the token, and what do you do in which order?

16.27 Sources

Primary sources

- GitHub Docs, read on 2 October 2026: [about authentication](#), [managing personal access tokens](#), [credential types](#), [token expiration and revocation](#), [caching credentials](#), [generating an SSH key](#), [testing your SSH connection](#), [GitHub's SSH key fingerprints](#), [SSH over the HTTPS port](#), [Permission denied \(publickey\)](#), [Host key verification failed](#), [troubleshooting cloning errors](#), [managing multiple accounts](#), [deploy keys](#), [single sign-on](#), [mandatory two-factor authentication](#).
- GitHub Blog and Changelog: [token authentication requirements](#), [improving Git protocol security](#), [the RSA host key](#), [security improvements for SSH](#).
- Git 2.55.0: [gitcredentials](#), [git-credential](#), [git-config](#) (`credential.*`, `transfer.credentialsInUrl`, `url.<base>.insteadOf`, `includeIf`), as installed (`git help -m <page>`); the source files [http.c](#) and [remote-curl.c](#) for what Git does on 401, 403 and 404.
- OpenSSH as installed (`man ssh`, `man ssh_config`, `man ssh-keygen`, `man ssh-add`); online at [man.openbsd.org](#).
- The GitHub CLI: `gh auth <command> --help` of 2.88.1, and [helper_config.go](#) for what `gh auth setup-git` writes.

Secondary sources

- The Phase 0 report of this course, section 2 (authentication, the error table, the flags) and section 4, and its research notes on the GitHub platform, section 4.
- Pro Git, [Credential Storage](#): the helper protocol with a custom helper. Caveat: predates the GitHub CLI and fine-grained tokens.

Videos (optional; the report's assessments rest on captions and chapter lists, not on full viewing)

- [Git & GitHub Crash Course 2025](#), Traversy Media, 49 minutes, 13 January 2025: SSH key setup. Caveat: beginner scope.
- [Complete git and Github course in Hindi](#), Chai aur Code, 8 June 2024: states that passwords no longer work and sets up SSH. Caveat: audited through auto-generated captions.
- No verified video covers credential helpers, token types or authentication diagnosis at this depth.

Further reading

- [Authenticating to the REST API](#) and [troubleshooting the REST API](#), for the 401, 403 and 404 rules that Git's HTTPS errors inherit.
- Chapter 21A for the workflow token and OIDC, Chapter 21B for credential blast radius, token forensics and the response to a leaked secret, Chapter 29 for the troubleshooting playbook.

Chapter 17: Pull Requests

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026, with the quoted documentation pages re-read on 2 October 2026. Transcripts are real output from `labs/ch17/`. Nothing in this chapter was run against GitHub: every statement about what GitHub does or displays is taken from its documentation or changelog and carries the link.

17.1 Why this matters

Four questions a CTO can ask in one week:

1. "The pull request changed one line. The page lists four commits and three files. Which is true?"
2. "CI was green on the pull request and red on `main` ten minutes after the merge. What did CI test?"
3. "The audit asks which commit the reviewer approved. The commit ID on `main` is not in the pull request. Where did it go?"
4. "We deleted the branch that contained the leaked key. Why can the commit still be fetched?"

Each is answered by knowing which part of a pull request is Git data, which part is a GitHub object, and which commits GitHub compares when it draws the page. The answers are in sections 17.12, 17.2, 17.8, and 17.2 again.

Hold on to one sentence through the chapter. **A pull request is a GitHub object that stores two names, a base and a head, and everything it shows is computed from three commits: the tip of the base, the tip of the head, and their merge base.** You met all three in Chapter 8: Merge. The page changes when one of the three changes, and for no other reason.

The sample project is `ticket-router`, a small service that reads a support ticket and names the queue that should answer it: `router/classify.py` holds the rules, `router/priority.py` the urgency scoring, `config/routing.yaml` the model settings. A bare repository on disk plays the shared repository. You are the contributor. Asha maintains the repository and merges. Ravi is a teammate.

17.2 What GitHub creates when a pull request opens

In one sentence. Opening a pull request moves no branch; it creates a GitHub object and a little Git data in the base repository: a read-only ref for the head commit and, when the merge is clean, a test merge commit.

Analogy. A pull request is a change request clipped to two bookmarks in a shared book. The form (who asks, who must sign, the discussion) lives in the office's filing system. The analogy breaks at one point that matters: the office also keeps a copy of the proposed pages in its own drawer under your request number, where you can read it and cannot change it, and where it stays after you remove your bookmark.

Precisely. You should be able to say which layer owns each item.

Item	Layer	Where it lives
Your commits, the head branch, the base branch	Git	objects and <code>refs/heads/...</code> in the head and base repositories
The pull request number, title, description, labels, reviewers, reviews, comments, review threads	GitHub	GitHub's database; not in any clone

Item	Layer	Where it lives
<code>refs/pull/<number>/head</code>	Git data written by GitHub	a ref in the base repository, read-only for you
The test merge commit and <code>refs/pull/<number>/merge</code>	Git data written by GitHub	a commit and a ref in the base repository, "when possible"
Checks and commit statuses	GitHub (and Actions)	attached to commit IDs in GitHub's database
The "Commits" and "Files changed" tabs	GitHub	computed from the three commits; stored nowhere in Git

GitHub's reference: "When you open a pull request, GitHub creates temporary Git references that point to the pull request's head branch and, when possible, to a simulated merge result. These refs help GitHub and integrations evaluate the pull request without changing the base branch" ([pull requests reference](#)). The page no longer spells out the ref names. Two other pages confirm them: the checkout instructions use `git fetch origin pull/ID/head:BRANCH_NAME` ([checking out pull requests locally](#)), and the Actions documentation sets `GITHUB_REF` to `refs/pull/PULL_REQUEST_NUMBER/merge` for `pull_request` events ([events that trigger workflows](#)).

See it. The Git part can be reproduced with a bare repository. The commands on the "server" below are plain Git that I chose to produce the documented result; GitHub does not publish the commands it runs.

Your branch `feature/priority-routing` has three commits and is pushed. `main` has moved on by one commit since you branched. On the server, "pull request 1" becomes a ref:

```
# On the server. Opening pull request 1 for feature/priority-routing, as Git data:
$ cd server/ticket-router.git
$ git update-ref refs/pull/1/head refs/heads/feature/priority-routing
# Clients may read refs/pull/ but not write it:
$ git config set receive.hideRefs refs/pull
$ git for-each-ref --format="%(objectname:short) %(refname)"
16d4788 refs/heads/feature/priority-routing
9aa221a refs/heads/main
16d4788 refs/pull/1/head
```

`receive.hideRefs` is the stock Git setting that hides a ref namespace from pushes while leaving it readable. Both refs name the same commit, `16d4788`. A normal clone never sees the new one, because the default `refspec` fetches `refs/heads/*` only (Chapter 12, section 12.12):

```
$ cd ../../asha/ticket-router
$ git ls-remote origin
9aa221a9fb05716040012e33f1e7c3074f948edc HEAD
16d4788572b31e17e4c3104a1d9861a5ce2c47ea refs/heads/feature/priority-routing
9aa221a9fb05716040012e33f1e7c3074f948edc refs/heads/main
16d4788572b31e17e4c3104a1d9861a5ce2c47ea refs/pull/1/head
# A clone fetches refs/heads/* only, so the pull request ref is not in Asha's clone:
$ git config get --all remote.origin.fetch
+refs/heads/*:refs/remotes/origin/*
$ git fetch
$ git for-each-ref --format="% (refname)" refs/remotes refs/pull
refs/remotes/origin/HEAD
refs/remotes/origin/feature/priority-routing
refs/remotes/origin/main
```

The server advertises `refs/pull/1/head`; the clone has nothing under `refs/pull`. To take a pull request into your clone you name the ref, as the documentation shows:

```
$ git fetch origin pull/1/head:pr-1
From ../../server/ticket-router
* [new ref] refs/pull/1/head -> pr-1
```

```
$ git log --oneline -3 pr-1
16d4788 Fix the name of the escalations queue
12ae95d Route high-priority tickets to escalation
44c1e7b Add priority scoring
$ git rev-parse pr-1 origin/feature/priority-routing
16d4788572b31e17e4c3104a1d9861a5ce2c47ea
16d4788572b31e17e4c3104a1d9861a5ce2c47ea
```

`pull/1/head` is shorthand for `refs/pull/1/head`. `gh pr checkout 1` does this fetch for you (section 17.16).

The namespace is read-only. GitHub's page quotes the error; plain Git produces the identical line when a ref is hidden from pushes:

```
$ git push origin pr-1:refs/pull/1/head
To ../../server/ticket-router.git
! [remote rejected] pr-1 -> refs/pull/1/head (deny updating a hidden ref)
error: failed to push some refs to ' ../../server/ticket-router.git'
[exit status: 1]
$ git push origin main:refs/pull/9/head
To ../../server/ticket-router.git
! [remote rejected] main -> refs/pull/9/head (deny updating a hidden ref)
error: failed to push some refs to ' ../../server/ticket-router.git'
[exit status: 1]
```

Now the second piece of Git data. GitHub "creates a merge commit to test whether the pull request can be automatically merged into the base branch. This test commit is not added to the base branch or the head branch" ([REST: get a pull request](#)). Its parents are "the latest commit on the base branch and the pull request's head commit" ([available rules, signed commits](#)). A server has no working tree, so the merge is computed in the object database, which stock Git exposes as `git merge-tree` (Chapter 8, section 8.17):

```
# On the server. A merge without a working tree (Chapter 8, section 8.17):
$ cd ../../server/ticket-router.git
$ git merge-tree --write-tree main refs/pull/1/head
beff5a60b5d7f4a2b93e41130391fc485f5970e0
[exit status: 0]
$ tree=$(git merge-tree --write-tree main refs/pull/1/head)
$ merge=$(git commit-tree $tree -p main -p refs/pull/1/head -m "Merge refs/pull/1/head into main")
$ git update-ref refs/pull/1/merge $merge
$ git log --oneline --graph -5 refs/pull/1/merge
* 135aad1 Merge refs/pull/1/head into main
|\
| * 16d4788 Fix the name of the escalations queue
| * 12ae95d Route high-priority tickets to escalation
| * 44c1e7b Add priority scoring
* | 9aa221a Raise the confidence threshold to 0.7
|/
$ git for-each-ref --format="% (objectname:short) % (refname)" refs/heads refs/pull
16d4788 refs/heads/feature/priority-routing
9aa221a refs/heads/main
16d4788 refs/pull/1/head
135aad1 refs/pull/1/merge
```

`135aad1` is a real merge commit with two parents, reachable only from `refs/pull/1/merge`. Neither branch moved:

```
# The test merge commit is on neither branch:
$ git branch --contains refs/pull/1/merge
$ git show -s --format="%h parents: %p" refs/pull/1/merge
135aad1 parents: 9aa221a 16d4788
$ git rev-parse main refs/pull/1/head
9aa221a9fb05716040012e33f1e7c3074f948edc
16d4788572b31e17e4c3104a1d9861a5ce2c47ea
```

Picture.

```
base repository on the server

fbbcc8d---53e7f57---f3e7ca9---9a383e5---9aa221a      refs/heads/main
                                     \
                                     135aad1      refs/pull/1/merge  (test merge; on no
branch)                                     /
                                     44c1e7b---12ae95d---16d4788
                                     |
                                     +-- refs/heads/feature/priority-routing
                                     +-- refs/pull/1/head    (read-only)

GitHub's database: pull request #1 { base: main, head: feature/priority-routing, title, reviews,
checks ... }
```

This answers question 2 of section 17.1. For a `pull_request` event, the Actions documentation states that `actions/checkout` "checks out the merge branch. Your CI tests run against the merged result, not just the head branch alone", and that `GITHUB_SHA` "is the SHA of the merge commit on the merge branch" ([events](#)). CI tested `135aad1`: your three commits merged with `main` as `main` was when the test merge was made. Chapter 20A: GitHub Actions fundamentals covers the workflow side.

The test merge is a stored commit, so it can be stale. After one more commit lands on `main`:

```
# On the server, after one more commit landed on main:
$ git rev-parse --short main
028b1de
$ git show -s --format="%h parents: %p" refs/pull/1/merge
135aad1 parents: 9aa221a 16d4788
# The merge ref is a stored commit. It is as fresh as its first parent.
$ git merge-base --is-ancestor main refs/pull/1/merge
[exit status: 1]
```

The first parent of the stored merge is still `9aa221a`, and `main` is at `028b1de`. Until the merge is regenerated, whatever reads the merge ref sees an older `main`.

Version note. Older behavior: the test merge commit was also regenerated when somebody viewed the pull request page. Current behavior: it is generated only when changes are pushed to the pull request branch, when the merge base changes, or when the current test merge commit is older than 12 hours. Since: 19 February 2026 ([changelog](#)). Recommended: do not build automation on the freshness of `refs/pull/N/merge`; the [REST guide](#) warns that "this content becomes outdated without warning".

Unverified. The same REST guide still says a test merge commit "is created when you view the pull request in the UI". The Phase 0 report records this as a conflict with the changelog, which is later and which this course follows.

When the merge conflicts there is no merge ref. A second pull request changes a line that `main` has changed since:

```
# Pull request 2 changes a line that main has changed since. On the server:
$ git merge-tree --write-tree --name-only main refs/pull/2/head
25dcf7083388484e218d7dc1b73f8e560e6c0909
config/routing.yaml

Auto-merging config/routing.yaml
CONFLICT (content): Merge conflict in config/routing.yaml
```

```
[exit status: 1]
# No clean tree, so no test merge commit and no refs/pull/2/merge:
$ git for-each-ref --format="%(%(refname))" refs/pull
refs/pull/1/head
refs/pull/1/merge
refs/pull/2/head
```

Exit status 1 and no clean tree: the "when possible" in GitHub's sentence. Two documented consequences follow: the merge button is deactivated until the conflict is resolved ([merge conflicts](#)), and "Workflows will not run on `pull_request` activity if the pull request has a merge conflict" ([events](#)). A pull request with no CI run at all is often a conflicting pull request.

The head ref outlives the head branch. This answers question 4:

```
# The head branch is deleted. The pull request ref still names the commits:
$ git update-ref -d refs/heads/feature/priority-routing
$ git for-each-ref --format="%(%(objectname:short))" refs/heads refs/pull
e996b41 refs/heads/fix/threshold
d8c296f refs/heads/main
16d4788 refs/pull/1/head
135aad1 refs/pull/1/merge
e996b41 refs/pull/2/head
$ cd ../../ravi/ticket-router
$ git fetch --prune
From ../../server/ticket-router
- [deleted]          (none)      -> origin/feature/priority-routing
  028b1de..d8c296f  main        -> origin/main
$ git fetch origin pull/1/head
From ../../server/ticket-router
* branch            refs/pull/1/head -> FETCH_HEAD
$ git log --oneline -1 FETCH_HEAD
16d4788 Fix the name of the escalations queue
```

The branch is gone and the commits are still one `git fetch origin pull/1/head` away. GitHub documents the principle: "After a pull request is opened, GitHub stores all of the changes remotely. Commits in a pull request are available in a repository even before the pull request is merged" ([checking out pull requests locally](#)). Nobody can rewrite or delete these refs with Git. For a leaked secret, deleting the branch therefore removes nothing; the credential must be rotated (Chapter 21B: Repository security).

The base is pinned. "When you open a pull request, GitHub sets the base to the commit that branch references. If the branch is updated in the future, GitHub does not update the base branch's commit" ([changing the base branch](#)). The pull request page keeps describing your change from where it started, while a compare page of the same branches follows the current tips; the two "can calculate changed files from different merge bases. As a result, the same branches can sometimes show different diffs in each place" ([pull requests reference](#)).

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
Open a pull request (<code>gh pr create</code>) ●	unchanged	unchanged	unchanged	unchanged	unchanged (<code>gh</code> may first push the branch and set its upstream)	base repository gains <code>refs/pull/N/head</code> and, when the merge is clean, a test merge commit under <code>refs/pull/N/merge</code> ; no branch moves	pull request object created; review requests, checks and notifications start
<code>git fetch origin pull/N/head:pr-N</code> ●	unchanged	unchanged	unchanged	unchanged	new local branch <code>pr-N</code> ; <code>FETCH_HEAD</code>	unchanged	unchanged

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
					rewritten; objects added		

In production. An evaluation job that posts a metric to every pull request must record which commit it evaluated. With the default checkout that is the test merge, a commit in nobody's clone. Record `github.event.pull_request.head.sha` next to the metric, or the number cannot be reproduced.

17.3 What a pull request shows: `base..head` and `base...head`

In one sentence. The "Commits" tab is the output of `git log base..head`, and the "Files changed" tab is the output of `git diff base...head`, a diff from the merge base to the head.

Precisely. Chapter 14A: History investigation taught both notations (sections 14A.2 and 14A.8). For `git log`, `A..B` means "reachable from B and not from A". For `git diff`, which compares two snapshots and knows nothing about ranges, `A...B` means "from the merge base of A and B to B". GitHub's documentation states which one a pull request uses: "Pull requests on GitHub show a three-dot diff", and gives the reason: "Because the three-dot comparison uses the merge base, it focuses on 'what a pull request introduces'" ([branches reference](#)).

See it. You have not fetched since you pushed. One fetch, then the two branches:

```
$ git fetch
From ../../server/ticket-router
   9a383e5..9aa221a  main      -> origin/main
$ git log --oneline --graph origin/main feature/priority-routing
* 9aa221a Raise the confidence threshold to 0.7
| * 16d4788 Fix the name of the escalations queue
| * 12ae95d Route high-priority tickets to escalation
| * 44c1e7b Add priority scoring
|/
* 9a383e5 Add classifier test
* f3e7ca9 Add routing config
* 53e7f57 Add keyword classifier
* fbbcc8d Add README
```

The commit list of the pull request, and the commit both sides share:

```
# The "Commits" tab: commits reachable from the head branch and not from the base branch.
$ git log --oneline origin/main..feature/priority-routing
16d4788 Fix the name of the escalations queue
12ae95d Route high-priority tickets to escalation
44c1e7b Add priority scoring
$ git rev-list --count origin/main..feature/priority-routing
3
```

```
$ git merge-base origin/main feature/priority-routing
9a383e547a1c8840f3c5c7b23bfab6120f366ed0
$ git log --oneline -1 $(git merge-base origin/main feature/priority-routing)
9a383e5 Add classifier test
```

Three commits. `9aa221a`, the newer commit on `main`, is not among them: it is reachable from the base. The diff the reviewer sees:

```
# The "Files changed" tab: merge base compared with the head. Three dots.
$ git diff --stat origin/main...feature/priority-routing
```

```
router/classify.py | 6 ++++~
router/priority.py | 6 +++++
2 files changed, 11 insertions(+), 1 deletion(-)
```

Two files, both yours. Compare the two-dot diff of the same branches:

```
# Two dots compare the two tips. The newer commit on main appears, reversed.
$ git diff --stat origin/main..feature/priority-routing
config/routing.yaml | 2 +-
router/classify.py   | 6 ++++~
router/priority.py   | 6 +++++
3 files changed, 12 insertions(+), 2 deletions(-)
$ git diff origin/main..feature/priority-routing -- config/routing.yaml
diff --git a/config/routing.yaml b/config/routing.yaml
index b97f8de..a84cfc9 100644
--- a/config/routing.yaml
+++ b/config/routing.yaml
@@ -1,3 +1,3 @@
 model: router-small-v1
 -confidence_threshold: 0.7
 +confidence_threshold: 0.6
 fallback_queue: general
```

A third file appears, backwards: the diff claims your branch lowers the threshold from 0.7 to 0.6. Your branch never touched that file. A two-dot diff compares the two tips, so every change that landed on `main` after you branched shows up as its own reversal. It is what `git diff main` shows on your machine, and the usual source of "GitHub shows something different from my terminal". The three-dot diff is a two-dot diff whose left side is the merge base:

```
# A three-dot diff is a two-dot diff whose left side is the merge base.
$ git diff --stat $(git merge-base origin/main feature/priority-routing) feature/priority-routing
router/classify.py | 6 ++++~
router/priority.py | 6 +++++
2 files changed, 11 insertions(+), 1 deletion(-)
```

Picture.

```

16d4788          9aa221a          main          two-dot: 9aa221a compared with
...f3e7ca9---9a383e5
16d4788          \
                  44c1e7b---12ae95d---16d4788      three-dot: 9a383e5 compared with
                  \                                (merge base)
                   \_____/                        feature/priority-routing
                   git log main..feature: the "Commits" tab
```

GitHub's advice for keeping the two views equal is to merge the base into the topic branch: "When you merge the base branch, the diffs shown by two-dot and three-dot comparisons are the same" (same page). The run confirms it, and shows the price:

```
# After the base branch is merged into the head branch, the tip of main is the merge base.
$ git merge -q origin/main
$ git log --oneline -1 $(git merge-base origin/main feature/priority-routing)
9aa221a Raise the confidence threshold to 0.7
$ git diff --stat origin/main..feature/priority-routing
router/classify.py | 6 ++++~
router/priority.py | 6 +++++
2 files changed, 11 insertions(+), 1 deletion(-)
$ git diff --stat origin/main...feature/priority-routing
router/classify.py | 6 ++++~
router/priority.py | 6 +++++
2 files changed, 11 insertions(+), 1 deletion(-)
```

```
$ git log --oneline origin/main..feature/priority-routing
8335b40 Merge remote-tracking branch 'origin/main' into feature/priority-routing
16d4788 Fix the name of the escalations queue
12ae95d Route high-priority tickets to escalation
44c1e7b Add priority scoring
```

The merge base moved to `9aa221a`, the two diffs agree, and the commit list gained a merge commit. Section 17.7 weighs that against rebasing.

Root cause. A three-dot diff is not "what will change on `main` when I merge". It is "what the head changed since the merge base". The two differ exactly when both sides changed the same thing. The test merge of section 17.2 is the object that answers the first question.

In production. Before you open a pull request, run `git fetch`, `git log --oneline origin/main..HEAD` and `git diff --stat origin/main...HEAD`. A commit you did not write will be seen by the reviewer too (section 17.12).

17.4 The lifecycle: draft, ready, review, merge or close

In one sentence. A pull request moves from draft to ready for review, collects reviews and checks, and ends merged or closed; each state is GitHub data, and only the merge writes to a branch.

Draft. "Draft pull requests cannot be merged, and code owners are not automatically requested to review them." Marking the pull request ready "will request reviews from any code owners", and you "can convert a pull request to a draft at any time" ([pull requests reference](#)). Use a draft when you want CI and early comments but not a formal review.

Outdated advice. Older material says draft pull requests need a paid plan for private repositories. Since 1 May 2025 they are available in every repository ([changelog](#)).

Reviews. A review has one of three outcomes ([pull request reviews](#)):

Outcome	Meaning	Does it block the merge?
Comment	feedback without a verdict	no
Approve	the reviewer accepts the change	counts toward required approvals, if a rule requires any
Request changes	the author should address feedback first	only if a rule requires a pull request

GitHub's text on the third row: "The **Request changes** option is purely informational and will not prevent merging unless a ruleset or classic branch protection rule is configured with the 'require a pull request' option." When such a rule exists and the reviewer has write access, "the pull request cannot be merged until the same collaborator submits another review approving the changes" ([reviewing proposed changes](#)). Without a rule, "changes requested" is a request, not a lock (Chapter 18).

Two more documented facts: anyone with read access can review and comment, and "pull request authors cannot approve their own pull requests" (same page), which is why the review labs need a second account.

Suggestions. A reviewer can propose replacement lines in a comment. When the author applies one suggestion or a batch, the documentation describes the resulting Git data exactly: it "creates a single commit on the compare branch of the pull request. Each person who suggested a change included in the commit will be a co-author of the commit. The person who applies the suggested changes will be a co-author and the committer of the

commit" ([incorporating feedback](#)). That commit is made on GitHub's side. Your local branch lacks it until you `git pull`, and a push without pulling is rejected as a non-fast-forward.

Closing keywords. Fixes #12 in the description closes the issue at merge time, but "only when the pull request targets the repository's *default* branch" ([linking a pull request to an issue](#)). A pull request into `release/1.0` will not close the issue (Chapter 15: GitHub).

Automated reviewers. Since 1 September 2026, in public preview and off by default, Copilot code review can submit an approving review that counts toward required approvals ([changelog](#)). When you design a review policy, count the humans.

17.5 Stale approvals, dismissal, and the most recent push

In one sentence. An approval is attached to the diff as it was when the reviewer approved; two optional settings decide what happens to that approval when the diff changes afterwards.

The risk they address. Without either setting, the sequence "approve, then push one more commit, then merge" lands a commit that nobody reviewed. GitHub's documentation calls this a pull request being "hijacked (where unapproved content is added to approved pull requests)".

Setting 1: dismiss stale approvals. "GitHub records the state of the diff at the point when a pull request is approved ... If the diff changes from this state (for example, because a contributor pushes new changes to the pull request branch or clicks **Update branch**, or because a related pull request is merged into the target branch), the approving review is dismissed as stale, and the pull request cannot be merged until someone approves the work again" ([available rules](#), [pull request rule](#)). Note the last cause: the diff can change without anyone touching your branch. The same page adds that approvals "will be dismissed as stale if the merge base introduces new changes after the review was submitted".

Setting 2: require approval of the most recent reviewable push. It requires "an approval from someone other than the last person to push to a branch". With this option "'stale' reviews are not dismissed, and the pull request remains approved as long as someone other than the person who made the most recent changes approves it". GitHub presents it as a compromise for large pull requests with many reviewers, and says which one is stricter: "it is safer to dismiss stale reviews."

The first costs a re-review after every update, including an "Update branch". The second can leave earlier approvals standing on a diff that has since changed.

A side effect in Git terms. With either setting, GitHub notes that "manually creating the merge commit for a pull request and pushing it directly to a protected branch will fail, unless the contents of the merge exactly match the merge generated by GitHub for the pull request". The server compares your merge with its own test merge.

Manual dismissal. People with write access can dismiss a review; "you must add a comment explaining why", and the review becomes a plain comment ([dismissing a review](#)). A ruleset can restrict who may do that (generally available in rulesets since 7 July 2026, [changelog](#)).

In production. A team that rebases pull request branches and also dismisses stale approvals re-approves after every rebase. That is the control working as designed; name the cost when you propose the setting.

17.6 Checks, status checks and mergeability

In one sentence. Mergeability is two independent questions: can Git merge the two tips (the test merge), and do the repository's rules allow it (reviews, checks and the rest of Chapter 18).

Two kinds of status checks. GitHub distinguishes checks, which carry detailed output and are created by GitHub Apps including Actions, from commit statuses, a simpler state set through the API by external services. "GitHub Actions generates checks, not commit statuses" ([status checks](#)). Both are attached to a commit ID, not to the pull request. A new push creates a new head commit with no checks yet.

Which commit must be green. "Required checks must pass on the latest commit SHA. Checks from earlier commits don't satisfy the requirement." And when the test merge commit has a status, that is the commit that must pass, which the merge box indicates with Showing checks for the merge commit ([troubleshooting required status checks](#)).

What the API reports. The `mergeable` field "can be true, false, or null. If the value is null, then GitHub has started a background job to compute the mergeability." When it is true, `merge_commit_sha` "will be the SHA of the test merge commit" ([REST: get a pull request](#)). A script that treats `null` as "not mergeable" is wrong; it must ask again.

Optional is not required. A red check blocks nothing by itself; only a rule that names the check does. Chapter 18, section 18.8 covers the traps of required checks, and Chapters 20A and 20B the workflows that produce them.

Unverified. The status-checks page says checks data is retained for 400 days and that archived required checks must be re-run. A changelog entry states that from 1 October 2026 checks, workflow runs and statuses follow the Actions retention setting, 90 days by default and not retroactive ([changelog](#)). The Phase 0 report flags the two as conflicting. Practical consequence either way: an old pull request may need its checks re-run before it can merge.

17.7 Conflicts in a pull request

In one sentence. A pull request "has conflicts" when the test merge of head into base cannot be computed cleanly, and you repair it by changing the head branch, either by merging the base into it or by rebasing it onto the base.

Precisely. It is the ordinary three-way conflict of Chapter 8, detected on the server, where nobody can resolve it. The resolution has to arrive as new commits on the head branch. GitHub offers three routes ([merge conflicts](#)):

Route	What it does to the head branch	Limits
The web conflict editor	"merges the entire base branch into the head branch" (resolving on GitHub)	only "simple competing line change conflicts"
The command line	whatever you choose: a merge of the base, or a rebase	you need a clone and push access to the head branch
Copilot (Fix with Copilot in the merge box)	commits made by the agent	needs Copilot cloud agent (changelog, 26 March 2026); review the result like any resolution

See it. Ravi's branch `fix/threshold` lowers the confidence threshold. Asha raised it on `main` in the meantime. The test merge:

```
# Ravi. His pull request fix/threshold -> main, and the test merge:
$ git log --oneline origin/main..fix/threshold
0e18437 Document how the threshold was tuned
be88439 Lower the confidence threshold to 0.65
$ git merge-tree --write-tree --name-only origin/main fix/threshold
e7f43627eaf0524ac48167b60df524db4bde078c
config/routing.yaml
```

```
Auto-merging config/routing.yaml
CONFLICT (content): Merge conflict in config/routing.yaml
[exit status: 1]
```

Repair 1: merge the base into the head. This is what the web editor and the **Update branch** button do, and what `gh pr update-branch` does by default.

```
# Repair 1: merge the base branch into the head branch.
$ git merge origin/main
Auto-merging config/routing.yaml
CONFLICT (content): Merge conflict in config/routing.yaml
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git diff
diff --cc config/routing.yaml
index 338a664,b97f8de..0000000
--- a/config/routing.yaml
+++ b/config/routing.yaml
@@@ -1,3 -1,3 +1,7 @@@
    model: router-small-v1
++<<<<<< HEAD
+confidence_threshold: 0.65
+=====
+ confidence_threshold: 0.7
++>>>>>> origin/main
+    fallback_queue: general
$ printf 'model: router-small-v1\nconfidence_threshold: 0.65\nfallback_queue: general\n' >
config/routing.yaml
$ git add config/routing.yaml
$ git commit -q -m "Merge main into fix/threshold"
$ git push
To ../../server/ticket-router.git
    0e18437..ce88db6  fix/threshold -> fix/threshold
```

```
$ git log --oneline --graph -5
*   ce88db6 Merge main into fix/threshold
|\
| * 4b21efb Raise the confidence threshold to 0.7
* | 0e18437 Document how the threshold was tuned
* | be88439 Lower the confidence threshold to 0.65
|/
* 9a383e5 Add classifier test
$ git log --oneline origin/main..fix/threshold
ce88db6 Merge main into fix/threshold
0e18437 Document how the threshold was tuned
be88439 Lower the confidence threshold to 0.65
$ git merge-tree --write-tree origin/main fix/threshold
47f5aab81203b78e95a028e97e22c7935c94b122
[exit status: 0]
```

The push was a fast-forward: no force, nobody's clone of the branch is disturbed. The pull request now lists three commits, one of them a merge that carries the resolution.

Repair 2: rebase the head onto the base. In a second copy of the same situation:

```
# Repair 2, in a copy of the clone: rebase the head branch onto the base.
$ cd ../../rebase-copy/ravi/ticket-router
$ git rebase origin/main
Rebasing (1/2)
Auto-merging config/routing.yaml
CONFLICT (content): Merge conflict in config/routing.yaml
error: could not apply be88439... Lower the confidence threshold to 0.65
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
```

```

hint: Disable this message with "git config set advice.mergeConflict false"
Could not apply be88439... # Lower the confidence threshold to 0.65
[exit status: 1]
$ printf 'model: router-small-v1\nconfidence_threshold: 0.65\nfallback_queue: general\n' >
config/routing.yaml
$ git add config/routing.yaml
$ git rebase --continue
[detached HEAD e2639c7] Lower the confidence threshold to 0.65
 1 file changed, 1 insertion(+), 1 deletion(-)
Rebasing (2/2)
Successfully rebased and updated refs/heads/fix/threshold.

```

```

$ git log --oneline --graph -5
* fd22cf1 Document how the threshold was tuned
* e2639c7 Lower the confidence threshold to 0.65
* 4b21efb Raise the confidence threshold to 0.7
* 9a383e5 Add classifier test
* f3e7ca9 Add routing config
$ git log --oneline origin/main..fix/threshold
fd22cf1 Document how the threshold was tuned
e2639c7 Lower the confidence threshold to 0.65
$ git push
To ../../server/ticket-router.git
 ! [rejected]          fix/threshold -> fix/threshold (non-fast-forward)
error: failed to push some refs to ' ../../server/ticket-router.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
[exit status: 1]
$ git push --force-with-lease
To ../../server/ticket-router.git
 + 0e18437...fd22cf1 fix/threshold -> fix/threshold (forced update)

```

Two commits in a straight line, both new: `be88439` became `e2639c7`. History was rewritten, so the push needs `--force-with-lease` (Chapter 12, section 12.8). The conflict was resolved inside the first commit; no commit records that it happened.

Both repairs give the reviewer the same diff:

```

# Both repairs give reviewers the same three-dot diff:
$ git diff origin/main...fix/threshold
diff --git a/README.md b/README.md
index 98a45e9..d8990f5 100644
--- a/README.md
+++ b/README.md
@@ -1,3 +1,5 @@
 # ticket-router

Sends each support ticket to the queue that can answer it.
+
+The threshold is tuned on the March ticket sample.
diff --git a/config/routing.yaml b/config/routing.yaml
index b97f8de..338a664 100644
--- a/config/routing.yaml
+++ b/config/routing.yaml
@@ -1,3 +1,3 @@
 model: router-small-v1
-confidence_threshold: 0.7
+confidence_threshold: 0.65
 fallback_queue: general
$ diff <(git -C ../../../ravi/ticket-router diff origin/main...fix/threshold) <(git diff
origin/main...fix/threshold) && echo identical
identical

```

	Merge base into head	Rebase head onto base
Push	fast-forward	forced (<code>--force-with-lease</code>)
Existing commit IDs on the branch	kept	replaced
Where the resolution is visible	in the merge commit (<code>git show --remerge-diff</code> , Chapter 8, section 8.16)	folded into the rewritten commits
Commit list of the pull request	grows by a merge commit	stays clean
Stale-approval dismissal (17.5)	triggered, the diff changed	triggered, the diff changed
Final history on <code>main</code> after a squash merge	identical	identical

The last row settles many arguments: under the squash method the shape of the branch disappears at merge time, so the cheaper merge is enough. Under the other two methods the shape lands on `main` , which is a reason to rebase.

GitHub, not Git. `gh pr update-branch` "updates with a merge commit (i.e., merging the base branch into the PR's branch)" by default and rebases with `--rebase` (`gh pr update-branch --help` , 2.88.1). The REST endpoint behind the button describes itself as "merging HEAD from the base branch into the pull request branch" ([REST: update a pull request branch](#)). Either way the head branch on GitHub moves and your local branch does not: pull before you continue.

In production. Do not resolve conflicts in lock files (`uv.lock`) or other generated files by picking a side. Take the base version, regenerate with the tool, commit the result.

17.8 The three merge methods

In one sentence. The three merge buttons write three different histories for the same content: a merge commit that keeps your commits, one new commit that replaces them, or new copies of your commits in a straight line.

Analogy. Three ways to file a report written in drafts: staple the drafts into the folder with a cover note (merge commit), retype one clean page (squash), or retype every draft in order (rebase). The folder's content is the same; what you can prove later about the drafts is not. The analogy breaks because retyped pages in Git are new objects with new IDs, and every tool that tracked the old IDs has lost them.

Precisely. These are GitHub server-side operations. Their documented semantics ([pull request merges](#), [about merge methods](#), [signature verification](#)):

	Create a merge commit	Squash and merge	Rebase and merge
Documented mechanics	"all commits from the feature branch are added to the base branch in a merge commit ... using the <code>--no-ff</code> option"	"the pull request's commits are squashed into a single commit", then merged "using the fast-forward option"	commits "are added onto the base branch individually without a merge commit"
New commits on the base	one merge commit	one commit	one per original commit
Your original commit IDs on the base	preserved	not present	not present: "always ... creates new commit SHAs"
Committer of what lands	your commits unchanged; the merge commit is committed by GitHub (see the	committed by GitHub	"always updates the committer information"

	Create a merge commit	Squash and merge	Rebase and merge
	note below)		
Signature on what lands	your commits keep theirs; the merge commit is signed by GitHub ("GitHub will automatically use GPG to sign commits you make using the web interface")	"GitHub would sign the final squash commit" (signed commits rule)	none: added "without commit signature verification"
Linear history rule (Chapter 18)	not allowed	allowed	allowed (at most 100 commits, limits)
Documented loss	none	"You lose information about when specific changes were originally made and who authored the squashed commits"	originally empty commits are dropped

On the committer: the Enterprise documentation for metadata rules says committer-email patterns "must also include `noreply@github.com` for web-based merges and other commits created on GitHub.com" ([metadata restrictions](#)).

Why rebase and merge cannot be signed, in GitHub's words: "GitHub creates a modified commit ... GitHub didn't truly create this commit, and can't therefore sign it as a generic system user. GitHub doesn't have access to the committer's private signing keys." The documented workaround is "to rebase and merge locally, and then push". Chapter 14B, section 14B.17 showed the mechanism: a rewritten commit keeps its author and loses its signature.

Unverified. Who is recorded as the *author* of a squash commit, and whether other contributors become Co-authored-by trailers, is not stated on the live documentation pages (the Phase 0 report flags it). The pages say only that an author email selector appears for squash merges "if you are the pull request author and you have more than one email address" ([merging a pull request](#)). Lab 22.1 has you read the real commit and record what you find.

See it. One pull request, three copies of the repository, the three local commands that correspond to the buttons. Asha merges. Two differences from GitHub are visible: the committer here is Asha, where GitHub records itself, and nothing here is signed.

```
# The same starting point in all three copies. Asha is the maintainer.
$ cd merge/asha
$ git log --graph --format="%h %an: %s" main origin/feature/priority-routing
* 9aa221a Asha Rao: Raise the confidence threshold to 0.7
| * 16d4788 Lab User: Fix the name of the escalations queue
| * 12ae95d Lab User: Route high-priority tickets to escalation
| * 44c1e7b Lab User: Add priority scoring
|/
* 9a383e5 Asha Rao: Add classifier test
* f3e7ca9 Asha Rao: Add routing config
* 53e7f57 Asha Rao: Add keyword classifier
* fbbcc8d Asha Rao: Add README
```

Method 1 is `git merge --no-ff` 🟡:

```
# Method 1, "Create a merge commit":
$ git merge --no-ff -m "Merge pull request #1 from feature/priority-routing" origin/feature/priority-routing
Merge made by the 'ort' strategy.
router/classify.py | 6 +++++-
router/priority.py | 6 ++++++
2 files changed, 11 insertions(+), 1 deletion(-)
create mode 100644 router/priority.py
$ git push -q origin main
$ git log --graph --format="%h %an / %cn: %s" -6
```

```
* 4c53f2e Asha Rao / Asha Rao: Merge pull request #1 from feature/priority-routing
|\
| * 16d4788 Lab User / Lab User: Fix the name of the escalations queue
| * 12ae95d Lab User / Lab User: Route high-priority tickets to escalation
| * 44c1e7b Lab User / Lab User: Add priority scoring
* | 9aa221a Asha Rao / Asha Rao: Raise the confidence threshold to 0.7
|/
* 9a383e5 Asha Rao / Asha Rao: Add classifier test
```

Method 2 is `git merge --squash` followed by an ordinary commit:

```
# Method 2, "Squash and merge":
$ cd ../../squash/asha
$ git merge --squash origin/feature/priority-routing
Automatic merge went well; stopped before committing as requested
Squash commit -- not updating HEAD
$ git commit -q -m "Route high-priority tickets to an escalations queue (#1)"
$ git push -q origin main
$ git log --graph --format="%h %an / %cn: %s" -3
* da48bba Asha Rao / Asha Rao: Route high-priority tickets to an escalations queue (#1)
* 9aa221a Asha Rao / Asha Rao: Raise the confidence threshold to 0.7
* 9a383e5 Asha Rao / Asha Rao: Add classifier test
$ git show --stat --format="%h parents: %p" HEAD
da48bba parents: 9aa221a

router/classify.py | 6 ++++--
router/priority.py | 6 +++
2 files changed, 11 insertions(+), 1 deletion(-)
```

Method 3 is `git rebase` followed by a fast-forward:

```
# Method 3, "Rebase and merge":
$ cd ../../rebase/asha
$ git switch -q -c pr-1 origin/feature/priority-routing
$ git rebase main
Rebasing (1/3)
Rebasing (2/3)
Rebasing (3/3)
Successfully rebased and updated refs/heads/pr-1.
$ git switch -q main
$ git merge --ff-only pr-1
Updating 9aa221a..ec49c9c
Fast-forward
 router/classify.py | 6 ++++--
 router/priority.py | 6 +++
2 files changed, 11 insertions(+), 1 deletion(-)
 create mode 100644 router/priority.py
$ git push -q origin main
$ git log --graph --format="%h %an / %cn: %s" -5
* ec49c9c Lab User / Asha Rao: Fix the name of the escalations queue
* 2303f3a Lab User / Asha Rao: Route high-priority tickets to escalation
* dbb1f36 Lab User / Asha Rao: Add priority scoring
* 9aa221a Asha Rao / Asha Rao: Raise the confidence threshold to 0.7
* 9a383e5 Asha Rao / Asha Rao: Add classifier test
```

What `main` gained in each copy, with author and committer:

```
# The feature commits as you wrote them:
$ git -C merge/you log --format="%h author=%an committer=%cn %s" main..feature/priority-routing
16d4788 author=Lab User committer=Lab User Fix the name of the escalations queue
12ae95d author=Lab User committer=Lab User Route high-priority tickets to escalation
44c1e7b author=Lab User committer=Lab User Add priority scoring
# What main gained in each copy (main@{1} is where main was before):
$ git -C merge/asha log --format="%h author=%an committer=%cn %s" main@{1}..main
4c53f2e author=Asha Rao committer=Asha Rao Merge pull request #1 from feature/priority-routing
16d4788 author=Lab User committer=Lab User Fix the name of the escalations queue
12ae95d author=Lab User committer=Lab User Route high-priority tickets to escalation
```

```

44c1e7b author=Lab User committer=Lab User Add priority scoring
$ git -C squash/asha log --format="%h author=%an committer=%cn %s" main@{1}..main
da48bba author=Asha Rao committer=Asha Rao Route high-priority tickets to an escalations queue (#1)
$ git -C rebase/asha log --format="%h author=%an committer=%cn %s" main@{1}..main
ec49c9c author=Lab User committer=Asha Rao Fix the name of the escalations queue
2303f3a author=Lab User committer=Asha Rao Route high-priority tickets to escalation
dbb1f36 author=Lab User committer=Asha Rao Add priority scoring

```

Merge: your three commits arrive under their own IDs, plus 4c53f2e. Squash: one commit, da48bba, and none of yours. Rebase: three commits with you as author, Asha as committer, and three new IDs (dbb1f36, 2303f3a, ec49c9c).

And yet:

```

# Three histories, one result: the tree of main is identical.
$ git -C merge/asha rev-parse main^{tree}
beff5a60b5d7f4a2b93e41130391fc485f5970e0
$ git -C squash/asha rev-parse main^{tree}
beff5a60b5d7f4a2b93e41130391fc485f5970e0
$ git -C rebase/asha rev-parse main^{tree}
beff5a60b5d7f4a2b93e41130391fc485f5970e0

```

The tree of `main` is `beff5a6` in all three copies, the tree of the test merge in section 17.2. **The merge method does not change what the code is after the merge. It changes what the history says about how the code got there.**

Picture.

Create a merge commit	Squash and merge	Rebase and merge
<pre> ...9a383e5---9aa221a-----4c53f2e --ec49c9c \ 44c1e7b--12ae95d--16d4788 / </pre>	<pre> ...9aa221a---da48bba </pre>	<pre> ...9aa221a---dbb1f36---2303f3a-- </pre>
(44c1e7b, 12ae95d, 16d4788 stay on the old branch, not on main)	(44c1e7b, 12ae95d, 16d4788 stay on the old branch, not on main)	(44c1e7b, 12ae95d, 16d4788 stay on the old branch, not on main)
main keeps your IDs and adds one	main gets one new commit	main gets three new commits

Two documented deviations of the rebase button. GitHub's rebase and merge "deviates slightly from `git rebase`": it "always updates the committer information and creates new commit SHAs, whereas `git rebase` does not change the committer information when the rebase happens on top of an ancestor commit", and it "drops commits that were empty to begin with ... whereas `git rebase` keeps originally-empty commits by default". Local Git shows both halves. A branch that already sits on top of `main`, with one empty commit:

```

# Asha, the maintainer. Nothing has landed on main since the branch was created:
$ git switch -q -c pr-1 origin/feature/priority-routing
$ git log --format="%h author=%an committer=%cn %s" main..pr-1
a4deec5 author=Lab User committer=Lab User Trigger CI again
9dcfb58 author=Lab User committer=Lab User Fix the name of the escalations queue
02820ea author=Lab User committer=Lab User Route high-priority tickets to escalation
3807b29 author=Lab User committer=Lab User Add priority scoring
$ git rebase main
Current branch pr-1 is up to date.
$ git log --format="%h author=%an committer=%cn %s" main..pr-1
a4deec5 author=Lab User committer=Lab User Trigger CI again
9dcfb58 author=Lab User committer=Lab User Fix the name of the escalations queue
02820ea author=Lab User committer=Lab User Route high-priority tickets to escalation
3807b29 author=Lab User committer=Lab User Add priority scoring

```

Nothing rewritten. To imitate the button you must force new commits and ask for empty ones to be dropped:

```
# What the documentation describes for the button: always new commits, new committer.
$ git rebase --no-ff main
Current branch pr-1 is up to date, rebase forced.
Rebasing (1/4)
Rebasing (2/4)
Rebasing (3/4)
Rebasing (4/4)
Successfully rebased and updated refs/heads/pr-1.
$ git log --format="%h author=%an committer=%cn %s" main..pr-1
123f252 author=Lab User committer=Asha Rao Trigger CI again
49f7387 author=Lab User committer=Asha Rao Fix the name of the escalations queue
9895c44 author=Lab User committer=Asha Rao Route high-priority tickets to escalation
216a2d1 author=Lab User committer=Asha Rao Add priority scoring
```

```
# And originally empty commits are dropped:
$ git rebase --no-ff --no-keep-empty main
Current branch pr-1 is up to date, rebase forced.
Rebasing (1/3)
Rebasing (2/3)
Rebasing (3/3)
Successfully rebased and updated refs/heads/pr-1.
$ git log --format="%h author=%an committer=%cn %s" main..pr-1
57537fc author=Lab User committer=Asha Rao Fix the name of the escalations queue
05bb2a6 author=Lab User committer=Asha Rao Route high-priority tickets to escalation
6da6459 author=Lab User committer=Asha Rao Add priority scoring
```

So after "Rebase and merge" the commits on `main` are never the commits you pushed, even when no rebase seemed necessary.

Operation (GitHub, at merge time)	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
Any of the three merge buttons 🟡	your clone: unchanged	unchanged	unchanged	unchanged	unchanged until you fetch	the base branch gains: your commits and a merge commit (merge), one new commit (squash), or new copies of your commits (rebase). After squash and rebase your original commits are on no branch, still under <code>refs/pull/N/head</code>	pull request becomes merged ; the head branch is deleted if the repository setting or the option says so

A repository's settings decide which methods are offered, and a ruleset can restrict them per branch (Chapter 18, section 18.6). With a merge queue "you no longer get to choose the merge method, as this is controlled by the queue" ([about merge methods](#)).

17.9 What the method means later: bisect, blame, revert, traceability

In one sentence. The method decides which commits exist on `main`, and every later tool (bisect, blame, revert, an audit that follows a commit ID) works on those commits and no others.

Is your work "on main"? Ask Git whether your head commit is an ancestor of `main`:

```
# Is your original head commit an ancestor of main?
$ git -C merge/asha merge-base --is-ancestor origin/feature/priority-routing main
[exit status: 0]
$ git -C squash/asha merge-base --is-ancestor origin/feature/priority-routing main
[exit status: 1]
```

```
$ git -C rebase/asha merge-base --is-ancestor origin/feature/priority-routing main
[exit status: 1]
```

Only the merge commit made your commits ancestors of `main`. After the other two methods the content arrived and the commits did not. Four observations follow.

Observation 1: `git branch -d` **refuses**. The maintainer deletes the head branch on the server, and you update your clone:

```
# You, the contributor, after the merge. Copy 1 (merge commit):
$ cd merge/you
$ git pull --ff-only --prune
From ../server
- [deleted]          (none)      -> origin/feature/priority-routing
 9aa221a..4c53f2e  main         -> origin/main
Updating 9aa221a..4c53f2e
Fast-forward
 router/classify.py | 6 +++++-
 router/priority.py | 6 ++++++
 2 files changed, 11 insertions(+), 1 deletion(-)
 create mode 100644 router/priority.py
$ git branch -vv
 feature/priority-routing 16d4788 [origin/feature/priority-routing: gone] Fix the name of the
 escalations queue
* main                    4c53f2e [origin/main] Merge pull request #1 from feature/priority-routing
$ git branch -d feature/priority-routing
Deleted branch feature/priority-routing (was 16d4788).
[exit status: 0]
```

```
# Copy 2 (squash):
$ cd ../../squash/you
$ git pull -q --ff-only --prune
$ git branch -vv
 feature/priority-routing 16d4788 [origin/feature/priority-routing: gone] Fix the name of the
 escalations queue
* main                    da48bba [origin/main] Route high-priority tickets to an escalations queue
(#1)
$ git branch -d feature/priority-routing
error: the branch 'feature/priority-routing' is not fully merged
hint: If you are sure you want to delete it, run 'git branch -D feature/priority-routing'
hint: Disable this message with "git config set advice.forceDeleteBranch false"
[exit status: 1]
```

Observed behavior	: After a squash merge (or a rebase merge) on GitHub, <code>git branch -d</code> refuses to delete the local branch: "not fully merged".
Git state	: The branch tip 16d4788 is not an ancestor of main. main has da48bba, a commit with one parent and the same content.
Mechanism	: <code>git branch -d</code> deletes only when the branch is merged into its upstream, or into HEAD when it has no upstream. "Merged" means reachable. The upstream is gone (pruned), so Git compares with HEAD and finds three unreachable commits.
Root cause	: Squash and rebase merges transfer content without ancestry.
Why Git does this	: <code>-d</code> exists to stop you from deleting the only ref to commits. Git cannot know that another commit carries the same changes.
Correct fix	: Verify that nothing would be lost, then delete with <code>git branch -D</code> .
Prevention	: None needed. Expect it under these two methods, and verify before <code>-D</code> .

The Phase 0 report lists this refusal as an inference from the documented new commit IDs; the transcript confirms it for the local equivalents. Lab 22.1 shows the other half of the rule: while the remote-tracking branch still exists, `-d` deletes with a warning, because the branch is merged into its upstream.

To verify before `-D`: after a rebase merge `git cherry` marks every commit `-` (a patch-equivalent is on `main`). After a squash no single commit matches. A test that works for both asks whether merging the branch would change `main` at all:

```
# Rebase copy: every commit of the branch has an equivalent patch on main ("-"):
$ git cherry -v main feature/priority-routing
- 44c1e7b21c49dd09f0df26e7f5aa147834faba31 Add priority scoring
- 12ae95d534559f98f9aa705ed852f5a4700f6edd Route high-priority tickets to escalation
- 16d4788572b31e17e4c3104a1d9861a5ce2c47ea Fix the name of the escalations queue
# Squash copy: no single commit on main matches a commit of the branch ("+"):
$ cd ../../squash/you
$ git cherry -v main feature/priority-routing
+ 44c1e7b21c49dd09f0df26e7f5aa147834faba31 Add priority scoring
+ 12ae95d534559f98f9aa705ed852f5a4700f6edd Route high-priority tickets to escalation
+ 16d4788572b31e17e4c3104a1d9861a5ce2c47ea Fix the name of the escalations queue
# A test that works for both: would merging the branch change main at all?
$ git merge-tree --write-tree main feature/priority-routing
beff5a60b5d7f4a2b93e41130391fc485f5970e0
$ git rev-parse main^{tree}
beff5a60b5d7f4a2b93e41130391fc485f5970e0
$ git branch -D feature/priority-routing
Deleted branch feature/priority-routing (was 16d4788).
```

The merged tree equals the tree of `main`: the branch has nothing left to give. `git branch -D`  removes the only ref to those commits. What it changes: the branch ref and its reflog are deleted. What it can destroy: nothing at once; the commits stay in the object database and, if you ever had the branch checked out, in the reflog of `HEAD`, for the periods given in Chapter 13. Preview: the tree comparison above. Recovery: `git branch <name> <id>`, with the ID that the command printed. When it is appropriate: after you have verified that the base contains the content.

Observation 2: blame and log name different commits.

```
# Who does blame name for the new lines of classify.py?
$ git -C merge/asha blame -s -L 1,3 router/classify.py
12ae95d5 1) from router.priority import priority
12ae95d5 2)
16d47885 3) QUEUES = ["billing", "technical", "general", "escalations"]
$ git -C squash/asha blame -s -L 1,3 router/classify.py
da48bba7 1) from router.priority import priority
da48bba7 2)
da48bba7 3) QUEUES = ["billing", "technical", "general", "escalations"]
$ git -C rebase/asha blame -s -L 1,3 router/classify.py
2303f3af 1) from router.priority import priority
2303f3af 2)
ec49c9cf 3) QUEUES = ["billing", "technical", "general", "escalations"]
```

After a squash every line points to one commit, `da48bba`; otherwise to the commit that introduced it (Chapter 14A, section 14A.18). Under squash the unit of history is the pull request, so the squash commit's message must link to it, as the `(#1)` in a default title does.

Observation 3: untested commits can land. The second commit of the branch used a wrong queue name and the third fixed it. Which commits on `main` contain the wrong name?

```
# The second feature commit used the queue name "escalation"; the third one fixed it.
# Which commits on main contain the wrong name?
$ for c in $(git -C merge/asha rev-list main@{1}..main); do git -C merge/asha grep -c
"\escalation\" \"$c -- router/classify.py; done
12ae95d534559f98f9aa705ed852f5a4700f6edd:router/classify.py:2
$ for c in $(git -C squash/asha rev-list main@{1}..main); do git -C squash/asha grep -c
"\escalation\" \"$c -- router/classify.py; done
$ for c in $(git -C rebase/asha rev-list main@{1}..main); do git -C rebase/asha grep -c
"\escalation\" \"$c -- router/classify.py; done
2303f3af71f7e0db4e5fbc21dcae7d7ba299cf72:router/classify.py:2
```

Under merge and rebase, `main` contains a commit whose tree has the bug. CI tested the final result, not each commit, and `git bisect` can stop on that intermediate commit. `git bisect start --first-parent` treats a

merged pull request as one step (Chapter 14A, section 14A.22); for the rebase method no flag helps, and every commit has to pass on its own. Squash removes the problem and the granularity with it.

Observation 4: reverting is three different operations.

```
# Undoing the pull request on main. Copy 1: one revert, and you must name the mainline.
$ cd ../../merge/you
$ git revert --no-edit -m 1 HEAD
[main 4e8155e] Revert "Merge pull request #1 from feature/priority-routing"
Date: Mon Sep 7 11:34:00 2026 +0530
2 files changed, 1 insertion(+), 11 deletions(-)
delete mode 100644 router/priority.py
$ git log --oneline -2
4e8155e Revert "Merge pull request #1 from feature/priority-routing"
4c53f2e Merge pull request #1 from feature/priority-routing
```

```
# Copy 2: one ordinary revert.
$ cd ../../squash/you
$ git revert --no-edit HEAD
[main 09f2610] Revert "Route high-priority tickets to an escalations queue (#1)"
Date: Mon Sep 7 11:37:00 2026 +0530
2 files changed, 1 insertion(+), 11 deletions(-)
delete mode 100644 router/priority.py
$ git log --oneline -2
09f2610 Revert "Route high-priority tickets to an escalations queue (#1)"
da48bba Route high-priority tickets to an escalations queue (#1)
```

```
# Copy 3: one revert per commit, and you must know where the pull request began.
$ cd ../../rebase/you
$ git revert --no-edit HEAD~3..HEAD
[main d1270df] Revert "Fix the name of the escalations queue"
Date: Mon Sep 7 11:40:00 2026 +0530
1 file changed, 2 insertions(+), 2 deletions(-)
[main 7e41c05] Revert "Route high-priority tickets to escalation"
Date: Mon Sep 7 11:40:00 2026 +0530
1 file changed, 1 insertion(+), 5 deletions(-)
[main 7335fb5] Revert "Add priority scoring"
Date: Mon Sep 7 11:40:00 2026 +0530
1 file changed, 6 deletions(-)
delete mode 100644 router/priority.py
$ git log --oneline -4
7335fb5 Revert "Add priority scoring"
7e41c05 Revert "Route high-priority tickets to escalation"
d1270df Revert "Fix the name of the escalations queue"
ec49c9c Fix the name of the escalations queue
```

A merge commit needs `-m 1` and carries the trap of Chapter 8, section 8.18: re-merging the branch brings nothing back until you revert the revert. A squash commit reverts like any commit. A rebased series has no marker of where the pull request began. On GitHub, the **Revert** button and `gh pr revert` create "a new pull request that reverts the original merge commit" ([reverting a pull request](#)).

Summary. No method is best. Each one buys something and pays for it.

Concern	Merge commit	Squash	Rebase
Commit IDs reviewed = commit IDs on <code>main</code>	yes	no	no
Bisect granularity	commit, or pull request with <code>- --first-parent</code>	pull request	commit
Revert	one command, <code>-m 1</code> , re-merge trap	one command	one per commit

Concern	Merge commit	Squash	Rebase
Reusing the branch afterwards	safe	old commits listed again, conflicts (17.12)	the same, but a plain <code>git rebase</code> skips the old commits

Contexts: merge commits where the reviewed commit IDs must be the deployed ones (audits, signed-commit policies, long-lived branches merged repeatedly); squash where pull requests are small and short-lived and commit hygiene inside a branch is not enforced; rebase where the team curates every commit, wants a linear history, and does not require signatures on the target. Chapter 27: Open source and team workflows returns to the choice.

17.10 Auto-merge

In one sentence. Auto-merge is a stored instruction on a pull request: merge with this method as soon as every requirement is met.

Precisely. "Auto-merge merges a pull request automatically after all required reviews and status checks pass." It must first be enabled for the repository, and the option "is shown only on pull requests that cannot be merged immediately", that is, when a rule has an unmet requirement. People with write permission can enable it; they and the author can disable it. One safety property is documented: "Auto-merge is disabled if someone without write permissions pushes new changes to the head branch or switches the base branch" ([automatically merging a pull request](#)). Availability: public repositories on GitHub Free, public and private on paid plans ([managing auto-merge](#)).

```
gh repo edit OWNER/REPO --enable-auto-merge      # repository setting, once
gh pr merge 12 --auto --squash                   # merge when the requirements are met
gh pr merge 12 --disable-auto                     # withdraw the instruction
```

What can go wrong. Auto-merge waits for *required* things only. In a repository with no required checks and no required reviews there is nothing to wait for. The documentation names one event that switches auto-merge off, a push by someone without write permission. For a commit pushed by someone with write permission after the approval, whether it merges unreviewed is therefore decided by the stale-approval settings of section 17.5 and not by auto-merge. Decide those settings before you allow auto-merge on a branch that deploys.

Outdated advice. The comment command `@dependabot merge` was removed on 27 January 2026 ([changelog](#)). The current mechanism is auto-merge.

17.11 Merge queue and the `merge_group` event

In one sentence. A merge queue merges pull requests one group at a time and runs the required checks on the exact commit that will become the new tip of the base branch.

The problem it solves. Without "strict" required checks, two pull requests can each be green against an older `main` and break `main` together (Chapter 8, section 8.15). With them, every merge makes all other pull requests out of date: "a race-to-merge situation that impacts developer productivity" ([repository limits](#)).

Precisely. "A merge queue creates temporary branches with a special prefix to validate pull request changes ... the changes in the pull request are grouped into a `merge_group` with the latest version of the `base_branch` as well as changes from pull requests ahead of it in the queue." The temporary branches begin with `gh-readonly-queue/{base_branch}` and "contain a different `sha` from the pull request". If a group fails its checks or conflicts, "the pull request will be removed from the queue", and the branches behind it are recreated without it ([managing a merge queue](#)).

See it. The idea in plain Git, with three approved one-commit pull requests. The branch prefix is the documented one; the rest is my imitation.

```
# The queue, in order 1, 2, 3. Each temporary branch contains main and everything ahead of it:
$ git switch -q -c gh-readonly-queue/main/pr-1 main
$ git merge -q --no-ff -m "Merge pull request #1" pr-1
$ git switch -q -c gh-readonly-queue/main/pr-2
$ git merge -q --no-ff -m "Merge pull request #2" pr-2
$ git switch -q -c gh-readonly-queue/main/pr-3
$ git merge -q --no-ff -m "Merge pull request #3" pr-3
$ git log --oneline --graph main..gh-readonly-queue/main/pr-3
* 0569767 Merge pull request #3
|\
| * 6c162ef Describe how to run the tests
* 7c2fd05 Merge pull request #2
|\
| * 1621dd9 Lower the confidence threshold to 0.65
* 2b79f02 Merge pull request #1
* 8c7493d Accept tickets without a subject
```

```
# The commits that the checks must test are none of the pull request heads:
$ git for-each-ref --format="%(%(objectname:short) %(refname:short))" refs/heads/pr-* refs/heads/gh-readonly-queue
2b79f02 gh-readonly-queue/main/pr-1
7c2fd05 gh-readonly-queue/main/pr-2
0569767 gh-readonly-queue/main/pr-3
8c7493d pr-1
1621dd9 pr-2
6c162ef pr-3
```

The checks must run on `2b79f02`, `7c2fd05` and `0569767`, commits that no contributor created and no `pull_request` run ever saw. Pull request 2 fails:

```
# The checks fail on the group of pull request 2. It leaves the queue; number 3 is rebuilt:
$ git branch -q -D gh-readonly-queue/main/pr-2
$ git switch -q -C gh-readonly-queue/main/pr-3 gh-readonly-queue/main/pr-1
$ git merge -q --no-ff -m "Merge pull request #3" pr-3
$ git log --oneline --graph main..gh-readonly-queue/main/pr-3
* 3bda42c Merge pull request #3
|\
| * 6c162ef Describe how to run the tests
* 2b79f02 Merge pull request #1
* 8c7493d Accept tickets without a subject
```

The entry for pull request 3 is a different commit now (`3bda42c`, not `0569767`) and must be tested again. When it passes, the base branch moves to the tested commit:

```
# The checks pass. The base branch moves to the tested commit, unchanged:
$ git switch -q main
$ git merge --ff-only gh-readonly-queue/main/pr-3
Updating 9a383e5..3bda42c
Fast-forward
 README.md          | 2 ++
 router/classify.py | 2 +-
 2 files changed, 3 insertions(+), 1 deletion(-)
$ git log --oneline --first-parent -3
3bda42c Merge pull request #3
2b79f02 Merge pull request #1
9a383e5 Add classifier test
```

The trap. Checks on queue branches are triggered by their own event. "You **must** use the `merge_group` event to trigger your GitHub Actions workflow when a pull request is added to a merge queue ... Otherwise, status checks

will not be triggered ... The merge will fail as the required status check will not be reported." The fix is one more trigger:

```
on:
  pull_request:
  merge_group:
```

Chapter 20A covers the event. Also documented: the queue fixes the merge method; `gh pr merge` on such a branch adds the pull request to the queue when the checks have passed and otherwise enables auto-merge, and `--admin` bypasses the queue (`gh pr merge --help`).

Availability. "Pull request merge queues are available in any public repository owned by an organization, or in private repositories owned by organizations using GitHub Enterprise Cloud" (same page), so not under a personal account.

Unverified. The merge-queue *ruleset rule* appears only in the Enterprise Cloud view of the documentation. The Phase 0 report could not confirm whether a Free or Team organization can enable the queue through a ruleset or only through a classic rule.

When not to use it. A queue adds a CI run per group and latency per merge. For a few merges a day, "strict" required checks cost less.

17.12 Why a pull request shows unexpected commits or a huge diff

In one sentence. The page shows `base...head` and `base...head`; when it shows too much, either the head contains commits that the base does not reach, or the base is not the branch the work started from.

GitHub documents five causes. The first two can be reproduced in plain Git.

Documented cause	What you see	Mechanism
The head branch was reused after a squash merge	commits that were "already squashed" listed again; repeated conflicts	the squash commit has no parent link to the branch, so the merge base never moved
Wrong or changed base branch	every commit of the branch the work was really based on	the range is taken against a base that does not contain those commits
The base branch moved	pull request page and compare page disagree	they "can calculate changed files from different merge bases"
Rewritten or force-pushed history	old and new copies of commits, outdated review comments	"Force pushing rewrites repository history and can ... corrupt pull requests"
The diff is truncated or hidden	files missing from "Files changed"	diff limits (17.18), or a <code>.gitattributes</code> rule hiding the file

Sources: [pull request merges](#), [changing the base branch](#), [pull requests reference](#), [troubleshooting commits](#), [branches reference](#).

Squash, then reuse the branch

Pull request 1 was squash-merged. Nobody deleted its branch.

```
# Pull request 1 was squash-merged. Its branch was not deleted.
$ git log --oneline --graph origin/main feature/priority-routing
* 1b2b8ed Route high-priority tickets to an escalations queue (#1)
* 9aa221a Raise the confidence threshold to 0.7
```

```
| * 16d4788 Fix the name of the escalations queue
| * 12ae95d Route high-priority tickets to escalation
| * 44c1e7b Add priority scoring
|/
* 9a383e5 Add classifier test
* f3e7ca9 Add routing config
* 53e7f57 Add keyword classifier
* fbbcc8d Add README
```

You make one small follow-up commit on the old branch, push, and open pull request 2. What it lists, what its diff shows, and what you actually changed:

```
# Pull request 2, same head branch, same base. Its commit list:
$ git log --oneline origin/main..feature/priority-routing
fa6e912 Treat data loss as urgent
16d4788 Fix the name of the escalations queue
12ae95d Route high-priority tickets to escalation
44c1e7b Add priority scoring
# Its diff (three dots), and what you believe you changed (your last commit):
$ git diff --stat origin/main...feature/priority-routing
router/classify.py | 6 ++++--
router/priority.py | 6 ++++++
2 files changed, 11 insertions(+), 1 deletion(-)
$ git show --stat --format="%h %s" HEAD
fa6e912 Treat data loss as urgent

router/priority.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

Four commits and two files for a one-line change. And the test merge:

```
# The test merge of pull request 2:
$ git merge-tree --write-tree --name-only origin/main feature/priority-routing
55b98ddb581c6463837afef116da1f66dd495a8a
router/priority.py

Auto-merging router/priority.py
CONFLICT (add/add): Merge conflict in router/priority.py
[exit status: 1]
```

A conflict in a file that only you ever edited.

```
$ git log --oneline -1 $(git merge-base origin/main feature/priority-routing)
9a383e5 Add classifier test
$ git show -s --format="%h parents: %p %s" origin/main
1b2b8ed parents: 9aa221a Route high-priority tickets to an escalations queue (#1)
```

Observed behavior	: Pull request 2 lists the three commits of pull request 1 again, shows their changes again, and conflicts with main in router/priority.py.
Git state	: merge-base(main, branch) is still 9a383e5, the original fork point. main's tip 1b2b8ed has one parent, 9aa221a.
Mechanism	: The squash commit copied the content of the branch and recorded no link to it. For Git, 44c1e7b, 12ae95d and 16d4788 were never merged. base..head lists them. The three-way merge sees "both sides added router/priority.py since 9a383e5, with different content": an add/add conflict.
Root cause	: Content merged without ancestry (Chapter 8, section 8.12), then more work on top of the old ancestry.
Why Git does this	: The merge base comes from parent links only. Git never compares patches to guess that a commit "is already there".
Correct fix	: Transplant the new commits onto main: git rebase --onto origin/main <last squashed commit>.
Prevention	: Delete the head branch after a squash or rebase merge and start new work from main. Enable automatic deletion of head branches in the repository.

GitHub's page says the same: "If you keep working on the same head branch after a squash merge, later pull requests can include commits that were already squashed into the base branch", and advises "using a merge commit or rebasing the branch before opening the next pull request".

Way out 1: move only what is new. `git rebase --onto` 🟡 (Chapter 9) takes the commits after `HEAD~1` and replays them on `main`:

```
# Way out 1: move only the new commit onto main. Everything up to HEAD~1 is already there.
$ git rebase --onto origin/main HEAD~1
Rebasing (1/1)
Successfully rebased and updated refs/heads/feature/priority-routing.
$ git log --oneline origin/main..feature/priority-routing
e73426a Treat data loss as urgent
$ git diff --stat origin/main...feature/priority-routing
router/priority.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git merge-tree --write-tree origin/main feature/priority-routing
084c79329c5ababb96ff69fb77466efba04e1ca0
[exit status: 0]
$ git push --force-with-lease
To ../../server/ticket-router.git
+ fa6e912...e73426a feature/priority-routing -> feature/priority-routing (forced update)
```

One commit, one file, a clean test merge, and a forced push. A plain `git rebase origin/main` is not the same thing. It replays all four commits and relies on Git to notice the duplicates:

```
# In a copy of the clone: a plain rebase replays all four commits.
$ cd ../../you-plain-rebase/ticket-router
$ git rebase origin/main
Rebasing (1/4)
dropping 44c1e7b21c49dd09f0df26e7f5aa147834faba31 Add priority scoring -- patch contents already
upstream
Rebasing (2/4)
Auto-merging router/classify.py
CONFLICT (content): Merge conflict in router/classify.py
error: could not apply 12ae95d... Route high-priority tickets to escalation
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
Could not apply 12ae95d... # Route high-priority tickets to escalation
[exit status: 1]
$ git status --short
UU router/classify.py
$ git rebase --abort
```

Git dropped the first commit because replaying it changed nothing. The second conflicts, because the squash already contains the third commit's fix of the same line.

Way out 2: merge main into the branch. One conflict, resolved once, and the merge base moves:

```
# In another copy. Way out 2: merge main into the branch and resolve once.
$ cd ../../you-merge/ticket-router
$ git merge origin/main
Auto-merging router/priority.py
CONFLICT (add/add): Merge conflict in router/priority.py
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
# The branch has everything main has plus the new word, so the branch side is the answer:
$ git restore --ours router/priority.py
$ git add router/priority.py
$ git commit -q -m "Merge main into feature/priority-routing"
$ git log --oneline -1 $(git merge-base origin/main feature/priority-routing)
1b2b8ed Route high-priority tickets to an escalations queue (#1)
```

```
$ git diff --stat origin/main...feature/priority-routing
router/priority.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline origin/main..feature/priority-routing
2710851 Merge main into feature/priority-routing
fa6e912 Treat data loss as urgent
16d4788 Fix the name of the escalations queue
12ae95d Route high-priority tickets to escalation
44c1e7b Add priority scoring
```

The diff is correct now. The commit list still shows five commits, which does not matter if the pull request is squash-merged.

Way out 3, the prevention: a new branch from `main` for every pull request, and automatic deletion of head branches ([documentation](#)), set by `gh repo edit --delete-branch-on-merge`.

A pull request against the wrong base

Ravi created `fix/empty-subject` from `main`, by habit. The fix is meant for the release branch, so he opened the pull request with base `release/1.0`:

```
# Ravi. One commit of his own, on a branch he created from main:
$ git log --oneline --graph origin/main origin/release/1.0 fix/empty-subject
* a1ae0eb Accept tickets without a subject
* 29be88c Describe how to run the tests
* 7e13c3d Raise the confidence threshold to 0.7
| * bc944fd Set version 1.0.0
|/
* 9a383e5 Add classifier test
* f3e7ca9 Add routing config
* 53e7f57 Add keyword classifier
* fbbcc8d Add README
```

```
# The pull request was opened with base release/1.0. What it lists and shows:
$ git log --oneline origin/release/1.0..fix/empty-subject
a1ae0eb Accept tickets without a subject
29be88c Describe how to run the tests
7e13c3d Raise the confidence threshold to 0.7
$ git diff --stat origin/release/1.0...fix/empty-subject
README.md      | 2 ++
config/routing.yaml | 2 +-
router/classify.py | 2 +-
3 files changed, 4 insertions(+), 2 deletions(-)
```

Three commits and three files for a one-line fix. The two extra commits are `main`'s, and merging would carry unreleased work into the release branch. In a real repository this is the pull request with hundreds of unrelated changes. Ask which base makes the branch small:

```
# Which base makes this branch a one-commit pull request?
$ for b in origin/main origin/release/1.0; do echo "$b: $(git rev-list --count $b..fix/empty-subject)
commits"; done
origin/main: 1 commits
origin/release/1.0: 3 commits
$ git log --oneline -1 $(git merge-base origin/release/1.0 fix/empty-subject)
9a383e5 Add classifier test
$ git log --oneline -1 $(git merge-base origin/main fix/empty-subject)
29be88c Describe how to run the tests
```

There are two repairs, and they are not interchangeable.

Fix A: the base is wrong. If the change belongs on `main`, change the base of the pull request (`gh pr edit --base main`, or **Edit title** next to the title and then the base branch menu, as the documentation describes). The

same branch is then a one-commit pull request:

```
# Fix A: the change belongs on main after all. Same branch, other base:
$ git log --oneline origin/main...fix/empty-subject
a1ae0eb Accept tickets without a subject
$ git diff --stat origin/main...fix/empty-subject
router/classify.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

GitHub warns about the side effect: "When you change the base branch of your pull request, some commits may be removed from the timeline. Review comments may also become outdated" ([changing the base branch](#)).

Fix B: the base is right and the branch started in the wrong place. Transplant the one commit:

```
# Fix B: the change does belong on release/1.0. Move the one commit there:
$ git rebase --onto origin/release/1.0 origin/main fix/empty-subject
Rebasing (1/1)
Successfully rebased and updated refs/heads/fix/empty-subject.
$ git log --oneline origin/release/1.0...fix/empty-subject
5c46029 Accept tickets without a subject
$ git diff --stat origin/release/1.0...fix/empty-subject
router/classify.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline --graph origin/main origin/release/1.0 fix/empty-subject
* 5c46029 Accept tickets without a subject
* bc944fd Set version 1.0.0
| * 29be88c Describe how to run the tests
| * 7e13c3d Raise the confidence threshold to 0.7
|/
* 9a383e5 Add classifier test
* f3e7ca9 Add routing config
* 53e7f57 Add keyword classifier
* fbbcc8d Add README
```

The command reads "take the commits of `fix/empty-subject` that are not on `origin/main` and replay them on `origin/release/1.0`". A plain `git rebase origin/release/1.0` would replay all three commits and change nothing about the pull request except the commit IDs (Lab 21.2).

Prevention. `gh pr create` uses the default branch as base unless you pass `--base` (section 17.16). Run `git log --oneline <base>..HEAD` first.

The other three causes

The base moved. Nothing is wrong; if the pull request page and a compare page must agree, update the branch (17.7). **Force-pushed history.** Review comments on the old commits become outdated; and if someone force-pushes the *base* branch, every open pull request against it lists the removed commits. Block force pushes on shared branches (Chapter 18). **Truncated or hidden diffs.** The file changed and the page does not render it; `git diff --stat base...head` on your machine has no such limit.

17.13 Indirect merges

In one sentence. GitHub marks a pull request as merged whenever its head commits become reachable from its base branch, by whatever route that happened.

Precisely. "A pull request can be marked as merged if its head branch commits become reachable from the base branch outside that pull request. This can happen when the same commits are merged through another pull request or pushed directly to the default branch ... Pull requests merged indirectly are marked as `merged` even if branch protection rules on that pull request were not satisfied" ([indirect merges](#)).

See it. Two branches where the upper one contains the lower one. Asha merges the *upper* pull request with a merge commit. The reachability test for the lower one:

```
# In a copy where nothing was merged yet. Asha merges the UPPER pull request into main:
$ cd ../../asha-indirect/ticket-router
$ git fetch -q
$ git merge -q --no-ff -m "Merge pull request #2 from feature/sla-timers" origin/feature/sla-timers
$ git push -q origin main
# Is the head of the LOWER pull request now reachable from main?
$ git merge-base --is-ancestor origin/feature/priority-routing main
[exit status: 0]
$ git log --oneline main..origin/feature/priority-routing
```

Exit status 0, and `main..origin/feature/priority-routing` is empty: every commit of the lower pull request is on `main`. Nobody pressed its merge button, and whatever review it still lacked was never given.

Why it matters. Reviews and checks are properties of a pull request; reachability is a property of commits. A rule that requires an approval is satisfied by the pull request that was actually merged, not by each pull request whose commits it contained. When an audit asks who approved a commit, answer from the pull request that carried it onto the base.

17.14 Stacked pull requests

In one sentence. A stack is a chain of pull requests in one repository in which each targets the branch of the one below, so that a large change is reviewed as small layers.

Version note. Older behavior: dependent pull requests were a convention maintained by hand. Current behavior: a GitHub feature in **public preview**, with the CLI extension `github/gh-stack`. Since: 30 July 2026 ([changelog](#), [reference](#)). Recommended: learn the plain-Git mechanics below first. `gh stack` is not part of `gh` 2.88.1 and none of its commands were run for this book.

Why the base decides what you see. Two layers, both pushed:

```
# The upper pull request against main lists both layers:
$ git log --oneline origin/main..feature/sla-timers
60a7f43 Alert before the SLA of a high-priority ticket expires
32a2f99 Add SLA minutes per priority
9dcfb58 Fix the name of the escalations queue
02820ea Route high-priority tickets to escalation
3807b29 Add priority scoring
# Against the branch below it, it lists its own layer:
$ git log --oneline feature/priority-routing..feature/sla-timers
60a7f43 Alert before the SLA of a high-priority ticket expires
32a2f99 Add SLA minutes per priority
$ git diff --stat feature/priority-routing...feature/sla-timers
config/routing.yaml | 1 +
router/priority.py   | 5 ++++
2 files changed, 6 insertions(+)
```

Against `main`, the upper pull request lists both layers, which is the wrong-base case of section 17.12. Against the branch below, it lists its own two commits.

What GitHub adds, from the reference page:

- All branches must be in the same repository: "Cross-fork stacks are not supported."
- "Every pull request in a stack is evaluated against rules for the **base of the stack**", typically `main`: required reviews, required status checks, CODEOWNERS and code scanning. Workflows trigger "as if each pull

request in the stack targets the base of the stack".

- A pull request can merge only when it and "all pull requests below it" meet the requirements, and when "the stack has a **fully linear history** between its branches".
- The asynchronous merge API, generally available since 1 October 2026, "is the only merge API that supports stacked pull requests" ([changelog](#)).

Linear, in Git terms, means the lower branch is an ancestor of the upper one:

```
# Linear stack: the lower branch is an ancestor of the upper one.
$ git merge-base --is-ancestor feature/priority-routing feature/sla-timers
[exit status: 0]
```

A review fix on the lower branch breaks that:

```
# A review fix lands on the lower branch:
$ git switch -q feature/priority-routing
$ printf '\n\ndef test_outage_goes_to_escalations():\n    assert classify({"subject": "Outage in
EU"}) == "escalations"\n' >> tests/test_classify.py
$ git commit -q -am "Test the escalations route"
$ git log --oneline --graph --decorate-refs=refs/heads feature/priority-routing feature/sla-timers -4
* c38d3bd Test the escalations route
| * 60a7f43 Alert before the SLA of a high-priority ticket expires
| * 32a2f99 Add SLA minutes per priority
|/
* 9dcfb58 Fix the name of the escalations queue
$ git merge-base --is-ancestor feature/priority-routing feature/sla-timers
[exit status: 1]
```

The upper branch still sits on the old tip. The repair is a rebase of the upper layer onto the new tip of the lower one, and a forced push of the rewritten branch:

```
# Restore the linear history: replay the upper layer on the new tip of the lower one.
$ git rebase feature/priority-routing feature/sla-timers
Rebasing (1/2)
Rebasing (2/2)
Successfully rebased and updated refs/heads/feature/sla-timers.
$ git merge-base --is-ancestor feature/priority-routing feature/sla-timers
[exit status: 0]
$ git log --oneline feature/priority-routing..feature/sla-timers
670af6f Alert before the SLA of a high-priority ticket expires
bf00849 Add SLA minutes per priority
$ git push -q origin feature/priority-routing
$ git push -q --force-with-lease origin feature/sla-timers
```

With more layers this cascades, which `gh stack rebase` and the **Rebase stack** button automate; the button's commits "are **not** signed" ([managing stacked pull requests](#)). In plain Git, `git rebase --update-refs` moves every branch of the chain in one run (Chapter 9, section 9.9).

When the bottom layer is squash-merged, the upper layer is in the squash-then-reuse situation of section 17.12, because it was built on commits that never reached `main`:

```
# The lower pull request was squash-merged and its branch deleted. The upper one now targets main:
$ git fetch --prune
From ../../server/ticket-router
- [deleted]          (none) -> origin/feature/priority-routing
 9a383e5..77fc160  main   -> origin/main
$ git log --oneline origin/main..feature/sla-timers
670af6f Alert before the SLA of a high-priority ticket expires
bf00849 Add SLA minutes per priority
c38d3bd Test the escalations route
9dcfb58 Fix the name of the escalations queue
02820ea Route high-priority tickets to escalation
```

```
3807b29 Add priority scoring
$ git merge-tree --write-tree --name-only origin/main feature/sla-timers
a9730e29529badbf0b65cf38bfcd5b884dde3c7b
router/priority.py
```

```
Auto-merging router/priority.py
CONFLICT (add/add): Merge conflict in router/priority.py
[exit status: 1]
```

```
# Only the upper layer is new. Transplant it: everything after the old lower branch, onto main.
$ git rebase --onto origin/main feature/priority-routing feature/sla-timers
Rebasing (1/2)
Rebasing (2/2)
Successfully rebased and updated refs/heads/feature/sla-timers.
$ git log --oneline origin/main..feature/sla-timers
41f9055 Alert before the SLA of a high-priority ticket expires
a791005 Add SLA minutes per priority
$ git merge-tree --write-tree origin/main feature/sla-timers
a59c7634e45ad06be44cb2ccab2a81fa40028b16
[exit status: 0]
```

The stack feature does this for you: "the remaining branches are automatically rebased so the next pull request targets the default base branch" ([about stacked pull requests](#)). Without it, GitHub only retargets: when a merged head branch is deleted, pull requests based on it switch to the merged pull request's base ([merging a pull request](#)). The transplant is then yours to do.

When not to stack. Layers that are not dependent should be independent pull requests. A stack multiplies force pushes and re-approvals, and a problem in the bottom layer delays every layer above.

17.15 The fork workflow end to end

In one sentence. In the fork-and-pull model you push to a repository you own and ask the upstream repository to take the commits; the pull request lives in the upstream repository.

Precisely. Chapter 12, section 12.10 built the triangular configuration: `origin` is your fork, `upstream` the shared repository. This section adds the pull request. Platform facts first ([pull requests reference](#), [forks reference](#)):

- "You do not need permission from the upstream repository to push to a fork you created."
- "A fork and its upstream share the same Git data. This means that all content uploaded to a fork is accessible from the upstream and all other forks of that upstream."
- The author can let maintainers push to the pull request branch. That works only for forks owned by a user: "You cannot give push permissions to a fork owned by an organization" ([allowing changes to a pull request branch](#)).
- For workflows triggered by a pull request from a fork, "secrets are not passed to the runner" except `GITHUB_TOKEN`, which "has read-only permissions" ([events, workflows in forked repositories](#)). Chapter 21A: GitHub Actions security explains why, and what `pull_request_target` changes.

Picture.




```
To ../../forks/you/ticket-router.git
0c6455c..ef0b9f0 pr-7 -> fix/empty-subject
```

Your branch moved without you. Pull before you add anything:

```
# You. Your branch in the fork moved without you:
$ git pull
From ../../forks/you/ticket-router
0c6455c..ef0b9f0 fix/empty-subject -> origin/fix/empty-subject
Updating 0c6455c..ef0b9f0
Fast-forward
 tests/test_classify.py | 4 +++
1 file changed, 4 insertions(+)
$ git log --format="%h %an: %s" upstream/main..fix/empty-subject
ef0b9f0 Asha Rao: Test a ticket without a subject
0c6455c Lab User: Accept tickets without a subject
```

Asha merges. Then the step that beginners skip, bringing clone and fork back in line with upstream:

```
# You. Bring your clone and your fork up to date with upstream:
$ git switch -q main
$ git fetch upstream
From ../../server/ticket-router
01822ed..fafe8b6 main -> upstream/main
$ git merge --ff-only upstream/main
Updating 9a383e5..fafe8b6
Fast-forward
 config/routing.yaml | 2 +-
 router/classify.py | 2 +-
 tests/test_classify.py | 4 +++
3 files changed, 6 insertions(+), 2 deletions(-)
$ git push origin main
To ../../forks/you/ticket-router.git
9a383e5..fafe8b6 main -> main
$ git rev-list --left-right --count upstream/main...origin/main
0 0
```

```
$ git branch -d fix/empty-subject
Deleted branch fix/empty-subject (was ef0b9f0).
$ git push origin --delete fix/empty-subject
To ../../forks/you/ticket-router.git
- [deleted] fix/empty-subject
$ git branch -a
* main
remotes/origin/HEAD -> origin/main
remotes/origin/main
remotes/upstream/HEAD -> upstream/main
remotes/upstream/main
```

`0 0`: fork and upstream agree. On GitHub the same sync is **Sync fork** or `gh repo sync`, which fast-forwards and, with `--force`, hard-resets the fork's branch (`gh repo sync --help`). If you committed on the fork's `main`, `--force` discards those commits. Lab 21.1 has you make that mistake and recover.

Three rules. Never commit on the fork's default branch. Branch from `upstream/main` after a fetch. One branch per pull request, deleted after the merge.

17.16 `gh pr`: the commands

In one sentence. `gh pr` drives the pull request object from the terminal; the Git work around it stays with `git`.

Every command and flag below was checked against `gh <command> --help` of the installed CLI, 2.88.1. None was run against GitHub here. Run them in your normal shell, not in `labs/shell`.

```

# Create
gh pr create --base main --title "Accept tickets without a subject" --body "Fixes #12"
gh pr create --draft --fill                # title and body from the commits; opens as a draft
gh pr create --dry-run                    # print what would be created (may still push)

# Inspect
gh pr status
gh pr list --state open --base main
gh pr view 12 --json baseRefName,headRefOid,mergeable,mergeStateStatus,reviewDecision
gh pr diff 12 --name-only
gh pr checks 12 --required                # exit status 8 while checks are pending

# Review and iterate
gh pr checkout 12                        # local branch for the head of pull request 12
gh pr review 12 --approve                 # or --request-changes / --comment, with --body
gh pr ready 12                           # draft -> ready; --undo goes back
gh pr edit 12 --base main                 # change the base branch
gh pr update-branch 12                   # merge the base into the head; --rebase to rebase

# Finish
gh pr merge 12 --squash --delete-branch
gh pr merge 12 --merge --match-head-commit "$(git rev-parse HEAD)"
gh pr merge 12 --auto --rebase
gh pr close 12 --comment "Superseded by #15" --delete-branch
gh pr revert 12

```

What the help text itself tells you:

- `gh pr create` takes the base from `--base`, else from the Git configuration value `branch.<current>.gh-merge-base`, else the default branch. Maintainers may push to the head branch by default; `--no-maintainer-edit` disables that.
- `--match-head-commit` merges only if the head is the given commit: the terminal's answer to the hijack problem of section 17.5.
- `gh pr merge --admin` 🚫 uses "administrator privileges to merge a pull request that does not meet requirements". What it changes: the base branch, without the required reviews or checks. Preview: `gh pr checks --required`. Recovery: `gh pr revert`. Appropriate: a documented emergency, by someone the bypass list names (Chapter 18).
- The JSON field names are those of the [current manual](#), which describes the newest CLI (2.102.0 on 1 October 2026).

17.17 Review practice

In one sentence. A review is a claim about a specific diff at a specific commit, so a good reviewer controls which diff and which commit they looked at.

This section is practice derived from the mechanics above, not GitHub documentation.

As an author. Keep the pull request small; GitHub's own guidance is that merging soon "encourages contributors to make pull requests smaller, which we recommend in general" ([branches reference](#)). Check `git log --oneline origin/main...HEAD` before you open it. Say what you tested: for an ML change, which evaluation set, which metric, which commit. Once review has started, add commits instead of rewriting; if you must rewrite, say so and keep the content change separate from the rebase.

As a reviewer. For anything risky, run the code: `gh pr checkout 12`, then the tests. After a force push, compare the old and new series with `git range-diff` (Chapter 9, section 9.14). Approve the commit you read: `--match-head-commit` on the command line, "dismiss stale approvals" in a ruleset. Give changes under `.github/workflows/`, to `CODEOWNERS`, to lock files and to deployment code a second reader; Chapter 19 turns that habit into a rule. Treat an automated approval as a signal about the diff, not about the intent.

What a review cannot see. Semantic conflicts with other open pull requests, files hidden by diff limits, and anything the test merge did not include. Checks on the merged result catch those, which is the argument for "strict" checks or a merge queue.

17.18 Display limits

In one sentence. GitHub truncates large pull requests; the Git data is complete, the page is not.

The documented limits ([repository limits](#)):

What	Limit
Total diff of a pull request	20,000 lines that you can load, or 1 MB of raw diff
One file's diff	20,000 loadable lines or 500 KB; 400 lines and 20 KB load automatically
Files in one diff	300 (25 for renderable files such as images)
Commits listed on compare and pull request pages	250, with a note that more exist
"Rebase and merge"	100 commits
Recommended maximums	1,000 open pull requests against one branch; 1 merged pull request per minute

Beyond a limit, review locally: `git diff --stat base...head` has no ceiling. A pull request that hits these limits is usually a case of section 17.12, or a generated file that should not be in the diff.

Version note. Older behavior: the classic "Files changed" page. Current behavior: a redesigned page is the default. Since: 22 January 2026 ([changelog](#)). Recommended: distrust older screenshots. Whether the classic opt-out still exists on 1 October 2026 is not confirmed in the Phase 0 report.

17.19 What can go wrong

Symptom	Diagnosis	Fix	Prevention
The pull request lists commits you did not write	<code>git log --oneline <base>..<code>head</code>; git merge-base</code>	change the base, or <code>git rebase --onto</code> (17.12)	check the range before opening
Old commits reappear after a squash merge	the merge base is the old fork point	<code>git rebase --onto origin/main <last squashed commit></code>	delete head branches on merge
No CI run on the pull request	a conflict (no test merge), a draft, or a workflow filter	resolve the conflict; Chapter 20B	update the branch early
CI green, <code>main</code> red after the merge	the test merge used an older base (17.2)	fix forward or revert	strict checks or a merge queue
"Changes requested", yet the merge button works	no rule requires a pull request (17.4)	add the rule (Chapter 18)	know which controls are advisory
<code>git branch -d</code> says "not fully merged"	squash or rebase method (17.9)	verify with <code>merge-tree</code> , then <code>-D</code>	expected
A pull request is "merged" that nobody merged	indirect merge (17.13)	review what landed	do not merge branches that contain unreviewed pull requests

Symptom	Diagnosis	Fix	Prevention
Fork cannot be synced	commits on the fork's default branch	move them to a branch (Lab 21.1)	never commit there

17.20 When not to use it, and dangerous edge cases

- **A pull request is not a durable record.** It is a GitHub object, in no clone, and it does not move to another host with the repository. Put the "why" into commit messages or files as well.
- **Anything pushed to a pull request is published.** Deleting the branch does not remove the commits (17.2).
- **Commit IDs are not stable evidence under squash and rebase.** If compliance needs "the reviewed commit is the deployed commit", only the merge-commit method satisfies it literally.
- **Rebase and merge discards signatures** and cannot satisfy a rule that requires signed commits on the base (Chapter 18, section 18.10).
- `--admin`, **bypass lists and indirect merges** are three ways a change reaches a protected branch without the reviews the rules describe. Know all three before you tell an auditor that every change was reviewed.
- **Do not gate a deployment on** `refs/pull/N/merge`. It can be stale. Gate on a check that ran on the commit that landed.
- **Stacked pull requests are a preview.** Do not make a release process depend on them.

17.21 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git fetch origin pull/N/head:pr-N</code>	● SAFE	adds objects and a local branch	<code>git ls-remote origin 'refs/pull/*'</code>	<code>git branch -D pr-N</code>
<code>git log base..head, git diff base...head, git merge-tree --write-tree</code>	● SAFE	nothing (merge-tree adds unreferenced objects)	not needed	not needed
<code>gh pr create</code>	● CAUTION	may push the branch; creates a GitHub object that notifies people	<code>--dry-run</code>	<code>gh pr close</code>
<code>git merge --no-ff, git merge --squash, git rebase, git rebase --onto</code>	● CAUTION	move or rewrite the current branch	<code>git merge-tree; git log <upstream>..HEAD</code>	<code>ORIG_HEAD</code> , the reflog, <code>git rebase --abort</code>
<code>git push --force-with-lease</code>	● DANGEROUS	replaces the remote branch if it is where you last saw it; can destroy commits on the server that no clone of yours has, and alone the check passes wrongly after a background fetch (Chapter 12, section 12.8)	<code>add --dry-run; git log HEAD..origin/<branch></code> after a fetch	push the old ID back from a reflog; appropriate for your own pull request branch after a rebase
<code>gh pr update-branch, gh pr edit --base, gh pr merge</code>	● CAUTION	move the head branch, change the base, or write to the base	<code>gh pr view, gh pr checks --required</code>	change the base back; <code>gh pr revert</code>

Command	Label	What it changes	Preview	Recovery
		branch; can dismiss approvals		
<code>gh pr merge --admin</code>	 DANGEROUS	merges without the required reviews and checks	as above	<code>gh pr revert</code> ; record the reason
<code>git push --force</code> to a shared base branch	 DANGEROUS	removes commits from the remote branch; corrupts open pull requests	<code>git log <branch>..origin/<branch></code>	Chapter 13; Chapter 30: Incident response
<code>git branch -D</code>	 DANGEROUS	deletes a ref and its reflog	the tree comparison of 17.9	<code>git branch <name> <id></code>
<code>gh repo sync --force</code>	 DANGEROUS	hard-resets a branch of the fork	<code>git rev-list --left-right --count upstream/main...origin/main</code>	push the old commits back from a clone that has them

17.22 Version notes

Topic	Older behavior	Current behavior	Since	Recommended
Test merge commits	also regenerated on page view	on push, on merge-base change, or when older than 12 hours	19 Feb 2026	do not rely on a fresh <code>refs/pull/N/merge</code>
Draft pull requests	paid plans for private repositories	all repositories	1 May 2025	use drafts freely
Stacked pull requests	manual convention	public preview, <code>github/gh-stack</code>	30 Jul 2026	learn the Git mechanics first
Programmatic merge	synchronous <code>PUT .../merge</code>	asynchronous merge API recommended; the synchronous one supports neither stacks nor merge queues	1 Oct 2026	new automation uses the asynchronous endpoint
Copilot as reviewer	comments only	can approve (preview, off by default)	1 Sep 2026	decide whether such approvals count
Pull request access	always open	a repository can disable pull requests or limit them to collaborators (changelog)	13 Feb 2026	know the setting exists
Documentation URLs	<code>/pull-requests/collaborating-with-pull-requests/...</code>	<code>/pull-requests/reference/...</code> and <code>/pull-requests/how-tos/...</code>	2025 to 2026	cite the new paths

17.23 Practice

Each lab has a local part with real transcripts and a GitHub part in your practice organization.

- Module 21 labs: Lab 21.1, a full fork and pull request cycle; Lab 21.2, a pull request against the wrong base; Lab 21.3, the squash-then-reuse problem.
- Module 22 labs: Lab 22.1, the three merge methods compared; Lab 22.2, a release from an annotated tag.
- Replay any transcript of this chapter with `labs/run`, for example `labs/run ch17/merge-methods`.
- Then read Chapter 18: Branch Protection and Rulesets, which turns the advisory controls of this chapter into enforced ones.

17.24 Interview questions

1. What exactly does GitHub create when a pull request is opened? Which parts are Git data, and in which repository do they live?
2. A pull request for a one-line change lists forty commits. Give three causes and the command that distinguishes them.
3. Why does a pull request show a three-dot diff and not a two-dot diff? Construct a case in which they differ.
4. CI passed on the pull request and failed on `main` right after the merge. Which commit did CI test, and what closes that gap?
5. Compare the three merge methods for a team that requires signed commits on `main` and must show an auditor that the reviewed commit is the deployed commit.
6. After "Squash and merge", `git branch -d` refuses to delete the local branch. Explain the refusal from the definition of "merged", and show how to verify that deleting is safe.
7. A reviewer approved, the author pushed another commit, and the pull request merged. Which two settings address this, and what does each cost?
8. What does a merge queue test that a `pull_request` workflow does not? Why can a required check stay unreported forever on a queue?
9. A pull request is shown as merged, yet nobody merged it and it had no approval. How is that possible?
10. Your fork's `main` cannot be fast-forwarded to upstream. What happened, how do you keep the stray work, and why is `gh repo sync --force` the wrong first move?
11. A contributor leaked a key in a pull request and deleted the branch. Is the commit gone?
12. You are asked to pick one merge method for a 40-person ML platform team. What do you ask before answering?

17.25 Sources

Primary sources (docs.github.com, the GitHub Changelog and cli.github.com; read on 1 and 2 October 2026)

- Reference pages: [Pull requests](#), [Branches](#), [Pull request merges](#), [Pull request reviews](#), [Status checks](#), [Merge conflicts](#), [Forks](#), [Stacked pull requests](#)
- How-to pages: [Checking out pull requests locally](#), [Merging a pull request](#), [Troubleshooting required status checks](#), [Syncing a fork](#); the others are linked where they are quoted.
- Repository pages: [About merge methods on GitHub](#), [Managing a merge queue](#), [Repository limits](#), [About commit signature verification](#), [REST API endpoints for pull requests](#), [Events that trigger workflows](#)
- Changelog entries, linked where they are used: 19 February 2026 (test merge commits), 30 July 2026 (stacked pull requests), 1 October 2026 (asynchronous merge API).
- GitHub CLI: `gh pr --help` and its subcommands (2.88.1, local); [gh pr manual](#).
- Git 2.55 manual pages: `git help merge-tree`, `git help rebase`, `git help branch`, `git help cherry`, `git help config (receive.hideRefs)`; the [gitfaq](#) entry on squash merges and long-lived branches.

Secondary sources

- The Phase 0 report of this course, sections 2, 3, 12 and 13, and its notes on the GitHub platform.
- GitHub Engineering, [Scaling merge-ort across GitHub](#) (2023): why a server needs a merge without a working tree.

Videos (from the Phase 0 report, with its caveats)

- ["The ultimate beginner's guide to GitHub in 2026"](#), GitHub, 22 September 2025: the pull request and merge chapters, 40:44 to 47:14. Mechanics only; compiled from 2024 episodes. The report found no verified video that teaches code review end to end in the 2026 interface.

Further reading

- Chapter 8: Merge, sections 8.12, 8.15 and 8.17; Chapter 9: Rebase, sections 9.9 and 9.14; Chapter 12: Remote Operations, sections 12.8, 12.10 and 12.12; Chapter 14A: History investigation, sections 14A.2, 14A.8, 14A.18 and 14A.22.
- [Pro Git](#), "[Contributing to a Project](#)": the fork-and-pull flow at a book's pace; its screenshots predate the current interface.

Chapter 18: Branch Protection and Rulesets

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026, with the quoted documentation pages re-read on 2 October 2026. Transcripts are real output from `labs/ch18/`. No ruleset in this chapter was created on GitHub by the author: every statement about what GitHub enforces or displays comes from its documentation or changelog and carries the link. Plan gates and user-interface labels change; re-check them on the day you rely on them.

18.1 Why this matters

Three sentences you will hear from a CTO after an incident:

1. " `main` is protected. How did a force push get through?"
2. "The pull request has two approvals and every check is green. Why is the merge button grey?"
3. "Who is allowed to skip our rules, and would we know if they did?"

Chapter 17 ended with a list of controls that are only advisory: a "changes requested" review, a red check, a CODEOWNERS file. This chapter is about the layer that makes them binding. On GitHub in 2026 that layer has two mechanisms that coexist, **rulesets** and **classic branch protection rules**, and most "why can't I merge" tickets come from not knowing that both are evaluated.

The chapter teaches rulesets first, because GitHub now says so: "Rulesets are the recommended way to protect your branches" ([changelog](#), 7 July 2026). Classic rules follow in section 18.14, with what still differs.

18.2 What a rule is: a check on a ref update

In one sentence. A rule is a condition that GitHub evaluates on its server before it lets a ref move, and neither your client nor your `--force` flag has a say in it.

Analogy. A bank teller will move money between accounts for anyone with the right card, but checks each transfer against the account's conditions first: two signatures above a limit, no withdrawals from a frozen account. The card is your write permission. The conditions are the rules. The analogy breaks in one place: on GitHub the conditions are published, and anyone who can read the repository can read them.

Precisely. Every change to a repository on a server is a ref update: a ref name, the old object ID, the new one. A push proposes ref updates. A merge button is GitHub performing one. Chapter 12, section 12.7 showed the plain-Git hook that sees each proposal first, `pre-receive`. Five ruleset rules are predicates on those three values:

Ruleset rule	The question it asks about (old, new, ref)
Restrict creations	is the old ID all zeros?
Restrict deletions	is the new ID all zeros?
Block force pushes	is old <i>not</i> an ancestor of new?
Require linear history	does <code>old..new</code> contain a commit with two parents?
Restrict updates	are both IDs non-zero, that is, does an existing ref move?

The remaining rules (a pull request, approvals, status checks, signatures) ask about GitHub objects attached to the new commits. They need the platform's database, which is why plain Git cannot imitate them.

See it. A bare repository plays the server. A hook of my own imitates the five rules; it is not GitHub's implementation.

```
$ cat rules/pre-receive
#!/bin/sh
# pre-receive: the server runs this before any ref moves.
# Standard input has one line per proposed ref update: <old id> <new id> <ref name>
zero=000000000000000000000000000000000000000000000000000
status=0
while read old new ref; do
  case "$ref" in
    refs/heads/main)                # target of the branch rules
      if [ "$new" = "$zero" ]; then
        echo "rule 'restrict deletions': $ref may not be deleted"; status=1; continue
      fi
      [ "$old" = "$zero" ] && continue # creation: nothing to compare with
      if ! git merge-base --is-ancestor "$old" "$new"; then
        echo "rule 'block force pushes': the update would remove commits from $ref"; status=1
      fi
      if [ -n "$(git rev-list --merges "$old..$new")" ]; then
        echo "rule 'require linear history': the update adds a merge commit to $ref"; status=1
      fi ;;
    refs/tags/v*)                  # target of the tag rules
      if [ "$new" = "$zero" ]; then
        echo "rule 'restrict deletions': $ref may not be deleted"; status=1
      elif [ "$old" != "$zero" ]; then
        echo "rule 'restrict updates': $ref already exists and may not move"; status=1
      fi ;;
  esac
done
exit $status
```

Once the hook is executable, a fast-forward still goes through:

```
$ printf '# Changelog\n\n## 1.1 (unreleased)\n' > CHANGELOG.md && git add CHANGELOG.md
$ git commit -q -m "Open the changelog for 1.1"
$ git push origin main
To ../../server/ticket-router.git
4e1f5fe..810dc2f  main -> main
```

A rewritten `main` does not:

```
$ git commit -q --amend -m "Open the changelog for 1.1.0"
$ git push --force origin main
remote: rule 'block force pushes': the update would remove commits from refs/heads/main
To ../../server/ticket-router.git
! [remote rejected] main -> main (pre-receive hook declined)
error: failed to push some refs to ' ../../server/ticket-router.git'
[exit status: 1]
$ git push --force-with-lease origin main
remote: rule 'block force pushes': the update would remove commits from refs/heads/main
To ../../server/ticket-router.git
! [remote rejected] main -> main (pre-receive hook declined)
error: failed to push some refs to ' ../../server/ticket-router.git'
[exit status: 1]
$ git reset -q --hard origin/main
```

Read the two rejections. `! [remote rejected]` means the server said no, as opposed to `! [rejected]`, which is your own Git refusing. `--force-with-lease` fared no better than `--force`: both only switch off *client* checks. Deletion is a ref update too:

```
$ git push origin --delete main
remote: rule 'restrict deletions': refs/heads/main may not be deleted
To ../../server/ticket-router.git
! [remote rejected] main (pre-receive hook declined)
```

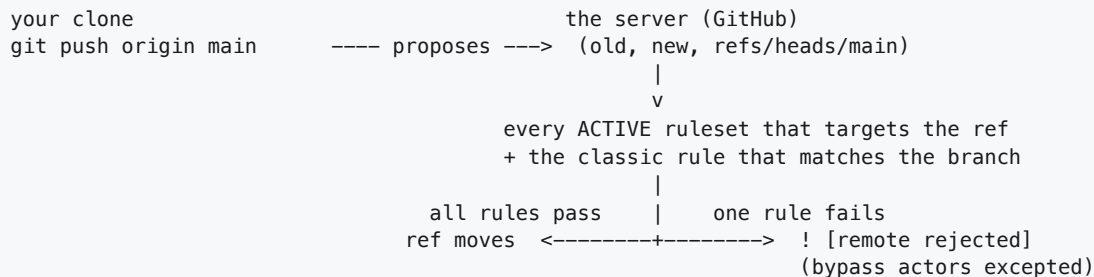
```
error: failed to push some refs to '../server/ticket-router.git'
[exit status: 1]
```

The linear-history rule ties this chapter to the merge methods of Chapter 17, section 17.8:

```
# A merge commit, the result of the "Create a merge commit" method, pushed to main:
$ git merge -q --no-ff -m "Merge pull request #1 from feature/priority-routing" feature/priority-
routing
$ git push origin main
remote: rule 'require linear history': the update adds a merge commit to refs/heads/main
To ../server/ticket-router.git
! [remote rejected] main -> main (pre-receive hook declined)
error: failed to push some refs to '../server/ticket-router.git'
[exit status: 1]
$ git reset -q --hard origin/main
# The squash method produces one ordinary commit, which the rule accepts:
$ git merge -q --squash feature/priority-routing
Automatic merge went well; stopped before committing as requested
Squash commit -- not updating HEAD
$ git commit -q -m "Route high-priority tickets to an escalations queue (#1)"
$ git push origin main
To ../server/ticket-router.git
810dc2f..8c7e96f main -> main
```

The merge commit is refused; the squash commit, an ordinary one-parent commit, is accepted. That is the whole content of GitHub's sentence that under this rule "any pull requests merged into the branch or tag must use a squash merge or a rebase merge" ([available rules](#)).

Picture.



Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
A push rejected by a rule	unchanged	unchanged	unchanged	unchanged	remote-tracking branch unchanged	no ref moves	Rule Insights records the failed evaluation
Creating or activating a ruleset 🟡	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged; no ref moves	later ref updates and merges are evaluated against it

In production. Asked "how did a force push get through", check in this order: was the ruleset Active at that time, did it target that branch name, was the actor on a bypass list, and was the rule in the ruleset at all. Sections 18.3 to 18.5 are those four questions.

18.3 Rulesets: targets and enforcement status

In one sentence. A ruleset is a named list of rules with a target (which refs), an enforcement status (on or off) and a bypass list (who is excepted).

Precisely. "You can have up to 75 rulesets per repository, and 75 organization-wide rulesets" ([about rulesets](#)). There are three kinds, chosen when you create one ([creating rulesets](#)):

Kind	Targets	Typical use
Branch ruleset	branches by name pattern, or "the default branch"	protect <code>main</code> and release branches
Tag ruleset	tags by name pattern	make release tags immutable (18.12)
Push ruleset	every push to the repository and its fork network	block file paths, extensions and sizes (18.12)

Target patterns. Branch and tag targets use `fnmatch` syntax. GitHub names the exact function: "Because GitHub uses the `File::FNM_PATHNAME` flag for the `File.fnmatch` syntax, the `*` wildcard does not match directory separators (`/`). For example, `qa/*` will match all branches beginning with `qa/` and containing a single slash, but will not match `qa/foo/bar`. You can include any number of slashes after `qa` with `qa/**/*`" (same page). That is a Ruby function, and macOS ships a Ruby, so the documented behavior can be run. The script calls `File.fnmatch(pattern, name, File::FNM_PATHNAME)` for each name:

```
$ /usr/bin/ruby match.rb "qa/*" qa/login qa/login/retry qa
qa/*      qa/login      match
qa/*      qa/login/retry -
qa/*      qa            -
$ /usr/bin/ruby match.rb "qa/**/*" qa/login qa/login/retry qa/a/b/c
qa/**/*   qa/login      match
qa/**/*   qa/login/retry match
qa/**/*   qa/a/b/c      match
```

```
$ /usr/bin/ruby match.rb "release/*" release/1.0 release/1.0/hotfix releases/1.0
release/*  release/1.0    match
release/*  release/1.0/hotfix -
release/*  releases/1.0   -
$ /usr/bin/ruby match.rb "*feature*" feature-x my-feature feature/x team/feature/x
*feature*  feature-x      match
*feature*  my-feature     match
*feature*  feature/x      -
*feature*  team/feature/x -
$ /usr/bin/ruby match.rb "**/*" main feature/x a/b/c
**/*       main          match
**/*       feature/x     match
**/*       a/b/c         match
$ /usr/bin/ruby match.rb "*" main feature/x
*          main          match
*          feature/x     -
```

Three results deserve attention. `release/*` does not cover `release/1.0/hotfix`. `*feature*` does not cover `feature/x`, although the documentation uses that very pattern as an example of "any branches matching", because `*` stops at the slash. And `*` alone matches no branch that has a slash in its name. A pattern that silently fails to match leaves a branch unprotected, with nothing to tell you. Section 18.16 shows how to ask GitHub which rules apply to a given branch name.

Backslash quoting, `[^...]` and extended globs are documented as unsupported. In the REST API two special targets exist, `~DEFAULT_BRANCH` and `~ALL` ([REST: rules](#)). Prefer the first for `main`: it follows a renamed default branch.

Enforcement status. The documentation for Free, Pro and Team lists two: **Active**, "your ruleset will be enforced upon creation", and **Disabled**. The Enterprise Cloud documentation adds **Evaluate**: "your ruleset will not be enforced, but you will be able to monitor which actions would or would not violate rules on the 'Rule Insights' page" ([Enterprise view](#)). The REST schema says the same in a parenthesis: "evaluate is only available with GitHub Enterprise". In the hook imitation the executable bit plays this part:

```
# Enforcement status "disabled": the same force push goes through.
$ chmod -x ../../server/ticket-router.git/hooks/pre-receive
$ git push --force origin v1.0.0
hint: The 'hooks/pre-receive' hook was ignored because it's not set as executable.
hint: You can disable this warning with `git config set advice.ignoredHook false`.
To ../../server/ticket-router.git
+ 13ad80e...d68a79d v1.0.0 -> v1.0.0 (forced update)
```

A disabled ruleset protects nothing and still looks reassuring in a settings page. Disabling is how rulesets get "temporarily" switched off during an incident and forgotten.

Unverified. Whether Evaluate mode is available to Team-plan organization rulesets is not stated; it appears only in the Enterprise Cloud documentation (flag carried from the Phase 0 report).

18.4 Layering: every applicable rule applies

In one sentence. Rulesets have no priority order: all active rulesets that target a ref, and the classic rule that matches it, are added together, and for each rule the strictest version wins.

Precisely. "A ruleset does not have a priority. Instead, if multiple rulesets target the same branch or tag in a repository, the rules in each of these rulesets are aggregated. If the same rule is defined in different ways across the aggregated rulesets, the most restrictive version of the rule applies. As well as layering with each other, rulesets also layer with protection rules targeting the same branch or tag" ([about rulesets](#), [rule layering](#)).

GitHub's own example: a ruleset requires signed commits and three reviews; a classic rule on the same branch requires linear history and two reviews. Result: signed commits, linear history, and three reviews.

```
organization ruleset    : block force pushes, 1 approval
repository ruleset A    : signed commits, 3 approvals
repository ruleset B    : required check "ci"
classic rule on main    : linear history, 2 approvals
-----
what a merge into main must satisfy: no force push, signed commits, "ci" green,
                                   linear history, 3 approvals
```

Two consequences. First, **you cannot loosen a branch by editing one layer**. Lab 23.1 reproduces this with two plain-Git layers: the hook is disabled and the force push is still refused, by `receive.denyNonFastForwards`.

Second, a repository ruleset can add to an organization ruleset and never subtract: "creating a new ruleset can make the rules targeting a branch or tag more restrictive, but never less restrictive" ([organization rulesets](#)).

Forks. "Forks do not inherit branch or tag rulesets from their upstream repositories", but they "do inherit push rulesets from their root repository" (same page). A contributor's fork of your protected repository is unprotected, which is fine: the rules guard your refs, not theirs.

18.5 Bypass and exempt actors

In one sentence. A ruleset applies to everyone, administrators included, except the actors on its bypass list.

Precisely. Eligible for a repository ruleset's bypass list are "Repository admins, organization owners, and enterprise owners", "the maintain or write role, or custom repository roles based on the write role", "Teams, excluding secret teams", "GitHub Apps" and "Dependabot" ([granting bypass permissions](#)). Individual users have been eligible since 7 May 2026 ([changelog](#)). Each entry has a mode; the REST schema names the three ([REST: rules](#)):

Mode	What the actor may do	Trace
always	push directly and merge despite the rules	a bypass is recorded
pull_request ("For pull requests only")	must open a pull request, and may then merge it despite unmet rules	"a clear trail of their changes in the pull request and audit log"
exempt	the rules are not run for this actor	none: "a bypass audit entry will not be created"

The exempt mode arrived on 10 September 2025. GitHub contrasts it with a standard bypass, "a 'break glass' action that requires an explicit actor bypass and generates prominent audit signals", whereas "an exemption silently skips enforcement" ([changelog](#)).

This answers question 3 of section 18.1. With an empty bypass list nobody can skip the rules, including the repository's administrators. With `pull_request` bypass you would know. With `exempt` you would not.

GitHub, not Git. The Maintain role's documented right to "push to protected branches" carries the remark "Doesn't apply to rulesets as these have a different bypass model" ([repository roles](#)). Under rulesets, roles grant nothing by themselves; only the bypass list does.

In production. Keep the bypass list short and made of roles or teams, not people. Prefer `pull_request` mode for humans, so that an emergency merge still leaves a pull request. Reserve `always` for the one automation that must push (a release bot), and give that automation its own GitHub App identity so that the entry names a thing you can audit.

18.6 The rules and their sub-options

In one sentence. A branch or tag ruleset can contain about a dozen kinds of rule; four of them (pull request, status checks, signed commits, linear history) cause nearly all blocked merges.

The catalogue, from [Available rules for rulesets](#) and its [Enterprise Cloud view](#). The last column is the `type` in the REST API ([REST: rules](#)).

Rule	What it enforces, and its sub-options	REST type
Restrict creations	only bypass actors may create matching refs	<code>creation</code>
Restrict updates	only bypass actors may push to matching refs	<code>update</code>
Restrict deletions	only bypass actors may delete matching refs. "This rule is selected by default."	<code>deletion</code>
Block force pushes	"This rule is enabled by default." Side effect: the default branch cannot be changed or renamed without bypass rights	<code>non_fast_forward</code>
Require linear history	no merge commits on the target; the repository must allow squash or rebase merging first	<code>required_linear_history</code>
Require a pull request before merging	"The pull request doesn't necessarily have to be approved, but it must be opened." Sub-options: number of approvals; dismiss stale approvals; code owner review; restrict who can dismiss reviews; approval of the most recent reviewable push; resolved conversations; allowed merge methods; required reviewers by path; extra approval for unattributed Copilot pull requests (preview)	<code>pull_request</code>
Require status checks to pass	a list of check names; "strict" (branch up to date) or "loose"; optional expected source app per check	<code>required_status_checks</code>

Rule	What it enforces, and its sub-options	REST type
Require signed commits	only signed and verified commits may be pushed to the target (18.10)	<code>required_signatures</code>
Require deployments to succeed	named environments must have deployed the change first	<code>required_deployments</code>
Require merge queue	merges go through a queue (Chapter 17, section 17.11); documented only in the Enterprise Cloud view; not available in organization-level rulesets	<code>merge_queue</code>
Require code scanning results; code quality results; restrict code coverage (preview); require secret scanning alerts are resolved (preview)	block the merge on findings, on analysis still running, or on a missing tool. Chapter 21B: Repository security	<code>code_scanning</code> , <code>code_quality</code> , <code>code_coverage</code>
Automatically request Copilot code review	requests a review; it does not block	<code>copilot_code_review</code>
Require workflows to pass	organization or enterprise level; Enterprise Cloud view. Chapter 20B	<code>workflows</code>
Metadata restrictions	Enterprise plan: patterns for commit messages, author and committer email, branch and tag names	<code>commit_message_pattern</code> and four more

Unverified. The REST name of the secret-scanning rule is given as `require_secret_scanning_alert_resolution` in the [changelog of 9 September 2026](#); the REST schema page read for this chapter does not list it yet.

A new ruleset therefore starts with two rules already selected: restrict deletions and block force pushes. Everything else is a decision. The next five sections take the rules that need explaining.

18.7 Required reviews

In one sentence. The pull request rule turns Chapter 17's advisory reviews into a gate, and each sub-option closes one specific loophole.

Sub-option (REST parameter)	Loophole it closes	Cost or caveat
Required approvals (<code>required_approving_review_count</code>)	merging without a second person	approvals count only from people with write permission; authors cannot approve their own pull request
Dismiss stale approvals (<code>dismiss_stale_reviews_on_push</code>)	commits pushed after the approval	re-approval after every change of the diff, including "Update branch" and a moved merge base (Chapter 17, section 17.5)
Approval of the most recent reviewable push (<code>require_last_push_approval</code>)	the last pusher approving their own addition	weaker than dismissal; GitHub: "it is safer to dismiss stale reviews"
Review from code owners (<code>require_code_owner_review</code>)	changes to owned paths without their owner	any one listed owner suffices (Chapter 19)
Restrict who can dismiss reviews (<code>dismissal_restriction</code>)	someone with write access dismissing a blocking review	available in rulesets since 7 July 2026

Sub-option (REST parameter)	Loophole it closes	Cost or caveat
Resolved conversations (<code>required_review_thread_resolution</code>)	merging over open questions (18.9)	the author can resolve threads too
Allowed merge methods (<code>allowed_merge_methods</code>)	history written in a form the team rejected	conflicts with repository settings block the merge
Required reviewers (<code>required_reviewers</code>)	paths that need a specific team, with a count	teams only; organization repositories only

Three rules of evaluation that the documentation states and people miss ([pull request rule](#)):

- **A request for changes now blocks.** "If someone chooses the **Request changes** option in a review, then that person must approve the pull request before the pull request can be merged." If that person is unavailable, "anyone with write permissions for the repository can dismiss the blocking review", unless dismissal is restricted.
- **Twin pull requests block each other.** "Collaborators cannot merge the pull request if there are other open pull requests that have a head branch pointing to the same commit with pending or rejected reviews."
- **Method conflicts block.** "If the repository has disabled a merge method and the ruleset required a different method, the merge will be blocked." A ruleset that allows only `squash` in a repository where squash merging is switched off leaves no way to merge.

Required reviewers is the newest sub-option, generally available since 17 February 2026: up to 15 teams, each with file patterns and a required number of approvals from 0 to 10, where zero means "the team will be added for visibility" ([changelog](#)). It "is not available on user-owned repositories as they do not contain teams". Chapter 19, section 19.10 compares it with CODEOWNERS.

Unverified. The documentation page describes the file patterns of required reviewers as "the same as a standard `.gitignore` file", with `!` negation. The REST schema says "File patterns use fnmatch syntax" and still labels the parameter beta. The two descriptions do not agree; test a pattern before relying on it.

18.8 Required status checks and their traps

In one sentence. The rule holds a list of check *names*; a merge is allowed when the newest commit has a passing result under every name, from whoever reported it.

Strict or loose. "Strict" means the checkbox **Require branches to be up to date before merging** is selected: "The topic branch **must** be up to date with the base branch before merging. This is the default behavior." Loose: fewer builds, and "status checks may fail after you merge your branch if there are incompatible changes with the base branch" ([available rules](#)). "Up to date" is a question about ancestry that you can ask locally:

```
# Rule: require branches to be up to date. Is the tip of the base an ancestor of the head?
$ git merge-base --is-ancestor origin/main HEAD
[exit status: 1]
# Commits only main has (left), commits only the branch has (right):
$ git rev-list --left-right --count origin/main...HEAD
1      3
```

Exit status 1: the tip of `main` is not an ancestor of the branch. One commit on `main` is missing from it.

The traps. All are documented in [Troubleshooting required status checks](#), [Troubleshooting rules](#) and [Skipping workflow runs](#).

Trap	What the documentation says	Consequence
A skipped workflow	"If a workflow is skipped due to path filtering, branch filtering or a commit message, then checks associated with that workflow will remain in a 'Pending' state."	the pull request waits forever ("Waiting for status to be reported")
A skipped job	"A job that is skipped will report its status as 'Success'." A job skipped because a job it needs failed "may not block merging"	a required check can be green without having run
Path filters	the documented example: a workflow with <code>paths: 'scripts/**'</code> and a required <code>build</code> job; a pull request that only touches the root is blocked	"Avoid requiring workflows that can be skipped."
<code>[skip ci]</code> in the head commit	the workflow does not run for <code>push</code> and <code>pull_request</code>	required checks stay pending; push a commit without the instruction
The seven-day rule	"A required status check must have completed successfully in the chosen repository during the past seven days"	a check name cannot be picked in the settings until it has run recently
The newest commit	"Required checks must pass on the latest commit SHA. Checks from earlier commits don't satisfy the requirement."	every push and every "Update branch" starts over
Same name twice	"If a check and a commit status have the same name, both must pass"	name collisions between tools block merges
The wrong event	checks from workflow jobs count only for runs triggered by <code>push</code> , <code>pull_request</code> , <code>pull_request_review</code> , <code>pull_request_target</code> , <code>deployment</code> , <code>deployment_status</code>	a green <code>workflow_dispatch</code> run on the branch satisfies nothing
Merge queue	queue groups trigger <code>merge_group</code> , a separate event	without that trigger the check is never reported
The wrong source	a check can be pinned to an expected GitHub App: "Required status check "build" was not set by the expected GitHub App."	protects against the next row
Anyone can report	"Any person or integration with write permissions to a repository can set the state of any status check"	an unpinned required check can be satisfied by an API call (Lab 23.3 does it)

Check names. The name to require depends on what produced the check: for a workflow job it "is `<job name>`"; for a job in a reusable workflow, "`<job name> / <reusable job name>`"; and "required status checks do not take workflow, matrix, or event trigger types into account". The classic-protection page adds: "make sure that job names are unique across all workflows", because the same job name in two workflows gives "ambiguous status check results".

Unverified. The exact check name that a matrix job reports is not stated on these pages (flag carried from the Phase 0 report). Do not guess it: read the names from a real run with `gh pr checks`, or from the merge box, and require what you read.

The robust pattern, an inference from the rows above and not a documented recipe: let the workflow start on every pull request, decide inside the workflow which jobs do real work, and require one final job that depends on the others, runs always, and fails if any of them failed or was cancelled. One stable name, never skipped at workflow level. Chapter 20B: Delivery, runners, cost and debugging builds it.

18.9 Conversation resolution

In one sentence. With this option, every review thread on the pull request must be marked resolved before the merge.

In a ruleset it is part of the pull request rule: "you can require all comments on the pull request to be resolved before it can be merged". In classic protection it is a separate setting, "Require conversation resolution before merging", which is why it is the one setting the conversion tool does not map one to one (18.14).

The control is weaker than it sounds. Resolving a thread is a click, and the author may do it: "You can resolve a conversation in a pull request if you opened the pull request or if you have write access to the repository" ([commenting on a pull request](#)). The rule ensures that every thread was *acknowledged*, not that it was *addressed*. Pair it with a team convention: the reviewer who opened a thread resolves it.

18.10 Signed commits and the merge methods

In one sentence. The rule accepts only commits with a verified signature on the target, and it is evaluated against the commits a pull request introduces, not only against the commit the merge creates.

Precisely. "Contributors and bots can only push commits that have been signed and verified." Then the part that surprises teams: "When GitHub evaluates whether a pull request can be merged, it creates a test merge commit ... GitHub checks the commits introduced by this test merge, including commits from the head branch. As a result, unsigned commits on the head branch can block a squash merge, even though GitHub would sign the final squash commit" ([signed commits rule](#)).

Merge method	What lands	Under "Require signed commits"
Merge commit	your commits and a merge commit signed by GitHub	works if every commit on the head branch is signed and verified
Squash	one commit signed by GitHub	the result would pass, but unsigned commits on the head branch block the merge anyway
Rebase	new copies of your commits, "without commit signature verification"	cannot satisfy the rule; documented workaround: "rebase and merge locally, and then push"

The local check for the first two rows:

```
# Rule: signed commits. %G? prints N for a commit without a signature:
$ git log --format="%h %G? %s" origin/main..HEAD
16d4788 N Fix the name of the escalations queue
12ae95d N Route high-priority tickets to escalation
44c1e7b N Add priority scoring
```

N means no signature. Chapter 14B, sections 14B.15 to 14B.17 set up signing; G in this column is a good signature *by your local trust settings*, which is not the same as "Verified" on GitHub, where the public key must be registered with the account. The documented repair for an unsigned commit already pushed: "rebase the commit to include a verified signature, then force push". With vigilant mode enabled, commits that GitHub marks "Partially verified" are permitted.

One more documented difference: when a branch is *created*, rulesets "check only the commits that aren't accessible from other branches", whereas classic protection does "not verify signed commits unless you restrict pushes that create matching branches".

When not to use it. The rule proves that someone holding a registered key made each commit. It does not prove the commit is good, and it costs every contributor and every bot a key. Server-side rebases (the rebase button, "Rebase stack") become unusable. Decide whether the audit value on your branch is worth that. Chapter 21B treats signing as one control among several.

18.11 Linear history, force pushes and deletions

Linear history. Section 18.2 showed the predicate. Two details. First, the rule constrains what lands on the target, not the shape of the pull request branch: a branch that contains "Merge main into ..." commits is fine if it is squash-merged, because one ordinary commit lands. Second, GitHub requires the repository to "allow squash merging or rebase merging" before the rule can be enabled. The local measure:

```
# Fix for "not up to date", variant 1: merge the base into the branch.
$ git merge -q origin/main
$ git merge-base --is-ancestor origin/main HEAD
[exit status: 0]
$ git rev-list --left-right --count origin/main...HEAD
0      4
$ git rev-list --count --merges origin/main..HEAD
1
```

After merging `main` in, the branch is up to date and carries one merge commit. Under the linear rule this pull request can still be squash-merged. Under the merge-commit method the rule would refuse it.

Block force pushes. GitHub's reasons, in one paragraph of the rules page: commits "that other collaborators have based their work on may be removed", which "may lead to merge conflicts or corrupted pull requests", and force pushing "can also be used to delete branches or point a branch to commits that were not approved in a pull request". The last clause is the security argument: without this rule, a required pull request can be undone after the fact. And: "Enabling force pushes will not override any other rules."

Restrict deletions is what keeps `main` and release branches from being deleted by a mistyped `git push origin --delete`. Deleting a branch destroys no commits by itself, but it removes the name everything else depends on, and every open pull request against it.

18.12 Tag rulesets and push rulesets

Tag rulesets. A release tag that moves is a supply-chain problem (Chapter 14B, section 14B.11). The hook imitation shows the two rules that prevent it:

```
$ git tag -a v1.0.0 -m "ticket-router 1.0.0" main~1
$ git push origin v1.0.0
To ../../server/ticket-router.git
 * [new tag]          v1.0.0 -> v1.0.0
# Moving a published tag, the supply-chain mistake of Chapter 14B:
$ git tag -f -a v1.0.0 -m "ticket-router 1.0.0" main
Updated tag 'v1.0.0' (was 13ad80e)
$ git push --force origin v1.0.0
remote: rule 'restrict updates': refs/tags/v1.0.0 already exists and may not move
To ../../server/ticket-router.git
 ! [remote rejected] v1.0.0 -> v1.0.0 (pre-receive hook declined)
error: failed to push some refs to ' ../../server/ticket-router.git'
[exit status: 1]
$ git push origin --delete v1.0.0
remote: rule 'restrict deletions': refs/tags/v1.0.0 may not be deleted
To ../../server/ticket-router.git
 ! [remote rejected] v1.0.0 (pre-receive hook declined)
error: failed to push some refs to ' ../../server/ticket-router.git'
[exit status: 1]
```

On GitHub that is a tag ruleset targeting `v*` with restrict updates, restrict deletions and block force pushes. Creation stays open, so a release can still be tagged; add restrict creations with a bypass entry for the release automation if only it may tag.

Outdated advice. "Tag protection rules" under the repository settings were retired on 30 August 2024 and migrated to tag rulesets ([sunset notice](#)). A second, different mechanism is the immutable release, which locks the tag of a published release (Chapter 15: GitHub).

Push rulesets. "With push rulesets, you can block pushes to a private or internal repository and that repository's entire fork network based on file extensions, file path lengths, file and folder paths, and file sizes. Push rules do not require any branch targeting because they apply to every push to the repository" ([about rulesets](#)).

Push rule	Documented detail
Restrict file paths	<code>fnmatch</code> patterns; at most 200 entries of up to 200 characters; "allowed exceptions" in preview since 25 August 2026
Restrict file path length	a character limit
Restrict file extensions	at most 200 entries
Restrict file size	a limit in megabytes; it "does not apply to Git Large File Storage"

Also documented: at most 1,000 ref updates per push, and the rules apply to the REST endpoints that create blobs, trees and file contents as well. For an ML team the obvious uses are "no `*.ckpt`, `*.safetensors` or `*.parquet` in Git" and a size ceiling well below GitHub's hard 100 MiB, so that large files go to Git LFS or to object storage (Chapter 22: Git LFS).

Plan gate. Push rulesets need the Team plan and a private or internal repository. You cannot practise them in a public repository on a Free organization.

18.13 Organization-level and enterprise-level rulesets

In one sentence. An organization ruleset is one ruleset that targets many repositories, selected by name pattern, by custom property, or by a filter.

Precisely. Organization rulesets are available on Team and Enterprise plans since 16 June 2025 ([changelog](#)). Repositories are targeted as "All repositories", "Only selected repositories", "Repositories matching a name", or, since 24 June 2025 and by default, "Repositories matching a filter" such as `visibility:private` `props.team:infra` ([changelog](#)). Only organization owners can edit them. Repository administrators can add stricter repository rulesets and cannot weaken the organization's (18.4). Enterprise-level rulesets, generally available since 24 March 2025, add one more layer on Enterprise Cloud.

Unverified. One sentence of About rulesets still says organization rulesets need the Enterprise plan. The June 2025 changelog and the organization article say Team. The Phase 0 report records this as a conflict inside GitHub's documentation and prefers the later sources.

The control chain at scale (an inference from the documented pieces, not one documented procedure): define repository custom properties such as `tier`; target organization rulesets by property; roll out in Evaluate mode where the plan has it; read Rule Insights; switch to Active with a narrow bypass list; watch the audit log. Chapter 15 covers custom properties and the audit log.

What a Free organization gets. No organization-level rulesets. Repository rulesets in each public repository, which is what the labs use.

18.14 Classic branch protection, and where it still differs

In one sentence. A classic branch protection rule is the older mechanism: one rule per branch-name pattern, visible to administrators, with its own bypass behavior, still supported and still evaluated alongside rulesets.

Version note. Older behavior: classic rules were the only way to protect a branch. Current behavior: rulesets are the recommended mechanism, and a **Convert to ruleset** control migrates one classic rule at a time ([changelog](#)). Since: 7 July 2026 for the recommendation, 11 August 2026 for the conversion tool; no end date for classic rules has been announced. Recommended: new protection as rulesets; when you inherit a repository, look for classic rules first.

Where the two differ ([about protected branches](#), [about rulesets](#)):

	Classic branch protection rule	Ruleset
How many apply to one branch	"Only a single branch protection rule can apply at a time"	all that target it, aggregated
Who can see it	people with admin access	"Anyone with read access to a repository can view its active rulesets"
Switching off	delete the rule	change the enforcement status
Administrators	not bound by default: restrictions "don't apply to people with admin permissions" unless Do not allow bypassing the above settings is selected	bound, unless on the bypass list
Scope	branches of one repository	branches, tags, pushes; repository, organization, enterprise
Conversation resolution	a setting of its own	inside the pull request rule
Restrict who can push	a setting, with a list of people, teams and apps	"Restrict updates" plus a bypass list
Lock branch (read-only), with "Allow fork syncing"	a setting	no rule of that name
Signed commits when a branch is created	not verified unless matching-branch creation is restricted	commits not reachable from other branches are checked
Rejection message seen by Git	remote: error: GH006: Protected branch update failed for refs/heads/main.	not documented on the pages read

The fourth row is the classic answer to "how did a force push get through": the pusher was an administrator, and the bypass setting was left at its default. Under classic rules the audit log records an administrator's override; the Phase 0 report names the event `protected_branch.policy_override`.

Conversion. The tool converts one classic rule at a time and "generates one or more rulesets that preserve the original rule's behavior". The new ruleset is Active at once. If you keep the classic rule, "an **Active** ruleset is enforced alongside it, so changes must satisfy both". The conversion "covers all branch protection types with the exception of the 'Require conversation resolution before merging' setting" ([converting branch protections](#)).

Unverified. How the classic "Lock branch" setting maps onto ruleset rules is not spelled out in the documentation (flag carried from the Phase 0 report). The report infers "Restrict updates" plus bypass settings.

18.15 Plan gates: what you can practise

Plan gates are the facts most likely to have changed by the time you read this. Each row is from a "Who can use this feature?" box or a changelog entry, read on 1 October 2026.

Feature	Where it is available
Repository rulesets; classic branch protection	public repositories on GitHub Free and Free for organizations; public and private on Pro, Team, Enterprise Cloud
Organization-level rulesets	Team and Enterprise (conflict noted in 18.13)
Enterprise-level rulesets; Evaluate status; metadata restrictions; ruleset history (180 days)	Enterprise Cloud documentation only
Push rulesets	Team plan, private or internal repositories
Merge queue	public repositories owned by an organization; private ones on Enterprise Cloud
Required reviewers by team	organization-owned repositories (teams are required)
Rule Insights dashboard	Team and Enterprise Cloud
Auto-merge	public repositories on Free; public and private on paid plans

For you this means: **a public repository in a free practice organization**. There you can create branch and tag rulesets, bypass lists, required reviews, required checks and code owner review. A private repository on a free plan accepts none of it, which is why the labs are public. Evaluate mode, push rulesets and organization rulesets are taught from the documentation.

18.16 Seeing and managing rules: `/rules`, `gh ruleset`, `gh api`

Seeing. "Anyone with read access to the repository can view the active rulesets." Three documented places: the Rulesets page reached from the branch list, the merge box "if there are rules blocking the merging of a pull request", and "by adding the `/rules` slug to the repository's URL", for example <https://github.com/github/docs/rules> ([managing rulesets](#)). Classic rules are not on that page; they are under the repository settings, **Branches**, for administrators.

`gh ruleset` is **read-only**. Its help says so: "These commands allow you to view information about them." Three subcommands exist in 2.88.1:

```
gh ruleset list           # rulesets of the current repository, including inherited ones
gh ruleset list --org ORG # organization rulesets (needs the admin:org scope)
gh ruleset view 123456    # one ruleset by ID; --web opens it in the browser
gh ruleset check main     # every rule that applies to a branch name
gh ruleset check release/2.0 # the branch "does not need to exist"
gh ruleset check --default
```

`gh ruleset check` is the answer to the silent-pattern problem of section 18.3: ask which rules would apply to a branch name before you trust a pattern.

Creating and changing go through the REST API ([REST: rules](#)):

```
gh api repos/ORG/REPO/rulesets           # list
gh api --method POST repos/ORG/REPO/rulesets --input ruleset.json # create
gh api repos/ORG/REPO/rulesets/123456    # read one, as JSON
gh api --method PUT repos/ORG/REPO/rulesets/123456 --input ruleset.json
gh api --method PUT repos/ORG/REPO/rulesets/123456 -f enforcement=disabled
gh api --method DELETE repos/ORG/REPO/rulesets/123456
gh api repos/ORG/REPO/rules/branches/main # active rules for one branch
```

```
gh api repos/ORG/REPO/rulesets/rule-suites          # evaluations: passed, failed, bypassed
gh api repos/ORG/REPO/branches/main/protection      # the classic rule, if any
```

`gh api --help` itself shows a ruleset body passed with `--input`. The endpoints are the documented ones; none of these commands was executed here. "Get rules for a branch" returns "all active rules that apply to the specified branch", from every level, and leaves out rulesets that are disabled or in evaluate mode. To prevent leaking information, `bypass_actors` is returned only to callers with write access to the ruleset.

Rulesets can be exported and imported as JSON in the web interface (**New ruleset**, then **Import a ruleset**), and GitHub maintains ready-made ones in [github/ruleset-recipes](#). Keeping the JSON in a repository gives you review and history for the rules themselves. The bypass list is excluded from the Enterprise history export, so record it separately.

Rule Insights lists ref updates that passed, failed or bypassed rulesets, and the `rule-suites` endpoint returns the same data. It is the evidence for "would we know if someone skipped the rules", with the exception of exempt actors (18.5).

18.17 "Why can't I merge?": a procedure

In one sentence. Work from the outside in: what the merge box says, which rules apply to the base branch from every layer, then one local Git question per rule.

1. **Read the merge box and the CLI's view.** `gh pr view N --json`
`mergeable`, `mergeStateStatus`, `reviewDecision` and `gh pr checks N --required`. The values come from the GraphQL API: `mergeable` is `MERGEABLE`, `CONFLICTING` or `UNKNOWN` (not computed yet: ask again); `mergeStateStatus` is one of `BEHIND` ("the head ref is out of date"), `BLOCKED`, `DIRTY` ("the merge commit cannot be cleanly created"), `DRAFT`, `UNSTABLE` ("mergeable with non-passing commit status"), `CLEAN`, `HAS_HOOKS`, `UNKNOWN` ([GraphQL reference](#)). `BLOCKED` says a rule is unmet and not which one; the remaining steps find it.
2. **List every rule on the base branch.** `gh ruleset check <base>` or the `/rules` page, then the classic rule under Settings, Branches (administrators) or `gh api repos/ORG/REPO/branches/<base>/protection`. Remember the organization level.
3. **Conflicts.** `git fetch`, then `git merge-tree --write-tree --name-only origin/<base> HEAD`. Exit status 1 names the files (Chapter 17, section 17.7).

```
# Mergeability: does the test merge succeed?
$ git merge-tree --write-tree --name-only origin/main HEAD
beff5a60b5d7f4a2b93e41130391fc485f5970e0
[exit status: 0]
```

4. **Up to date** (strict checks): `git merge-base --is-ancestor origin/<base> HEAD`. If not, update the branch, then expect the checks to run again and stale approvals to be dismissed.
5. **Checks.** For each required name: is there a result on the *newest* commit? Pending forever means a skipped workflow, a wrong name, or a wrong event (18.8). `gh pr checks N --required` shows what GitHub is waiting for.
6. **Reviews.** Count approvals from people with write access given after the last change of the diff. Look for an outstanding "changes requested", a missing code owner (Chapter 19), an approval by the last pusher, unresolved threads, a twin pull request on the same commit.
7. **Commits.** Which commits would land, and do they satisfy the commit-level rules?

```
# The commits a rule inspects: those the pull request introduces.
$ git log --format="%h parents=%p" origin/main..HEAD
16d4788 parents=12ae95d
12ae95d parents=44c1e7b
```

```
44c1e7b parents=9a383e5
# Rule: linear history. Merge commits among them:
$ git rev-list --count --merges origin/main..HEAD
0
```

Signatures (%G?, section 18.10), merge commits (linear history), and under Enterprise metadata rules the author and committer addresses.

8. **Method.** Is the method you chose allowed by the ruleset *and* enabled in the repository settings?
9. **You.** Are you allowed to merge at all (write permission), and is the pull request still a draft?

If everything passes and the button is still grey, a rule you cannot see is the likely cause: an organization or enterprise ruleset, or a classic rule visible only to administrators. Ask an administrator for the output of step 2.

The mirror case, "why could they merge?": a bypass actor, an administrator under a classic rule without the bypass restriction, `gh pr merge --admin`, a disabled ruleset, a pattern that does not match the branch, or an indirect merge (Chapter 17, section 17.13).

18.18 A worked design: protecting a production branch

Context. `inventory-api` deploys from `main` on every merge. Six engineers, pull requests of a few hundred lines, CI of about eight minutes, an ML platform team that owns the deployment workflow. The team has decided on squash merges. This is one defensible design for that context, with what each choice costs. Another team should make other choices.

The ruleset, as a file for the REST API. It is assembled from the documented schema and syntax-checked; it has not been sent to GitHub by the author.

```
{
  "name": "main: production branch",
  "target": "branch",
  "enforcement": "active",
  "conditions": { "ref_name": { "include": ["~DEFAULT_BRANCH"], "exclude": [] } },
  "bypass_actors": [],
  "rules": [
    { "type": "deletion" },
    { "type": "non_fast_forward" },
    { "type": "required_linear_history" },
    { "type": "pull_request",
      "parameters": {
        "required_approving_review_count": 1,
        "dismiss_stale_reviews_on_push": true,
        "require_code_owner_review": true,
        "require_last_push_approval": true,
        "required_review_thread_resolution": true,
        "allowed_merge_methods": ["squash"] } },
    { "type": "required_status_checks",
      "parameters": {
        "strict_required_status_checks_policy": true,
        "required_status_checks": [ { "context": "ci" } ] } }
  ]
}
```

The file is in the course as `labs/ch18/rulesets/main-production.json`.

Choice	Why here	What it costs	When to choose otherwise
Target <code>~DEFAULT_BRANCH</code>	survives a rename; no pattern to get wrong	protects one branch only	add a second ruleset for <code>release/**/*</code>

Choice	Why here	What it costs	When to choose otherwise
Deletions and force pushes blocked	<code>main</code> is deployed; its history is an audit record	a bad merge is undone by a revert, never by a reset	never, on a deployed branch
Linear history and squash only	one commit per pull request; trivial reverts	reviewed commit IDs are not the commit on <code>main</code> ; per-commit history is lost (Chapter 17, section 17.9)	merge commits if the reviewed IDs must be the deployed ones
One approval, stale approvals dismissed	nothing lands unseen; six people cannot afford two reviewers per change	re-approval after each update	two approvals for regulated code
Approval of the last push	covers a reviewer who pushes a fix and approves it	occasionally a second reviewer is needed	drop it for a team of two
Code owner review	the deployment workflow and CODEOWNERS have named owners (Chapter 19)	owners become a bottleneck if the file is too broad	no CODEOWNERS file yet
Conversations resolved	no merge over an open question	weak (18.9)	high-volume repositories where threads are used for chatter
One required check, strict	the tested commit includes current <code>main</code> ; one stable name (18.8)	with eight-minute CI and a few merges a day, some waiting	a merge queue when updates start to race
Empty bypass list	administrators follow the same path	an emergency needs a deliberate edit of the ruleset, which is recorded	add the repository admin role in <code>pull_request</code> mode if an on-call engineer must be able to merge at night
No signed-commit rule	squash commits are signed by GitHub anyway; requiring signatures on head branches would block contributors without keys (18.10)	commit authorship on branches is unverified	add it where provenance of every commit is a requirement

Two things the ruleset does not do, and what covers them. It does not protect release tags: that is a tag ruleset (18.12). It does not stop a required check from being reported by anyone with write access: pin the check to its GitHub App with `integration_id` once CI runs as Actions (18.8).

Rolling it out. Create it Disabled, run `gh ruleset check main` and read the result, open a test pull request, then switch to Active. On Enterprise Cloud, Evaluate mode does this properly. Lab 23.1 walks through it on your practice repository.

18.19 What can go wrong

Symptom	Diagnosis	Fix	Prevention
A force push or deletion reached a "protected" branch	Rule Insights and the audit log; was the ruleset Active, did the pattern match, was the actor a bypass or exempt actor, or an administrator under a classic rule	restore the ref (Chapter 13, Chapter 30: Incident response); close the gap	empty or <code>pull_request</code> -mode bypass lists; <code>gh ruleset check <branch></code> after every change
A required check is "Waiting for status to be reported" forever	the workflow was skipped by a path or branch filter or <code>[skip ci]</code> ; or the name is wrong; or the run used a non-counting event	run the workflow always and gate inside it; require the real name	one aggregate job as the required check
Merge blocked although approved	the approval was dismissed as stale, or given by the last pusher, or a code owner	re-approve; see section 18.17, step 6	explain the review settings in <code>CONTRIBUTING.md</code>

Symptom	Diagnosis	Fix	Prevention
	is missing, or another reviewer requested changes		
"Relaxed the ruleset, still blocked"	a second ruleset, an organization ruleset, or a classic rule also applies	list every layer (18.16)	one place for each rule; convert classic rules
Squash merge blocked under "signed commits"	unsigned commits on the head branch	rebase with signing and force-push, or a bypass actor merges	tell contributors before enabling the rule
No merge method works	the ruleset allows only a method that the repository has disabled	enable the method in the repository settings	change both in the same pull request to your infrastructure code
Default branch cannot be renamed	"Block force pushes" on it, and you are not a bypass actor	add a temporary bypass	plan renames
A new branch pattern is unprotected	* does not cross / (18.3)	**/* forms; re-check with <code>gh ruleset check</code>	test patterns before relying on them
Push rejected with GH006	a classic rule	follow it: open a pull request	know which branches are protected

18.20 When not to use it, and dangerous edge cases

- **Rules are not review.** A ruleset proves that a process ran. It does not prove that anyone understood the change.
- **A solo or two-person repository** can lock itself out: required approvals with nobody else to approve, and an empty bypass list. Start with "pull request required, zero approvals", which still gives you checks and a record.
- **Exempt actors leave no trace.** Use the mode only for automation whose every action is logged elsewhere.
- **Required checks without a pinned source** can be satisfied by anyone with write access.
- **Strict checks on a busy branch** create a race to merge. That is the signal for a merge queue, not for switching to loose checks without thought.
- **A disabled ruleset is invisible on the /rules page**, which shows active ones. An incident review must look at the settings page and at the ruleset history where the plan has it.
- **Rules guard refs, not objects.** A rejected push moved no ref. It is not a secret-removal tool and not a data-loss-prevention system; for metadata rules GitHub documents that rejected commits still enter the repository as retrievable, unreachable objects.
- **Preview rules** (coverage thresholds, secret-scanning resolution, the Copilot approval setting) can change without notice.

18.21 Command safety

Command	Label	What it changes	Preview	Recovery
<code>gh ruleset list, view, check; gh api repos/ORG/REPO/rulesets (GET); the /rules page</code>	🟢 SAFE	nothing	not needed	not needed
<code>git merge-base --is-ancestor, git rev-list --merges, git log --format='%G?', git merge-tree --write-tree</code>	🟢 SAFE	nothing	not needed	not needed

Command	Label	What it changes	Preview	Recovery
<code>gh api --method POST repos/ORG/REPO/rulesets --input file.json</code>	 DANGEROUS	creates a ruleset; if Active, it binds everyone at once, you included. It destroys nothing; the label is the one Chapter 15, section 15.22 gives every <code>gh api</code> call that is not a <code>GET</code> , because the call does whatever the endpoint and the JSON say, with all your permissions. Appropriate for rules kept as reviewed JSON	create it with <code>"enforcement": "disabled"</code> , then <code>gh ruleset check</code>	set <code>enforcement</code> to <code>disabled</code> , or delete the ruleset
<code>gh api --method PUT .../rulesets/ID</code>	 DANGEROUS	replaces settings of a ruleset. What it can destroy: the previous settings, which exist nowhere else unless you saved the JSON (ruleset history exists only on Enterprise). Appropriate for a reviewed change to a rule	<code>gh api .../rulesets/ID</code> first and keep the JSON	PUT the saved JSON back
<code>gh api --method DELETE .../rulesets/ID</code>	 DANGEROUS	removes the protection for every ref it targeted, immediately. What it can destroy: nothing directly; it makes force pushes and deletions possible	save the JSON first	re-create from the saved JSON; ruleset history exists only on Enterprise
<code>gh pr merge --admin, a bypass merge</code>	 DANGEROUS	writes to a protected branch without the required reviews or checks	<code>gh pr checks --required</code>	<code>gh pr revert</code> ; record why
<code>git push --force</code> to a branch whose protection was just removed	 DANGEROUS	removes commits from a shared branch	<code>git log origin/<branch>..<branch></code> and the reverse	Chapter 13, Chapter 30

For the two deletion and bypass rows: they are appropriate in a planned migration or a declared emergency, by a named person, with the reason written down where the audit will find it.

18.22 Version notes

Topic	Older behavior	Current behavior	Since	Recommended
Recommended mechanism	classic branch protection	rulesets; conversion tool	7 Jul and 11 Aug 2026	rulesets; check both layers
Tag protection rules	a repository setting	retired; tag rulesets	30 Aug 2024	tag rulesets
Organization rulesets	Enterprise only	Team and Enterprise	16 Jun 2025	re-verify plan gates

Topic	Older behavior	Current behavior	Since	Recommended
Merge method per branch	repository-wide setting only	<code>allowed_merge_methods</code> in the pull request rule (changelog)	24 Mar 2025	enforce the method where it matters
Bypass	role, team or app; always or pull requests only	plus the silent <code>exempt</code> mode; plus individual users	10 Sep 2025; 7 May 2026	prefer <code>pull_request</code> mode
Reviewers by path	CODEOWNERS only	required reviewers rule, per-team counts	17 Feb 2026	Chapter 19
Who may dismiss reviews	classic rules only	also in rulesets	7 Jul 2026	restrict on audited branches
Import, export, history	none	JSON import and export; history on Enterprise (changelog)	13 Feb 2025	keep ruleset JSON under version control
Push rule exceptions	none	allowed exceptions for paths and sizes (preview)	25 Aug 2026	preview: do not depend on it

18.23 Practice

- Module 23 labs: Lab 23.1, a ruleset on the default branch (with a local rehearsal on a bare server); Lab 23.3, three blocked merges to diagnose. Lab 23.2 belongs to Chapter 19.
- Replay the transcripts: `labs/run ch18/ref-updates`, `labs/run ch18/fnmatch-targets`, `labs/run ch18/merge-preflight`.
- Read the rules of a public repository you use, by adding `/rules` to its address. Explain each rule to yourself in terms of section 18.2.

18.24 Interview questions

1. What is a rule, mechanically? Which rules can be decided from the old ID, the new ID and the ref name alone, and which need platform data?
2. Two rulesets and a classic rule target `main` with different numbers of required approvals. How many approvals are required, and why?
3. `main` was force-pushed although a ruleset blocks force pushes. List the ways that can happen and the evidence for each.
4. What are the three bypass modes, and what trace does each leave?
5. A required check stays pending forever on documentation-only pull requests. Explain the cause and design a fix that keeps the path optimization.
6. Why can a squash merge be blocked by "Require signed commits" although GitHub signs the squash commit?
7. Compare "dismiss stale approvals" with "approval of the most recent reviewable push". Which attack does each stop?
8. Your ruleset targets `release/*`. Is `release/2.0/hotfix` protected? How do you check without creating the branch?
9. What can you not practise on a Free plan, and how would you learn it anyway?
10. Classic rule versus ruleset: name four behavioral differences that matter during an incident.
11. Walk through your procedure when a developer says "everything is green and I still cannot merge".
12. Design the protection of a branch that deploys to production for a team of six. Defend each rule and name its cost.

18.25 Sources

Primary sources (docs.github.com and the GitHub Changelog; read on 1 and 2 October 2026)

- Rulesets: [About rulesets](#) and its [Enterprise Cloud view](#), [Available rules for rulesets](#), [Creating rulesets for a repository](#), [Managing rulesets for a repository](#), [Troubleshooting rules](#), [Converting branch protections to rulesets](#), [Creating rulesets for repositories in your organization](#)
- Classic protection: [About protected branches](#)
- Checks: [Troubleshooting required status checks](#), [Status checks](#), [Skipping workflow runs](#)
- API and CLI: [REST API endpoints for rules](#), [rule suites](#), [branch protection](#); `gh ruleset --help` and `gh api --help` (2.88.1, local); [gh ruleset manual](#)
- Roles: [Repository roles for an organization](#)
- Changelog entries are linked where they are used.
- Ruby's `File.fnmatch`, the function GitHub's documentation links to; Git 2.55: `git help hooks` (`pre-receive`), `git help config` (`receive.denyNonFastForwards`).

Secondary sources

- The Phase 0 report of this course, sections 2, 3, 4 and 13, and its notes on the GitHub platform.
- [github/ruleset-recipes](#) : GitHub's importable example rulesets.

Videos (from the Phase 0 report, with its caveats)

- "[Introduction to GitHub Actions - Part 6 - Repository Rulesets](#)", Mickey Gousset, 6 December 2024, 15 minutes: a ruleset that requires a status check, the bypass list, rulesets versus classic rules. Current; slow and careful.
- "[Securely building GitHub on GitHub](#)", GitHub Universe 2024, 41 minutes: rulesets, Rule Insights, Evaluate mode and bypass lists at scale. Predates the 2025 and 2026 ruleset additions.

Further reading

- Chapter 12: Remote Operations, section 12.7 (server-side rules in plain Git); Chapter 14B, sections 14B.11 and 14B.15 to 14B.17; Chapter 17: Pull Requests, sections 17.5, 17.8 and 17.11.

Chapter 19: CODEOWNERS

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026, with the quoted documentation pages re-read on 2 October 2026. Transcripts are real output from `labs/ch18/` (the CODEOWNERS demos live there). Git cannot evaluate a CODEOWNERS file. Every statement about how GitHub matches patterns or requests reviews comes from its documentation and carries the link; where a transcript uses Git's ignore matcher, the text says exactly how the two syntaxes differ.

19.1 Why this matters

"We have a CODEOWNERS file. How did a change to the deployment workflow merge without the platform team seeing it?"

There are six ordinary answers: no rule required the review; a later line in the file took the path away from the team; the team has no explicit write access, so its line was skipped; the pull request was a draft; the file that counted was the one on the base branch; or the person who merged could bypass the rule. Each is documented behavior. None is a bug.

19.2 What CODEOWNERS is

In one sentence. CODEOWNERS is a text file in the repository that maps path patterns to users and teams, which GitHub uses to request reviews automatically and, if a rule says so, to require them.

Analogy. A building directory in the lobby tells a visitor whom to call. It locks no door. The lock is a separate device (a ruleset) that can be told to consult the directory. The analogy breaks because this directory is read top to bottom and the *last* line that fits wins.

Precisely. "Code owners are automatically requested for review when someone opens a pull request that modifies code that they own. Code owners are not automatically requested to review draft pull requests" ([about code owners](#)). Three layers again:

Item	Layer
The file, its content, its history	Git: an ordinary tracked file
Reading the file, matching changed paths, requesting reviewers	GitHub
Blocking the merge until an owner approves	GitHub, and only through a ruleset or a classic rule (Chapter 18, section 18.7)

Availability: code owners can be defined "in public repositories with GitHub Free and GitHub Free for organizations, and in public and private repositories" on paid plans (same page).

19.3 Where the file lives, and which one counts

In one sentence. GitHub looks in `.github/`, then the repository root, then `docs/`, uses the first file it finds, and reads it from the base branch of the pull request.

Precisely. "If `CODEOWNERS` files exist in more than one of those locations, GitHub will search for them in that order and use the first one it finds." And: "Each `CODEOWNERS` file assigns the code owners for a single branch", so `main` and `release/1.0` can have different owners. The file "must be under 3 MB in size"; a larger one "will not be loaded", which means no review requests at all.

See it. The search order and the size are plain Git questions about the base branch:

```
# The documented search order, applied to the base branch of the pull request:
$ for p in .github/CODEOWNERS CODEOWNERS docs/CODEOWNERS; do git cat-file -e origin/main:$p
2>/dev/null && echo "exists on main: $p"; done
exists on main: .github/CODEOWNERS
exists on main: docs/CODEOWNERS
# The first one found is used. Its size in bytes (the documented limit is 3 MB):
$ git cat-file -s origin/main:.github/CODEOWNERS
531
```

Two files exist on `main`. The one in `.github/` is used; the older `docs/CODEOWNERS` is dead text that will mislead whoever finds it. Delete it.

19.4 Syntax

In one sentence. Each line is a pattern followed by one or more owners; comments start with `#`.

The file of the sample repository, as it is on `main`:

```
$ git show origin/main:.github/CODEOWNERS
# Default owners for everything in the repository.
*                               @example-org/platform

# The routing code.
/router/                        @example-org/routing
/router/priority.py             @example-org/routing @example-org/on-call

# Configuration, wherever it lives.
*.yaml                          @example-org/platform @example-org/sre

# Documentation: files directly in docs/.
docs/*                           @example-org/docs

# The files that decide who must review, and what automation runs.
.github/                         @example-org/repo-admins
```

The documented rules ([CODEOWNERS syntax](#)):

- Owners are written `@username` or `@org/team-name`. An email address added to a user's account also works "in most cases", but not for managed user accounts.
- Several owners for one pattern go on **one line**. "If the code owners are not on the same line, the pattern matches only the last mentioned code owner."
- A pattern with **no** owner after it removes ownership: in the documented example `/apps/ @octocat` followed by `/apps/github`, changes under `apps/github` "can be made with the approval of any user who has write access".
- Inline comments are allowed after the owners.
- "CODEOWNERS paths are case sensitive, because GitHub uses a case sensitive file system", even when your Mac is not.
- "If any line in your CODEOWNERS file contains invalid syntax, that line will be skipped." The rest of the file still applies.

Owners need write access. This is the rule that fails silently most often. "The people you choose as code owners must have write permissions for the repository. When the code owner is a team, that team must be visible and it must have write permissions, even if all the individual members of the team already have write permissions directly, through organization membership, or through another team membership." And the consequence: "If you specify a user or team that doesn't exist or has insufficient access, a code owner will not be assigned." A secret team cannot be an owner, and a team whose members can all push through the organization's base permission is not an owner until the team itself is granted write access.

19.5 Pattern matching: last match wins, and how it differs from gitignore

In one sentence. For each changed path GitHub takes the last line of the file whose pattern matches it, and the patterns follow "most of the same rules used in gitignore files", with documented exceptions.

Last match wins. "Order is important; the last matching pattern takes the most precedence." In the sample file, `router/classify.py` matches `*` and `/router/`; the later one decides, so the routing team owns it. The trap is a general line placed *after* a specific one. `router/rules/eu.yaml` matches `*`, `/router/` and `*.yaml`. The last of the three is `*.yaml`, so the platform and SRE teams own it, and the routing team is not asked. Put general patterns first and specific ones last, and remember that "specific" means "later in the file", not "longer".

The pattern forms that GitHub documents with examples:

Pattern	Documented meaning
<code>*</code>	every file: the default owners
<code>*.js</code>	files with that extension, anywhere
<code>/build/logs/</code>	that directory at the repository root, and its subdirectories
<code>docs/*</code>	files directly in <code>docs/</code> , "but not further nested files like <code>docs/build-app/troubleshooting.md</code> "
<code>apps/</code>	"any file in an <code>apps</code> directory anywhere in your repository"
<code>/docs/</code>	the <code>docs</code> directory at the root "and any of its subdirectories"
<code>**/logs</code>	any <code>logs</code> directory at any depth, such as <code>/build/logs</code> and <code>/deeply/nested/logs</code>

What does not work. The documentation warns about three gitignore features: "Escaping a pattern starting with `#` using `\` so it is treated as a pattern and not a comment doesn't work", "Using `!` to negate a pattern doesn't work", and "Using `[ ]` to define a character range doesn't work".

See it, with a caution. Git has a matcher for gitignore syntax, `git check-ignore`, and it is tempting to use it as a CODEOWNERS tester. The following transcripts show Git's matcher, not GitHub's. Where the two agree, the transcript illustrates the shared rule. Where they differ, the difference is the lesson.

Within one file, Git also lets the last matching pattern decide, as long as only files are involved:

```
# Three patterns in one gitignore-format file. check-ignore -v names the pattern that decides:
$ printf '*.py\n/router/*.py\n/router/priority.py\n' > .gitignore
$ git check-ignore -v --no-index tests/test_classify.py router/classify.py router/priority.py
.gitignore:1:*.*py          tests/test_classify.py
.gitignore:2:/router/*.py    router/classify.py
.gitignore:3:/router/priority.py    router/priority.py
```

But gitignore works on directories first. When a pattern matches a directory, Git excludes the whole directory and never looks at the files inside it:

```
# Where the two part ways: a catch-all first line, then a more specific pattern.
$ printf '*\n*.py\n' > .gitignore
```

```
$ git check-ignore -v --no-index setup.py tests/test_classify.py
.gitignore:2:*.py      setup.py
.gitignore:1:*  tests/test_classify.py
# Git stops at the directory tests/, which the first line already matches.
# CODEOWNERS documentation: after "*", a later "*.js" line owns every JS file.
```

For `tests/test_classify.py` Git reports line 1, not line 2. CODEOWNERS is documented to behave differently: in GitHub's own example, `*` followed by `*.js` gives every JavaScript file to the JavaScript owner. The same mechanism explains the documented `docs/*` example:

```
# The same mechanism behind a documented example: docs/* and nested files.
$ printf 'docs/*\n' > .gitignore
$ git check-ignore -v --no-index docs/getting-started.md docs/build-app/troubleshooting.md
.gitignore:1:docs/*    docs/getting-started.md
.gitignore:1:docs/*    docs/build-app/troubleshooting.md
# Git ignores the nested file too, because docs/* matches the directory docs/build-app.
# CODEOWNERS documentation: docs/* does not match docs/build-app/troubleshooting.md.
```

```
Observed behavior : git check-ignore says docs/* matches docs/build-app/troubleshooting.md.
                   GitHub's CODEOWNERS documentation says it does not.
Git state          : none involved; this is pattern matching on path strings.
Mechanism          : gitignore patterns are applied to each directory on the way down. docs/*
                   matches the directory docs/build-app, and everything below an excluded
                   directory is excluded. CODEOWNERS assigns owners to files, one path at a time.
Root cause         : Two matchers with a shared pattern language and different jobs: one prunes
                   directory walks, the other labels files.
Why Git does this  : Skipping an ignored directory without reading it is what makes status fast.
Correct fix        : Do not test CODEOWNERS with check-ignore. Reason from the documented rules,
                   then confirm on GitHub (section 19.11).
Prevention         : To own a whole subtree write the directory form, /docs/ , not docs/* .
```

The three documented exceptions all work in Git, which is why a pattern that "works locally" can do nothing in CODEOWNERS. Negation, for one:

```
# Documented as unsupported in CODEOWNERS: ! negation. In gitignore it works:
$ printf '*.yaml\n!config/routing.yaml\n' > .gitignore
$ git check-ignore -v --no-index config/routing.yaml deploy/values.yaml
.gitignore:2:!config/routing.yaml    config/routing.yaml
.gitignore:1:*.yaml    deploy/values.yaml
```

Case sensitivity is the last difference. Git on a Mac usually ignores case in ignore patterns, because `git init` sets `core.ignoreCase` on a case-insensitive file system. GitHub never does:

```
# CODEOWNERS paths are always case sensitive. Git depends on core.ignoreCase, which git init
# switches on when the file system ignores case, as the default macOS file system does:
$ printf '/docs/\n' > .gitignore
$ git -c core.ignoreCase=false check-ignore -v --no-index Docs/a.md
[exit status: 1]
$ git -c core.ignoreCase=true check-ignore -v --no-index Docs/a.md
.gitignore:1:/docs/    Docs/a.md
[exit status: 0]
```

	gitignore (Git)	CODEOWNERS (GitHub, documented)
Last matching pattern decides	yes, among patterns that reach the file	yes
A matched directory ends the search	yes	no: owners are assigned per file
<code>docs/*</code> covers nested files	yes, through the matched subdirectory	no
<code>!</code> negation	yes	no
<code>[]</code> ranges	yes	no
<code>\#</code> to start a pattern with <code>#</code>	yes	no

	gitignore (Git)	CODEOWNERS (GitHub, documented)
Case	follows <code>core.ignoreCase</code>	always case sensitive

19.6 Why the file only requests reviews until a rule requires them

In one sentence. Without a rule, a code owner is a suggested reviewer whom the author may ignore; with the code-owner option of the pull request rule, the merge is blocked until one owner of every changed path approves.

Precisely. In a ruleset the option is "Require review from code owners" inside "Require a pull request before merging" (REST: `require_code_owner_review`). With it, "any pull request that modifies content with a code owner must be approved by that code owner before the pull request can be merged". Then the detail that weakens many designs: "if code has multiple owners, an approval from *any* of the code owners will be sufficient" ([available rules](#)). Two teams on one line means either team, not both.

Situation	What happens
No rule	owners are requested as reviewers; the pull request can merge without them
Rule with code owner review	every changed path that has an owner needs an approval from one of its owners
A changed path has no owner	the code-owner requirement asks nothing for that path
The pull request is a draft	owners are not requested until it is marked ready
The owner is the author	authors cannot approve their own pull requests (Chapter 17, section 17.4), so another owner of that path must; an inference from two documented rules, which Lab 23.2 lets you observe
The merger is a bypass actor	the rule does not bind them (Chapter 18, section 18.5)

That row is a reason to list a team, not one person, as owner.

19.7 The base branch decides

In one sentence. A pull request is judged by the CODEOWNERS file on its base branch, so a pull request that edits CODEOWNERS is reviewed under the old file.

Precisely. "To trigger review requests, pull requests use the version of `CODEOWNERS` from the base branch of the pull request." For a pull request from a fork into upstream, that is upstream's file. In a stack, CODEOWNERS is "evaluated from the stack base" ([stacked pull requests](#)).

See it. Your branch changes four paths, one of them the CODEOWNERS file, from which it removes the docs line:

```
# The paths this pull request changes (three dots, as on the Files changed tab):
$ git diff --name-only origin/main...HEAD
.github/CODEOWNERS
config/routing.yaml
docs/README.md
docs/runbooks/escalation.md
```

```
# The pull request also edits CODEOWNERS. On its own branch the docs line is gone:
$ git diff origin/main...HEAD -- .github/CODEOWNERS
diff --git a/.github/CODEOWNERS b/.github/CODEOWNERS
index a5e3975..6478d74 100644
--- a/.github/CODEOWNERS
```

```

+++ b/.github/CODEOWNERS
@@ -9,7 +9,6 @@
 *.yaml                @example-org/platform @example-org/sre

# Documentation: files directly in docs/.
-docs/*                @example-org/docs

# The files that decide who must review, and what automation runs.
.github/               @example-org/repo-admins
# Review requests come from the base version, which still has it:
$ git show origin/main:.github/CODEOWNERS | grep -n "^docs"
12:docs/*              @example-org/docs
[exit status: 0]
$ git show HEAD:.github/CODEOWNERS | grep -n "^docs"
[exit status: 1]

```

The base version still has `docs/*`, so the docs team is requested for `docs/README.md` although your branch deleted their line. You cannot remove a reviewer in the pull request that needs them. For the same reason a fix to CODEOWNERS takes effect one merge later than people expect.

19.8 Protecting CODEOWNERS and the workflows directory

In one sentence. The file that names the reviewers, and the directory that defines what automation runs, need owners themselves, or anyone with write access can change both in one pull request.

GitHub's advice: "To protect a repository fully against unauthorized changes, you also need to define an owner for the CODEOWNERS file itself. The most secure method is to define a CODEOWNERS file in the `.github` directory of the repository and define the repository owner as the owner of either the CODEOWNERS file (`/.github/CODEOWNERS @owner_username`) or the whole directory (`/.github/ @owner_username`)."

Owning the whole `.github/` directory, as the last line of the sample file does, also covers `.github/workflows/`. A workflow file decides what runs with the repository's token and secrets, so a change to it is a change to your deployment and security posture (Chapter 21A: GitHub Actions security). Three conditions make the protection real: the line must come last, so that no later pattern such as `*.yaml` takes workflow files away; the code-owner option must be required by a rule on the default branch; and the owning team must hold explicit write access.

19.9 Monorepo patterns

In one sentence. In a monorepo the file is the map of the organization, and its order has to follow the rule "general first, specific last".

```

# 1. Default: somebody always owns a new directory.
*                @example-org/platform

# 2. One block per component, anchored at the root.
/services/router/    @example-org/routing
/services/billing/   @example-org/billing
/libs/eval/          @example-org/ml-eval

# 3. Cross-cutting file types that must override the components.
/services/**/migrations/ @example-org/data
**/Dockerfile        @example-org/platform

# 4. Last: the files that control review and automation.
.github/             @example-org/repo-admins

```

Use the directory form (`/services/router/`) for subtrees; `dir/*` covers one level only. Anchor component paths with a leading slash, or `router/` also matches a directory of that name inside another component. Prefer teams

to people, so that ownership survives holidays and departures. The patterns with `**` follow the gitignore rules and are not among GitHub's documented examples except `**/logs` : confirm them with the instruments of section 19.11.

What the file cannot express: "both teams must approve", "two members of this team", or "everyone except this directory". The first two belong to the rule in the next section. The third has only the documented workaround of an owner-less line.

19.10 The newer "required reviewers" rule

In one sentence. Since February 2026 a ruleset can require approvals from named teams for file patterns, with a count per team, which covers the cases CODEOWNERS cannot express.

Precisely. In the pull request rule, "you can require review or approval from specific teams when a pull request changes certain files or directories. You can specify up to 15 different teams, and for each team you can require a certain number of approvals." Counts run from 0 to 10; zero adds the team "for visibility". The team "must have write permissions (or higher)". The rule "is not available on user-owned repositories as they do not contain teams" ([available rules](#), [required reviewers](#)). It became generally available on 17 February 2026, and GitHub positions it with care: it "augments CODEOWNERS files but doesn't replace them. CODEOWNERS files remain the best way to manage ownership, support individuals as reviewers, and request reviews even when not required" ([changelog](#)).

	CODEOWNERS	Required reviewers rule
Lives in	a file in the repository, reviewed like code	a ruleset, edited by administrators
Owners	users, teams, emails	teams only
Several owners	any one suffices	each listed team has its own required count
Negation	no	documented with <code>!</code>
Applies across repositories	one file per repository and branch	an organization ruleset can cover many
Visible to contributors	in the tree, and by hovering over a file	on the rules page
Takes effect	when on the base branch	when the ruleset is active

Chapter 18, section 18.7 records an unresolved point: the documentation page and the REST schema describe the pattern syntax of this rule differently. Test before you depend on it.

19.11 Finding out what GitHub thinks

In one sentence. Three documented instruments tell you how GitHub read the file: error highlighting on the file's page, the owner shown for each file, and a REST endpoint that lists errors.

- "When you navigate to the CODEOWNERS file in your repository, you can see any errors highlighted."
- When you browse to a file, hovering over the shield icon shows "a tool tip with codeownership details", from the file "for whichever branch in whichever repository you're looking at".
- "A list of errors in a repository's CODEOWNERS file is also accessible via the API": `GET /repos/{owner}/{repo}/codeowners/errors` , with an optional `ref` ([REST: list CODEOWNERS errors](#)).

```
gh api repos/ORG/REPO/codeowners/errors
gh api --method GET repos/ORG/REPO/codeowners/errors -f ref=my-branch
gh pr view 12 --json reviewRequests
```

Each error has a line, a column, a kind, a message and sometimes a suggestion (same page). The CLI has no dedicated command for linting CODEOWNERS; the Phase 0 report looked for one. The commands above use documented endpoints and were not executed here.

The second command is how you test an edit before it merges: ask for the errors of the file as it is on your branch.

19.12 What can go wrong: common mistakes

Mistake	Symptom	Diagnosis	Fix
No rule requires code owner review	owners are requested and ignored	the <code>/rules</code> page shows no such option for the branch	add the option to the pull request rule
A general pattern after a specific one	the wrong team is requested	read the file bottom-up for the path	move general lines to the top
A team without explicit write access, or a secret team	nobody is requested; the errors endpoint reports the line	<code>gh api .../codeowners/errors</code>	grant the team write access; make it visible
A user or team name misspelled	as above	as above	correct the name
Owners of one pattern on two lines	only the later line counts	read the file	one line per pattern
<code>!</code> , <code>[]</code> or <code>\#</code> in a pattern	the line does not do what it does in <code>.gitignore</code>	section 19.5	rewrite with supported forms or an owner-less line
<code>docs/*</code> meant as "everything under docs"	nested files fall to the default owner	section 19.5	<code>/docs/</code>
Wrong case in a path	no match	compare with <code>git ls-files</code>	match the case in the repository
Two CODEOWNERS files	the edited one is not the used one	section 19.3	keep one, in <code>.github/</code>
The fix is on a branch, not on the base	old owners are still requested	section 19.7	merge the fix first
CODEOWNERS itself has no owner	anyone with write access can rewrite it	read the last lines	own <code>/.github/</code>
A sole owner	they cannot approve their own change	section 19.6	own by team

19.13 When not to use it, and dangerous edge cases

- **Do not use it as a notification list.** Every pattern is a review request on every matching pull request. One busy person as owner of `*` becomes the bottleneck of the repository.
- **Do not treat it as access control.** It decides who is asked, and with a rule who must approve. It does not stop anyone with write access from pushing to an unprotected branch.
- **"Any one owner" is not "all owners".** If a change needs two teams, CODEOWNERS on one line does not give you that.
- **Bypass actors and indirect merges** go around it, like every review requirement (Chapter 17, section 17.13; Chapter 18, section 18.5).

19.14 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git show <base>:.github/CODEOWNERS, git diff --name-only <base>...HEAD, git cat-file -e</code>	● SAFE	nothing	not needed	not needed
<code>git check-ignore -v --no-index; gh api repos/ORG/REPO/codeowners/errors</code>	● SAFE	nothing; the first answers a different question (19.5)	not needed	not needed
Editing <code>CODEOWNERS</code> in a pull request	● CAUTION	who is requested, and under a rule who can block, from the next merge on	the errors endpoint with <code>ref</code>	revert the commit
Removing the owner of <code>/.github/</code>	● DANGEROUS	lets any writer change reviewers and workflows without owner review. Destroys no data; removes a control	review the diff of the file	restore the line; audit what merged in between

19.15 Version notes

Topic	Older behavior	Current behavior	Since	Recommended
Reviewers by path	CODEOWNERS only	plus the required reviewers rule with per-team counts	17 Feb 2026	CODEOWNERS for ownership; the rule where counts or several teams are needed
Dependabot reviewers	the <code>reviewers</code> option in <code>dependabot.yml</code>	removed; code owners are used (changelog)	8 Aug 2025	own dependency manifests in CODEOWNERS
Stacks	not applicable	CODEOWNERS evaluated from the stack base (preview)	30 Jul 2026	expect the base's file

19.16 Practice: pattern exercises

Use the sample file of section 19.4. For each path, name the line that decides and the owners. Reason from the documented rules of section 19.5; do not use `git check-ignore`. Answers are in the Module 23 answers.

- `router/classify.py`
- `router/priority.py`
- `router/rules/eu.yaml`
- `config/routing.yaml`
- `docs/README.md`
- `docs/runbooks/escalation.md`
- `Docs/guide.md`
- `.github/workflows/ci.yaml`
- `README.md`
- The team wants `docs/runbooks/` owned by `@example-org/on-call` and everything else under `docs/` by `@example-org/docs`. Write the lines and say where in the file they go.
- Someone appends `*.py @example-org/python-guild` as the last line "so the guild sees all Python". Which existing owners lose which paths?

12. The line `/.github/ @example-org/repo-admins` is moved to the top of the file. Who owns `.github/workflows/ci.yaml` now?

Then do Lab 23.2, which proves enforcement with a second account.

19.17 Interview questions

1. What does a CODEOWNERS file do when no rule refers to it?
2. In which locations is the file looked up, in which order, and from which branch is it read for a given pull request?
3. Explain "last match wins" with an example where a later, more general pattern takes a path away from a specific team.
4. Name the gitignore features that do not work in CODEOWNERS, and one behavior that differs although the syntax is accepted.
5. A team is listed as owner and is never requested. Give three documented causes.
6. A path has two owning teams on one line and code owner review is required. Whose approval is needed?
7. Why can a pull request not remove its own required reviewer by editing CODEOWNERS?
8. How do you protect the CODEOWNERS file and the workflows directory, and which three conditions make that protection real?
9. When would you use the required reviewers rule instead of, or together with, CODEOWNERS?
10. How do you find out what GitHub thinks of your CODEOWNERS file before merging a change to it?

19.18 Sources

Primary sources (docs.github.com and the GitHub Changelog; read on 1 and 2 October 2026)

- [About code owners](#): locations, syntax, the example file, size limit, write-access requirement, forks, branch protection.
- [Available rules for rulesets](#): code owner review and the required reviewers sub-option.
- [REST API: list CODEOWNERS errors](#).
- [Changelog, 17 February 2026: required reviewer rule generally available](#).
- [Stacked pull requests \(reference\)](#).
- Git 2.55: `gitignore pattern format` (`git help ignore`), `git help check-ignore`.

Secondary sources

- The Phase 0 report of this course, section 2 ("CODEOWNERS requests reviews; only a rule makes them mandatory"), and its notes on the GitHub platform, section 3.

Videos

- The Phase 0 report lists no verified video that teaches CODEOWNERS; its survey of popular beginner courses found that none covers it.

Further reading

- Chapter 17: Pull Requests, sections 17.3 to 17.5; Chapter 18: Branch Protection and Rulesets, sections 18.5 and 18.7; Chapter 4: The Working Tree, for `.gitignore` itself.

Chapter 20A: GitHub Actions Fundamentals

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026, with the quoted documentation pages and action READMEs re-read on 2 October 2026. Transcripts are real output from `labs/ch20a/`. Nothing in this chapter was run on GitHub. The eight workflow files in `workflows/` were parse-checked with PyYAML and assembled from documented syntax and from each action's `action.yml` at the pinned version; they were **not executed on GitHub by the author**. The labs of Module 26 have you run them. Every statement about what GitHub Actions does carries the link it was taken from.

20A.1 Why this matters

Four questions a CTO can ask about one pull request:

1. "The tests passed on your laptop and failed in CI on the same commit. Was it the same commit?"
2. "The build stamped the release as `6fe5455` and not as a version. Who removed the tags?"
3. "This workflow has a `paths` filter and is a required check. Why is the pull request waiting forever?"
4. "The job is green. Did the tests run, or did a pipe swallow the failure?"

None of these is answered by knowing YAML keys. Each is answered by knowing what a runner has on disk and in its environment when your command starts: which commit, how much history, which shell with which flags, which token, which variables. This chapter builds that picture from the documentation and, wherever plain Git can show the mechanism, from a local transcript. The answers: question 1 in section 20A.8, question 2 in 20A.8, question 3 in 20A.4 and 20A.16, question 4 in 20A.7.

Two neighbors complete the subject. Chapter 20B: Delivery, runners, cost and debugging covers environments, deployments, concurrency in depth, reusable workflows, runners, billing and the investigation order for a failing run. Chapter 21A: GitHub Actions security covers the job token, fork pull requests, `pull_request_target`, injection, and pinning. This chapter mentions those topics only where a fundamental cannot be stated without them.

GitHub, not Git. Everything in this chapter except the repository content is GitHub Actions: a service that reacts to events on GitHub by running commands on machines. Git appears in exactly one place, when a job fetches commits onto the runner. That place causes most of the surprises.

20A.2 The model: workflow, event, job, step, action, runner, shell

In one sentence. A workflow is a YAML file in the repository that says "when this event happens, run these jobs"; each job is a list of steps that run in order on one fresh machine called a runner; a step is either a shell command or an action, which is a packaged program.

Analogy. A workflow is a standing order at a print shop: "whenever a manuscript arrives (event), give one copy to the proofreader and one to the typesetter (two jobs), each at a clean desk (runner), each following a numbered checklist (steps), some items of which say 'use the house stamping machine' (an action)". The analogy breaks in two places. The desks are destroyed after each job, so nothing left on one is found by the next. And the manuscript is not handed over: the worker must fetch it, and by default fetches only the last page (section 20A.8).

Precisely. The terms, as the [workflow syntax reference](#) uses them:

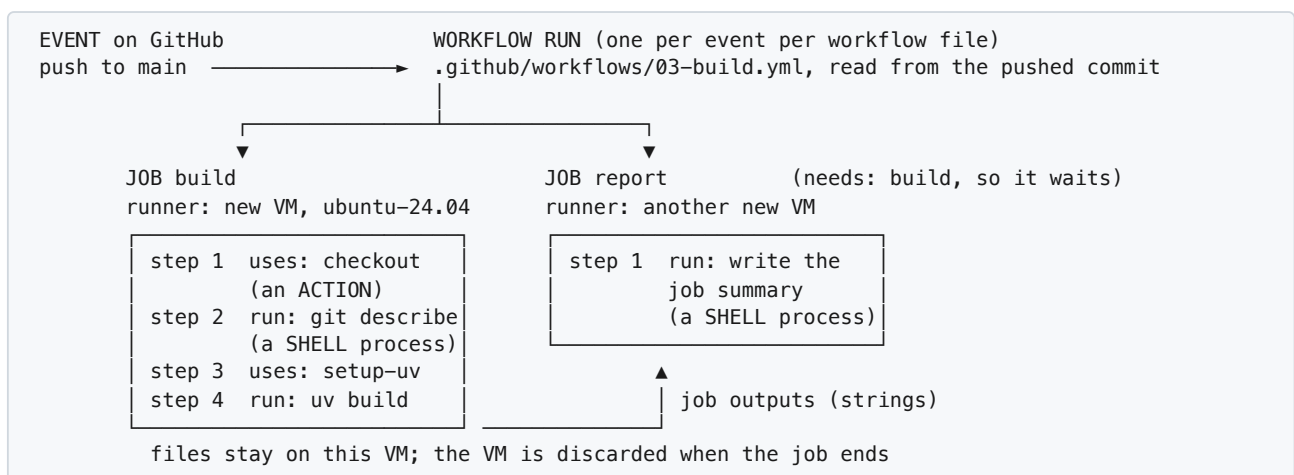
Term	Definition	Where it is written
Workflow	A configurable automated process defined by a YAML file. The file must be in <code>.github/workflows</code> and end in <code>.yaml</code> or <code>.yml</code>	the file itself
Event	An activity on GitHub (a push, a pull request activity, a release), a schedule, or a manual or API request	not written; it happens
Trigger	The <code>on</code> key of a workflow: which events, with which filters, start a run	<code>on:</code>
Workflow run	One execution of a workflow for one event. It has a run ID, a run number and, when re-run, an attempt number	created by GitHub
Job	A set of steps that execute on the same runner. Jobs of one run execute in parallel unless <code>needs</code> orders them	<code>jobs.<job_id></code>
Step	One <code>run</code> command line (or script) or one <code>uses</code> reference to an action. Steps of a job execute in order	<code>jobs.<job_id>.steps[*]</code>
Action	A reusable program referenced by <code>uses: owner/repo@ref</code> . It is a JavaScript program, a composite of steps, or a Docker container, declared by the <code>action.yaml</code> in its repository	another repository
Runner	The machine that executes one job. A GitHub-hosted runner is a new virtual machine per job, chosen by <code>runs-on</code>	<code>jobs.<job_id>.runs-on</code>
Shell	The program that executes a <code>run</code> step. Each <code>run</code> step is a new process	<code>shell:</code> , or the default

Three properties follow from the definitions and explain most of what later sections show:

- **A job is the unit of isolation.** A job starts on a fresh machine ([GitHub-hosted runners](#)). Files, installed tools and environment variables do not carry from job to job. Data crosses jobs only as job outputs (strings), artifacts (files) or caches (files, best effort).
- **A step is the unit of process.** Steps of one job share the filesystem of the runner, but each `run` step is a new shell process. A `cd` or an `export` in one step is gone in the next (section 20A.7).
- **The workflow file is versioned with the code.** For `push` and `pull_request` the workflow definition is read from the commit the event refers to, so a branch can change its own CI. For several other events it is read from the default branch only (section 20A.4).

Inside `.git`. A workflow is an ordinary blob at the path `.github/workflows/<name>.yaml` in a commit's tree. Nothing else in the repository's Git data describes Actions. Runs, jobs, logs, artifacts, caches, secrets and variables are GitHub objects: a clone does not contain them and `git log` cannot show them.

Picture.



20A.3 YAML read carefully

In one sentence. A workflow file is data, not a script: a YAML parser turns it into nested maps, lists and scalars before GitHub Actions interprets any key, so indentation and quoting decide what GitHub sees.

Precisely. Four rules cover the mistakes that occur in workflow files.

1. **Indentation is structure, and tabs are not indentation.** A key belongs to the map whose column it sits in. A tab character in indentation is a syntax error.
2. **An unquoted scalar is typed by its shape.** `3.10` is a number, and the number is 3.1. `v1.0` is a string because of the `v`. Quote everything that is a version, a label or a glob: `"3.10"`, `"v*"`, `"**/*.py"`. The [filter pattern cheat sheet](#) adds that patterns starting with `*`, `[` or `!` must be quoted, because those characters are YAML syntax.
3. **| keeps line breaks, > folds them into spaces.** A multi-line `run` script needs `|`.
4. `&name` marks a node and `*name` reuses it. Anchors and aliases are standard YAML and have been accepted in workflow files since 18 September 2025 ([changelog](#), [reference](#)).

See it. The parser below is PyYAML, the library that parse-checks the course workflows. It implements YAML 1.1. GitHub uses its own parser, so read each result with the note that follows it.

```
$ cat scalars.yml
python: [3.9, 3.10, "3.10", 3.11]
version: 1.0
enabled: yes
country: NO
tag: v1.0
$ python3 -c "$show" scalars.yml
{
  "python": [
    3.9,
    3.1,
    "3.10",
    3.11
  ],
  "version": 1.0,
  "enabled": true,
  "country": false,
  "tag": "v1.0"
}
```

`3.10` became `3.1`: a matrix written `python: [3.9, 3.10]` asks for Python 3.1. This is a property of numbers, not of one parser, and it is why the READMEs of `actions/setup-python` and `astral-sh/setup-uv` quote every version. The values `yes` and `NO` became booleans because YAML 1.1 says so.

Unverified. Whether GitHub's workflow parser also reads unquoted `yes`, `no`, `on` and `off` values as booleans is not stated on the documentation pages read for this chapter. Quote such values and the question does not arise.

```
$ cat on-key.yml
name: Tests
on:
  push:
    branches: [main]
  pull_request:
jobs: {}
$ python3 -c "$show" on-key.yml
{
  "name": "Tests",
  "true": {
```

```

    "push": {
      "branches": [
        "main"
      ]
    },
    "pull_request": null
  },
  "jobs": {}
}

```

The key `on` came out as the boolean `true`. GitHub Actions documents `on` as the trigger key and reads it as such; a generic YAML 1.1 tool does not. The practical consequence is for your own tooling: a script that loads workflow files with PyYAML must look up the key `True`, which is what `tools/check_course.py` of this course does. Notice also `pull_request: null`. An event with no configuration is a key with an empty value, and the documentation requires the colon on every event once any event in the map has configuration ([on](#)).

```

$ cat anchors.yml
jobs:
  test:
    env: &common_env
    PYTHONPATH: src
    CI_PROFILE: fast
    steps: &setup
    - uses: actions/checkout@SHA
  lint:
    env: *common_env
    steps: *setup
$ python3 -c "$show" anchors.yml
{
  "jobs": {
    "test": {
      "env": {
        "PYTHONPATH": "src",
        "CI_PROFILE": "fast"
      },
      "steps": [
        {
          "uses": "actions/checkout@SHA"
        }
      ]
    },
    "lint": {
      "env": {
        "PYTHONPATH": "src",
        "CI_PROFILE": "fast"
      },
      "steps": [
        {
          "uses": "actions/checkout@SHA"
        }
      ]
    }
  }
}

```

After parsing, the alias is indistinguishable from a copy. An anchor therefore removes repetition inside one file and nothing more; it cannot cross files. For that, Chapter 20B covers reusable workflows and composite actions.

Unverified. YAML merge keys (`<<: *name`) are not mentioned by the anchors documentation or the changelog entry. The course's research notes mark their support as unverified. The examples in this course do not use them.

```
$ cat blocks.yml
literal: |
  uv sync --locked
  uv run pytest
folded: >
  uv sync --locked
  uv run pytest
$ python3 -c "$show" blocks.yml
{
  "literal": "uv sync --locked\nuv run pytest\n",
  "folded": "uv sync --locked uv run pytest\n"
}
```

With `>` the two commands became one line, `uv sync --locked uv run pytest`, which is one wrong command and not two right ones.

```
$ python3 -c "$show" tab.yml 2>&1 | tail -3
yaml.scanner.ScannerError: while scanning for the next token
found character '\t' that cannot start any token
in "tab.yml", line 5, column 1
[exit status: 0]
```

20A.4 Events and filters

In one sentence. The `on` key lists the events that start a run; each event fixes which commit and ref the run is about and which copy of the workflow file is used, and filters narrow an event to branches, tags or paths.

Precisely. `on` takes one event (`on: push`), a list (`on: [push, pull_request]`), or a map from event to its configuration. The eight events a backend or ML team uses constantly, with the values the [events reference](#) gives for the commit (`GITHUB_SHA`) and the ref (`GITHUB_REF`):

Event	Starts when	GITHUB_SHA	GITHUB_REF	Notes from the reference
<code>push</code>	commits or tags are pushed	tip commit pushed to the ref	the updated ref	runs for any branch, including workflows not merged into the default branch
<code>pull_request</code>	a pull request is opened, gets new commits (<code>synchronize</code>) or is reopened; other activity types need <code>types:</code>	last merge commit on the <code>GITHUB_REF</code> branch	<code>refs/pull/N/merge</code>	does not run while the pull request has a merge conflict
<code>workflow_dispatch</code>	someone starts it by hand, the CLI or the API	last commit on the chosen branch or tag	the branch or tag that received the dispatch	only if the workflow file exists on the default branch; up to 25 inputs
<code>schedule</code>	a POSIX cron expression matches	last commit on the default branch	the default branch	UTC unless a <code>timezone</code> is given; shortest interval 5 minutes; can be delayed or dropped under load; disabled after 60 days without repository activity in a public repository

Event	Starts when	GITHUB_SHA	GITHUB_REF	Notes from the reference
<code>workflow_call</code>	another workflow calls this one	same as the caller	same as the caller	makes the workflow reusable (Chapter 20B)
<code>workflow_run</code>	another workflow is requested, in progress or completed	last commit on the default branch	the default branch	only if the file exists on the default branch; can read secrets even when the first workflow could not (Chapter 21A)
<code>merge_group</code>	a pull request enters a merge queue (<code>checks_requested</code>)	the commit of the merge group	the ref of the merge group	required checks must listen to it or the queue never gets a result (Chapter 17, section 17.11)
<code>release</code>	a release is published, created, edited and so on	last commit in the tagged release	<code>refs/tags/<tag_name></code>	subscribe to <code>published</code> to cover stable and pre-releases

Read the table as three families. `push`, `pull_request`, `merge_group` and `release` are about a specific commit that somebody produced. `schedule` and `workflow_run` are about the default branch, whatever happened elsewhere. `workflow_dispatch` is about whatever ref the person chose.

One event is deliberately not in the table: `pull_request_target`. It starts on the same pull request activity as `pull_request`, but it is about the base repository: the workflow file comes from the default branch and the job receives the base repository's token and secrets. Chapter 18, section 18.8 counts its checks, and Chapter 21A, section 21A.5 explains when it is safe; do not use it before reading that section.

Filters. `push` accepts `branches`, `branches-ignore`, `tags`, `tags-ignore`, `paths` and `paths-ignore`. `pull_request` accepts `branches` and `branches-ignore`, which match the **base** branch, and the two path filters. The rules that matter ([filters](#)):

- A positive and an `-ignore` filter of the same kind cannot be combined for one event. Use `!pattern` inside the positive filter; order matters.
- When a branch filter and a path filter are both present, the workflow runs only when both are satisfied.
- Path filters are not evaluated for pushes of tags.
- Path filters compare with a **three-dot diff for pull requests and a two-dot diff for pushes** ([diff comparisons](#)). A push with more than 1,000 commits, or a diff that times out, always runs the workflow; a diff of more than 3,000 files in which the match is not among the first 3,000 does not.

See it. Those two diffs are plain Git (Chapter 14A covers range notation), so you can predict a path filter before pushing. The repository has a pull request branch that touches only Python files, while `main` moved on.

```
# Pull request: base main, head feature/reorder-report. Three dots: what the branch changed.
$ git diff --name-only origin/main...feature/reorder-report
src/inventory_api/report.py
tests/test_report.py
# Does any changed path match java-service/** ?
$ git diff --quiet origin/main...feature/reorder-report -- "java-service/**"
[exit status: 0]
# Exit status 0 means no difference under that path: the Java workflow would not start.
```

```
# Two dots compare the two tips, so the change made on main shows up as if it were yours:
$ git diff --name-only origin/main..feature/reorder-report
```

```
src/inventory_api/report.py
src/inventory_api/stock.py
tests/test_report.py
tests/test_stock.py
```

The three-dot form compares the merge base with the head, so it lists what the branch did. The two-dot form compares the tips and blames the branch for `stock.py`, which changed on `main`. For a pull request GitHub uses the first. For a push it uses two dots between the old and new tip of the pushed branch, which is the same thing as "the commits you just pushed":

```
# Push: the diff from the old tip of the branch to the new one.
$ git switch --quiet feature/reorder-report
$ before=$(git rev-parse HEAD)
$ git commit --quiet -am "Bump java-service to 0.2.0"
$ git diff --name-only $before..HEAD
java-service/pom.xml
$ git diff --quiet $before..HEAD -- "java-service/**"
[exit status: 1]
# Exit status 1: a path under java-service/ changed, so the Java workflow would start.
```

In production. A monorepo with a Python service and a Java service gives each its own workflow with a `paths` filter, so a documentation change does not spend twenty runner minutes on Maven. The saving has a price that question 3 of section 20A.1 names: a workflow that a filter skips never creates its check, and a ruleset that requires that check waits for a result that will not come ([troubleshooting required status checks](#)). Section 20A.16 gives the fix. The same applies to commits whose message contains `[skip ci]` and its documented variants, which skip `push` and `pull_request` workflows ([skipping workflow runs](#)).

GitHub, not Git. Events created by a workflow's own job token do not start new workflow runs, with documented exceptions (`workflow_dispatch`, `repository_dispatch`, and pull request events, which wait for approval since 11 June 2026) ([GITHUB_TOKEN](#), [changelog](#)). A job that pushes a commit with the job token does not trigger the `push` workflows for that commit.

20A.5 Contexts and expressions

In one sentence. A context is a named object of facts about the run (`github`, `matrix`, `steps`, and others), and an expression, written `${{ ... }}`, is a small formula over contexts that GitHub Actions replaces with its value before your step starts.

Analogy. An expression is a mail-merge field. The letter (your step) is printed with the field already filled in; the recipient (the shell) never sees the field, only the text. The analogy is exact about the danger and incomplete about the timing: text substituted into a script becomes part of the script, which is how untrusted pull request titles become shell commands (Chapter 21A), and not every field is available in every place.

Precisely. The contexts, from the [contexts reference](#):

Context	Holds	Typical use
<code>github</code>	the event and the run: <code>event_name</code> , <code>event</code> (the full webhook payload), <code>ref</code> , <code>ref_name</code> , <code>sha</code> , <code>head_ref</code> and <code>base_ref</code> (pull request events only), <code>actor</code> , <code>repository</code> , <code>run_id</code> , <code>run_number</code> , <code>run_attempt</code> , <code>workflow</code>	conditions, names, concurrency groups
<code>env</code>	variables set with <code>env:</code> in the workflow, job or step	passing configuration to steps
<code>vars</code>	configuration variables stored in GitHub settings	non-secret settings per repository or

Context	Holds	Typical use
		environment
secrets	secrets, including <code>GITHUB_TOKEN</code>	credentials
inputs	inputs of <code>workflow_dispatch</code> or <code>workflow_call</code>	manual and reusable workflows
matrix, strategy	the current matrix combination and the strategy settings	matrix jobs
steps	per earlier step with an <code>id</code> : <code>outputs</code> , <code>outcome</code> , <code>conclusion</code>	using a step's result
needs	per job listed in <code>needs</code> : <code>outputs</code> , <code>result</code>	using a job's result
job, runner	the running job (<code>status</code> , service ports) and its machine (<code>os</code> , <code>arch</code> , <code>temp</code> , <code>environment</code>)	paths, conditions on platform
jobs	job results, in reusable workflows only	outputs of a called workflow

Rules of the expression language ([expressions](#)):

- Literals are booleans, `null`, numbers and strings. **Strings use single quotes** inside `${{ }}`; double quotes are an error.
- Operators are `( ) [ ] . ! < <= > >= == != && ||`. String comparison ignores case. When the two sides of `==` differ in type, both are converted to numbers, and a string that is not a number becomes `NaN`.
- The falsy values are `false`, `0`, `-0`, `""`, `'` and `null`.
- A property that does not exist evaluates to an empty string, without an error. A typo in `steps.buld.outputs.version` is therefore silent.
- Functions: `contains`, `startsWith`, `endsWith`, `format`, `join`, `toJSON`, `fromJSON`, `hashFiles`, `case`, and the status functions `success()`, `failure()`, `cancelled()`, `always()`.
- `hashFiles(pattern)` computes a SHA-256 per matched file and then one SHA-256 over those; the pattern is relative to the workspace, and no match gives an empty string.
- `case(pred1, val1, pred2, val2, ..., default)` returns the value for the first true predicate, and has existed since 29 January 2026 ([changelog](#)). Older files imitate it with `cond && a || b`, which returns `b` whenever `a` is falsy.

Context availability is per key. Not every context exists everywhere ([context availability](#)). A job-level `if` can read `github`, `needs`, `vars` and `inputs`, but not `matrix`, `steps`, `env` or `secrets`. A step-level `if` can read `matrix`, `steps` and `env`, but still not `secrets`. `hashFiles` works only in step keys. The reason is timing: a job-level `if` is decided before a runner exists, so nothing that lives on the runner can take part.

The trap in `if`. In an `if`, the `${{ }}` wrapper is optional unless the expression starts with `!`. Text outside the wrapper turns the whole value into a non-empty string, which is truthy, so the condition is always true. Since 29 January 2026 the editor validation flags this and the run shows an annotation (same changelog as above).

```
if: ${{ github.event_name == 'push' }}      # an expression
if: github.event_name == 'push'             # the same expression
if: ${{ github.event_name == 'push' }} && true  # a string, always truthy
```

Picture. Where each piece is evaluated:

workflow file		GitHub Actions service		runner
on:, jobs.<id>.if,	→	evaluated before a runner		
strategy.matrix		is assigned		
steps[*].if, with:,	→		→	evaluated per step, on the
env:, run: \${{ ... }}				runner, BEFORE the shell starts
run: echo "\$NAME"			→	expanded by the shell, at run time

In production. The difference between the last two rows is a security boundary. `run: echo "${{ github.event.pull_request.title }}"` pastes the title into the script text. `env: TITLE: ${{ github.event.pull_request.title }}` followed by `run: echo "$TITLE"` hands the title to the shell as data. Every course workflow uses the second form, including for values that are not attacker-controlled, so that the habit is uniform. Chapter 21A explains the attack.

20A.6 `env`, `vars` and `secrets`

In one sentence. `env` is defined in the workflow file and becomes environment variables of the process; `vars` and `secrets` are stored in GitHub settings and are read through contexts; only `secrets` are encrypted and masked.

Precisely.

	<code>env</code>	<code>vars</code>	<code>secrets</code>
Defined in	the workflow file, at workflow, job or step level	settings of the organization, repository or environment	settings of the organization, repository or environment
Visible in the repository	yes, it is in the file	no, but readable in settings and logs	no; write-only in settings
Reaches a step as	an environment variable, and <code>\${{ env.NAME }}</code>	<code>\${{ vars.NAME }}</code> only; map it into <code>env</code> to get a variable	<code>\${{ secrets.NAME }}</code> only; map it into <code>env</code> or with
Same name at several levels	the most specific wins (step over job over workflow)	the lowest level wins: environment over repository over organization	the environment-level secret wins
Limits	part of the file	48 KB each; 1,000 per organization, 500 per repository, 100 per environment	48 KB each; 1,000 per organization, 100 per repository, 100 per environment
Masked in logs	no	no	yes, when the exact value appears; not a guarantee

Sources: [variables](#), [secrets](#). Names of variables and secrets may contain letters, digits and underscores, must not start with `GITHUB_` or a digit, and are case-insensitive when referenced.

Three behaviors that cause real failures:

- **An unset secret is an empty string, not an error.** A deploy step that receives an empty credential fails later and elsewhere, with a message about authentication ([using secrets](#)).
- **Secrets are not passed to workflows triggered from forks or by Dependabot**, apart from a read-only job token (same page; the security model is Chapter 21A). A fork pull request that "fails only in CI" is often a job that needed a secret.
- `secrets` **cannot be used in** `if`. Map the secret into `env` and test `env.NAME != ''`.

`GITHUB_TOKEN` is a secret that GitHub creates for each job. What it may do is set by the `permissions` key. All eight course workflows declare `permissions: contents: read` at the top, and when any permission is named, every unnamed one becomes `none` ([permissions](#)). Workflow 6 adds `packages: write` for one job. Least privilege is the subject of Chapter 21A; here it is enough to know that the block is not optional decoration.

The runner also sets default environment variables. The ones this chapter uses: `CI` (always `true`), `GITHUB_SHA`, `GITHUB_REF`, `GITHUB_REF_NAME`, `GITHUB_EVENT_NAME`, `GITHUB_WORKSPACE` (the default working directory of steps), `GITHUB_EVENT_PATH` (a file holding the webhook payload), `RUNNER_OS`, `RUNNER_TEMP`. Each has a `github.` or `runner.` context twin; use the variable inside scripts and the context in YAML keys.

20A.7 Shells, and passing data between steps and jobs

In one sentence. Every `run` step is a new shell process started from a documented command template, so state survives a step only if it is written to a file the runner provides: `GITHUB_OUTPUT` for named outputs, `GITHUB_ENV` for environment variables, `GITHUB_STEP_SUMMARY` for a report.

Precisely: the shell. The runner writes your `run` text to a temporary script file and executes it with a template in which `{0}` is that file ([shell](#)):

You write	Platform	Command the runner executes
nothing	Linux, macOS	<code>bash -e {0}</code>
<code>shell: bash</code>	any	<code>bash --noprofile --norc -eo pipefail {0}</code>
nothing	Windows	PowerShell Core (<code>pwsh</code>)
nothing, inside a job container	Linux	<code>sh</code> , not <code>bash</code>
<code>shell: python</code>	any	<code>python {0}</code>

`-e` stops the script at the first command that fails. It does not look inside a pipeline: the status of a pipeline is the status of its last command unless `pipefail` is set. The implicit default therefore lets `failing-tests | tee log.txt` pass.

See it. The two templates, run locally on the same three-line script:

```
$ cat step.sh
set -u
false | tee /dev/null
echo "the step reached its last line"
# No shell key on Linux or macOS: bash -e {0}
$ bash -e step.sh
the step reached its last line
[exit status: 0]
# shell: bash: bash --noprofile --norc -eo pipefail {0}
$ bash --noprofile --norc -eo pipefail step.sh
[exit status: 1]
```

Under `bash -e` the step is green although `false` failed, because `tee` succeeded. With the `shell: bash` template the same script stops with status 1. This answers question 4 of section 20A.1: a green job proves only that the last command of every pipeline succeeded, unless the shell was declared. Workflow 3 sets `defaults.run.shell: bash` for this reason.

A new process per step:

```
$ cat step1.sh
export BUILD_ID=build-42
cd build
pwd
$ bash -e step1.sh
$LAB/ch20a/step-shell/build
$ cat step2.sh
echo "BUILD_ID is [${BUILD_ID:-}]"
pwd
$ bash -e step2.sh
BUILD_ID is []
$LAB/ch20a/step-shell
```

The `export` and the `cd` of the first script did not reach the second. On a runner the same happens between steps, which is why `defaults.run.working-directory` and `GITHUB_ENV` exist.

Precisely: the files. The runner creates files and puts their paths in environment variables ([workflow commands](#)):

File	You append	Effect	Limits
GITHUB_OUTPUT	name=value	the step (it needs an <code>id</code>) gets <code>steps.<id>.outputs.name</code>	job outputs at most 1 MB per job and 50 MB per run
GITHUB_ENV	NAME=value	later steps of the job get the environment variable; the writing step does not	cannot override <code>GITHUB_*</code> or <code>RUNNER_*</code>
GITHUB_PATH	a directory	prepended to <code>PATH</code> for later steps	
GITHUB_STEP_SUMMARY	GitHub-flavored Markdown	shown on the run's summary page	1 MiB per step; 20 step summaries shown per job

A step output is local to its job. To cross to another job it must be re-exported as a **job output** (`jobs.<id>.outputs`) and read through `needs.<id>.outputs`. An output that looks like a secret is dropped with a message. Multi-line values use a delimiter.

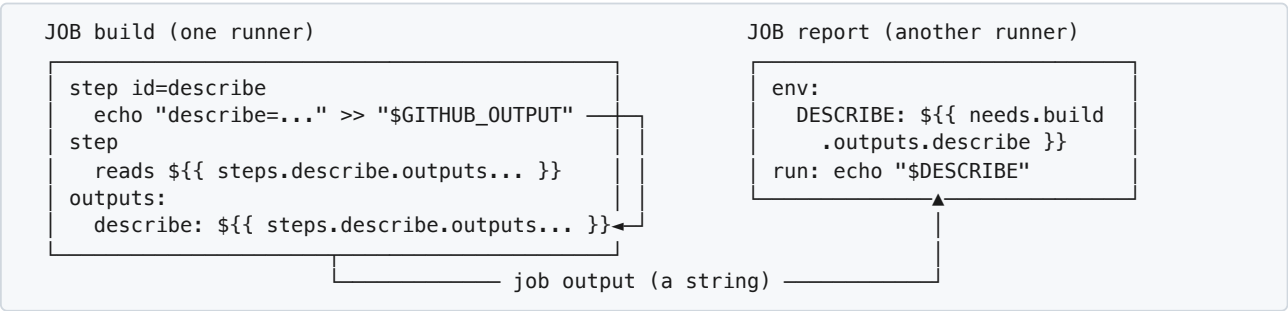
```
# The runner gives each step a file path in GITHUB_OUTPUT. A file stands in for it here.
$ export GITHUB_OUTPUT="$PWD/step-describe.output"
$ echo "describe=v0.1.0-1-g$(printf abc1234)" >> "$GITHUB_OUTPUT"
$ echo "files=inventory_api-0.1.0.tar.gz inventory_api-0.1.0-py3-none-any.whl" >> "$GITHUB_OUTPUT"
$ cat "$GITHUB_OUTPUT"
describe=v0.1.0-1-gabc1234
files=inventory_api-0.1.0.tar.gz inventory_api-0.1.0-py3-none-any.whl
# A multi-line value needs the delimiter form:
$ printf "notes<<EOF_NOTES\nline one\nline two\nEOF_NOTES\n" >> "$GITHUB_OUTPUT"
$ cat "$GITHUB_OUTPUT"
describe=v0.1.0-1-gabc1234
files=inventory_api-0.1.0.tar.gz inventory_api-0.1.0-py3-none-any.whl
notes<<EOF_NOTES
line one
line two
EOF_NOTES
```

The file is an append-only list of assignments that the runner reads when the step ends. Always quote `"$GITHUB_OUTPUT"` and always use `>>`: a single `>` discards what earlier lines of the same step wrote.

Outdated advice. `echo "::set-output name=x::value"` and `::save-state` are deprecated since 11 October 2022. They still work with a warning; the planned removal was postponed on 24 July 2023, and the current workflow commands page no longer documents them ([deprecation](#), [postponement](#)). The reason for the change is visible in the transcript below: the old form was a magic line on standard output, so any program whose output a step printed, including a test that echoed untrusted input, could set outputs.

```
# The deprecated form wrote a command to standard output, where any program can print it:
$ echo "::set-output name=describe::v0.1.0"
::set-output name=describe::v0.1.0
```

Picture.



20A.8 What `actions/checkout` does by default

In one sentence. A runner starts with no repository at all; `actions/checkout` fetches **one commit, without tags**, for the ref of the event, and on `pull_request` events that ref is the test merge commit `refs/pull/N/merge`, checked out with a detached HEAD.

Analogy. You asked a courier for "the contract" and received the last page, unstapled, with a cover sheet that merges your draft and the other party's latest draft. It is the right thing to sign and the wrong thing to consult about the history of clause 4. The analogy breaks because the last page here is a complete snapshot of every file: only the history is missing, not the content.

Precisely. From the action's [README](#) and [action.yml](#) at v7.0.1, the version pinned in this course:

Input	Default	Consequence
<code>ref</code>	the ref or commit of the event; the default branch for another repository	on <code>pull_request</code> , <code>refs/pull/N/merge</code>
<code>fetch-depth</code>	<code>1</code>	"Only a single commit is fetched by default"; <code>0</code> fetches all history for all branches and tags
<code>fetch-tags</code>	<code>false</code>	no tags, so nothing for <code>git describe</code> to find
<code>persist-credentials</code>	<code>true</code>	the job token is kept for later <code>git</code> commands of the job and removed in post-job cleanup; since v6 it is stored under <code>\$RUNNER_TEMP</code> and not in <code>.git/config</code>
<code>lfs</code> , <code>submodules</code>	<code>false</code>	LFS pointers are not replaced by content; submodule directories are empty
<code>clean</code>	<code>true</code>	matters on self-hosted runners that reuse a workspace
<code>allow-unsafe-pr-checkout</code>	<code>false</code>	v7 refuses to check out fork pull request code under <code>pull_request_target</code> and <code>workflow_run</code> (Chapter 21A)

The course workflows set `persist-credentials: false` because none of them runs an authenticated `git` command after the checkout: a credential that is not on disk cannot be read by a later step.

Inside `.git`. A shallow fetch writes the fetched commit's ID to the file `.git/shallow`. That file tells Git to treat the commit as having no parents, although the commit object itself still names them (Chapter 26, section 26.11 covers shallow clones and their limits). No tag refs are created. HEAD is a branch for `push` events and a bare commit ID for pull requests.

See it: one commit, no tags. Your clone of the fixture has three commits and an annotated tag:

```
# Your clone: three commits, one annotated tag.
$ cd you/inventory-api
$ git log --oneline --decorate
6fe5455 (HEAD -> main, origin/main) Document how to run the tests
ae299cb (tag: v0.1.0) Add Maven module for price calculation
05d3477 Add stock functions and tests
$ git describe --tags
v0.1.0-1-g6fe5455
$ cd ../../
```

The documented defaults, reproduced as a clone from a `file://` URL. (A plain path would make Git copy the repository and ignore `--depth`; the URL form goes through the transfer protocol, as a runner does. The runner's own sequence of Git commands is an implementation detail and is not what is shown.)

```
# HUB is a file:/// URL of the bare repository that plays GitHub.
# fetch-depth: 1 and fetch-tags: false, as a clone:
$ git clone --depth 1 --no-tags "$HUB" runner
Cloning into 'runner'...
$ cd runner
$ git log --oneline --decorate
6fe5455 (grafted, HEAD -> main, origin/main, origin/HEAD) Document how to run the tests
$ git rev-parse --is-shallow-repository
true
$ cat .git/shallow
6fe5455926c26261fe2c73104018ca2d96fdb554
$ git tag --list
$ git rev-list --count HEAD
1
```

grafted in the log decoration and the ID in `.git/shallow` say the same thing: history is cut here. The commit count is 1 and the tag list is empty.

```
$ git describe --tags
fatal: No names found, cannot describe anything.
[exit status: 128]
$ git describe --tags --always
6fe5455
[exit status: 0]
# History questions get wrong answers, not errors:
$ git log --oneline -- java-service/pom.xml
6fe5455 Document how to run the tests
$ git merge-base HEAD origin/main~1
fatal: Not a valid object name origin/main~1
[exit status: 128]
```

`git describe` fails with "No names found". With `--always` it does not fail: it prints the abbreviated commit ID `6fe5455`, and a build stamps that as the version. That is question 2 of section 20A.1. Worse than the error are the two lines after it. `git log -- java-service/pom.xml` claims the file was last changed by the only commit there is, because a parentless commit "adds" every file. The true answer, in the full clone, is `ae299cb`. Any tool that asks history a question (changed-file detection, blame-based analysis, changelog generators, version-from-tag plugins) gets a confident wrong answer.

```
# fetch-tags: true with depth 1: the tag arrives, its commit is not an ancestor we have.
$ git fetch --depth 1 origin "refs/tags/*:refs/tags/*"
From file:///LAB/ch20a/shallow-checkout/hub/inventory-api
* [new tag]          v0.1.0      -> v0.1.0
$ git tag --list
v0.1.0
$ git describe --tags
fatal: No tags can describe '6fe5455926c26261fe2c73104018ca2d96fdb554'.
Try --always, or create some tags.
[exit status: 128]
```

Fetching tags alone does not repair `describe`. The tag now exists, but `describe` walks from HEAD through parents to find a tagged ancestor, and the walk ends at the graft. Tags need the history between them and HEAD.

```
# fetch-depth: 0, as a repair of this clone:
$ git fetch --unshallow --tags
$ git rev-parse --is-shallow-repository
false
$ git rev-list --count HEAD
3
$ git describe --tags
v0.1.0-1-g6fe5455
$ git log --oneline -- java-service/pom.xml
ae299cb Add Maven module for price calculation
```

```

Observed behavior : the build names the release 6fe5455 instead of v0.1.0-1-g6fe5455
Git state         : .git/shallow lists HEAD; one commit; no refs under refs/tags
Mechanism         : git describe walks parents looking for a tagged commit; the graft ends the walk
Root cause        : actions/checkout defaults: fetch-depth 1, fetch-tags false
Why it does this  : a job that only compiles and tests needs the snapshot, not the history;
                   fetching one commit is the fastest correct default for that job
Correct fix       : fetch-depth: 0 in the job that needs history or tags (workflow 3)
Prevention        : decide per job whether it asks history a question; never use --always
                   to silence describe in a release build

```

State table for the two Git operations of this demonstration, on the runner's clone:

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
 <code>git clone --depth 1 --no-tags</code>	files of one commit	matches that commit	main (or detached for a pull request)	the fetched commit	.git/shallow written; origin/main; no tags	unchanged	unchanged
 <code>git fetch --unshallow --tags</code>	unchanged	unchanged	unchanged	unchanged	.git/shallow removed; missing commits and tag refs added	unchanged	unchanged

See it: the merge ref. Chapter 17, section 17.2 showed that GitHub keeps, for a mergeable pull request, a test merge commit under `refs/pull/N/merge`. The events reference states that `pull_request` workflows set `GITHUB_REF` to that ref and `GITHUB_SHA` to that commit, and that the checkout action therefore "checks out the merge branch", so that "CI tests run against the merged result, not just the head branch alone" ([how the merge branch affects your workflow](#)).

The fixture: your branch adds a reorder report whose test assumes the low-stock threshold is 5. Meanwhile a colleague raised the threshold to 10 on `main`. The two changes touch different files, so they merge without a conflict.

```

# Your clone, on the pull request branch. The tests pass.
$ cd you/inventory-api
$ git switch feature/reorder-report
Switched to branch 'feature/reorder-report'
Your branch is up to date with 'origin/feature/reorder-report'.
$ git log --oneline -2
80c9cfa Add reorder report
6fe5455 Document how to run the tests
$ PYTHONPATH=src python3 -m unittest discover -s tests 2>&1 | tail -1
OK
$ cd ../..

```

The server side below is a bare repository in which the fixture created the two pull request refs with plain Git, imitating the documented behavior.

```

# The repository on the platform after pull request 7 was opened:
$ git ls-remote "$HUB"
29b222ba49c3886f27c205a7ec4c982d141052c3      HEAD
80c9cfa3a369de2fb126f79a0e34f6139c858a76      refs/heads/feature/reorder-report
29b222ba49c3886f27c205a7ec4c982d141052c3      refs/heads/main
80c9cfa3a369de2fb126f79a0e34f6139c858a76      refs/pull/7/head
d04ef31e3ed5a74c9a89af11e2cbb441599e84c7      refs/pull/7/merge
192d2bc34d491d9b35cc1c00e50dde959dd9df55      refs/tags/v0.1.0
ae299cbfba5ad3d9388d3dded0eeee37b09946aa      refs/tags/v0.1.0^{*}

```

`refs/pull/7/head` is your commit `80c9cfa`. `refs/pull/7/merge` is `d04ef31`, a commit that neither you nor your colleague created. The runner fetches that one commit:

```
# The runner: fetch one commit, the one refs/pull/7/merge names, and detach HEAD on it.
$ git init --quiet runner && cd runner
$ git remote add origin "$HUB"
$ git fetch --no-tags --depth=1 origin +refs/pull/7/merge:refs/remotes/pull/7/merge
From file://$LAB/ch20a/merge-ref/hub/inventory-api
* [new ref]          refs/pull/7/merge -> pull/7/merge
$ git checkout --detach refs/remotes/pull/7/merge
HEAD is now at d04ef31 Merge 80c9cfa3a369de2fb126f79a0e34f6139c858a76 into
29b222ba49c3886f27c205a7ec4c982d141052c3
```

```
$ git status --short --branch
## HEAD (no branch)
$ git log -1 --format="GITHUB_SHA would be %H%n%an: %s"
GITHUB_SHA would be d04ef31e3ed5a74c9a89af11e2cbb441599e84c7
GitHub: Merge 80c9cfa3a369de2fb126f79a0e34f6139c858a76 into 29b222ba49c3886f27c205a7ec4c982d141052c3
$ git rev-list --count HEAD
1
$ git branch --show-current
$ grep LOW_STOCK_THRESHOLD src/inventory_api/stock.py
LOW_STOCK_THRESHOLD = 10
def low_stock(stock, threshold=LOW_STOCK_THRESHOLD):
$ ls src/inventory_api
__init__.py
report.py
stock.py
```

HEAD is detached, there is no current branch, the history is one commit deep, and the working tree contains both changes: your `report.py` and the threshold of 10 from `main`.

```
# The merged code is what CI tests. main changed the threshold; the new test assumed 5.
$ PYTHONPATH=src python3 -m unittest discover -s tests 2>/dev/null
[exit status: 1]
$ PYTHONPATH=src python3 -m unittest discover -s tests 2>&1 | tail -1
FAILED (failures=1)
```

The tests fail on the runner and pass on your laptop. Both results are correct. They tested different commits.

```
# Back in your clone: the commit CI tested is not there, and your branch is unchanged.
$ cd ../you/inventory-api
$ merge=$(git ls-remote "$HUB" refs/pull/7/merge | cut -f1)
$ git cat-file -t $merge
fatal: git cat-file: could not get object info
[exit status: 128]
# Reproduce what CI tested: merge the current base into a throwaway copy of your branch.
$ git fetch --quiet origin
$ git switch --quiet --detach feature/reorder-report
$ git merge --quiet --no-edit origin/main
$ git rev-parse HEAD^{tree}
745aaaaed0f47d2429feeb200e9776b329be83ef
$ git -C ../../runner rev-parse HEAD^{tree}
745aaaaed0f47d2429feeb200e9776b329be83ef
$ PYTHONPATH=src python3 -m unittest discover -s tests 2>&1 | tail -1
FAILED (failures=1)
$ git switch --quiet feature/reorder-report
```

The commit that CI tested does not exist in your clone, so `git show` of the ID in the CI log fails there. Merging the current base into a detached copy of your branch produces the same tree (the two tree IDs are equal), and the failure reproduces locally.

```
Observed behavior : tests pass locally, fail in CI "on the same commit"
Git state          : laptop HEAD = 80c9cfa (pull request head); runner HEAD = d04ef31 (test merge)
Mechanism          : pull_request sets GITHUB_REF to refs/pull/N/merge; checkout uses GITHUB_REF
Root cause         : the base branch gained a commit that changes behavior the new code relies on
Why it does this   : the question a pull request asks is "is the result of merging safe",
```

Correct fix : which only the merged tree can answer
 Prevention : update the branch (merge or rebase onto the base), fix the code, push
 : reproduce with `git fetch origin && git merge origin/main` before debugging;
 a merge queue or "require branches to be up to date" closes the remaining gap

Picture, with the IDs of the transcript:



Three consequences, all from the same pages:

- **The test merge can be older than main**. It is regenerated when the pull request branch is pushed, when the merge base changes, or when it is older than 12 hours (Chapter 17, section 17.2). A re-run of a job reuses the original commit ([re-running](#)), so a re-run does not pick up a newer base.
- **A conflicted pull request has no merge ref, and its pull_request workflows do not run at all**. The symptom is missing checks, not failed ones.
- **To test the head commit alone**, the README documents `ref: ${github.event.pull_request.head.sha}`. You then test something that will never be on the base branch; do it knowingly, for example for a linter that should judge only the author's files.

In production. An ML team's pull request adds a metric and passes locally. CI fails in an unrelated data-loader test. The author spends an hour on the loader before looking at the first lines of the checkout step's log, where the fetched ref is `refs/pull/412/merge`. Someone had merged a loader change twenty minutes earlier. The habit that prevents the lost hour: read which commit the job checked out before reading anything else. Workflow 1 prints it in its own step for that reason.

20A.9 Controlling jobs: `needs`, `if`, `timeout-minutes`, `continue-on-error`

In one sentence. `needs` orders jobs and carries their results, `if` decides whether a job or step runs, `timeout-minutes` bounds how long it may run, and `continue-on-error` lets a failure pass without failing what contains it.

Precisely. From the [workflow syntax reference](#):

Key	Level	Rule	Default
<code>needs</code>	job	the job waits for the listed jobs. If one of them fails or is skipped, this job is skipped, unless its <code>if</code> says otherwise	none: jobs run in parallel
<code>if</code>	job, step	the job or step runs when the expression is truthy. An <code>if</code> without a status function behaves as if <code>success()</code> && were in front of it	<code>success()</code>
<code>timeout-minutes</code>	job, step	the job or step is cancelled after this many minutes	360 for a job
<code>continue-on-error</code>	job, step	a failure does not fail the job (step level) or the run (job level)	<code>false</code>

Status functions change the implicit `success()`: `failure()` is true when an earlier step or needed job failed; `always()` is true even when the run was cancelled; `cancelled()` is true on cancellation. The expressions page advises against `always()` for anything that could fail critically and recommends `if: ${!cancelled() }` for "run whether or not the earlier steps passed".

`continue-on-error` at step level splits a step's result in two: `steps.<id>.outcome` is the result before the setting is applied and `steps.<id>.conclusion` the result after. A failed step with `continue-on-error: true` has outcome `failure` and conclusion `success` ([steps context](#)).

Two rules about skipping decide whether a pull request can merge, and they point in opposite directions:

- A **job** skipped by its `if` reports success for its check, so a required check is satisfied.
- A **workflow** that never starts, because of a path or branch filter or a skip instruction, reports nothing, and a required check stays pending.

Both are from [troubleshooting required status checks](#). They make `if` on a job a way to bypass a required check by accident, and `paths` on a workflow a way to block every unrelated pull request.

In production. Set `timeout-minutes` on every job. The default of 360 minutes is also the hard limit for a GitHub-hosted job ([limits](#)), so a test that hangs on a network call bills six hours unless you say otherwise. The course workflows use 5 to 20 minutes.

20A.10 Matrix strategies

In one sentence. A matrix turns one job definition into one job per combination of the values you list, each with its own runner and its own `matrix` context.

Precisely. `strategy.matrix` maps variable names to lists; the jobs are the Cartesian product ([matrix](#)).

- `exclude` removes combinations; a partial match is enough.
- `include` is processed after `exclude`. An entry is added to every existing combination it can extend without overwriting an original matrix value; if it would overwrite one, it becomes a new combination of its own.
- `fail-fast` defaults to `true`: when one matrix job fails, the in-progress and queued ones are cancelled. `max-parallel` caps how many run at once.
- A run may generate at most 256 matrix jobs.
- A job-level `if` is evaluated before the matrix is expanded, so it cannot read `matrix`.
- Outputs of matrix jobs are merged into one set and the last writer wins, in no guaranteed order.

Workflow 10 lists `os: [ubuntu-24.04, macos-15]`, `python: ["3.11", "3.12", "3.13"]` and `experimental: [false]`. That is six combinations. `exclude` removes macOS with 3.11, leaving five. The `include` entry (Ubuntu, 3.14, `experimental: true`) would overwrite `python` and `experimental` of any existing combination, so it is added as a sixth job. `continue-on-error: ${matrix.experimental}` is `true` only for that job.

Unverified. The names of the checks that matrix jobs report are not specified on an official page that the course's research could find. Workflow 10 therefore sets `name:` explicitly from the matrix values, and adds a final job with a fixed name to serve as the required check.

20A.11 Dependency caching

In one sentence. A cache stores a directory under a key so that a later run with the same key can restore it and skip the download; it is an optimization that may be absent, never a place to keep something you need.

Analogy. A cache is the shared fridge at work with labelled boxes. If a box with your label is there, lunch is quick. If it was cleared out, you cook again. You do not keep your passport in it, and you do not eat from a box whose label you did not write. The analogy holds for eviction and for trust; it breaks on mutability, because a cache entry can never be changed once saved.

Precisely. From the [dependency caching reference](#) and the [actions/cache README](#):

- **Key.** At most 512 characters, usually built from the operating system and `hashFiles` of a lock file. An existing key is never overwritten: "You cannot change the contents of an existing cache."
- **Restore order.** The exact `key`; then entries whose key starts with `key`; then each `restore-keys` prefix in order, the most recently created match winning.
- **Save.** On a miss, the cache is saved at the end of the job, and only if the job completes successfully.
- **cache-hit output.** `true` only for an exact match on the primary key; `false` when a restore key matched or nothing was restored.
- **Scope.** A run can restore caches created on its own branch or on the default branch; a pull request run can also restore from its base branch. Sibling branches cannot read each other's caches. A cache saved by a pull request run belongs to the merge ref and is restorable only by re-runs of that pull request.
- **Eviction.** Entries not accessed for 7 days are removed. A repository gets 10 GB without charge; above the limit the least recently used entries are deleted. Administrators can raise the limit, billed, since 20 November 2025 ([changelog](#)).
- **Trust.** Anyone who can open a pull request can read caches of the base branch, and cache contents are not signed. Since 26 June 2026, events that people without write access can cause get a read-only token for the default branch's caches, and since 10 September 2026 the `cache-mode` key (`read`, `write`, `write-only`, `none`) states the access explicitly ([read-only caches](#), [cache-mode](#)). Chapter 21A covers cache poisoning.

The explicit form, as the README shows it, for a tool whose download directory you know:

```
- uses: actions/cache@55cc8345863c7cc4c66a329aec7e433d2d1c52a9 # v6.1.0
  id: cache
  with:
    path: ~/.m2/repository
    key: maven-${{ runner.os }}-${{ hashFiles('java-service/pom.xml') }}
    restore-keys: |
      maven-${{ runner.os }}-
```

Most projects never write this block, because the setup actions contain it. `actions/setup-java` with `cache:maven` stores `~/.m2/repository` under a key that hashes `**/pom.xml` by default ([README](#)). `astral-sh/setup-uv` has `enable-cache`, default `auto`: enabled on GitHub-hosted runners except for release, tag push, `pull_request_target` and `workflow_run` events, and keyed on the files matched by `cache-dependency-glob` ([README](#)). Workflows 4 and 5 use these.

```
Observed behavior : a dependency was upgraded, CI still installs the old version from cache
Mechanism         : the key had no lock-file hash, or a broad restore key matched an old entry,
                   which was then extended and saved under the new key
Root cause        : caches are immutable per key and restore keys match by prefix
Correct fix       : put hashFiles of the lock file in the key; install from the lock file
                   (uv sync --locked) so that a stale cache cannot change what is installed
Prevention        : a manual prefix (v2-) to invalidate everything at once
```

In production. Cache package downloads and small test models, never multi-gigabyte checkpoints: 10 GB per repository is shared by all branches, and a model cache evicts the dependency caches that made CI fast.

20A.12 Artifacts

In one sentence. An artifact is a set of files that a job uploads to GitHub, where it is stored with the run, can be downloaded by later jobs or by people, and expires after a retention period.

Precisely. From the READMEs of [upload-artifact v7.0.1](#) and [download-artifact v8.0.1](#):

Property	Rule
Immutability	an artifact cannot be changed after upload. A second upload with the same name in one run fails unless <code>overwrite: true</code> , which replaces it
Availability	scoped to the job: downloadable by a later job of the same run as soon as the upload step ends
Retention	90 days by default; <code>retention-days</code> from 1 to 90, never above the repository or organization setting. Since 1 October 2026 the same setting also deletes checks, workflow runs and commit statuses (changelog)
Limits	500 artifacts per job; hidden files are excluded unless <code>include-hidden-files: true</code>
Permissions	zipped uploads do not keep file permissions (files become 644). Tar the files first if the executable bit matters
Outputs	<code>artifact-id</code> , <code>artifact-url</code> , <code>artifact-digest</code> (SHA-256)
Download	by name into <code>path</code> ; without <code>name</code> , all artifacts, each in a directory of its name. v8 fails on a digest mismatch (<code>digest-mismatch: error</code> is the default)
Since v7	<code>archive: false</code> uploads one file without zipping it

Artifact or cache? The [documentation's rule](#): a cache for files that are reused between runs and can be recreated (dependencies); an artifact for files a job produced that must be passed to another job or kept after the run (build outputs, test reports).

The reason to pass build outputs as an artifact, and not to rebuild in the deploy job, is the one a CTO cares about: **the files that were tested are the files that are deployed**. A rebuild in a second job resolves dependencies again, possibly to different versions.

In production. Upload test reports with `if: ${ failure() }` and a short retention. Never upload a directory that may contain credential files or a `.git` folder with persisted credentials: an artifact is a copy of those files that outlives the job.

20A.13 The eight workflows, line by line

The files are in `workflows/`. Each carries a header comment, declares `permissions` at the top, and pins every action to a full commit ID from `workflows/ACTION_PINS.md` with the version as a comment (Chapter 21A explains why a tag is not enough). They were parse-checked and assembled from documented syntax; the author did not execute them on GitHub. Line numbers below refer to the files. Lines that repeat in every file are explained once, in workflow 1.

Workflow 1: `01-tests.yml`, run the tests

```
name: Tests

on:
  push:
    branches: [main]
  pull_request:

# The job token may read the repository contents and nothing else.
permissions:
  contents: read

jobs:
  test:
    name: Unit tests
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    steps:
      - name: Check out the repository
        uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
```

```

    persist-credentials: false

- name: Show what was checked out
  run: |
    echo "event      : $GITHUB_EVENT_NAME"
    echo "GITHUB_REF : $GITHUB_REF"
    echo "GITHUB_SHA : $GITHUB_SHA"
    git log -1 --format='HEAD      : %H %s'
    git rev-parse --is-shallow-repository
    git status --short --branch

- name: Set up Python
  uses: actions/setup-python@5fda3b95a4ea91299a34e894583c3862153e4b97 # v7.0.0
  with:
    python-version: "3.13"

- name: Run the tests with the standard library
  env:
    PYTHONPATH: src
  run: python -m unittest discover -s tests -v

```

Lines	What they say	Why
10	<code>name</code> is what the Actions tab and the check show	without it the file path is shown
12 to 15	run on pushes to <code>main</code> and on pull request activity (opened, synchronize, reopened)	<code>branches: [main]</code> on <code>push</code> stops a second run for every push to a pull request branch, which <code>pull_request</code> already covers
18, 19	the job token may read repository contents; every other permission is <code>none</code>	least privilege, and independence from the repository's default setting
22 to 25	one job, ID <code>test</code> , display name <code>Unit tests</code> , on the fixed image <code>ubuntu-24.04</code> , at most 10 minutes	a fixed label does not move when <code>ubuntu-latest</code> migrates (section 20A.14)
27 to 30	step 1 is an action: fetch the event's commit into the workspace, and do not keep the token on disk	defaults of section 20A.8
32 to 39	step 2 is a shell script that prints the event, ref, commit, whether the clone is shallow, and the branch status	the first thing to read in any log: what was checked out
41 to 44	step 3 installs CPython 3.13 and puts it on <code>PATH</code> ; the version is quoted	section 20A.3
46 to 49	step 4 sets <code>PYTHONPATH</code> for this step only and runs the tests; a non-zero exit status fails the step, the job and the run	<code>env</code> at step level is the narrowest scope

On a push to `main`, step 2 prints `refs/heads/main`, the pushed commit, `true` for shallow, and a status line that names a branch. On a pull request it prints `refs/pull/N/merge`, the test merge commit, `true`, and `## HEAD (no branch)`, as in the transcript of section 20A.8. That prediction is from the documentation; Lab 26.1 has you confirm it.

Workflow 2: `02-lint.yml`, linting

```

- name: Install uv
  uses: astral-sh/setup-uv@c18668ad3cf93ea998bef934396af7bb5c839dc7 # v10.2.0
  with:
    python-version: "3.13"

- name: Install the project and the dev group from the lock file
  run: uv sync --locked

- name: Lint
  id: lint
  run: uv run ruff check .

```

```

# Runs even when the lint step failed, so one push reports both problems.
- name: Check formatting
  id: format
  if: ${{ !cancelled() }}
  run: uv run ruff format --check .

- name: Write the job summary
  if: ${{ !cancelled() }}
  env:
    LINT_OUTCOME: ${{ steps.lint.outcome }}
    FORMAT_OUTCOME: ${{ steps.format.outcome }}
  run: |
    {
      echo "### ruff"
      echo ""
      echo "| Check | Outcome |"
      echo "|---|---|"
      echo "| ruff check | $LINT_OUTCOME |"
      echo "| ruff format --check | $FORMAT_OUTCOME |"
    } >> "$GITHUB_STEP_SUMMARY"

```

- Lines 32 to 35: `astral-sh/setup-uv` installs `uv`. Its `python-version` input sets the variable `UV_PYTHON` for the rest of the job (README, "Python version"); `uv` then provides that interpreter when a command needs it.
- Lines 37, 38: `uv sync --locked` creates the virtual environment from `uv.lock` and fails if the lock file does not match `pyproject.toml` ([uv guide for GitHub Actions](#)). CI installs what was reviewed, or stops.
- Lines 40 to 48: two checks with IDs. The second has `if: ${{ !cancelled() }}`, which replaces the implicit `success()`, so formatting is checked even when linting failed. The job still fails, because a failed step fails the job unless `continue-on-error` is set.
- Lines 50 to 63: the summary step reads `steps.<id>.outcome` through `env` and appends a Markdown table to `$GITHUB_STEP_SUMMARY`. The braces group the `echo` commands so that one redirection covers all of them.

Workflow 3: `03-build.yml`, build the application

```

on:
  push:
    branches: [main]
    tags: ["v*"]
  workflow_dispatch:

permissions:
  contents: read

defaults:
  run:
    shell: bash

jobs:
  build:
    name: Build sdist and wheel
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    outputs:
      describe: ${{ steps.describe.outputs.describe }}
      files: ${{ steps.list.outputs.files }}
    steps:
      # All history and all tags: `git describe` needs the tags and the commits between them.
      - name: Check out the repository with full history
        uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          fetch-depth: 0
          persist-credentials: false

      - name: Describe the commit
        id: describe
        run: |

```

```
describe="$(git describe --tags --always)"
echo "describe=$describe" >> "$GITHUB_OUTPUT"
echo "Building $describe"
```

- Lines 13 to 17: pushes to `main`, pushes of tags matching `v*` (quoted, because `*` is YAML syntax at the start of a scalar), and manual runs. `branches` and `tags` under one `push` are alternatives: a push matches if it is a matching branch or a matching tag.
- Lines 22 to 24: every `run` step of the file uses the `shell: bash` template, so pipelines fail properly (section 20A.7).
- Lines 31 to 33: the job publishes two outputs, each defined as the output of a step.
- Lines 36 to 40: `fetch-depth: 0` fetches all history and tags, which line 45 needs (section 20A.8).
- Lines 42 to 47: the step has `id: describe`; line 46 appends `describe=<value>` to `$GITHUB_OUTPUT`.

The second job, `report` (lines 63 to 80), has `needs: build`, no checkout, and reads `needs.build.outputs.*` through `env`. It proves the point of section 20A.2: the only things that reached the second machine are two strings.

Workflow 4: `04-python-tests.yml`, Python tests with uv and caching

```
concurrency:
  group: ${github.workflow}-${github.ref}
  cancel-in-progress: true
# ...
- name: Install uv with its cache enabled
  id: setup-uv
  uses: astral-sh/setup-uv@c18668ad3cf93ea998bef934396af7bb5c839dc7 # v10.2.0
  with:
    python-version: "3.13"
    enable-cache: true
    cache-dependency-glob: uv.lock

- name: Report the cache result
  env:
    CACHE_HIT: ${steps.setup-uv.outputs.cache-hit}
  run: echo "uv cache hit - $CACHE_HIT"

- name: Install the project and the dev group from the lock file
  run: uv sync --locked

- name: Run the tests
  run: uv run pytest
```

- Lines 22 to 24: one run at a time per workflow and ref. A new push to the same branch or pull request cancels the run in progress. `github.ref` is `refs/pull/N/merge` for pull requests, so each pull request is its own group. Chapter 20B treats concurrency in full.
- Lines 37 to 43: `enable-cache: true` turns on the cache built into `setup-uv`, and `cache-dependency-glob: uv.lock` makes the lock file the only input to the key. A changed lock file gives a new key and a miss; any other change gives a hit.
- Lines 45 to 48: the action's `cache-hit` output, printed, so the log states which case occurred.
- Lines 50 to 54: install from the lock file, then run `pytest`, which collects the `unittest` classes.

Workflow 5: `05-java-tests.yml`, Java tests with Maven

```
on:
  push:
    branches: [main]
    paths:
      - "java-service/**"
      - ".github/workflows/05-java-tests.yml"
  pull_request:
```

```

paths:
  - "java-service/**"
  - ".github/workflows/05-java-tests.yml"
# ...
# Applies to `run` steps only. `uses` steps still resolve paths from the workspace root.
defaults:
  run:
    working-directory: java-service
# ...
- name: Set up the JDK and the Maven cache
  uses: actions/setup-java@de7274f081f381c8f8158605e0321c36c376e2e6 # v6.0.1
  with:
    distribution: temurin
    java-version: "21"
    cache: maven
    cache-dependency-path: java-service/pom.xml

- name: Build and test
  run: mvn -B verify

- name: Upload the test reports when the job failed
  if: ${ failure() }
  uses: actions/upload-artifact@043fb46d1a93c77aae656e7c1c64a875d1fc6a0a # v7.0.1
  with:
    name: surefire-reports
    path: java-service/target/surefire-reports/
    if-no-files-found: warn
    retention-days: 7

```

- Lines 14 to 23: the workflow starts only when a file under `java-service/` or the workflow file itself changed. Listing the workflow file means that an edit to the CI is tested by the CI. Do not make this workflow a required check (section 20A.16).
- Lines 34 to 36: `run` steps start in `java-service`. The comment on line 33 states the limit: `defaults.run` applies to `run` steps only, so the `path` on line 59 is written from the workspace root.
- Lines 43 to 49: `actions/setup-java` installs Temurin 21 (`distribution` is required) and restores `~/.m2/repository` from a cache keyed on the given `pom.xml`.
- Line 52: `-B` is Maven's batch mode: no interactive prompts and no color codes in the log.
- Lines 54 to 61: `if: ${ failure() }` runs the upload only when an earlier step failed. The reports are kept for 7 days.

The Maven module and the plugin versions in its `pom.xml` were not built by the author. The versions were confirmed to exist in Maven Central on 2 October 2026.

Workflow 6: `06-docker-image.yml`, build and push an image to ghcr.io

```

permissions:
  contents: read

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }

jobs:
  image:
    name: Build and push
    runs-on: ubuntu-24.04
    timeout-minutes: 20
    permissions:
      contents: read
      packages: write
# ...
- name: Log in to ghcr.io
  if: ${ github.event_name != 'pull_request' }
  uses: docker/login-action@dbcb813823bdd20940b903addbd779551569679f # v4.6.0
  with:

```

```

    registry: ${ env.REGISTRY }
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }

- name: Compute tags and labels
  id: meta
  uses: docker/metadata-action@dc802804100637a589fabce1cb79ff13a1411302 # v6.2.0
  with:
    images: ${ env.REGISTRY }/${ env.IMAGE_NAME }
    tags: |
      type=ref,event=branch
      type=ref,event=pr
      type=semver,pattern={{version}}
      type=sha

- name: Build, and push unless this is a pull request
  id: build
  uses: docker/build-push-action@c3c9e263c25d99ce0380d002d59b67737d91b0dc # v7.4.0
  with:
    context: .
    push: ${ github.event_name != 'pull_request' }
    tags: ${ steps.meta.outputs.tags }
    labels: ${ steps.meta.outputs.labels }
    cache-from: type=gha
    cache-to: type=gha,mode=max

```

- Lines 21, 22 and 33 to 35: the workflow default is read-only; the one job that pushes adds `packages: write`. A job-level `permissions` block replaces the workflow-level one for that job, so `contents: read` is repeated.
- Lines 24 to 26: workflow-level `env`. `github.repository` is `owner/name`; `docker/metadata-action` lowercases image names (README, "Image name and tag sanitization"), which a registry requires.
- Lines 46 to 52: log in with the job token. GitHub's guide recommends the token over a personal access token, and states that a package published this way inherits the visibility and permissions of the repository ([publishing with Actions](#)). The step is skipped on pull requests.
- Lines 54 to 63: tags are computed from Git facts. From the action's README: `type=ref,event=branch` gives the branch name (`main`); `type=ref,event=pr` gives `pr-N`; `type=semver,pattern={{version}}` gives `0.2.0` for a tag `v0.2.0`; `type=sha` gives `sha-` plus the short commit ID. With the default `flavor`, a semver tag also produces `latest`.
- Lines 65 to 74: `context: .` builds from the checked-out workspace. Without it the action builds from the Git context, which skips the checkout and ignores local file changes (README, "Git context"). `push` is an expression that is `false` on pull requests. `type=gha` stores BuildKit layers in the Actions cache ([Docker documentation](#)), subject to the scope and eviction rules of section 20A.11.

The Dockerfile was not built by the author. Chapter 20B returns to this file for delivery.

Workflow 7: `07-artifact.yml`, create an artifact and consume it

```

outputs:
  artifact-id: ${ steps.upload.outputs.artifact-id }
  artifact-digest: ${ steps.upload.outputs.artifact-digest }
# ...
- name: Upload the build outputs
  id: upload
  uses: actions/upload-artifact@043fb46d1a93c77aee656e7c1c64a875d1fc6a0a # v7.0.1
  with:
    name: inventory-api-dist
    path: dist/
    if-no-files-found: error
    retention-days: 7
# ...
deploy-simulated:
  name: Simulated deployment
  needs: build

```

```

runs-on: ubuntu-24.04
timeout-minutes: 5
# ...
- name: Download the build outputs
  uses: actions/download-artifact@3e5f45b2cfb9172054b4087a40e8e0b5a5461e7c # v8.0.1
  with:
    name: inventory-api-dist
    path: dist

- name: Show what arrived
  env:
    ARTIFACT_ID: ${ needs.build.outputs.artifact-id }
    ARTIFACT_DIGEST: ${ needs.build.outputs.artifact-digest }
  run: |
    echo "artifact ID      : $ARTIFACT_ID"
    echo "artifact digest : $ARTIFACT_DIGEST"
    ls dist

- name: Deploy (simulated)
  run: bash scripts/deploy.sh staging dist

```

- Lines 45 to 52: `if-no-files-found: error` turns an empty `dist/` into a failure at the upload, not a mystery at the download. The step's outputs become job outputs on lines 28 to 30.
- Lines 54 to 58: the second job waits for `build`. It is a new machine, so it checks out the repository again to get `scripts/deploy.sh`.
- Lines 66 to 70: download by name into `dist`. A single artifact is extracted directly into `path`. Version 8 of the action compares the digest and fails on a mismatch.
- Lines 81, 82: the simulated deployment lists exactly the downloaded files. The script changes nothing anywhere; Lab 26.7 shows its real output.

Workflow 10: `10-matrix.yml`, matrix testing

```

test:
  name: py${{ matrix.python }} on ${{ matrix.os }}
  runs-on: ${{ matrix.os }}
  timeout-minutes: 15
  continue-on-error: ${{ matrix.experimental }}
  strategy:
    fail-fast: false
    matrix:
      os: [ubuntu-24.04, macos-15]
      python: ["3.11", "3.12", "3.13"]
      experimental: [false]
      exclude:
        - os: macos-15
          python: "3.11"
      include:
        - os: ubuntu-24.04
          python: "3.14"
          experimental: true
# ...
all-tests:
  name: All matrix tests
  if: ${ always() }
  needs: test
  runs-on: ubuntu-24.04
  timeout-minutes: 5
  steps:
    - name: Check the result of the matrix
      env:
        RESULT: ${ needs.test.result }
      run: |
        echo "matrix result: $RESULT"
        test "$RESULT" = "success"

```

Section 20A.10 counted the jobs: six, of which one is experimental. Line 24 names each job from its matrix values. Line 25 chooses the runner per combination. Line 27 lets the Python 3.14 job fail without failing the run.

The last job is the pattern to remember. A ruleset that requires "All matrix tests" needs no edit when a Python version is added. `if: ${{ always() }}` makes it run even when a matrix job failed, because a skipped job would report success and satisfy the required check (section 20A.9). It then fails unless `needs.test.result` is `success`.

Unverified. How `needs.<job>.result` aggregates the results of a matrix whose only failure had `continue-on-error: true` is not spelled out on the pages read for this chapter. The expectation, from the definition of `continue-on-error`, is `success`. Lab 26.8 has you observe it.

20A.14 Action versions and the Node 24 runtime

Node 24 is the only JavaScript action runtime on github.com runners since 23 September 2026 ([changelog](#)). A JavaScript action declares its runtime in `action.yml` (`runs.using: node24`). Older majors that declare `node20` are not rejected: according to the runner's source they are executed on Node 24, a configuration their authors did not test ([runner source](#)). The rule for a workflow author is to use the Node 24 majors.

The versions this chapter's workflows pin, with the behavior change each brought (Phase 0 report, section 2, and the release notes linked there):

Action	Pinned	What changed that you must know
<code>actions/checkout</code>	v7.0.1	v6 moved the persisted credential under <code>\$RUNNER_TEMP</code> ; v7 refuses fork pull request checkouts under <code>pull_request_target</code> and <code>workflow_run</code>
<code>actions/setup-python</code>	v7.0.0	Node 24, ESM. The release notes and the README disagree about whether the <code>pip-install</code> input was removed; the course does not use it
<code>actions/setup-java</code>	v6.0.1	<code>distribution</code> stays required; downloaded JDKs are verified against vendor checksums
<code>actions/cache</code>	v6.1.0	handles read-only cache access; versions before v4.2.0 and v3.4.0 fail since the cache service was rewritten in February 2025
<code>actions/upload-artifact</code> , <code>actions/download-artifact</code>	v7.0.1, v8.0.1	immutable artifacts since v4; <code>archive: false</code> in upload v7; download v8 fails on a digest mismatch
<code>astral-sh/setup-uv</code>	v10.2.0	no floating major tag since v8, so <code>@v10</code> does not exist: pin a full version or a commit; cache disabled by default for <code>pull_request_target</code> , <code>workflow_run</code> and <code>release</code> since v10
<code>docker/login-action</code> , <code>setup-buildx-action</code> , <code>metadata-action</code> , <code>build-push-action</code>	v4.6.0, v4.4.1, v6.2.0, v7.4.0	the Node 24 majors, released March 2026

Runner images move too. On 1 October 2026 `ubuntu-latest` is Ubuntu 24.04, and it migrates to Ubuntu 26.04 between 19 October and 19 November 2026 ([changelog](#)). The course workflows say `ubuntu-24.04` and `macos-15`, so that a change of image is a commit you make and not a date you discover. Chapter 20B covers runners, labels and retirements.

20A.15 Syntax added in 2025 and 2026

The core model has not changed, but a file written this year can contain keys that no older course mentions ([workflow syntax](#); each row links its changelog entry):

Since	Addition	What it replaces
18 Sep 2025	YAML anchors and aliases (changelog)	copy and paste inside one file
4 Dec 2025	25 <code>workflow_dispatch</code> inputs, up from 10 (changelog)	
29 Jan 2026	the <code>case()</code> function (changelog)	the <code>a && b c</code> idiom
19 Mar 2026	a <code>timezone</code> next to <code>cron</code> ; <code>environment</code> with <code>deployment: false</code> (changelog)	UTC arithmetic in cron lines
2 Apr 2026	<code>entrypoint</code> and <code>command</code> for service containers (changelog)	custom service images
7 May 2026	<code>concurrency.queue: max</code> , up to 100 pending runs per group (changelog)	only one pending run
25 Jun 2026	parallel steps: <code>background</code> , <code>wait</code> , <code>wait-all</code> , <code>cancel</code> , <code>parallel</code> (changelog)	<code>&</code> and <code>wait</code> in shell
30 Jul 2026	<code>uses: \$/path</code> , the same repository at the running commit, without a checkout; github.com only, runner 2.336.0 or later (changelog)	<code>uses: ./path</code> after a checkout
3 Sep 2026	the <code>vulnerability-alerts</code> permission; <code>job.workflow_*</code> context fields (changelog)	
10 Sep 2026	<code>cache-mode</code> at workflow or job level (changelog)	trusting the trigger-based default

None of the eight workflows needs these keys, and the course did not add them for show. Know them so that you can read them. Parallel steps break one statement of section 20A.2 in a controlled way: steps with `background: true` do not run in order.

Unverified. A workflow-level dependency lock announced in GitHub's 2026 security roadmap does not appear in the syntax reference as of 1 October 2026 ([roadmap](#)). Do not write it.

20A.16 What can go wrong

Symptom	Diagnosis	Fix	Prevention
Passes locally, fails in CI on a pull request	the checkout log shows <code>refs/pull/N/merge</code> ; reproduce with <code>git fetch origin && git merge origin/main</code> on a detached copy	update the branch and fix the interaction	read the checked-out commit first; print it as workflow 1 does
Version is a bare commit ID, or <code>git describe</code> fails	<code>git rev-parse --is-shallow-repository</code> prints <code>true</code> ; <code>git tag</code> prints nothing	<code>fetch-depth: 0</code> in that job	decide per job whether it asks history a question
A required check never reports and the pull request cannot merge	the workflow has <code>paths</code> , <code>branches</code> or a skip instruction, so no run exists for this commit	remove the filter from required workflows, or require a job that always runs	require one aggregate job; filter inside jobs, not on the workflow

Symptom	Diagnosis	Fix	Prevention
The job is green but a command in a pipeline failed	no <code>shell:</code> key, so <code>bash -e</code> without <code>pipefail</code>	<code>shell: bash</code> , or <code>defaults.run.shell: bash</code>	set the default once per workflow
A step's output is empty in a later step	missing <code>id</code> , a typo in the name (no error for a missing property), <code>></code> instead of <code>>></code> , or a read from another job without a job output	fix the reference; add <code>outputs:</code> to the job	print outputs in the step that writes them
An <code>if</code> is always true	text outside <code>\${{ }}</code> made the value a string	wrap the whole expression	read the annotation that the run shows since January 2026
The old dependency version is installed	a restore key matched a stale cache	key on the lock-file hash	install from the lock file
The Python matrix runs 3.1	<code>3.10</code> was not quoted	quote it	quote every version
A secret-dependent step fails only for fork pull requests	secrets are not passed to fork runs; the value is an empty string	make the job work without the secret, or skip that step for forks	never switch to <code>pull_request_target</code> as a shortcut (Chapter 21A)

The investigation order for a failing run, and the debug-logging and re-run tools, are Chapter 20B.

20A.17 When not to use it, and dangerous edge cases

- **Do not put the only copy of a procedure in a workflow.** A build that exists only as YAML steps cannot be run on a laptop or on another CI system. Keep the commands in the repository (`uv run pytest`, `scripts/deploy.sh`) and let the workflow call them.
- **Do not use a cache as storage or an artifact as a registry.** Caches disappear after 7 days without access; artifacts after the retention period. A release belongs in a package registry or a GitHub Release.
- **Do not use `schedule` for anything that must happen at a given minute.** The documentation says runs can be delayed and, under high load, dropped.
- **Do not add `fetch-depth: 0` everywhere.** On a repository with a long history it turns a one-second fetch into a full clone for every job. Add it to the job that needs history.
- `continue-on-error` and `if: always()` **hide failures by design.** Each use should have a comment that says why. An aggregate job with `always()` must test the results explicitly, as workflow 10 does, or it turns red runs green.
- **A workflow file is code that runs with a token.** Anyone who can push a branch can change what `push` and `pull_request` workflows do on that branch. Protect `.github/workflows/` with CODEOWNERS and a ruleset (Chapter 19).

20A.18 Command safety

This chapter's own commands are local reads and additive fetches. The GitHub CLI commands are used in the labs; flags were checked with `gh <command> --help` of version 2.88.1.

Command	Label	What it changes	Preview	Recovery
<code>git clone --depth 1 --no-tags <url></code>	🟢 SAFE	creates a new shallow repository	none needed	delete the directory
<code>git fetch --unshallow --no-tags</code>	🟢 SAFE	adds objects and tag refs; removes <code>.git/shallow</code>	<code>git rev-parse --is-shallow-</code>	none needed

Command	Label	What it changes	Preview	Recovery
			repository	
git diff --name-only A...B	● SAFE	nothing		
gh run list, gh run view --log-failed, gh run watch, gh cache list	● SAFE	nothing on GitHub		
gh workflow run <file> --ref <branch>	● CAUTION	starts a run, which uses minutes and may deploy if the workflow deploys	read the workflow file at that ref	cancel with gh run cancel <run-id>
gh run rerun <run-id> --failed	● CAUTION	starts a new attempt on the original commit	gh run view <run-id>	cancel the attempt
git push origin v0.2.0	● CAUTION	creates a tag on GitHub; starts every workflow with a matching tag filter, here an image push	git ls-remote --tags origin	deleting the tag does not unpublish the image

No command in this chapter is ● DANGEROUS.

20A.19 Version notes

Version note. Older behavior: JavaScript actions ran on Node 16, then Node 20. Current behavior: Node 24 only. Since: 23 September 2026. Recommended: current majors of every action, pinned by commit ID.

Version note. Older behavior: actions/checkout wrote the job token into .git/config. Current behavior: stored in a separate file under \$RUNNER_TEMP. Since: v6 (20 November 2025). Recommended: persist-credentials: false unless a later step needs authenticated Git.

Version note. Older behavior: step outputs through ::set-output on standard output. Current behavior: append to \$GITHUB_OUTPUT. Since: 11 October 2022 (deprecation; the old form still works with a warning). Recommended: the file.

Version note. Older behavior: any trigger could write the default branch's caches. Current behavior: read-only for triggers that people without write access can cause; cache-mode to be explicit. Since: 26 June and 10 September 2026. Recommended: leave the default; do not add cache-mode: write to make a warning disappear.

Version note. Older behavior: checks, runs and statuses were kept for 400 days or more. Current behavior: they follow the artifact and log retention setting, 90 days by default, not retroactively. Since: 1 October 2026. The status checks page still says 400 days; the Phase 0 report records the conflict and follows the changelog.

20A.20 Practice

- Module 26 labs: Labs 26.1 to 26.8, one per workflow. Each has you add the file, predict when it runs and what it checks out, run it, read the log, then break it and diagnose.
- Replay any transcript of this chapter, for example `labs/run_ch20a/merge-ref` or `labs/run_ch20a/shallow-checkout`.
- Then read Chapter 20B: Delivery, runners, cost and debugging and Chapter 21A: GitHub Actions security.

20A.21 Interview questions

1. A pull request's CI fails and the author says "it passes on my machine, same commit". What do you check first, and why can both be right?
2. What does `actions/checkout` put on the runner by default? Name three tools or commands that give wrong results there, and say whether each fails or lies.
3. Explain the difference between `run: echo "${{ github.event.pull_request.title }}"` and passing the title through `env`. When is each evaluated, and by what?
4. A job is green although `pytest | tee report.txt` had failing tests. Give the exact shell command line the runner used, and two ways to fix it.
5. For each of `push`, `pull_request`, `schedule` and `workflow_dispatch`: which commit does the run refer to, and from which commit is the workflow file read?
6. A required check stays "expected" forever on documentation-only pull requests. Explain the mechanism and design a fix that keeps the path filter's saving.
7. How do you pass a value from a step to a later step, and from a job to a later job? What cannot be passed this way, and what do you use for it?
8. What is the difference between a cache and an artifact in purpose, scope, lifetime and trust? Which one may a release job consume?
9. A matrix has `os` with two values, `python` with three, one `exclude` and one `include`. How do you count the jobs, and how do you make one of them allowed to fail?
10. Why does this course pin `actions/checkout` by a 40-character commit ID, and what does the Node 24 change of September 2026 mean for a file that says `@v4`?
11. A teammate proposes `if: always()` on the deploy job "so it is not skipped". What happens, and what do you propose?
12. You inherit a repository with twelve workflow files. In what order do you read one of them to predict when it runs, where, with what code, and with what permissions?

20A.22 Sources

Primary sources (docs.github.com, the GitHub Changelog, and each action's repository at the pinned tag; read on 1 and 2 October 2026)

- Reference pages: [workflow syntax](#), [events that trigger workflows](#), [contexts](#), [expressions](#), [variables](#), [secrets](#), [workflow commands](#), [dependency caching](#), [limits](#), [GitHub-hosted runners](#), [reusing workflow configurations](#).
- Other documentation: [workflow artifacts](#), [GITHUB_TOKEN](#), [skipping workflow runs](#), [troubleshooting required status checks](#), [publishing a package with Actions](#).
- Action READMEs and `action.yml` files at the pinned versions: [checkout v7.0.1](#), [setup-python v7.0.0](#), [setup-java v6.0.1](#), [cache v6.1.0](#), [upload-artifact v7.0.1](#), [download-artifact v8.0.1](#), [setup-uv v10.2.0](#), [docker/login-action v4.6.0](#), [docker/metadata-action v6.2.0](#), [docker/build-push-action v7.4.0](#), [docker/setup-buildx-action v4.4.1](#).
- Tool documentation: [uv with GitHub Actions](#), [Docker: cache management with GitHub Actions](#).
- Changelog entries, linked where they are used.

- Git 2.55 manual pages: `git help clone`, `git help fetch`, `git help describe`, `git help diff`. GitHub CLI 2.88.1: `gh run --help`, `gh workflow --help`, `gh cache --help`.

Secondary sources

- The Phase 0 report of this course, sections 2, 3, 4, 12 and 13, and its notes on GitHub Actions, including the flags carried into this chapter as "Unverified".

Videos (from the Phase 0 report, with its caveats)

- ["Complete GitHub Actions Course - From BEGINNER to PRO"](#), Sid Palas, DevOps Directive, 3 h 43 min, 24 September 2025. The most complete current free course on workflow mechanics. It predates the Node 24-only runtime and `actions/checkout v7`, says "branch protections" and never mentions rulesets.
- ["GitHub Actions tutorial in Hindi"](#), CODERS NEVER QUIT, 1 h 51 min, 27 September 2024, Hindi. Jobs, expressions, checkout, caching and artifacts. Mostly current in concept; the action versions shown were not checked; no pinning.

Further reading

- Chapter 17: Pull Requests, sections 17.2, 17.6 and 17.11; Chapter 18: Branch Protection and Rulesets on required status checks; Chapter 19: CODEOWNERS on protecting the workflows directory; Chapter 26: Performance, section 26.11 on shallow clones; Chapter 14A: History investigation on two-dot and three-dot ranges.

Chapter 20B: GitHub Actions: delivery, runners, cost and debugging

Baseline. Git 2.55.0 and GitHub CLI 2.88.1 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch20b/`. Everything about GitHub Actions is described from the linked documentation: the workflow files of this chapter were parse-checked and assembled from documented syntax, but the author did not execute them on GitHub. You run them in the labs.

Chapter 20A taught the parts of a workflow and built eight of them. This chapter uses those parts to deliver software, and then teaches the skill a CTO pays for: finding out why a run failed, in a fixed order, with evidence. Security of workflows is Chapter 21A; it is referred to here and not repeated.

20B.1 Why this matters

Three questions reach a senior engineer every month.

"Who approved what is running in production, and which commit is it?" The answer is not in Git. It is in GitHub objects: an environment, its protection rules, a deployment record. If you cannot say which rules gate the production job and who can bypass them, you do not control your releases.

"The tests pass on my machine. Why is the pull request red?" Almost always because the job did not run what you ran: another commit, another shell, another tool version, no secrets, an empty history. Each cause is a documented default. An engineer who knows the defaults finds the cause in minutes; one who does not re-runs the job and hopes.

"Why did our Actions bill triple, and why does the merge button say it is waiting for a check that never comes?" Both are consequences of how runs are started, cancelled, billed and reported. Both are explainable from the documentation.

GitHub, not Git. Nothing in this chapter is a Git feature. Git supplies the commit, the ref and the tag. GitHub Actions decides when to run, on which machine, with which token, and GitHub decides what a "deployment" and a "check" are. Where Git explains a failure (a shallow history, a file-name case, a line ending), the chapter shows it with a local transcript.

20B.2 Environments

In one sentence. An environment is a named GitHub object that a job can reference, and that holds protection rules the job must pass before it starts and secrets and variables the job can read only after that.

Analogy. An environment is the locked door of a server room with its own key cabinet inside. The job is a technician. The door has rules: someone must sign the visitor in, the visitor must come from a known department, there may be a waiting period. The keys in the cabinet are only reachable after the door has opened. The analogy breaks in one place: the door is guarded by GitHub, not by the server room. If your cloud account also accepts credentials from somewhere else, the environment protects nothing.

Precisely. A job references an environment with `jobs.<job_id>.environment` , either as a name or as a mapping with `name` and `url` . The [environments reference](#) defines the rules:

Rule or content	What it does	Documented detail
Required reviewers	The job waits until a listed person or team approves	Up to six users or teams; one approval is enough; "prevent self-review" stops the person who started the run from approving it
Wait timer	The job waits a fixed time after it is triggered	1 to 43,200 minutes (30 days); waiting is not billed
Deployment branches and tags	Only runs on matching refs may deploy	"No restriction", "Protected branches only", or "Selected branches and tags"; patterns are matched against the ref, and <code>*</code> does not match <code>/</code>
Allow administrators to bypass	Lets a repository administrator force the deployment	On by default; can be switched off per environment
Custom deployment protection rules	A GitHub App decides (for example an observability product)	Public preview; at most six protection rules enabled per environment
Environment secrets	Secrets that exist only for jobs that reference the environment	Not readable before a required approval is given
Environment variables	Non-secret configuration in the <code>vars</code> context	Same scoping as the secrets

Two details decide how safe this is. First, the [workflow syntax](#) says all protection rules must pass before a job that references the environment is sent to a runner. The gate is in front of the machine, not a step inside the job that the job could skip. Second, [managing environments](#) says that a workflow that names an environment that does not exist creates it, without any rule, and that anyone who can edit workflows can do so, while only repository administrators can configure an environment. A typing error in the name (`production`) therefore gives you an unprotected environment, not an error.

Plan gates. These come straight from the same reference and are the first thing to check before you design a gate:

Feature	Public repository, any plan	Private repository on Free	Private on Pro or Team	Private on Enterprise
Required reviewers, wait timers, custom rules, disabling admin bypass	yes	no	no	yes
Deployment branch and tag rules	yes	no	yes	yes
Environment secrets	yes	no	yes	yes
Environment variables	yes	not stated for Free	yes	yes

Unverified. Whether a private repository on a Free plan can use environments at all is a conflict inside GitHub's own material. A [changelog entry of 15 May 2025](#) says environments are available on all plans in public and private repositories; the [managing environments](#) page on 1 October 2026 still says users on Free plans can configure environments only for public repositories. The Phase 0 report could not resolve it. The practice repository of the Actions labs, `YOUR-ORG/inventory-api` , is public, so the labs are not affected. For a private repository, test it before you promise a gate.

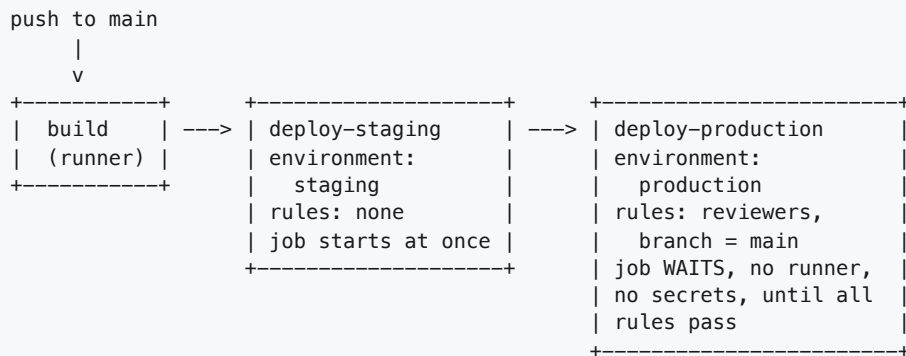
Inside `.git`. Nothing. An environment, its rules, its secrets and the deployment records it produces are GitHub objects. A clone of the repository contains none of them. The only trace in Git is the word after `environment:` in the workflow file. That is why a repository that is mirrored to another host loses its gates, and why "the workflow file says `environment: production` " proves nothing until you have read the environment's settings.

See it. You cannot see an environment with Git. What Git can show is the thing a deployment branch rule is matched against: the ref of the run. In this transcript a hotfix branch is checked out; the commands print the ref and test it against `refs/heads/main` the way a rule "Selected branches: `main`" would.

```
# An urgent fix on a branch. The rule on production says: selected branches, main.
$ git switch --quiet hotfix/lead-days
$ git symbolic-ref HEAD
refs/heads/hotfix/lead-days
$ test "$(git symbolic-ref HEAD)" = refs/heads/main
[exit status: 1]
$ git merge-base --is-ancestor HEAD main
[exit status: 1]
```

The first test fails because the ref is `refs/heads/hotfix/lead-days`. The second shows that the commit is not yet contained in `main` either. A branch rule looks at the first fact only: the name of the ref the run was started for. Section 20B.4 comes back to what that does and does not guarantee.

Picture.



In production. A team serving an LLM application keeps two cloud roles: one that can update the staging service and one that can update production. The production role's credentials are reachable only from the `production` environment, which requires one approval from the on-call group, prevents self-review and accepts only `main`. An engineer who edits a workflow on a feature branch to "quickly deploy" gets a job that fails the branch rule before any runner starts. The gate did its work without anybody reading the diff. With federated credentials the same idea is enforced by the cloud provider as well, because the token's subject names the environment; that is part of Chapter 21A.

Creating environments

The documented interface path is the repository's **Settings**, then **Environments**, then **New environment** ([managing environments](#); the interface changes, so trust the page in front of you). The same can be done with the REST API, which is what you want for infrastructure that is reviewed as code. The endpoint is `PUT /repos/{owner}/{repo}/environments/{environment_name}` and its body accepts `wait_timer`, `prevent_self_review`, `reviewers` (objects with `type` `User` or `Team` and a numeric `id`) and `deployment_branch_policy` (an object with the two booleans `protected_branches` and `custom_branch_policies`, which must have opposite values) ([REST: deployment environments](#)).

```
# Create (or update) an environment without rules
gh api -X PUT repos/YOUR-ORG/inventory-api/environments/staging

# Secrets and variables scoped to it (gh asks for the secret value; it is not put on the command
line)
gh secret set DEPLOY_TOKEN --env staging
gh variable set STAGING_URL --env staging --body "https://staging.example.com"

# Read back what exists
```

```
gh api repos/YOUR-ORG/inventory-api/environments --jq '.environments[].name'
gh secret list --env staging
```

`gh secret set without --body` reads the value from standard input or prompts for it, which keeps it out of your shell history (`gh secret set --help`). Lab 27.4 adds the reviewer and the branch rule to `production` with one `gh api` call and a JSON file.

20B.3 Deploying to staging: workflow 8

Read `workflows/08-deploy-staging.yml` beside this section. It has two jobs.

```
jobs:
  build:
    runs-on: ubuntu-24.04
    steps:
      # checkout, setup-uv, then:
      - run: uv sync --locked
      - run: uv run pytest
      - run: uv build
      - uses: actions/upload-artifact@043fb46d1a93c77aae656e7c1c64a875d1fc6a0a # v7.0.1
        with:
          name: inventory-api-dist
          path: dist/

  deploy-staging:
    needs: build
    runs-on: ubuntu-24.04
    environment:
      name: staging
      url: ${vars.STAGING_URL}
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
      - uses: actions/download-artifact@3e5f45b2cfb9172054b4087a40e8e0b5a5461e7c # v8.0.1
        with:
          name: inventory-api-dist
          path: dist/
      - run: bash scripts/deploy.sh staging
    env:
      DEPLOY_TOKEN: ${secrets.DEPLOY_TOKEN}
```

(The file itself has more: `timeouts`, `persist-credentials: false`, a job summary. This excerpt keeps what the argument needs.)

Four decisions are worth defending in a review.

Build once, deploy what was built. The deploy job does not run `uv build` again. It downloads the artifact the build job uploaded. Every job starts on a fresh machine, so the artifact is the only way the bytes travel, and it is also the point: the thing that was tested is the thing that is deployed. `actions/download-artifact v8` fails by default when the downloaded content does not match the digest recorded at upload ([README](#)). The same idea in plain Git: the same commit always produces the same archive bytes, and a digest detects any change.

```
$ git archive --format=tar --prefix=warehouse-api/ HEAD | shasum -a 256
28e6e91f0be5134ce817c81ab4dfb7e7e0ac69ac7cd756cb8859f27c4746d1e2 -
$ git archive --format=tar --prefix=warehouse-api/ HEAD | shasum -a 256
28e6e91f0be5134ce817c81ab4dfb7e7e0ac69ac7cd756cb8859f27c4746d1e2 -
$ git archive --format=tar --prefix=warehouse-api/ v1.1.0 | shasum -a 256
1581f1ac4facda70544419ab7165fd84af4c05ce715e387aa9c44d4a38eddd44 -
```

The first two digests are equal because `git archive` of one commit is deterministic; the third differs because `v1.1.0` is another commit. A build tool is not always this reproducible, which is the reason to move the built file between jobs and not to rebuild it.

The environment belongs to the job that deploys. Only `deploy-staging` names the environment. The build job cannot read `DEPLOY_TOKEN`, because an environment secret is "only available to workflow jobs that reference the environment". Test code and build scripts, including those of your dependencies, never run in a job that holds a deployment credential.

The secret reaches the script through `env`. The expression `${{ secrets.DEPLOY_TOKEN }}` is placed in the step's `env` mapping and the script reads `$DEPLOY_TOKEN`. It is never interpolated into the text of a `run` command. Chapter 21A explains why that difference is a security boundary.

`environment.url`. The URL is shown with the deployment on GitHub. It may be an expression; here it comes from an environment variable, so the workflow file contains no host name.

What changes when you push to `main` with this workflow installed:

Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
unchanged	unchanged	unchanged	unchanged	<code>refs/remotes/origin/main</code> moves to the pushed commit	<code>refs/heads/main</code> moves	a workflow run for the <code>push</code> event at the new commit; one artifact; one deployment record and status for <code>staging</code> ; a check on the commit for each job

Every job that references an environment creates a deployment object, unless `deployment: false` is set under `environment` (added on 19 March 2026; required reviewers and wait timers still apply, and it cannot be combined with custom protection rules) ([control deployments](#)). Use that form for a job that needs an environment's secrets but deploys nothing, for example an integration test against a staging database.

What a deployment contains. Before you approve or trigger a deployment, ask Git what it adds to what is running. If staging runs `v1.1.0` and the run is for the tip of `main`:

```
# Staging runs v1.1.0. A push to main is about to deploy the tip of main.
$ git log --oneline v1.1.0..main
57c8425 Add the lock file
197d992 Document the release runbook
$ git diff --stat v1.1.0 main
 docs/runbook.md | 4 ++++
 uv.lock          | 6 ++++++
 2 files changed, 10 insertions(+)
$ ./scripts/deploy.sh staging
would deploy 57c8425 to staging
```

Two commits and two files. This is the question "what am I about to ship?" answered with a two-dot range, and it is worth one line in every deployment's job summary.

20B.4 Promotion to production: workflow 9

`workflows/09-environments.yml` adds a third job.

```

deploy-production:
  needs: deploy-staging
  runs-on: ubuntu-24.04
  concurrency:
    group: production
    cancel-in-progress: false
  environment:
    name: production
    url: ${vars.DEPLOY_URL}
  steps:
    # checkout and download-artifact as in deploy-staging, then:
    - run: bash scripts/deploy.sh production
      env:
        DEPLOY_TOKEN: ${secrets.DEPLOY_TOKEN}

```

The two deploy jobs are textually almost the same. The difference is on GitHub: `staging` and `production` each have a variable `DEPLOY_URL` and a secret `DEPLOY_TOKEN` with different values, and only `production` has rules. That is the pattern to aim for: the workflow describes the procedure, the environments hold what differs.

What the reviewer sees. From the documentation ([reviewing deployments](#)): the run shows the job as waiting; a required reviewer opens the run, chooses **Review deployments**, selects the environment and chooses **Approve and deploy** or **Reject**. A rejection fails the workflow. A job that nobody approves fails automatically after 30 days. Nothing in this was captured for the book; the labels are GitHub's and may change.

What an approval releases. The reviewer approves a job of a run, and a run is bound to one commit. The useful habit is to look at the range between what production has and what the run carries. In this transcript two refs under `refs/deployed/` stand in for GitHub's deployment records. They are the author's bookkeeping for the demonstration; GitHub keeps deployments as platform objects, not as refs.

```

$ git for-each-ref --format="%(refname) %(objectname:short)" refs/deployed
refs/deployed/production c4b5de2
refs/deployed/staging 57c8425
# What an approval of the production job would release:
$ git log --oneline refs/deployed/production..refs/deployed/staging
57c8425 Add the lock file
197d992 Document the release runbook

```

What the branch rule guarantees. A deployment branch rule compares the run's ref with name patterns. Since 8 December 2025, for runs triggered by `pull_request` events the ref that is evaluated is `refs/pull/<number>/merge`, and for `pull_request_target` it is the default branch ([changelog](#)). A rule "Selected branches: `main`" therefore rejects pull request runs and runs on other branches. It says nothing about review of the commits on `main`; that is the job of the ruleset on `main` (Chapter 18). The gate is the combination: a ruleset that forces changes to `main` through reviewed pull requests, plus an environment that accepts only `main`, plus a required reviewer. Remove any one and there is a path around the other two.

```

Observed behavior : A job that names "production" started without waiting for anybody.
Git state          : Irrelevant. The workflow file names the environment correctly.
Mechanism          : Protection rules are properties of the environment object on GitHub. The
                    : environment had been created by the first run that named it, with no rules;
                    : or the repository is private on a plan where reviewers do not apply.
Root cause         : The gate was assumed from the YAML and never configured or verified.
Why GitHub does it: Naming a missing environment creates it, so that a first deployment works
                    : without an administrator; rules are an administrator's decision.
Correct fix        : Configure the rules; read them back with
                    : gh api repos/OWNER/REPO/environments/production.
Prevention         : Create environments before the workflow that uses them; keep their
                    : configuration in a reviewed script; test the gate with a harmless run.

```

20B.5 Concurrency groups

In one sentence. A concurrency group is a name; GitHub lets at most one run or job with that name execute at a time and decides what happens to the others.

Analogy. A single-track railway section with one waiting siding. One train is on the track. A second train waits in the siding. If a third arrives, the second is sent away and the third takes the siding. With `cancel-in-progress: true` the arriving train also removes the one on the track. The analogy breaks with `queue: max`, which turns the siding into a yard for up to 100 trains.

Precisely. From the [workflow syntax](#):

- `concurrency` can be set for the whole workflow or for one job. Its value is a group name, or a mapping with `group`, `cancel-in-progress` and `queue`.
- Default behavior: when a run is queued and another run in the same group is in progress, the new one is pending, and **any run already pending in that group is cancelled**. So by default at most one runs and one waits.
- `cancel-in-progress: true` also cancels the run in progress. It may be an expression.
- `queue: max` (since 7 May 2026) keeps up to 100 pending runs and processes them in order; combining it with `cancel-in-progress: true` is a validation error ([changelog](#)).
- The group expression may use only the `github`, `inputs` and `vars` contexts at workflow level (a job-level group may also use `needs`, `strategy` and `matrix`).
- Group names are case-insensitive, and a group is not private to a workflow: two workflows that use the group name `deploy` share one track.
- The documentation states that ordering is not guaranteed.

Two patterns cover most needs, and they want opposite settings.

```
# Continuous integration: only the newest commit of a branch or pull request matters.
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

```
# Deployment: never interrupt one, never run two at once.
concurrency:
  group: production
  cancel-in-progress: false
```

The first includes the workflow name and the ref, so that only runs of the same workflow on the same ref compete. The second is deliberately the same for every ref. Workflow 8 uses the second form for the whole workflow; workflow 9 uses one group per environment at job level, so a staging deployment does not wait for a production approval.

Inside `.git`. Nothing changes. But the ref is the usual ingredient of a group name, and the refs of a repository are what make groups distinct:

```
# Workflow 5. Each of these refs gets pushes; a concurrency group should tell them apart:
$ git for-each-ref --format="%(%refname)" refs/heads
refs/heads/docs/rollback-steps
refs/heads/feature/safety-stock
refs/heads/main
```

A group that contains `github.ref` gives each of these three its own track. A group named `ci` gives all of them one.

Picture. Default behavior, three pushes in quick succession into one group:

```
time --->
run 1 [===== in progress =====]
run 2   [ pending ]--X  cancelled when run 3 is queued
run 3   [ pending .....][=== in progress ===]
```

In production. The default has a consequence for deployments that surprises teams: with three merges in ten minutes, the middle one is never deployed on its own. That is usually fine, because the third run contains the second merge. It is not fine when each run must happen, for example a database migration per commit. Then use `queue: max`, or design the deployment to be cumulative. The second trap is in reusable workflows: a called workflow sees the caller's name in `github.workflow`, so the same `${{ github.workflow }}-${{ github.ref }}` group with `cancel-in-progress: true` in caller and called workflow makes the called job cancel its own caller ([reusing workflow configurations](#)). The documentation also notes that `concurrency` and `environment` are not connected: an environment does not serialise anything by itself.

20B.6 Reuse: reusable workflows, composite actions, container actions

Three mechanisms remove duplication. They differ in what they replace.

	Reusable workflow	Composite action	Docker container action
Replaces	one or more whole jobs	several steps inside a job	one step
Defined in	a workflow file with <code>on: workflow_call</code>	<code>action.yml</code> with <code>runs.using: "composite"</code>	<code>action.yml</code> with <code>runs.using: "docker"</code>
Called with	<code>jobs.<job_id>.uses</code>	<code>steps[*].uses</code>	<code>steps[*].uses</code>
Chooses the runner	yes, each of its jobs has <code>runs-on</code>	no, runs on the caller's runner	no; needs a Linux runner
Can name an environment	yes	no	no
Secrets	declared under <code>on.workflow_call.secrets</code> , or <code>secrets: inherit</code>	cannot read the <code>secrets</code> context; pass values as inputs	receives inputs and <code>env</code>
Shows in the log as	separate jobs	one step	one step

Sources: [reusing workflow configurations](#), [custom actions](#), [metadata syntax](#). A JavaScript action is the fourth kind; since 23 September 2026 it runs on Node 24 only, and Chapter 20A covers it.

Reusable workflows: workflow 11

In one sentence. A reusable workflow is a workflow file that another workflow calls as if it were a single job.

Analogy. A subcontractor who brings a whole crew, their own tools and their own site rules, and works to a written order (the inputs). You cannot give the crew individual instructions; you can only place the order and read the delivery note (the outputs). The analogy breaks on trust: the subcontractor works with your access badge, and can use less of its access than you gave, never more.

Precisely. `workflows/11-reusable-workflow.yml` declares its interface:

```
on:
  workflow_call:
    inputs:
      environment:
        required: true
```

```

    type: string
  python-version:
    required: false
    type: string
    default: "3.13"
  secrets:
    deploy-token:
      required: false
  outputs:
    revision:
      description: Abbreviated ID of the commit that was deployed
      value: ${ jobs.deploy.outputs.revision }

```

and `workflows/11-caller.yml` uses it:

```

jobs:
  staging:
    uses: ../github/workflows/11-reusable-workflow.yml
    with:
      environment: staging
    secrets:
      deploy-token: ${ secrets.SHARED_DEPLOY_TOKEN }

  report:
    needs: staging
    runs-on: ubuntu-24.04
    steps:
      - run: echo "Staging now runs revision $REVISION" >> "$GITHUB_STEP_SUMMARY"
        env:
          REVISION: ${ needs.staging.outputs.revision }

```

The rules, all from the two documentation pages linked above and the [how-to](#):

- **Inputs** have a `type` of `boolean`, `number` or `string`. **Outputs** are declared at workflow level and mapped from job outputs; the caller reads them as `needs.<calling job>.outputs.<name>`.
- **A calling job is not a normal job.** It may contain only `name`, `uses`, `with`, `secrets`, `strategy`, `needs`, `if`, `concurrency`, `permissions` and `cache-mode`. It has no `runs-on` and no `steps`. No context or expression is allowed in `uses`.
- **Where the file comes from.** `../github/workflows/file.yml` is the same repository at the commit of the run. `owner/repo/..github/workflows/file.yml@ref` is another repository; pin `ref` to a full commit ID for the reason every action is pinned. Since 30 July 2026 the documentation recommends `$/github/workflows/file.yml` for the same repository on github.com; it needs runner 2.336.0 or later and does not exist on GitHub Enterprise Server ([changelog](#)). Files in subdirectories of `../github/workflows` cannot be called.
- **Secrets** pass only one level: a workflow called by a called workflow needs them passed again. `secrets: inherit` passes all of the caller's secrets and works within one organization or enterprise. Environment secrets cannot be passed by the caller, "as `on.workflow_call` does not support the `environment` keyword": the called job names the environment, as `deploy` does with `environment: ${ jobs.inputs.environment }`.
- `env` **does not cross**. Variables in the caller's workflow-level `env` are not visible in the called workflow. Use inputs, or configuration variables in `vars`.
- **Permissions only narrow.** The called workflow's `GITHUB_TOKEN` permissions can be reduced, not raised, relative to the caller's.
- **The github context is the caller's.** `Event`, `ref`, `commit` and `github.workflow` are those of the calling workflow. Billing is the caller's too.
- **Limits.** Up to ten levels of workflows (the top-level caller and nine below it) and at most 50 unique reusable workflows called from one workflow file, since 6 November 2025 ([changelog](#)).
- **Check names.** A required status check for a job in a called workflow is named `<calling job name> / <called job name>` (Chapter 18, section 18.8). With the two files above, the check is `Staging / Test`,

`build` and `deploy`. Renaming either job silently breaks a rule that requires the old name.

Unverified. How secrets of the environment that a called job names appear in that job when the caller passes named secrets and does not use `secrets: inherit` is not spelled out on the pages read for this chapter. Workflow 11 avoids the question: the caller passes a repository secret, and the called file documents it. Verify the behavior in your repository before you depend on it.

Inside `.git`. Both files are ordinary blobs in the commit. With the `./` form, the run uses the called file from the same commit as the caller. So a branch that changes the called file tests its own version, and `main` keeps using the old one until the merge:

```
$ git ls-files .github/workflows
.github/workflows/deploy.yml
.github/workflows/reusable-deploy.yml
$ git grep -n "uses:" main -- .github/workflows/deploy.yml
main:.github/workflows/deploy.yml:9:   uses: ../github/workflows/reusable-deploy.yml
$ git diff --stat main ci/python-version
.github/workflows/reusable-deploy.yml | 4 +++
1 file changed, 4 insertions(+)
$ git grep -c "python-version" main ci/python-version -- .github/workflows/reusable-deploy.yml
ci/python-version:.github/workflows/reusable-deploy.yml:1
```

`git grep -c` finds the new input only on the branch. A caller on `main` that already passes `python-version` would be passing an input that the called file on `main` does not declare. Caller and called file must agree in one commit when they live in one repository, and across a pinned ref when they do not.

Re-runs. Re-running all jobs resolves a reference that is not a commit ID again; re-running failed or selected jobs uses the same commit of the called workflow as the first attempt ([reference](#)). A reusable workflow referenced as `@main` can therefore differ between a first run and a full re-run.

In production. A platform team owns one deploy workflow in a central repository. Forty service repositories call it at a pinned commit and pass an environment name. A change to the deployment procedure is one reviewed pull request, rolled out by Dependabot updates of the pin. That one file then runs with deployment credentials in forty repositories, so its repository needs the strictest ruleset in the organization.

Composite actions

A composite action packages steps. A minimal `action.yml`, placed for example in `.github/actions/setup-project/action.yml`:

```
name: Set up the project
description: Install uv and the locked dependencies
inputs:
  python-version:
    description: Python version to use
    required: false
    default: "3.13"
runs:
  using: "composite"
  steps:
    - uses: astral-sh/setup-uv@c18668ad3cf93ea998bef934396af7bb5c839dc7 # v10.2.0
      with:
        python-version: ${ inputs.python-version }
    - run: uv sync --locked
      shell: bash
```

A job uses it after a checkout with `uses: ../github/actions/setup-project`. The documented rules that differ from workflow steps: every `run` step must state its `shell`; inputs are read as `${ inputs.name }`; outputs

need a `value` that maps to a step output; the `secrets` context is not available, so a secret must arrive as an input; and the parallel-step keywords of June 2026 cannot be used inside a composite action. In the log the whole action is one step, which makes a failure inside it harder to locate than the same steps written out.

Docker container actions

```
name: Check the migration files
description: Run the migration linter in its own image
inputs:
  directory:
    description: Directory that holds the migrations
    required: true
runs:
  using: "docker"
  image: "Dockerfile"
  args:
    - "${{ inputs.directory }}"
```

The runner builds the image from the `Dockerfile` beside `action.yml` and runs it with the arguments; inputs also arrive as environment variables named `INPUT_<NAME>`. It needs a Linux runner, and the documentation notes it is slower than the other kinds because the image is built or pulled first. Use it when the tool needs an operating system environment you do not want on the runner; otherwise prefer a composite action.

Which one. Steps that repeat inside jobs: composite action. A whole job, or a job that needs an environment, a runner choice or secrets: reusable workflow. Neither, when the duplication is two short blocks in one file: YAML anchors (supported since 18 September 2025) or plain repetition are easier to read and to debug.

20B.7 Publishing a container image, and release automation in outline

The image. Workflow 6 from Chapter 20A, `workflows/06-docker-image.yml`, builds and pushes an image to the GitHub container registry at `ghcr.io`. The delivery-relevant facts, from [publishing Docker images](#) and the Phase 0 report:

- The job authenticates with the `GITHUB_TOKEN` and needs `packages: write`; the documentation recommends the token over a personal access token. Publishing from a workflow with that token is also, in the documentation's words, the easiest way to connect the package to the repository.
- `docker/metadata-action` derives tags and labels from the Git ref and commit; `docker/build-push-action` builds and, with `push: true`, pushes. By default that action builds from the Git context, not from the checked-out directory, so files changed by earlier steps are ignored unless `context: .` is set ([README](#)).
- An image is identified by its digest, not by a tag. A tag such as `latest` or `main` is a movable name, exactly like a branch. Deploy by digest, and record the digest and the commit together. `actions/attest` can attach signed build provenance to the digest (Chapter 21B).

An image tag should identify one commit. Git can tell you whether a working tree is exactly a commit and what that commit is called relative to the release tags:

```
# A build from a working tree that differs from the commit it claims to be:
$ printf '\n# local tweak\n' >> src/warehouse/rules.py
$ git status --short
M src/warehouse/rules.py
$ git describe --tags --match 'v*' --dirty
v1.1.0-2-g57c8425-dirty
```

`-dirty` is Git saying that the working tree differs from HEAD. A runner's checkout is clean, which is one reason to build release images in CI and not on a laptop. The sample project's `Dockerfile` could not be built while this book

was written (no image pulls); Lab 27.1 has you run it.

Releases, in outline. A release pipeline is three facts and one trap.

- A release is a GitHub object on a Git tag (Chapter 15). In a job, `gh release create "$TAG" dist/* --verify-tag --generate-notes` with `GH_TOKEN` set and `contents: write` creates it; `--verify-tag` refuses to invent a tag that does not exist ([using the GitHub CLI in workflows](#), `gh release create --help`).
- With immutable releases, create a draft, attach every asset, then publish ([immutable releases](#)).
- Tools such as [release-please](#) derive the release from conventional commit messages. It is named as a concept and was not run for this book.
- **The trap.** Events caused by the `GITHUB_TOKEN` do not start new workflow runs, with narrow exceptions ([GITHUB_TOKEN](#)). A workflow that creates a tag or a release with the job token will not trigger your separate `on: release` or `tag-push` workflow. Either chain the work as jobs of one workflow with `needs`, or create the tag with a GitHub App token.

A release job that derives the version from a tag also needs the tags, which leads to section 20B.12.

20B.8 GitHub-hosted runners

In one sentence. A GitHub-hosted runner is a fresh virtual machine, built from a published image, that runs one job and is then destroyed.

Precisely. `runs-on` selects it by label. The facts below are from the [hosted runners reference](#) and the [runner-images repository](#) as recorded in the Phase 0 report.

Label on 1 October 2026	Image	Change in 2026
<code>ubuntu-latest</code>	Ubuntu 24.04	migrates to Ubuntu 26.04 between 19 October and 19 November 2026 (changelog)
<code>windows-latest</code>	Windows Server 2025 with Visual Studio 2026	Visual Studio 2026 since June 2026
<code>macos-latest</code>	macOS 26 on arm64	moved from macOS 15 in June and July 2026
<code>ubuntu-24.04</code> , <code>ubuntu-26.04</code> , <code>windows-2022</code> , <code>macos-15</code> , <code>macos-26</code>	the named version	fixed labels
<code>ubuntu-22.04</code>	Ubuntu 22.04	deprecated since 17 September 2026, unsupported from 17 April 2027 (announcement)
<code>macos-14</code>	macOS 14	unsupported from 2 November 2026, with brownouts in October (announcement)
<code>ubuntu-24.04-arm</code> , <code>windows-11-arm</code>	arm64	usable in private repositories since 29 January 2026
<code>ubuntu-slim</code>	one CPU, runs the job in a container	jobs limited to 15 minutes

`ubuntu-20.04`, `windows-2019` and `macos-13` no longer exist.

A `-latest` label is a moving name, the runner equivalent of a branch. A migration is rolled out gradually over weeks, so two runs of one workflow on the same day can get different images. Images are also rebuilt weekly, so tool versions inside a fixed label move too: in May 2026 Node 20 left the images and the default `node` became 22 ([announcement](#)). The workflows of this course therefore use `ubuntu-24.04` and install their tools with `setup` actions at stated versions. The cost of pinning is that you must move the label yourself before the image retires; put the retirement dates in your calendar. The "Set up job" section of a run's log names the image the job received; the Phase 0 notes did not verify that section separately, so read it in your own run.

Hardware. Standard Linux and Windows runners have 4 CPUs and 16 GB of memory in public repositories and 2 CPUs and 8 GB in private ones, with 14 GB of disk in both; macOS arm64 runners have 3 CPUs and 7 GB. A test suite that fits in a public repository can run out of memory or time after the repository is made private. Linux and macOS runners allow `sudo` without a password.

In production. A model-evaluation job that fits in the team's public mirror is killed in the private repository, with identical code. Ask first for the visibility of the repository, then for the label. GPU and larger runners exist for this; they are billed per minute on Team and Enterprise plans and are never covered by included minutes ([larger runners](#)).

20B.9 Self-hosted runners

In one sentence. A self-hosted runner is the same runner program on a machine you operate, which asks GitHub for jobs and runs them with whatever access that machine has.

Analogy. Lending your workshop to anyone who holds a work order. A GitHub-hosted runner is a rented workshop that is demolished after each job. Yours stays: tools, leftovers and everything a previous visitor hid there. The analogy breaks in that you choose who may write work orders, and that choice is the whole security question.

Precisely.

- **Risk.** GitHub's [secure use reference](#) says self-hosted runners "should almost never be used for public repositories", because anyone can open a pull request that runs code on them, and it extends the warning to private repositories where anyone with read access can fork and open a pull request. A hosted runner is a clean machine per job; a persistent self-hosted runner "can be persistently compromised by untrusted code in a workflow". On a self-hosted runner an environment does not isolate secrets from other jobs on the same machine.
- **Ephemeral runners.** Registering with `./config.sh --ephemeral` gives a runner that takes one job and is removed. Just-in-time runners are created through the REST API and also run at most one job. GitHub recommends autoscaling with ephemeral runners and advises against autoscaling persistent ones ([self-hosted runners reference](#)). The documentation adds that reusing hardware for just-in-time runners can still expose information from the environment: "ephemeral" must include the disk.
- **Actions Runner Controller** is the Kubernetes operator GitHub documents as the reference implementation for scale sets of ephemeral runners ([concept](#)).
- **Runner groups** restrict which repositories may send jobs to which runners, and a job selects one with `runs-on: { group: name }`. They are available to organizations on every plan since 17 October 2024 ([changelog](#)). The risk is a group open to "all repositories": every repository in the organization, including the least reviewed one, can run code on the machines that can reach production.
- **Versions.** Since 29 September 2026 a runner older than 2.329.0 cannot register on github.com, and a runner must install each new release within 30 days or it stops receiving jobs ([changelog](#)). This bites fleets built from a fixed image with updates disabled.
- **Queueing.** A job waits until a matching runner is online and fails after 24 hours in the queue; a self-hosted job may run for up to five days ([limits](#)).

In production. An ML team runs GPU evaluation on its own machines because hosted GPU minutes are expensive. The defensible design: ephemeral runners in a runner group that only the evaluation repository may use; workflows on those runners triggered only by `push` to protected branches and by `workflow_dispatch`, never by `pull_request` from forks; no long-lived cloud credentials on the machine.

20B.10 Limits and billing

Limits that shape designs, from [Actions limits](#) unless linked otherwise:

Limit	Value
Job on a GitHub-hosted runner	6 hours (<code>timeout-minutes</code> defaults to 360)
Workflow run, including waiting for approval	35 days
Environment approval	fails after 30 days without approval
Matrix	256 jobs per run
Re-runs	50 per workflow run, within 30 days of the first run (changelog)
Reusable workflows	10 levels; 50 unique called workflows per file
Concurrent jobs on standard runners	Free 20, Pro 40, Team 60, Enterprise 500; macOS 5 (50 on Enterprise)
Cache	10 GB per repository without charge; entries unused for 7 days are evicted
Artifacts and logs	90 days by default; since 1 October 2026 the same setting also removes checks, runs and statuses (changelog)

Billing, from [GitHub Actions billing](#) and [runner pricing](#):

- Standard hosted runners are free in public repositories. Self-hosted runners are free.
- Private repositories include 2,000 minutes per month on Free, 3,000 on Pro and Team, 50,000 on Enterprise Cloud, and 500 MB, 1 GB, 2 GB and 50 GB of artifact storage, which is shared with GitHub Packages.
- List prices per minute since 1 January 2026: Linux 2-core \$0.006, Linux arm64 2-core \$0.005, `ubuntu-slim` \$0.002, Windows 2-core \$0.010, macOS \$0.062. Each job is rounded up to a whole minute.
- Storage beyond the allowance: artifacts \$0.25 and cache \$0.07 per GB per month.
- Larger runners are always billed, in public repositories too.
- A reusable workflow is billed to the caller. Time spent in a wait timer is not billed.

Unverified. How Windows and macOS minutes consume the *included* minutes is not stated on any 2026 page the Phase 0 research could fetch. Older material gives multipliers of 2 for Windows and 10 for macOS; the old multiplier page now redirects to the price list. Do not quote a multiplier. Read your own usage in the billing settings.

Version note. Older behavior: tutorials quote pre-2026 prices. Current behavior: prices were cut by up to 39% on 1 January 2026. Announced and not in effect: a charge of \$0.002 per minute for self-hosted runners, announced on 16 December 2025 for 1 March 2026 and then postponed without a new date ([announcement and postponement](#)). Recommended: check the price page before you write a cost estimate.

The cost levers follow from the rules: cancel superseded CI runs with a concurrency group; prefer one job with several steps over many one-step jobs, because each job rounds up; keep macOS for what needs macOS, at roughly ten times the Linux price; use `ubuntu-slim` for glue jobs; shorten artifact retention. Path filters also save minutes, and section 20B.13 shows what they cost you.

20B.11 The investigation order for a failing workflow

A failing run tempts you to open the red step and start reading. Do that last but three. The order below goes from "did the right thing start at all" to "what did it say", because an answer early in the list makes everything after it irrelevant.

#	Question	What to look at
1	Workflow. Which workflow file, from which commit, defined this run?	<code>gh run view <id> --json workflowName,headSha,headBranch,event</code> ; <code>gh workflow view <file> --yaml --ref <branch></code> . <code>schedule</code> , <code>issue_comment</code> and <code>workflow_run</code> use the file on the default branch. <code>workflow_dispatch</code> can be triggered only if the file exists on the default branch, and the run is dispatched against the branch or tag you choose (Chapter 20A, section 20A.4)
2	Event. What triggered it, and which ref and commit did the job check out?	<code>push</code> builds the pushed commit; <code>pull_request</code> builds <code>refs/pull/N/merge</code> ; a re-run reuses the original commit and ref
3	Permissions. What could the token do?	top-level and job-level <code>permissions</code> ; unlisted scopes are <code>none</code> ; fork and Dependabot runs get a read-only token
4	Runner. Which image, which size?	<code>runs-on</code> , the image named at the top of the job log, public or private repository
5	Environment. Did the job reference one, and did its rules pass?	waiting, rejected, wrong branch, or an environment created by accident
6	Dependencies. Were the same versions installed as locally?	lock file, <code>--locked</code> , versions printed by setup steps
7	Secrets. Were they present?	an unset or withheld secret is an empty string, not an error
8	Action versions. Which commit of each action ran?	the pins; an old major on Node 24; a moved tag
9	Logs. What did the failed step print?	<code>gh run view <id> --log-failed</code>
10	Artifacts. Were the expected files produced and passed on?	<code>gh run download <id></code> ; names; expiry
11	Cache. What was restored, under which key?	the cache step's log; <code>gh cache list --key <prefix></code>
12	Concurrency. Was the run cancelled or replaced by another?	conclusion <code>cancelled</code> ; the group name; <code>gh run list --workflow <file></code>

Steps 1 and 2 remove the most confusion for the least effort. `gh run view` prints the event, the branch and the commit ID; compare that ID with `git rev-parse HEAD` on your machine before you compare anything else. The method is applied to a constructed case in Lab 28.2 and summarised as a runbook in the GitHub Actions guide.

20B.12 "Passes locally, fails on GitHub Actions"

The documented causes, from section 13 of the Phase 0 report. All rows are GitHub Actions behavior except the last two, which are Git.

Cause	Mechanism	Remedy
Shallow, tagless clone	<code>actions/checkout</code> fetches one commit and no tags, so <code>git describe</code> and tag-derived versions fail (README)	<code>fetch-depth: 0</code>

Cause	Mechanism	Remedy
The job is not testing the pushed commit	On <code>pull_request</code> the job checks out <code>refs/pull/N/merge</code> , a merge of the head into the current base, in detached HEAD; it does not run at all while the pull request conflicts (events)	merge or rebase the base locally to reproduce
Missing secrets	Not passed to runs from forks or from Dependabot; the token is read-only; an unset secret is an empty string (using secrets)	fork-safe jobs; never <code>pull_request_target</code> as a shortcut (Chapter 21A)
Shell differences	The implicit shell on Linux and macOS is <code>bash -e</code> without <code>pipefail</code> ; <code>shell: bash</code> adds <code>-o pipefail</code> ; in a job container the default is <code>sh</code> ; on Windows it is PowerShell (shell)	state the shell
Moving images and tools	Weekly image rebuilds; every <code>-latest</code> label moved in 2026	fixed labels, setup actions with versions
Old action majors	Actions written for Node 20 now run on Node 24 (changelog)	current majors
Required check stays pending	A workflow skipped by a filter or <code>[skip ci]</code> never reports (skipping runs)	require an aggregate job (20B.13)
Runs cancelled	A newer run in the same concurrency group	workflow name and ref in the group
Stale or missing cache	A cache is immutable per key; <code>restore-keys</code> restore the most recent prefix match; pull request caches are scoped to the merge ref (dependency caching)	hash of the lock file in the key
Out of memory or time	Smaller runners in private repositories; six-hour limit	split or resize
Environment differences	<code>CI=true</code> is set; steps share no shell state; every job is a new machine (variables)	<code>GITHUB_ENV</code> , outputs, artifacts
A workflow that never fires	Events made with the <code>GITHUB_TOKEN</code> start no runs; scheduled workflows are disabled after 60 days without repository activity in public repositories	a GitHub App token, or jobs chained with <code>needs</code>
Case sensitivity (Git)	macOS and Windows filesystems ignore case by default; a Linux runner does not	fix the names in Git
Line endings (Git)	CRLF committed, or converted on checkout by <code>core.autocrlf</code>	<code>.gitattributes</code>

Unverified. The official Actions pages read for the Phase 0 report do not state the time zone and locale of hosted runners, whether Windows runners check out with CRLF by default, or whether the hosted runners' filesystems are case-sensitive. Only the Git mechanisms below are sourced, from Git's documentation and from real runs. Linux filesystems being case-sensitive is the general rule this chapter relies on.

Three of these are Git, and can be reproduced on your machine.

A shallow clone has no tags to describe

On your laptop the version comes out of the history:

```
# Your clone: full history, all tags.
$ git log --oneline --decorate
57c8425 (HEAD -> main) Add the lock file
197d992 Document the release runbook
c4b5de2 (tag: v1.1.0) Add the deploy workflows
3c8340d Add reorder thresholds
e797c71 (tag: v1.0.0) Add the deploy script
91fe9ab Add stock rules and their checks
$ git describe --tags --match 'v*'
v1.1.0-2-g57c8425
```

v1.1.0-2-g57c8425 reads: two commits after the tag v1.1.0, at commit 57c8425. Now the clone a job gets by default, imitated with `git clone --depth 1 --no-tags` (the action's documented defaults are `fetch-depth: 1` and `fetch-tags: false`):

```
# A clone with one commit and no tags, which is what the checkout action fetches by default:
$ cd ..
$ git clone --quiet --depth 1 --no-tags "file://$PWD/warehouse-api" runner
$ cd runner
$ git log --oneline --decorate
57c8425 (grafted, HEAD -> main, origin/main, origin/HEAD) Add the lock file
$ git tag --list
$ git rev-parse --is-shallow-repository
true
$ git describe --tags --match 'v*'
fatal: No names found, cannot describe anything.
[exit status: 128]
```

One commit, marked `grafted` because its parent is deliberately missing, no tags, and `git describe` exits with status 128. Fetching the tags alone does not help:

```
# Fetching the tags alone, still at depth 1, brings the tag objects but not the path to them:
$ git fetch --quiet --depth 1 origin "refs/tags/*:refs/tags/*"
$ git tag --list
v1.0.0
v1.1.0
$ git describe --tags --match 'v*'
fatal: No tags can describe '57c8425908f43c74c14b2642edb59e0f99dac38c'.
Try --always, or create some tags.
[exit status: 128]
$ git rev-list --count HEAD
1
```

The tags exist now, but `git describe` walks from HEAD through parents to find a tagged commit, and the history is one commit long. The error changed from "No names found" to "No tags can describe"; both mean the same root cause. With full history it works:

```
# With the whole history the tag is reachable from HEAD again:
$ git fetch --quiet --unshallow --tags
$ git rev-parse --is-shallow-repository
false
$ git rev-list --count HEAD
6
$ git describe --tags --match 'v*'
v1.1.0-2-g57c8425
```

Observed behavior : The version step fails in the job; the same command works on every laptop.
Git state : `.git/shallow` lists the one fetched commit; `refs/tags` is empty.
Mechanism : `git describe` needs a tag that is reachable from HEAD through parent links.
Root cause : actions/checkout defaults to `fetch-depth: 1` and `fetch-tags: false`.
Why it does this : One commit is all most jobs need, and it is fast on a large repository.
Correct fix : `fetch-depth: 0` on the checkout step of the job that needs history.
Prevention : Derive versions in one job; never add `"|| echo 0.0.0"` or `--always` to make the error disappear, because that ships a wrong version.

A name that differs only in case

```
# Git recorded the name with a capital T. The loader asks for a lower-case name.
$ git config get core.ignorecase
true
$ git ls-files configs
configs/Thresholds.yaml
$ cat configs/thresholds.yaml
reorder:
```

```
lead_days: 5
safety_stock: 10
```

Git recorded `Thresholds.yaml`. The code asks for `thresholds.yaml`, and on a default macOS volume the file opens. Git set `core.ignorecase` to `true` when it created the repository because it detected such a filesystem. Git's own lookup compares bytes, as a Linux filesystem does:

```
# Git itself compares names byte by byte, as a case-sensitive filesystem does:
$ git cat-file -e HEAD:configs/thresholds.yaml
fatal: path 'configs/thresholds.yaml' exists on disk, but not in 'HEAD'
[exit status: 128]
$ git cat-file -e HEAD:configs/Thresholds.yaml
[exit status: 0]
```

The fix must be a rename in Git, `git mv configs/Thresholds.yaml configs/thresholds.yaml`, committed and pushed. Renaming in Finder changes nothing that Git notices. The opposite accident is a commit made on Linux that contains both spellings; a Mac clone warns and checks out only one:

```
# The other half of the problem: a commit made on Linux that holds both spellings.
$ git ls-files | grep -i readme
README.md
Readme.md
$ cd ..
$ git clone --quiet warehouse-api second-clone
warning: the following paths have collided (e.g. case-sensitive paths
on a case-insensitive filesystem) and only one from the same
colliding group is in the working tree:

  'README.md'
  'Readme.md'
$ ls second-clone | grep -i readme
Readme.md
```

This transcript depends on a case-insensitive volume, the macOS default. On a case-sensitive volume the first `cat` fails and there is no collision warning.

A script with CRLF line endings

```
$ ./scripts/deploy.sh staging
env: bash\r: No such file or directory
[exit status: 127]
```

The kernel read the first line and looked for an interpreter named `bash` followed by a carriage return. `git ls-files --eol` shows where the carriage returns live:

```
# i/ is the index (what is committed), w/ the working tree, attr/ the attributes in force.
$ git ls-files --eol scripts/deploy.sh README.md
i/lf    w/lf    attr/                README.md
i/crlf  w/crlf  attr/                scripts/deploy.sh
$ git config get core.autocrlf
```

`i/crlf` means the committed blob itself contains CRLF. No attribute applies and `core.autocrlf` is unset, so nothing normalised it. The repair is a rule in the repository and a renormalisation of the index:

```
# Declare the rule in the repository, so that it does not depend on anybody's configuration:
$ printf '* text=auto\n*.sh text eol=lf\n' > .gitattributes
$ git add --renormalize .
$ git status --short
M  scripts/deploy.sh
?? .gitattributes
$ git commit -q -m "Normalise line endings; shell scripts are always LF"
```

```
$ git ls-files --eol scripts/deploy.sh
i/lf      w/crlf  attr/text eol=lf      scripts/deploy.sh
```

After the commit the index holds LF (`i/lf`) while this working tree still has the old bytes (`w/crlf`) until the file is checked out again. A fresh clone, which is what a runner makes, gets LF. Chapter 14C covers attributes in depth.

20B.13 Required checks that stay pending

The rules, from [troubleshooting required status checks](#) and [skipping workflow runs](#); Chapter 18, section 18.8 gives the ruleset side.

Situation	What the check reports	Merge
The workflow never started: path filter, branch filter, or <code>[skip ci]</code> in the commit message	nothing; the required check stays "pending"	blocked
The workflow started and the job was skipped by its <code>if</code>	success (skipped counts as passing)	allowed
A job was skipped because a job it needs failed	skipped, so it may not block	allowed, wrongly
The check ran on a <code>workflow_dispatch</code> run of the head branch	not evaluated for the pull request	blocked
A merge queue is used and the workflow lacks <code>on: merge_group</code>	nothing in the queue	blocked in the queue

A path filter is evaluated on the pull request's three-dot diff. Git shows the list GitHub would test:

```
# Workflow 2. For a pull request, a path filter is evaluated on the three-dot diff:
$ git diff --name-only main...docs/rollback-steps
docs/runbook.md
$ git diff --name-only main...feature/safety-stock
src/warehouse/rules.py
uv.lock
```

A workflow with `paths: ["src/**"]` starts for the second branch and not for the first. If its job is a required check, the documentation-only pull request waits forever. The filter also has edges: a push with more than 1,000 commits always runs the workflow, and with more than 3,000 changed files a match beyond the first 3,000 is not seen ([syntax](#)).

The robust design, which the report labels an inference from these rules: do not filter the workflow. Start it always, decide inside which jobs to run with `if`, and require one aggregate job that always runs and fails when something it needs failed or was cancelled.

```
all-checks:
  name: all-checks
  if: ${ !cancelled() }
  needs: [unit-tests, lint]
  runs-on: ubuntu-24.04
  steps:
    - name: Fail if a needed job failed or was cancelled
      if: ${ contains(needs.*.result, 'failure') || contains(needs.*.result, 'cancelled') }
      run: exit 1
```

Require `all-checks` in the ruleset and nothing else. The matrix and the job list can then change without touching the rule. The `*` object filter, `contains` and `cancelled` are documented in the [expressions reference](#), which also recommends `!cancelled()` over `always()`; `needs.<job_id>.result` is in the [contexts reference](#).

20B.14 The debugging instruments

Read the failure first.

```
gh run list --workflow 08-deploy-staging.yml --limit 5
gh run view RUN_ID
gh run view RUN_ID --log-failed
gh run view RUN_ID --json event,headBranch,headSha,conclusion,jobs
gh run watch RUN_ID --exit-status
gh pr checks --watch
```

`--log-failed` prints the log of the failed steps only. `gh run watch` follows a run until it ends and, with `--exit-status`, exits non-zero when it fails, so it can be chained in a script. `gh pr checks` exits with status 8 while checks are pending. All flags are from the `--help` output of GitHub CLI 2.88.1.

Re-run. A re-run is not a new run. It uses "the same `GITHUB_SHA` and `GITHUB_REF` of the original event" and the privileges of the actor who triggered the original, is possible for 30 days, and at most 50 times ([re-running workflows and jobs](#)).

```
gh run rerun RUN_ID --failed
gh run rerun RUN_ID --failed --debug
gh run rerun --job JOB_ID
```

Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
unchanged	unchanged	unchanged	unchanged	unchanged	unchanged	a new attempt of the same run, at the original commit and ref; new checks on that commit; for a deploy workflow, a new deployment of that commit

The last cell is the danger. Re-running last week's deploy run deploys last week's commit over today's. Git can tell you whether a commit is behind what is already deployed:

```
# Somebody re-runs an old workflow run. A re-run uses the commit of the original run:
$ git switch --quiet --detach v1.1.0
$ ./scripts/deploy.sh staging
would deploy c4b5de2 to staging
# Is that commit behind what staging already had? (exit status 0 means yes)
$ git merge-base --is-ancestor HEAD main
[exit status: 0]
$ git log --oneline HEAD..main
57c8425 Add the lock file
197d992 Document the release runbook
```

Exit status 0 from `git merge-base --is-ancestor HEAD main` means the commit of the re-run is an ancestor of `main`: the deployment would go backwards by the two commits listed. Use a re-run to retry a flaky step of the latest run, and a fresh run for everything else.

Debug logging. Two switches, each a repository secret or variable set to `true` (the secret wins if both exist): `ACTIONS_STEP_DEBUG` adds debug lines to step logs, and `ACTIONS_RUNNER_DEBUG` adds runner and worker diagnostic logs in the `runner-diagnostic-logs` folder of the downloaded log archive ([enable debug logging](#)). For one run, `gh run rerun --debug` or the checkbox on the re-run dialog does the same without leaving the switch on. Anyone who may run the workflow may enable it.

Skipped jobs. Since 29 January 2026 the log of a skipped job shows the original `if` expression and its expanded values ([troubleshooting](#)).

Run and manage workflows.

```
gh workflow list --all
gh workflow view 09-environments.yml --yaml
gh workflow run 09-environments.yml --ref main
gh workflow disable "Deploy through environments"
gh cache list --key Linux-inventory-api
gh cache delete --all
```

`gh workflow run` needs `on: workflow_dispatch` in the file on the default branch, and a run started this way does not satisfy a required check of a pull request.

Do not print contexts carelessly. Dumping `toJSON(github)` into a log is a documented debugging aid, and that context contains `github.token`. Print the fields you need.

20B.15 Linting, and what a local emulator cannot reproduce

Catch what can be caught before the push. `actionlint` checks workflow syntax, expression types and embedded shell ([README](#)); the GitHub Actions language service in the editor does part of that while you type. Neither is installed for this course, and the course's own check is only that each file parses as YAML and has `on` and `jobs`, which finds indentation errors and nothing else:

```
python3 -c "import yaml,sys; yaml.safe_load(open(sys.argv[1]))" workflows/08-deploy-staging.yml
```

All six broken workflows of Lab 28.1 pass this check, and a linter would pass most of them too. A linter proves that a file is well formed. It cannot know that the history is shallow or that a secret is withheld.

`act` runs workflows locally in Docker containers. Its own documentation lists what it does not implement: `concurrency`, `job permissions`, `environment`, `OIDC`, `timeout-minutes`, `continue-on-error`, step summaries, and a complete `github` context ([unsupported functionality](#)). It is useful for the shell logic of steps and useless for exactly the topics of this chapter: gates, tokens, groups, runner images. Treat a green local run as evidence about your scripts, never about your deployment.

20B.16 What can go wrong

Symptom	Diagnosis	Fix	Prevention
Production deployed without approval	<code>gh api repos/OWNER/REPO/environments/production</code> shows no rules, or the name in the workflow is misspelled	configure the rules; correct the name; delete the stray environment	create environments first; review workflow changes through CODEOWNERS
A middle deployment never happened	default concurrency cancelled the pending run	<code>queue: max</code> , or cumulative deployments	choose the queue policy on purpose
CI runs of different branches cancel each other	the group lacks <code>github.ref</code>	add workflow name and ref	copy the two patterns of 20B.5
Empty secret in a called workflow	secrets pass one level only, or the secret is an environment secret	pass it again; read it in the job that names the environment	keep the secret path short
Pull request waits for a	the workflow was filtered out	aggregate job	never require a filterable workflow

Symptom	Diagnosis	Fix	Prevention
check forever			
Old code deployed	a re-run of an old run	start a new run	<code>git merge-base --is-ancestor</code> guard in the deploy script
Job killed after the repository became private	2 CPUs and 8 GB	split the job or pay for a larger runner	note visibility in the runbook

20B.17 When not to use it, and dangerous edge cases

- **Do not use an environment as your only production control.** It is enforced by GitHub. Credentials that also work from a laptop bypass it. Bind the credential to the environment at the cloud provider (Chapter 21A).
- **Administrators can bypass** protection rules unless that is switched off, and whoever can edit the environment can remove the rule. Know who that is.
- **Do not put** `cancel-in-progress: true` **on a deployment.** A deployment cancelled half-way leaves a state nobody designed.
- **Do not extract a reusable workflow for two callers.** Indirection costs debugging time; the check name changes and can break required checks.
- **Do not use self-hosted runners for public repositories,** and do not give a persistent runner credentials you would not give every contributor.
- **Do not make a job green by weakening it:** `continue-on-error`, `|| true`, `--always`, broader `restore-keys`. Each hides a cause from the next engineer.
- **A re-run is a time machine.** On a deploy workflow it is a rollback nobody announced.

20B.18 Command safety

Command	Label	What it changes	Preview	Recovery
<code>gh run view</code> , <code>gh run list</code> , <code>gh run watch</code> , <code>gh pr checks</code> , <code>gh workflow view</code> , <code>gh cache list</code>	● SAFE	nothing	not needed	not needed
<code>git describe</code> , <code>git ls-files --eol</code> , <code>git diff --name-only A...B</code> , <code>git merge-base --is-ancestor</code>	● SAFE	nothing	not needed	not needed
<code>git fetch --unshallow --tags</code>	● SAFE	adds objects and tags; removes <code>.git/shallow</code>	<code>git rev-parse --is-shallow-repository</code>	not needed
<code>git add --renormalize .</code>	● CAUTION	rewrites index entries to normalised line endings	<code>git ls-files --eol</code> ; <code>git status</code> afterwards	<code>git restore --staged .</code> before committing
<code>gh workflow run</code>	● CAUTION	starts a run; on a deploy workflow, a deployment	read the file with <code>gh workflow view --yaml</code>	<code>gh run cancel</code> ; deploy the previous commit

Command	Label	What it changes	Preview	Recovery
<code>gh run rerun</code>	⚠ CAUTION	new attempt at the original commit	<code>gh run view RUN_ID --json headSha</code> and compare with <code>main</code>	start a new run from the current commit
<code>gh run cancel</code>	⚠ CAUTION	stops a run, possibly mid-deployment	<code>gh run view</code>	re-run; check the target's state by hand
<code>gh secret set</code> , <code>gh variable set</code>	⚠ CAUTION	overwrites the stored value; the old secret value cannot be read back	<code>gh secret list --env NAME</code>	set the previous value again from your secret store
<code>gh api -X PUT .../environments/NAME</code>	🔴 DANGEROUS	replaces the environment's protection settings with the body sent. What it can destroy: the previous settings, required reviewers included, which exist nowhere else unless you saved them; the label is the one Chapter 15, section 15.22 gives every <code>gh api</code> call that is not a <code>GET</code> . Appropriate for environment rules kept as code	<code>gh api .../environments/NAME</code>	send the previous configuration again
<code>gh cache delete --all</code>	⚠ CAUTION	removes every cache of the repository	<code>gh cache list</code>	caches are rebuilt by the next runs, slowly

No command in this chapter destroys Git history. The irreversible effects are outside Git: a deployment that ran, a secret value overwritten.

20B.19 Version notes

Version note. Older behavior: `concurrency` kept one pending run per group. Current behavior: the same by default, with `queue: max` for up to 100. Since: 7 May 2026. Recommended: `queue: max` only where every run must execute.

Version note. Older behavior: same-repository workflows and actions referenced with `./`. Current behavior: `$/` is the recommended form on github.com. Since: 30 July 2026, runner 2.336.0. Recommended: `./` where GitHub Enterprise Server or older self-hosted runners must run the file.

Version note. Older behavior: a deployment branch rule on a pull request run was evaluated against the head branch. Current behavior: against `refs/pull/N/merge`, and against the default branch for `pull_request_target`. Since: 8 December 2025.

Version note. Older behavior: `ubuntu-latest` meant Ubuntu 22.04, then 24.04. Current behavior: 24.04, moving to 26.04 from 19 October 2026. Recommended: a fixed label.

Version note. Older behavior: checks, runs and statuses were kept 400 days or more. Current behavior: they follow the Actions retention setting, 90 days by default. Since: 1 October 2026. The status-checks page that still says 400 days is flagged as a conflict in the Phase 0 report.

Outdated advice. "Use `git reset --hard` and a force push to redo a deployment", and "add `pull_request_target` so that fork pull requests get the secrets". The first rewrites shared history to fix something that is not in Git; the second is the vulnerability class of Chapter 21A.

20B.20 Practice

- Module 27 labs: Labs 27.1 to 27.5 run workflows 6, 7, 8, 9 and 11 in your practice repository `YOUR-ORG/inventory-api`.
- Module 28 labs: Lab 28.1 has six broken workflows from `workflows/broken/`, each with a different documented root cause; Lab 28.2 applies the investigation order to a described failing run.
- Keep the GitHub Actions guide open while you work: it lists all twelve workflows, the authoring checklist and the debugging runbook.

20B.21 Interview questions

1. A job names `environment: production`. List everything that must be true on GitHub, outside the workflow file, for that to be a real gate.
2. Three merges land on `main` within five minutes and the deploy workflow uses `concurrency: production`. Which runs deploy, and why?
3. When would you choose a reusable workflow over a composite action, and what does each choice do to the names of required status checks?
4. Why can a caller not pass an environment secret to a reusable workflow, and where must the secret be read?
5. `git describe` works on every laptop and fails in the job. Explain the state of the runner's repository and give the minimal fix.
6. A pull request shows "waiting for status to be reported" and no run exists. Name three causes and the design that avoids all of them.
7. What exactly does `gh run rerun` re-run, and why is that dangerous for a deploy workflow?
8. Your CI bill rose after a repository became private and one job started timing out. What changed?
9. Make the case for and against self-hosted GPU runners for model evaluation, including the controls you would require.
10. A test passes on a Mac and fails on `ubuntu-24.04` with a file-not-found error. How do you prove the cause with Git alone?
11. In which order do you investigate a failed run, and why is the log not first?
12. What can a local emulator tell you about a deployment workflow, and what can it not?

20B.22 Sources

Primary sources

- GitHub Docs: [deployments and environments](#), [managing environments](#), [control deployments](#), [reviewing deployments](#), [REST: deployment environments](#).
- GitHub Docs: [workflow syntax](#), [reusing workflow configurations](#), [reuse workflows](#), [metadata syntax](#), [custom actions](#).
- GitHub Docs: [GitHub-hosted runners](#), [self-hosted runners](#), [secure use](#), [limits](#), [billing](#), [runner pricing](#).
- GitHub Docs: [dependency caching](#), [using secrets](#), [enable debug logging](#), [re-running workflows and jobs](#), [troubleshooting workflows](#), [skipping workflow runs](#), [troubleshooting required status checks](#).
- Action documentation at the pinned versions: [actions/checkout v7.0.1](#), [actions/cache v6.1.0](#), [actions/upload-artifact v7.0.1](#), [actions/download-artifact v8.0.1](#), [astral-sh/setup-uv v10.2.0](#).
- GitHub CLI manual: [gh run view](#), [gh run rerun](#), [gh run watch](#), [gh workflow run](#), and the `--help` output of the installed 2.88.1.
- The local Git manual: `git help describe`, `git help ls-files`, `git help gitattributes`, `git help config` (`core.ignoreCase`).

Secondary sources

- [actionlint](#) and [act: unsupported functionality](#), third-party tools, not installed for this course.
- The GitHub changelog entries linked in the text for each dated change.

Videos (from the Phase 0 report, with its caveats)

- ["Complete GitHub Actions Course - From BEGINNER to PRO"](#), Sid Palas, DevOps Directive, 3 h 43 min, 24 September 2025. The most complete current free course; it has sponsor segments, says "branch protections" and never mentions rulesets, and pre-dates Node 24-only runners and `actions/checkout v7`.
- ["Introduction to GitHub Actions - Part 6 - Repository Rulesets"](#), Mickey Gousset, 15 min, 6 December 2024. A ruleset that requires a status check.

Further reading

- Chapter 18, section 18.8, for required status checks from the ruleset side; Chapter 21A for the security model of everything in this chapter.

Chapter 21A: GitHub Actions security

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026, with the quoted documentation pages re-read on 2 October 2026. Transcripts are real output from `labs/ch21a/`. They show plain Git and `grep` on this course's workflow files. Nothing in this chapter was run on GitHub: `workflows/12-secure.yml` and the five files in `workflows/vulnerable/` were assembled from documented syntax and parse-checked with PyYAML, but not executed. Every statement about how GitHub Actions behaves comes from the documentation, a changelog entry, or a published post-mortem, and carries the link.

This chapter is defensive. It shows unsafe workflow patterns next to their fixes, at the level of detail GitHub's own security documentation uses, so that you can review and harden your team's workflows. It contains no working attack. Workflow mechanics (events, contexts, caching, environments, runners) are taught in Chapter 20A and Chapter 20B. Repository security and the response to a leaked secret are Chapter 21B.

21A.1 Why this matters

"A contributor we have never heard of opened a pull request on our public evaluation library. Eight hours later there were malicious versions of our package on the registry, published by our own release workflow. No password was stolen. How?"

That is the shape of the TanStack incident of 11 May 2026 (section 21A.18), and each step in it was documented behavior: a workflow trigger that runs with the base repository's privileges, a checkout of the contributor's code, a cache shared with the release job, and a release job that could request a publishing identity. A CTO who asks this question does not want a list of settings. They want to know which line of which file gave an outsider code execution next to a credential, why the platform allowed it, what the smallest change is that closes it, and how you will know it stays closed.

The whole chapter reduces to one question that you ask of every job:

Whose code runs in this job, on whose text does it operate, and what can the job reach?

If the answer to the first two parts is "somebody outside the team" and the answer to the third is anything other than "nothing", you have found a vulnerability.

21A.2 The model in five parts

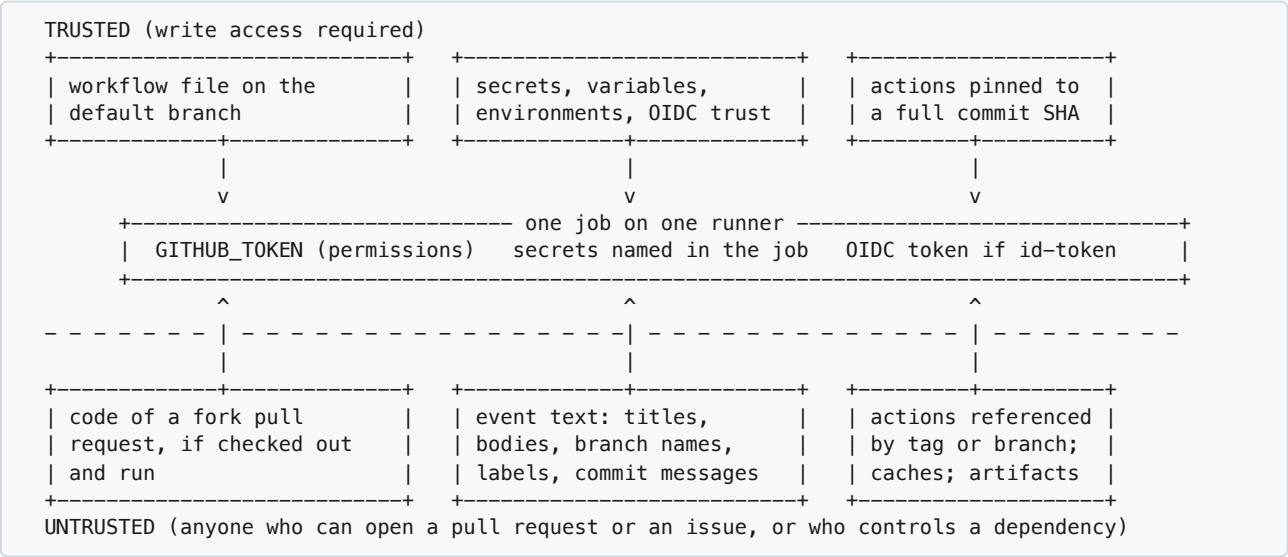
In one sentence. A workflow run is a program that GitHub starts on your behalf, with a credential for your repository in its pocket, and its safety depends on who controls the program's code and its inputs.

Analogy. A contractor is let into your office each time a parcel arrives. The front desk gives the contractor a badge; the badge opens whatever doors the desk configured. The contractor follows a written procedure (the workflow file), uses tools bought from other firms (actions), and reads the label on the parcel (event data). Three things can go wrong: the badge opens too many doors, the procedure was written by the person who sent the parcel, or the contractor reads the label aloud as if it were an instruction. The analogy breaks in one place that matters: this contractor executes text literally and instantly, so a label that contains an instruction is carried out before anyone can notice.

Precisely. The Phase 0 report condenses GitHub's documentation into five parts, and the rest of this chapter takes them one at a time:

Part	The fact	Section
1. The job token	Each job receives a <code>GITHUB_TOKEN</code> limited to the workflow's repository; the <code>permissions</code> key sets what it may do	21A.3
2. Fork pull requests	A <code>pull_request</code> run from a fork gets a read-only token and no secrets; by default only first-time contributors need approval	21A.4
3. Privileged triggers	<code>pull_request_target</code> (and <code>workflow_run</code> , <code>issue_comment</code>) run with the base repository's token and secrets; they are safe only while they do not run the fork's code	21A.5
4. Expression injection	<code>\${{ }}</code> is substituted into the script text before the shell starts, so attacker-controlled fields become code	21A.6
5. Third-party actions	An action runs with the job's token and secrets; a tag can be moved; only a full commit SHA is immutable	21A.7

Picture. Trust flows downward. Everything above the dashed line is controlled by people with write access; everything below it can be controlled by anyone on the internet.



GitHub Actions, not Git. Nothing in this model is a property of Git. Git contributes exactly two facts that the model relies on: a tag is a ref that can be moved, and a commit ID names content that cannot change (section 21A.7).

21A.3 The job token and `permissions`

In one sentence. Every job gets its own short-lived token for the repository that contains the workflow, and the `permissions` key decides what that token may do.

Analogy. A visitor badge printed at the start of each shift and shredded at the end. The desk can print it with access to one room or to the whole building. The analogy breaks because every tool the visitor picks up can use the badge without asking: "An action can access the `GITHUB_TOKEN` through the `github.token` context even if the workflow does not explicitly pass the `GITHUB_TOKEN` to the action" ([authenticate with GITHUB_TOKEN](#)).

Precisely. "At the start of each workflow job, GitHub automatically creates a unique `GITHUB_TOKEN` secret to use in your workflow." It is a GitHub App installation access token, and "the token's permissions are limited to the

repository that contains your workflow". It expires when the job finishes; on GitHub-hosted runners it "can live for a maximum of 6 hours" ([GITHUB_TOKEN](#)).

What the token may do is decided in three layers:

Layer	Where	What it decides
The repository or organization default	The "Workflow permissions" setting	What a workflow with no <code>permissions</code> key gets: either "read and write access for all permissions" or "just read access for the <code>contents</code> and <code>packages</code> permissions" (repository settings)
The workflow	Top-level <code>permissions</code>	The token of every job that does not override it
The job	<code>jobs.</code> <code><job_id>.permissions</code>	The token of that job only

Three rules of the `permissions` key carry most of the weight ([workflow syntax](#), [permissions](#)):

1. "If you specify the access for any of these permissions, all of those that are not specified are set to `none`." Writing `contents: read` therefore also removes `issues`, `packages`, `pull-requests` and every other scope.
2. `permissions: {}` removes everything. `permissions: read-all` and `permissions: write-all` are the two shorthands.
3. The scopes listed on 1 October 2026 are `actions`, `artifact-metadata`, `attestations`, `checks`, `code-quality`, `contents`, `deployments`, `discussions`, `id-token`, `issues`, `packages`, `pages`, `pull-requests`, `security-events`, `statuses` and `vulnerability-alerts`; each takes `read`, `write` or `none`, and `write` includes `read`.

Version note. Older behavior: a workflow without a `permissions` key received a read-write token. Current behavior: the default is read-only for *new* repositories, organizations and enterprises. Since: 2 February 2023, and "this change will not impact any existing enterprises, organizations or repositories" ([changelog](#)). Recommended: never rely on the default. Declare `permissions` at the top of every workflow, because a repository created before February 2023 may still have the permissive setting. The Nx project's post-mortem names exactly this as one of three causes (section 21A.18).

The unsafe and the safe form, side by side:

```
# Unsafe: every job, and every action in every job, can push, tag, release and edit issues.
permissions: write-all
```

```
# Safe: read-only for the workflow; one job adds the one scope it needs.
permissions:
  contents: read

jobs:
  label:
    permissions:
      issues: write      # contents is now none for this job, which is fine: it checks nothing out
```

GitHub's guidance is the same in prose: "It's good security practice to set the default permission for the `GITHUB_TOKEN` to read access only for repository contents. The permissions can then be increased, as required, for individual jobs within the workflow file" ([secure use](#)).

Two properties limit what a stolen job token can do. First, "events triggered by the `GITHUB_TOKEN` will not create a new workflow run", with `workflow_dispatch` and `repository_dispatch` as the documented exceptions ([GITHUB_TOKEN](#)). The exception is not academic: in the Nx incident a stolen read-write token was used to dispatch the publish workflow. Second, the token dies with the job.

Inside `.git`. The token also reaches Git. The `actions/checkout` README says: "The auth token is persisted in the local git config. This enables your scripts to run authenticated git commands. The token is removed during post-job cleanup. Set `persist-credentials: false` to opt-out" ([checkout README](#)). Since version 6 of the action the credential is stored "in a separate file under `$RUNNER_TEMP` instead of directly in `.git/config`" (same page). Either way, until the job ends, every later step and every process it starts can run authenticated Git commands against your repository with whatever `contents` permission the job has. A job that only builds and tests has no reason to leave it there.

21A.4 Fork pull requests and the approval gate

In one sentence. A workflow triggered by `pull_request` from a fork runs the contributor's version of the code with a read-only token and without your secrets.

Analogy. A visitor may use the lobby computer, which has no access to the internal network. The analogy breaks in two places: by default a visitor who has been in the building once is waved through without a check from then on, and the lobby computer may be one of your own machines (a self-hosted runner, section 21A.12).

Precisely. "With the exception of `GITHUB_TOKEN`, secrets are not passed to the runner when a workflow is triggered from a forked repository. The `GITHUB_TOKEN` has read-only permissions in pull requests from forked repositories" ([events, workflows in forked repositories](#)). Pull requests opened by Dependabot are treated the same way.

Whether such a run starts at all is a separate setting. For public repositories, "by default, all first-time contributors require approval to run workflows", and the options are approval for first-time contributors who are new to GitHub, for all first-time contributors, or for all external contributors. The documentation attaches a warning: a user "that has had any commit or pull request merged into the repository will not require approval", and "a malicious user could meet this requirement by getting a simple typo" fix accepted ([controlling changes from forks](#)).

What the fork model protects	What it does not protect
Your secrets: they are not sent to the runner	The runner itself: the contributor's code still executes on it
Your repository: the token cannot write	Readable data: a public repository's contents, and the caches a pull request may restore (section 21A.11)
	A self-hosted runner, which is your machine
	Workflows on privileged triggers, which "will always run, regardless of approval settings" (same page)

The last row is the hinge of the next section. The approval gate applies to `pull_request`. It does not apply to `pull_request_target`.

In production. An open-source evaluation harness wants to show benchmark scores on contributor pull requests. The scores need a provider API key. Under `pull_request` the key is absent, so the job fails for every outside contributor. This is the fork model working. The wrong fix is in the next section; the right ones are to run the keyed evaluation after merge, or on demand by a maintainer on a branch inside the repository, or to split the work into an unprivileged run and a privileged follow-up that never executes the contributor's code.

21A.5 Privileged triggers and the "pwn request"

In one sentence. `pull_request_target` runs your workflow, from your default branch, with your token and your secrets, in response to a stranger's pull request; it is safe exactly as long as it never executes that stranger's

code.

Analogy. The front desk has a procedure for parcels from unknown senders: log the parcel, put a sticker on it, send a receipt. The procedure is safe because the desk never opens the parcel. A "pwn request" is the day someone amends the procedure to "open the parcel and follow the instructions inside", while the clerk still holds the master key. The analogy breaks because the parcel can also influence the clerk without being opened: its label is event text (section 21A.6).

Precisely. GitHub's reference says such workflows "run with elevated trust: the job receives the base repository's `GITHUB_TOKEN` and access to repository and organization secrets". The trigger is safe by default because "the workflow, and any subsequent `actions/checkout` call that does not specify a `ref`, is taken from the base repository's default branch". Then: "You introduce risk when a workflow author overrides this default to run the fork's code." And: "The checkout step alone does not execute untrusted code... The vulnerability is completed by the *next* step that runs code checked out into the current working directory." "This pattern is known as a 'pwn request' and has been the root cause of multiple supply-chain compromises" ([securely using pull_request_target](#)).

The token is not read-only here: "When a workflow is triggered by the `pull_request_target` event, the `GITHUB_TOKEN` is granted read/write repository permission, even when it is triggered from a public fork" ([workflow syntax, permissions](#)).

The unsafe pattern, as the documentation prints it. The `actions/checkout@v6` line is the documentation's own example, quoted unchanged; the workflows of this course pin `actions/checkout` v7.0.1 by commit ID (section 21A.7), and since v7 and its back-ports the action refuses this checkout unless `allow-unsafe-pr-checkout: true` is set (the table at the end of this section):

```
# INSECURE. Provided as an example only.
on:
  pull_request_target:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
        with:
          ref: ${ github.event.pull_request.head.sha }
      - name: Test
        run: make test
```

Read it with the question from section 21A.1. Whose code runs? `make test` runs the `Makefile` of the pull request, so the contributor's. What can the job reach? A read-write token and every repository and organization secret the workflow names. What an attacker achieves is whatever those credentials allow: pushing commits, publishing packages, reading secrets. "Run" is wider than it looks. A build, an install step with lifecycle scripts, a test runner, a linter that loads a configuration file from the working directory: each executes or is steered by files from the checkout.

The same reference lists the shapes to look for in review:

Shape	Example
Checking out the pull request's head or merge commit	<code>ref: \${ github.event.pull_request.head.sha }</code> , or <code>ref: refs/pull/\${ github.event.pull_request.number }/merge</code>
Pointing the checkout at the fork	<code>repository: \${ github.event.pull_request.head.repo.full_name }</code>
Fetching the code another way and running it	<code>git fetch</code> of the pull request ref, <code>gh pr checkout</code> , or downloading an artifact built from it

And it widens the scope: "Pwn requests are also not unique to `pull_request_target` ... an `issue_comment` or `workflow_run` workflow that fetches and runs a fork's pull request code is vulnerable in the same way."

The fix, in order of preference.

- 1. Do not use the trigger. "If the workflow does not need `pull_request_target`, update it to use a safer event where appropriate, such as `pull_request`" (same reference). Most CI needs no secrets and no write token.

```
# Safe: the contributor's code runs, but with a read-only token and no secrets.
on:
  pull_request:

permissions:
  contents: read

jobs:
  build:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          persist-credentials: false
      - name: Test
        run: make test
```

- 2. If a privileged action is needed (a comment, a label), keep `pull_request_target` but never check out or run the pull request's code: operate on metadata only, with the narrowest permissions.
- 3. If the privileged action needs a result computed from the contributor's code, separate the two. GitHub Security Lab's design is an unprivileged `pull_request` workflow that builds and tests and uploads its result as an artifact, and a privileged `workflow_run` workflow that downloads the artifact and comments. Artifact data is safe there when used "in a safe manner, like reading PR numbers or reading a code coverage text to comment on the PR" (preventing pwn requests). The secure-use reference agrees: "For privilege separation between workflows, `workflow_run` is a better trigger", and such workflows "should treat artifacts uploaded from other workflows with caution" (untrusted code checkout).



A label as a gate ("run only when a maintainer adds `safe-to-test`") is weaker than it looks. The Security Lab article notes it "is still prone to a race condition in which the attacker may push new changes after the workflow was approved (labeled), but has not started yet".

The platform now leans against the pattern. Four changes matter in review, and none of them removes the need for it:

Change	Effect	What it does not cover
8 December 2025 (changelog)	The workflow file for <code>pull_request_target</code> always comes from the default branch	A vulnerable workflow that is on the default branch
<code>actions/checkout</code> v7, 18 June 2026, back-ported to older lines on 20 July 2026 (changelog)	The action refuses to fetch a fork pull request's head or merge ref under <code>pull_request_target</code> , and under	<code>git fetch</code> or <code>gh pr checkout</code> in a <code>run</code> step;

Change	Effect	What it does not cover
	<code>workflow_run</code> started by a pull request event, unless <code>allow-unsafe-pr-checkout: true</code>	other events such as <code>issue_comment</code>
Read-only caches for low-trust triggers, 26 June 2026 (section 21A.11)	A privileged-trigger run cannot write the default branch's cache	Restoring a cache that was already poisoned
Workflow execution protections, generally available 17 September 2026 (section 21A.8)	A default rule will block <code>pull_request_target</code> in public repositories from 2 November 2026 unless a maintainer allows it	Private and internal repositories

The opt-out input deserves one sentence of its own. The changelog says "the flag is intentionally named to be easy to spot in code review and static analysis", and the reference says: "Only do this after confirming the checked-out code is never executed." In a pull request, `allow-unsafe-pr-checkout: true` is a line that needs a written justification.

21A.6 Script injection

In one sentence. An expression in `${{ }}` is replaced by its value in the text of the script before the shell starts, so a value that an outsider controls becomes part of your program.

Analogy. Mail merge. You write "Dear {name}, your order shipped" and the software pastes the name in. If the letter is then read aloud by someone who obeys every sentence, a customer whose "name" contains a sentence gets it obeyed. The analogy is close; it breaks only in that a shell is far more obedient than any reader, and treats quotes and separators in the pasted text as structure.

Precisely. "The `run` command executes within a temporary shell script on the runner. Before the shell script is run, the expressions inside `${{ }}` are evaluated and then substituted with the resulting values, which can make it vulnerable to shell command injection" ([script injections](#)). The order is the whole mechanism:

workflow file	GitHub Actions	runner
-----	-----	-----
<pre>run: title="\${{ X }}"</pre>	<pre>1. evaluate \${{ ... }} 2. paste the VALUE of X into the script text</pre>	<pre>3. write the text to a temporary script file 4. start the shell on it</pre>

The shell never sees `"${{ X }}"`. It sees whatever X contained, as source code.

Which values are attacker-controlled? The documentation: "Attackers can add their own malicious content to the `github` context, which should be treated as potentially untrusted input. These contexts typically end with `body`, `default_branch`, `email`, `head_ref`, `label`, `message`, `name`, `page_name`, `ref`, and `title`" (same page). Branch names are on the list (`head_ref`), and Git allows characters in a branch name that a shell treats as syntax. Commit messages, author names and email addresses are Git data that anyone can set to anything (Chapter 21B shows the spoofing locally).

The unsafe step, as the documentation prints it:

```
- name: Check PR title
run: |
  title="${{ github.event.pull_request.title }}"
  if [[ $title =~ ^octocat ]]; then
    echo "PR title starts with 'octocat'"
    exit 0
  else
    echo "PR title did not start with 'octocat'"
    exit 1
  fi
```

The author's quotes do not help, because the value arrives before the shell parses anything: a title that contains a double quote closes the string, and what follows it is parsed as commands. The documentation's own demonstration uses a title that closes the quote and lists the workspace directory. What an attacker achieves is arbitrary commands in that job, with that job's token and secrets. On `pull_request` from a fork that is contained by the fork model. On `pull_request_target`, `issues`, `issue_comment` or `discussion`, it is not: the Nx compromise began with exactly this, a pull request title echoed in a `pull_request_target` workflow (section 21A.18).

The fixed step uses the documented mitigation, an intermediate environment variable ([secure use, script injection](#)):

```
- name: Check PR title
  env:
    TITLE: ${github.event.pull_request.title}
  run: |
    if [[ "$TITLE" =~ ^octocat ]]; then
      echo "PR title starts with 'octocat'"
      exit 0
    else
      echo "PR title did not start with 'octocat'"
      exit 1
    fi
```

Why the environment-variable form is safe. The expression is still evaluated, but its value no longer goes into the script text. In the documentation's words, the value "is stored in memory and used as a variable, and doesn't interact with the script generation process". The script that the shell parses is now a constant: it is identical for every pull request, and you can review it once. The title reaches the shell as the *value* of `$TITLE` after parsing is over, and the shell does not re-parse the value of a variable as commands when it expands it. The double quotes around `"$TITLE"` are still needed, for the ordinary shell reason that an unquoted expansion is split into words and glob-expanded; the documentation tells you to quote for that reason.

Observed behavior	: a step that only prints a pull request title runs commands nobody wrote
Git state	: irrelevant; the title is GitHub data, a branch name or commit message is Git data
Mechanism	: <code>\${{ }}</code> is substituted into the script text before the shell starts
Root cause	: attacker-controlled text was placed where the shell expects source code
Why Actions does this	: expressions are a templating layer over the whole workflow file; the template engine does not know that the result will be parsed by a shell
Correct fix	: pass the value through <code>env:</code> (or as an action input with <code>with:</code>) and use <code>"\$VAR"</code>
Prevention	: treat every <code>\${{ }}</code> inside <code>run:</code> as a finding until proven constant; run CodeQL for workflows, zizmor or actionlint in CI (section 21A.14)

The same reference gives a second mitigation, which it prefers when available: "Use an action instead of an inline script", passing the value through `with:`. Two notes for review. The rule is about where the value lands, not which context it comes from; a pull request number cannot carry code, but a reviewer who decides case by case will eventually decide wrongly, so many teams adopt the mechanical rule of no `${{ }}` inside `run:` at all. And `env:` does not make later misuse safe: `eval "$TITLE"`, or writing the value unvalidated into `$GITHUB_ENV`, brings the problem back one layer down.

In production. A prompt-evaluation workflow prints the pull request title in a job that holds a provider API key because it runs on `pull_request_target`. That one line is the Nx pattern (section 21A.18). The fix is two edits: the `env:` form, and removing the privileged trigger so that the key is not in the job at all.

21A.7 Third-party actions: mutable tags and commit pinning

In one sentence. `uses: owner/repo@ref` downloads and runs someone else's code inside your job, and only a full commit ID guarantees that it is the code you reviewed.

Analogy. A tag is a street address; a commit ID is a fingerprint. Whoever owns the plot can put up a different building at the same address overnight. The analogy breaks in your favor: a fingerprint can in principle be shared by two people, while forging a commit ID means producing "a SHA-1 collision for a valid Git object payload" ([secure use](#), [third-party actions](#)).

Precisely. A compromised action "would have access to all secrets configured on your repository, and may be able to use the `GITHUB_TOKEN` to write to the repository". "Pinning an action to a full-length commit SHA is currently the only way to use an action as an immutable release." Tags are a matter of trust: "Pin actions to a tag only if you trust the creator... a tag can be moved or deleted if a bad actor gains access to the repository" (same page). The same applies to reusable workflows.

See it. This is Git, not GitHub Actions, so it can be shown locally. A bare repository stands for an action's repository; `v1` is an annotated tag.

```
# What a consumer gets for "report-size@v1" today. The line ending in ^{} is the commit.
$ git ls-remote --tags hosting/report-size.git
71c87ced80cd16d0206b29b9a557f2bdf5e3d3b2      refs/tags/v1
80827f43993c0dc6fc8324685d01e720707e5429      refs/tags/v1^{}
```

The first line is the tag object, the second (ending in `^{}`) the commit it points at. Now someone with push access, a maintainer or whoever holds a maintainer's token, moves the tag:

```
# Anyone who can push to the action repository can point v1 somewhere else.
$ git tag -f -a v1 -m "report-size v1"
Updated tag 'v1' (was 71c87ce)
$ git push --force origin v1
To $LAB/ch21a/mutable-tag/hosting/report-size.git
+ 71c87ce...0ca190d v1 -> v1 (forced update)
```

```
# Same name, different commit. No consumer workflow changed.
$ git ls-remote --tags hosting/report-size.git
0ca190da3c6db48b75af6a358aef5b189725e9a4      refs/tags/v1
5de1e2d145bc0b3fbb28e6d457f7220359d63f84      refs/tags/v1^{}
```

```
# The commit that was reviewed still exists, and its ID still names exactly that content.
$ git -C hosting/report-size.git cat-file -p 80827f43993c0dc6fc8324685d01e720707e5429:action.yml
name: report-size
description: Prints the size of the build output
runs:
  using: composite
  steps:
    - run: du -sh dist
      shell: bash
```

A workflow that says `@v1` now runs `5de1e2d`. A workflow that says `@80827f43993c0dc6fc8324685d01e720707e5429` runs what it ran yesterday. 🚫 **DANGEROUS:** `git push --force origin v1` replaces a published tag. It changes the remote tag ref; it can silently change what every consumer of the tag runs; preview with `git ls-remote --tags origin v1`; recover by pushing the old tag object back, if you still have it; it is appropriate only for a deliberate "moving major tag" policy that consumers know about.

	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git tag -f -a v1</code>	unchanged	unchanged	unchanged	unchanged	<code>refs/tags/v1</code> points at a new tag object	unchanged	unchanged
<code>git push --force</code>	unchanged	unchanged	unchanged	unchanged	unchanged	<code>refs/tags/v1</code> replaced	every <code>@v1</code> reference resolves to the

	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
origin v1							new commit on its next run

Writing and maintaining a pin.

```
# Unsafe: resolved again on every run, to whatever the tag points at then.
- uses: actions/setup-python@v7

# Safe: the commit, with the version as a comment for humans and for Dependabot.
- uses: actions/setup-python@5fda3b95a4ea91299a34e894583c3862153e4b97 # v7.0.0
```

Take the commit ID from the action's own repository (`workflows/ACTION_PINS.md` shows the `git ls-remote` command): "you should verify it is from the action's repository and not a repository fork" (same page). Pins do not update themselves, which is the point, so let Dependabot propose the updates. The documented configuration ([keeping your actions up to date](#)):

```
version: 2
updates:
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

Since 14 July 2026, version updates wait a default cooldown of three days after a release before a pull request is opened; security updates are immediate, and the `cooldown` option configures it ([changelog](#)). A cooldown gives the ecosystem time to notice a malicious release before you adopt it.

The limits of pinning. State these whenever you recommend it:

- A pin protects against a moved tag, "but not against a malicious commit pinned deliberately or against unpinned actions nested inside composite actions" (Phase 0 report). Read the diff of a pin update.
- "Dependabot only creates alerts for vulnerable actions that use semantic versioning and will not create alerts for actions pinned to SHA values" ([secure use](#)). Pinning trades automatic alerts for immutability; version updates and advisories you read yourself fill the gap.
- A remote script fetched and executed in a `run` step (`curl ... | bash`) is the same risk class with no pin available. That is the Codecov case.
- Immutable releases (generally available since 28 October 2025) make a published release's tag unchangeable once a maintainer enables the feature ([changelog](#)); `astral-sh/setup-uv` has used them since v8.0.0. You cannot see from a `uses:` line whether a tag is protected this way, so the pin remains the rule.

Unverified. "Immutable actions" (actions published as packages) had no general-availability announcement that the research could find by 1 October 2026, and the secure-use page still calls commit pinning "currently the only way". GitHub's 2026 roadmap describes workflow-level dependency locking as future work ([roadmap](#)); it has not shipped.

21A.8 Organization policies

In one sentence. Policies move three of this chapter's rules from "every author must remember" to "the platform refuses".

Policy	What it enforces	Source
Allowed actions and reusable workflows	Allow all; only those in the owner's organization or enterprise; or the owner plus a selection, with switches for actions created by GitHub and by verified Marketplace creators and a pattern list. Available on all plans since 5 February 2026	repository settings , changelog
Blocking	"Prefix an entry with <code>!</code> to block a specific action or version" (since 15 August 2025)	changelog
"Require actions to be pinned to a full-length commit SHA"	"any workflow that attempts to use an action that isn't pinned will fail", including GitHub's own actions; "Reusable workflows can still be referenced by tag"	same changelog; repository settings
Workflow permissions default	Read-only token for workflows without a <code>permissions</code> key; a restrictive organization default disables the permissive option below it	section 21A.3
Fork pull request approval	Who needs approval before a <code>pull_request</code> run starts	section 21A.4
Workflow execution protections	Actor rules (who may trigger workflows) and event rules (which events may run), built on rulesets, at enterprise, organization and repository level	changelog , about Actions policies
"Allow GitHub Actions to create and approve pull requests"	Off by default for new personal repositories; keep it off so that a workflow cannot approve its own change	repository settings

The labels are quoted from the documentation; the interface changes, so trust the linked page over this table.

The change dated 2 November 2026. With execution protections, GitHub introduced a default event rule: "For public repositories that do not already have an applicable event policy, GitHub is introducing a default rule that disables `pull_request_target` ... It initially runs in evaluate mode... On November 2, 2026, we'll automatically enforce the default rule for affected repositories" ([changelog](#); the [reference](#) words it as enforcement "for affected repositories that were using the default `pull_request_target` policy before general availability"). Private and internal repositories are not affected. A maintainer can explicitly allow the event, optionally for named workflow files. If your public repository has a labeler or a welcome bot on `pull_request_target`, decide before that date: migrate it, or allow it deliberately after reviewing it against section 21A.5.

Policies do not replace review of the files. The control for that is a code-owner rule on `.github/workflows/` enforced by a ruleset (Chapter 19, section 19.8): "If all your workflow files are stored `.github/workflows`, you can add this directory to the code owners list, so that any proposed changes to these files will first require approval" ([secure use](#)).

21A.9 Secrets: scope, and why masking is not a boundary

In one sentence. A secret is as exposed as the least trustworthy code in any job that receives it.

Precisely. Secrets exist at organization, repository and environment level, and "GitHub Actions can only read a secret if you explicitly include the secret in a workflow" ([secrets](#)). Where you include it sets the exposure:

```
# Unsafe: every step of every job, including third-party actions and your dependencies'
# install scripts, has the token in its environment.
env:
  DEPLOY_TOKEN: ${ secrets.STAGING_DEPLOY_TOKEN }

jobs:
  lint: ...
  test: ...
  deploy: ...
```

```
# Safe: one step of one job, behind an environment.
jobs:
  deploy:
    environment:
      name: staging
    steps:
      - name: Deploy
        env:
          DEPLOY_TOKEN: ${ secrets.STAGING_DEPLOY_TOKEN }
        run: bash scripts/deploy.sh staging
```

Two facts are routinely misunderstood.

1. **Write access is secret access.** "Any user with write access to your repository has read access to all secrets configured in your repository" ([secure use](#)): they can push a branch with a workflow that uses them. GhostAction and Shai-Hulud are this sentence at scale. Environment secrets behind required reviewers (section 21A.13) are the exception that makes the sentence false for your most valuable credentials.
2. **Masking is a courtesy for logs.** Registered secrets are redacted from logs, but "because there are multiple ways a secret value can be transformed, this redaction is not guaranteed"; "never use structured data as a secret"; and "if an unredacted secret is sent to a workflow run log, you should delete the log and rotate the secret" (same page). Redaction matches known strings in log output. It does nothing about a process that sends the value elsewhere, and nothing about code that reads the runner's memory, the technique the report finds recurring from tj-actions to TanStack. The boundary is which code runs in the job and which secrets the job holds.

21A.10 OIDC federation

In one sentence. Instead of storing a cloud key as a secret, the job asks GitHub for a signed statement of what it is, and the cloud exchanges that statement for a credential valid for that job only.

Analogy. A notarized letter of introduction in place of a copied house key. The cloud reads the letter (repository, branch, environment) and decides whether to open. The analogy breaks at one point: any code running in the job can ask the notary for the letter.

Precisely. "The job or workflow must grant the `id-token: write` permission to allow GitHub's OIDC provider to create a JSON Web Token"; the setting "only enables fetching and setting the OIDC token; it does not grant write access to other resources". The issuer is `https://token.actions.githubusercontent.com`, the default audience is the URL of the repository owner, and the cloud returns "a short-lived access token that is only valid for a single job" ([OIDC reference](#), [OpenID Connect](#)).

The token's claims describe the run: besides the standard `aud`, `iss`, `sub`, `exp`, `iat`, `jti` and `nbf` there are `repository`, `repository_id`, `repository_owner`, `repository_owner_id`, `repository_visibility`, `ref`, `ref_type`, `environment`, `event_name`, `workflow_ref`, `job_workflow_ref`, `runner_environment`, `actor` and others. The cloud's trust policy matches on them, above all on the subject:

Run	Default subject (sub)
Job that references an environment	<code>repo:ORG-NAME/REPO-NAME:environment:ENVIRONMENT-NAME</code>
Pull request event, no environment	<code>repo:ORG-NAME/REPO-NAME:pull_request</code>
Branch	<code>repo:ORG-NAME/REPO-NAME:ref:refs/heads/BRANCH-NAME</code>
Tag	<code>repo:ORG-NAME/REPO-NAME:ref:refs/tags/TAG-NAME</code>

"You **must** define at least one condition, so that untrusted repositories can't request access tokens for your cloud resources" (OIDC reference). The trust policy is where the security lives. GitHub's AWS guide gives this condition

for an environment ([OIDC in AWS](#)):

```
"Condition": {
  "StringEquals": {
    "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
    "token.actions.githubusercontent.com:sub": "repo:octo-org/octo-repo:environment:prod"
  }
}
```

The same guide also shows a wildcard form, `"StringLike"` with `repo:octo-org/octo-repo:*`. Read that one as: every branch, every tag and every pull request workflow of the repository may assume the role. Prefer the exact form.

Version note. Older behavior: the subject contains only names, so a deleted and re-registered organization or repository name could mint the same subject. Current behavior: "repositories created after July 15, 2026 now use an immutable default subject format that includes both the owner ID and repository ID", for example `repo:octo-org@123456/octo-repo@456789:ref:refs/heads/main`; renames and transfers after that date switch as well, and existing repositories can opt in. Since: opt-in 23 April 2026, default 15 July 2026 ([changelog](#), [reference](#)). Recommended: check which format your repository issues before writing the trust policy; a policy in the old format will not match a new repository.

Subject customization lets an organization or repository change which claims form the subject, through the REST API's `include_claim_keys` (for example to require `job_workflow_ref`, so that only a particular reusable workflow can obtain the role). The AWS guide notes that custom claims are not supported there, which is why customizing the subject matters on that cloud.

What OIDC does not do. It removes the long-lived key. It does not make the job trustworthy. In the TanStack post-mortem's words: "OIDC trusted-publisher binding has no per-publish review. Once configured, any code path in the workflow can mint a publish-capable token." Grant `id-token: write` to one job, bind the subject to an environment with protection rules, and let no untrusted code or cache into that job.

21A.11 Caches and artifacts are untrusted input

In one sentence. A cache is a shared, unsigned directory keyed by a string; whoever can write an entry under the key your release job computes controls what that job restores.

Precisely. "Anyone with read access can create a pull request on a repository and access the contents of a cache", and "cache contents are not signed or verified" ([dependency caching](#)). A run can restore caches from its own branch and from the default branch. A `pull_request` run saves to its own merge ref, which other branches cannot restore. A privileged trigger runs in the default branch's context, and that was the opening: the TanStack post-mortem records that "cache writes use a runner-internal token, not the workflow `GITHUB_TOKEN` ... Setting `permissions: contents: read` does not block cache mutation."

Two platform changes followed:

Date	Change
26 June 2026	Only <code>push</code> , <code>workflow_dispatch</code> , <code>repository_dispatch</code> , <code>delete</code> , <code>registry_package</code> , <code>page_build</code> and <code>schedule</code> can create or overwrite caches in the default branch's scope. Other triggers that resolve to it get "read-only access" (changelog , reference)
10 September 2026	<code>cache-mode</code> , at workflow or job level, with the values <code>read</code> , <code>write</code> , <code>write-only</code> and <code>none</code> . The job value overrides the workflow value. "Explicitly declaring <code>cache-mode: write</code> or <code>cache-mode: write-only</code> reintroduces the risk of cache-poisoning" on a low-trust trigger (changelog , syntax)

```
jobs:
  publish:
    cache-mode: none      # a job that holds a publishing identity restores nothing it did not build
```

Setup actions cache implicitly, so check them too: `astral-sh/setup-uv` v10 disables its cache by default for `pull_request_target`, `workflow_run` and `release` events ([release notes](#)). Artifacts follow the same rule: "A `workflow_run` workflow should treat artifacts uploaded by other workflows as untrusted data, since their contents can come from a fork" ([securely using pull_request_target](#)).

21A.12 Self-hosted runners

In one sentence. A self-hosted runner is your machine, on your network, executing whatever the workflow says, and it keeps its state between jobs unless you make it ephemeral.

Precisely. "Self-hosted runners should almost never be used for public repositories on GitHub, because any user can open pull requests against the repository and compromise the environment." Private repositories are not exempt: "anyone who can fork the repository and open a pull request (generally those with read access to the repository) are able to compromise the self-hosted runner environment". GitHub-hosted runners are "ephemeral and clean isolated virtual machines"; self-hosted ones "can be persistently compromised by untrusted code in a workflow" ([hardening for self-hosted runners](#)). The fork approval gate does not help once a contributor is past it: the code "will execute automatically if the user is allowed to bypass approval in the set approval policy or if the pull request is approved" ([repository settings](#)).

The controls, all documented: ephemeral or just-in-time runners that take one job and are removed ([reference](#)); runner groups that restrict which repositories may use a runner; the organization setting that restricts or disables repository-level runners ([organization settings](#)); approval for all external contributors. Note also that on self-hosted runners environments do not isolate secrets ([deployments and environments](#)). Chapter 20B covers runner operation.

21A.13 Environment protection

In one sentence. An environment puts a human decision, a branch condition, or both between a job and the secrets and cloud identity that belong to a deployment target.

Precisely. "All deployment protection rules must pass before a job referencing the environment is sent to a runner", and environment secrets are available only to jobs that reference the environment, after approval if reviewers are required ([deployments and environments](#)). For security review, three details matter:

- Referencing a name that does not exist creates an environment with no rules, and "anyone that can edit workflows in the repository can create environments via a workflow file, but only repository admins can configure the environment" ([managing environments](#)). A typo in the name silently removes the gate.
- Deployment branch rules are evaluated against the ref the run executes on: since 8 December 2025 that is `refs/pull/number/merge` for `pull_request` events and the default branch for `pull_request_target` ([changelog](#)).
- Required reviewers and wait timers are plan-gated: on Free, Pro and Team they exist only for public repositories. Chapter 20B has the plan table.

Combined with OIDC, an environment name in the subject means the cloud role can be assumed only by a job that passed that environment's rules.

21A.14 Static analysis of workflows

Tool	Layer	What it finds	Source
CodeQL for workflows	GitHub code scanning; generally available since 22 April 2025; enabled by default setup when workflow files exist on the default branch	"missing required permissions, dangerous inputs without proper validation, and script injection vulnerabilities"; since CodeQL 2.26.4, mutable references to reusable workflows	changelog , changelog
zizmor	Third party; v1.30.1 observed on 1 October 2026	Audits named <code>template-injection</code> , <code>dangerous-triggers</code> , <code>excessive-permissions</code> , <code>unpinned-uses</code> , <code>cache-poisoning</code> , <code>artipacked</code> , <code>overprovisioned-secrets</code> , <code>secrets-inherit</code> , <code>self-hosted-runner</code> , <code>dependabot-cooldown</code> and others	audits
actionlint	Third party; v1.7.12	Syntax and expression type checks, shellcheck integration, and "script injection by untrusted inputs, hard-coded credentials"	README
OpenSSF Scorecard	Third party; results can surface in code scanning	The checks Dangerous-Workflow, Token-Permissions and Pinned-Dependencies	checks

Neither zizmor nor actionlint is installed in this course's environment, so no output is shown. A scanner finds patterns; it does not know which of your jobs holds the credential that matters. Run one in CI as a required check, and still read the file.

21A.15 The platform changes of 2025 and 2026

The dates are sourced. Pairing each change with an incident is the researchers' inference, not GitHub's statement; GitHub's changelog posts do not name the incidents.

Date	Change	Section
22 April 2025	CodeQL analysis of workflow files generally available	21A.14
15 August 2025	Allowed-actions policy can block entries and require full commit pins	21A.8
28 October 2025	Immutable releases generally available	21A.7
8 December 2025	<code>pull_request_target</code> always uses the workflow from the default branch; environment branch rules follow the execution ref	21A.5, 21A.13
5 February 2026	Action allow-listing on all plans	21A.8
23 April and 15 July 2026	Immutable OIDC subject claims: opt-in, then default for new repositories	21A.10
18 June and 20 July 2026	<code>actions/checkout</code> v7 refuses fork pull request code under privileged triggers; back-ported	21A.5
26 June 2026	Read-only default-branch cache for low-trust triggers	21A.11
14 July 2026	Dependabot version updates get a default cooldown	21A.7
28 July 2026	Runs "identified as potentially malicious" are held until a collaborator with write access approves them; public repositories only, no configuration (changelog)	21A.18
10 September 2026	<code>cache-mode</code>	21A.11
17 September 2026	Workflow execution protections generally available	21A.8
2 November 2026	The default rule that disables <code>pull_request_target</code> in affected public repositories is enforced	21A.8

Unverified. The heuristics behind the holds on suspicious runs are not documented. Do not count on them as a control.

21A.16 Workflow 12: a secure-by-default workflow

`workflows/12-secure.yml` puts the controls in one file: a test job, a job that reads the pull request title, a dependency review, and a staging deployment through OIDC. It was assembled from documented syntax and input names and parse-checked; it was not executed on GitHub, and Lab 29.2 has you run it. Read it by asking five questions, which `grep` can answer for any workflow.

```
# Question 1: which events start it? Question 2: what may the token do?
$ grep -n -A4 '^on:' 12-secure.yml
15:on:
16-  pull_request:
17-  push:
18-    branches: [main]
19-
$ grep -n -A2 'permissions:' 12-secure.yml
21:permissions:
22-  contents: read
23-
--
60:    permissions: {}
61-    steps:
62-      - name: Title must not be empty or longer than 72 characters
--
80:    permissions:
81-      contents: read
82-    steps:
--
104:    permissions:
105-      contents: read
106-      id-token: write
```

`pull_request`, not `pull_request_target`: fork code runs without secrets. The workflow token is read-only; the title job has none (`{}`); only the deployment job adds `id-token: write`.

```
# Question 3: whose code runs, and is every reference a full commit ID?
$ grep -n 'uses:' 12-secure.yml
37:    uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
43:    uses: astral-sh/setup-uv@c18668ad3cf93ea998bef934396af7bb5c839dc7 # v10.2.0
84:    uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
89:    uses: actions/dependency-review-action@a1d282b36b6f3519aa1f3fc636f609c47dddb294 # v5.0.0
114:    uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
120:    uses: aws-actions/configure-aws-credentials@e1253824e5c10ff9df46874f81ed3ec929e19cfd #
v6.3.0
# References that are not 40 hexadecimal characters (no output means none):
$ grep -n 'uses:' 12-secure.yml | grep -vE '@[0-9a-f]{40}(\ |$)'
[exit status: 1]
```

Six references, six commit IDs, all from `workflows/ACTION_PINS.md`. Exit status 1 from the second command means `grep -v` found nothing to print.

```
# Question 4: where does an expression appear, and is any of them inside a script?
$ grep -n '${{' 12-secure.yml
26:  group: ${{{ github.workflow }}}-${{{ github.ref }}}
27:  cancel-in-progress: ${{{ github.event_name == 'pull_request' }}}
56:    if: ${{{ github.event_name == 'pull_request' }}}
66:      PR_TITLE: ${{{ github.event.pull_request.title }}}
77:    if: ${{{ github.event_name == 'pull_request' }}}

```

```

96:     if: ${ github.event_name == 'push' && github.ref == 'refs/heads/main' && vars.AWS_ROLE_ARN !=
'' }}
122:         role-to-assume: ${ vars.AWS_ROLE_ARN }}
123:         aws-region: ${ vars.AWS_REGION }}

```

Classify each line by where the value lands. Lines 26, 27, 56, 77 and 96 are evaluated by Actions and never reach a shell. Lines 122 and 123 are action inputs taken from repository variables, which only people with repository access set. Line 66 is the one untrusted value, and it sits under `env:`. No expression appears inside a script.

```

# Question 5: which stored secrets does it read? (no output means none)
$ grep -n 'secrets\' 12-secure.yml
[exit status: 1]
$ grep -n -B1 -A1 'id-token' 12-secure.yml
105-     contents: read
106:     id-token: write
107-     # Control 8: a job that holds a cloud identity neither restores nor saves a cache.

```

The workflow reads no stored secret. Its only credential beyond the job token is the OIDC token of the deployment job.

Control in the file	Threat it answers	Section
permissions: contents: read at the top; {} and per-job additions below	A compromised step using a write token	21A.3
on: pull_request	Fork code running next to secrets	21A.4, 21A.5
Every uses: is a commit ID with a version comment	A moved tag	21A.7
persist-credentials: false on every checkout	Later steps using the token through Git	21A.3
The title only under env:	Script injection	21A.6
The deployment if: requires a push to main	Unreviewed code reaching the cloud identity	21A.10
environment: staging	A deployment without the environment's rules; it also fixes the OIDC subject	21A.13
id-token: write on one job only	Any other job requesting a cloud token	21A.10
cache-mode: none on the deployment job	A poisoned cache executing next to the cloud identity	21A.11
timeout-minutes and concurrency	Runaway or overlapping runs	Chapter 20B

What it does not do: it does not attest the build, it does not scan itself (add CodeQL default setup), and the staging role's trust policy lives in the cloud account, where the workflow cannot enforce it.

21A.17 AI agents inside workflows

An agent in a workflow (an LLM reviewer, a triage bot, a coding agent) combines parts 3 and 4 of the model in a new form: it reads attacker-controlled text and holds credentials, and for an agent, text is instruction. No `env:` trick separates data from code in a prompt.

Two findings from the report. An automated account ran a week-long campaign in February and March 2026 that combined `pull_request_target` abuse, branch-name and filename injection, and prompt injection against an AI reviewer ([StepSecurity](#)). And researchers reported in April 2026 that AI coding agents run as GitHub Actions could be steered by text in pull request titles, issue bodies or comments into revealing CI secrets ([Techzine](#)); this is secondary reporting only, and the primary write-up was not fetched.

The report's recommendation, labelled there as an inference: an agent in a workflow gets a dedicated low-privilege, spend-capped key; no write token unless required; and it does not run automatically on untrusted contributions. In this chapter's terms: give the agent's job `permissions` as if the pull request author had written the job, because through the prompt they partly did.

21A.18 Case studies

Each case is given as what happened, root cause, lesson, from the Phase 0 report and its sources. Flags from the report are carried in the last column. No GitHub output or attack detail beyond the published summaries is reproduced.

Incident	What happened	Root cause	Lesson	Flags
tj-actions/changed-files, March 2025 (CISA, advisory)	Version tags were repointed to a malicious commit that printed secrets from runner memory into workflow logs; more than 23,000 repositories used the action	A stolen bot token; the chain began with a <code>pull_request_target</code> flaw in an upstream project (Unit 42)	Tags are mutable; only commit-pinned workflows were unaffected; public logs are world-readable	Conflict: CISA gives the window as 12 March 00:00 UTC to 15 March 12:00 UTC 2025, the advisory says 14 to 15 March, Unit 42 places the mass tag override at 14 March 16:57 UTC. Use CISA's window for audits. How many repositories leaked secrets is not stated by either; a "218 repositories" figure is secondary and unverified
Nx "s1ngularity", August 2025 (post-mortem)	Malicious versions of eight packages were live for about four hours and harvested developer credentials	A <code>pull_request_target</code> workflow that echoed an unsanitised pull request title; a legacy read-write default token; a publish workflow that could be dispatched	One injectable line plus a write token reaches a publish credential	None
GhostAction, September 2025 (GitGuardian)	Compromised maintainer accounts pushed a workflow that sent secrets to an attacker's server: 3,325 secrets from 817 repositories	Write access equals secret access	Protect workflow files with code-owner review; prefer environment-scoped secrets and OIDC	Vendor report

Incident	What happened	Root cause	Lesson	Flags
Shai-Hulud and its sequel, September and November 2025 (GitHub Blog , Wiz)	A self-replicating npm worm stole tokens, pushed secret-dumping workflows, and in its second wave registered infected machines as self-hosted runners	Stolen developer and CI tokens	A token with workflow write access is equivalent to every secret it can reach; monitor for new workflows, runners and repositories	The size of the second wave is approximate: the notes cite the same Wiz post for about 700 malicious versions and about 800 packages
PyTorch runners (disclosed January 2024) and Ultralytics (December 2024) (researcher write-up , PyPI)	Persistent self-hosted runners on a public ML repository could be reached by a pull request; a poisoned Actions cache led to malicious PyPI releases published through the legitimate workflow	Self-hosted runners with weak approval settings; an insecure trigger plus cache trust	ML projects are prime targets; use ephemeral runners and treat caches as untrusted	PyTorch is a researcher disclosure, not an observed attack
Trivy and LiteLLM, February to March 2026 (advisory , Aqua notice , Datadog)	A <code>pull_request_target</code> flaw leaked a token; weeks later a malicious scanner release was published and almost all action tags were force-pushed to malicious commits; a downstream LLM gateway library that ran the scanner unpinned had its publishing credential stolen, and two malicious versions were on PyPI for about three hours	Incomplete, non-atomic credential rotation after the first incident; tag-pinned security tooling	Rotate everything at once; security tools in CI are high-value targets; pin by commit	Conflict: 75 of 76 tags (Wiz) versus 76 of 77 (Datadog); write-ups name the account behind the first exploit differently. Use the advisory and the vendor notice for exact figures
TanStack, 11 May 2026 (post-mortem)	84 malicious versions of 42 packages were published through the project's own trusted-publisher identity; no npm token was stolen	A <code>pull_request_target</code> workflow built fork code; the fork poisoned the shared cache; the release job restored it; malware read the job's OIDC token from runner memory	Read-only permissions did not block cache writes; OIDC is not safe if untrusted code runs in the job; <code>pull_request_target</code> bypassed the first-time-contributor gate	Whether the versions carried valid provenance attestations, and the "Mini Shai-Hulud" name, come from secondary reports; the post-mortem does not say so. No GitHub-authored post-mortem was found
Codecov, 2021 (Codecov)	A modified uploader script exported CI environment variables for two months	A mutable script fetched and executed in CI	Remote scripts are the same risk class as mutable tags	History only

A further tag hijack, of the actions-cool actions in May 2026, is known to this course only through secondary reporting ([The Hacker News](#)); it is mentioned for completeness and nothing is built on it.

Three patterns recur. Mutable references were repointed after a maintainer credential was stolen (tj-actions, Trivy). A privileged trigger ran or interpolated outsider input (the upstream of tj-actions, Nx, Trivy, TanStack). Stolen

tokens were used to push workflows (GhostAction, Shai-Hulud). And one technique recurs across them: reading the runner process's memory to collect secrets and OIDC tokens. The report's conclusion is the sentence to carry into a design review: masked logs are irrelevant once attacker code runs in a job; "the robust controls are preventing untrusted code from running in privileged jobs and limiting which secrets a job holds."

21A.19 A review checklist for workflow pull requests

Use it on every pull request that touches `.github/workflows/`, an action definition, or a script that a workflow runs. Lab 29.3 applies it.

1. **Trigger.** Does the change add or keep `pull_request_target`, `workflow_run`, `issue_comment`, `issues` or `discussion`? If so, which outsider-controlled input reaches the job?
2. **Checkout.** Any `ref:` or `repository:` taken from the event? Any `allow-unsafe-pr-checkout`? Any `git fetch` or `gh pr checkout` of pull request code in a privileged job?
3. **Permissions.** Is there a top-level `permissions` block, read-only? Is each `write` scope on the one job that needs it? Did the change widen anything?
4. **Expressions.** Is there any `${{ }}` inside `run:`, or inside an input that is evaluated as code? Including multi-line scripts?
5. **Actions.** Is every `uses:` a 40-character commit ID from the action's own repository, with a version comment? Is a new action necessary, and who maintains it?
6. **Remote code.** Any download piped into a shell, or an installer fetched without a checksum?
7. **Secrets.** Is each secret referenced in the narrowest place: one step, in a job behind an environment? Could OIDC replace it? Does any secret appear on a command line?
8. **Privileged jobs.** For each job that publishes, deploys or holds `id-token: write`: does it run only on trusted refs, behind an environment, with `cache-mode: none` or `read`, and with no artifact from an untrusted run executed?
9. **Runner.** Did `runs-on` change to a self-hosted label? For which events?
10. **Credentials in Git.** `persist-credentials: false` wherever the job does not push?
11. **Process.** Was the change reviewed by a code owner of `.github/workflows/`, and does a ruleset require that? Does a workflow scanner run on the pull request?

21A.20 What can go wrong

Symptom	Diagnosis	Fix	Prevention
A step fails with "Resource not accessible by integration" after you add <code>permissions</code>	Naming one scope set the others to <code>none</code>	Add the missing scope to that job only	Decide scopes per job when the job is written
CI works for the team and fails for outside contributors	The job needs a secret; fork runs get none	Split secret-dependent work out of pull request CI (section 21A.5)	Design pull request CI to need no secrets
A public repository's <code>pull_request_target</code> workflow stops running after 2 November 2026	The default event rule is enforced	Migrate to <code>pull_request</code> , or allow the event explicitly after review	Review the evaluate-mode insights before the date
The cloud rejects the OIDC token	The subject does not match the trust policy: wrong environment or branch, or the immutable format	Compare the token's <code>sub</code> with the policy	Keep trust policies in code, next to the workflow
A secret appears in a log	Redaction is not guaranteed	Delete the log and rotate the secret	Keep secrets out of jobs that print

Symptom	Diagnosis	Fix	Prevention
			untrusted data
An unknown workflow file or runner appears	A token with write access was used	Chapter 21B: contain, rotate everything at once	Code-owner rule on workflows; audit log alerts

21A.21 When not to use it, and dangerous edge cases

- `pull_request_target`: not for building, testing or linting. If you cannot state why the job needs a write token or a secret, it does not need the trigger.
- **Self-hosted runners**: not for public repositories, and not persistent ones for anything a pull request can reach.
- `secrets: inherit` **and workflow-level** `env: secrets:` not when any job in scope runs third-party code.
- **The edge case behind most incidents**: two safe-looking workflows that share something. A low-privilege workflow that can write a cache, an artifact, a label or a branch, and a high-privilege workflow that trusts it. Review workflows as a set.

21A.22 Command safety

Editing workflow files is ordinary Git work; the risk lies in what the files do once pushed.

Command	Label	What it changes	Preview	Recovery
<code>git ls-remote --tags <url></code>	● SAFE	Nothing; lists a remote's tags	Not needed	Not needed
<code>grep -n 'uses:' .github/workflows/*.yml</code>	● SAFE	Nothing	Not needed	Not needed
<code>git tag -f -a v1</code>	● CAUTION	Moves a local tag	<code>git rev-parse 'v1^{commit}'</code>	Re-create the tag at the old commit
<code>git push --force origin v1</code>	● DANGEROUS	Replaces a published tag (section 21A.7)	<code>git ls-remote --tags origin v1</code>	Push the old tag object back, if you still have it
Pushing a change under <code>.github/workflows/</code>	● CAUTION	Git: an ordinary commit. GitHub Actions: the next matching event runs the new file with the repository's token and secrets	Review the diff with section 21A.19	Revert the commit; rotate any secret the changed workflow could read

21A.23 Version notes

Section 21A.15 lists every dated change. The four that change how you write a workflow:

Topic	Older	Current	Since	Recommended
Default token	Read and write	Read-only for new repositories and organizations	2 February 2023	Declare <code>permissions</code> in every workflow
Checkout of fork code under privileged triggers	Allowed	Refused without an explicit flag	checkout v7, 18 June 2026	Never set the flag for code that runs

Topic	Older	Current	Since	Recommended
Default-branch cache	Writable by any run in its scope	Read-only for low-trust triggers; <code>cache-mode</code>	26 June and 10 September 2026	<code>cache-mode: none</code> or <code>read</code> in release jobs
<code>pull_request_target</code> in public repositories	Runs	Blocked by a default rule unless allowed	Enforced 2 November 2026	Decide per workflow before that date

Outdated advice. "Use `pull_request_target` so that CI works for forks" and "pin to the major tag so that you get fixes automatically" are both still common in tutorials. The first is the pattern behind most of section 21A.18. The second is the tj-actions incident.

21A.24 Practice

Do the Module 29 labs: Lab 29.1 (five planted weaknesses in `workflows/vulnerable/`), Lab 29.2 (justify each control of workflow 12), Lab 29.3 (review a pull request diff). Then re-read Chapter 19, section 19.8, and add a code-owner line for `.github/workflows/` to your practice repository. Chapter 21B continues with repository security and the response to a leaked secret.

21A.25 Interview questions

1. A workflow has no `permissions` key. What can its token do, and what does the answer depend on?
2. Why does a fork pull request under `pull_request` not receive secrets, and what remains exposed anyway?
3. Explain a "pwn request" without using the word. Which step completes the vulnerability?
4. Why does moving `${{ github.event.pull_request.title }}` from `run:` to `env:` fix an injection, when the same value still reaches the same shell?
5. Your team pins every action to a commit. Name three things that this does not protect against.
6. What does `id-token: write` grant, and where is the access decision made?
7. In the TanStack incident no credential was stolen from storage. Walk through how a fork pull request led to a publish, and name the control that would have broken each link.
8. A colleague says "the secret is masked in the logs, so it is safe". Respond.
9. When is `pull_request_target` the right trigger, and what changes on 2 November 2026?
10. Why are self-hosted runners and public repositories a dangerous combination, and what makes ML projects particularly exposed?
11. How would you give an LLM review agent access to pull requests from outside contributors?
12. You may enable one organization-level control today. Which one, and why?

21A.26 Sources

Primary sources

- GitHub Docs: [secure use reference](#); [securely using `pull\_request\_target`](#); [script injections](#); [GITHUB_TOKEN](#); [workflow syntax](#); [OIDC reference](#); [OIDC in AWS](#); [dependency caching](#); [secrets reference](#); [managing GitHub Actions settings for a repository](#); [about Actions policies](#); [keeping your actions up to date with Dependabot](#).
- GitHub changelog entries, linked where each change is described (sections 21A.3, 21A.5, 21A.7, 21A.8, 21A.10, 21A.11, 21A.15).
- [actions/checkout README at v7.0.1](#).
- Advisories and post-mortems: [CISA on tj-actions](#); [GHSA-mrrh-fwg8-r2c3](#); [Nx post-mortem](#); [GHSA-69fq-xp46-6x23](#); [Aqua incident notice](#); [TanStack post-mortem](#); [PyPI on Ultralytics](#); [Codecov security update](#).

Secondary sources

- [GitHub Security Lab: preventing pwn requests](#) (2021; parts 2 to 4 of the series were not consulted).
- Vendor and researcher analyses: [Unit 42](#); [GitGuardian on GhostAction](#); [Wiz on Shai-Hulud 2.0](#); [Datadog Security Labs](#); [StepSecurity on hackerbot-claw](#); [John Stawinski on PyTorch](#).
- Secondary reporting only: [Techzine](#); [The Hacker News](#).
- Tools: [zismor audits](#); [actionlint](#); [OpenSSF Scorecard checks](#).

Videos (from the Phase 0 report, with its caveats)

- [Adnan Khan, "The dark side of GitHub Actions"](#), RomHack 2024, 53 min. The vulnerability classes are current; GitHub defaults have changed since.
- [Niek Palm, "Beyond the Commit: Weaponizing and Hardening GitHub Actions"](#), NDC Security 2026, 55 min. The most current defender-oriented talk; it pre-dates the June to September 2026 platform changes.
- ["Tag, You're Leaked: Surviving the tj-actions Supply Chain Attack"](#), BSides PDX 2025, 24 min. An incident-response case study.

Further reading

- [GitHub's 2026 security roadmap for Actions](#): announced work, not shipped features.
- `workflows/ACTION_PINS.md` in this course: the pins and how to re-verify them.

Chapter 21B: Repository security, identity, the Git client, and secret-leak response

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch21b/`. Every secret in this chapter is a dummy such as `DUMMY-KEY-not-a-real-secret-12345`, spelled so that no scanner pattern matches it. GitHub Actions security is Chapter 21A; the incident drills are Chapter 30.

21B.1 Why this matters

A CTO asks four questions after a security scare, and none of them is about a command.

1. "A key was in the repository. We deleted it. Are we safe?"
2. "This commit says Asha wrote it. Did she?"
3. "An intern cloned a repository from a paper's footnote. What could that have run on the laptop?"
4. "If one token leaks, what can the holder reach, and how would we know?"

Each question has a wrong answer that sounds reasonable. Deleting a file adds a commit and removes nothing. An author name is text. A clone is safe and an unpacked archive is not. A token's reach depends on its type, and most teams cannot say which type their automation uses.

This chapter gives the mechanisms behind the right answers, at three layers that you must keep apart: Git on your machine (what runs, what a commit claims, what history contains), GitHub (what is displayed, scanned, blocked and retained), and the issuer of a credential (whether a leaked secret still works).

The third layer is the one tutorials leave out, and it is where incidents are decided: a secret is harmless from the moment its issuer revokes it, and dangerous until then, whatever you do to the repository.

21B.2 The Git client: what a clone runs, and what it does not

In one sentence. Cloning a repository copies its objects and refs and nothing that Git would execute, so cloning and reading is generally safe; running Git inside a `.git` directory that somebody else wrote is not.

Analogy. A clone is a photocopy of a book: you get every page, and none of the previous owner's sticky notes that say "when you open this, also call this number". An unpacked archive that contains `.git` is the previous owner's own copy, sticky notes included, and Git follows them. The analogy breaks in one place: the pages can still contain code that you later choose to run (a `Makefile`, a notebook). Git protects you from Git running something, not from yourself running the project.

Precisely. Two kinds of file under `.git` can make Git execute a program: hooks (`.git/hooks/*`, or wherever `core.hooksPath` points) and configuration (`.git/config`, where an alias beginning with `!`, `core.pager`, `core.editor`, `core.fsmonitor`, `core.sshCommand`, a credential helper, a filter driver and several other settings name commands). Git's manual states the rule in its SECURITY section: configuration and hooks are not copied by `git clone`, so cloning a remote repository with untrusted content and inspecting it with `git log` is generally safe; running Git commands in a `.git` directory, or the working tree around it, that came from an untrusted source is not, and the documented way to get a clean copy of such a directory is `git clone --no-local (git help git, section SECURITY)`.

Inside .git. A clone creates a new .git from the template directory of your own Git installation. Its hooks/ holds only the inactive *.sample files, and its config holds what git clone writes: the repository format, the origin remote and the branch's upstream. From the source it receives objects (in a pack) and refs. Nothing else crosses.

See it. A repository prepared by somebody else contains a post-checkout hook and an alias that runs a shell command. Both are harmless here: the hook appends a line to a log file, and the alias prints a sentence.

```
$ cat vendor-tool/.git/hooks/post-checkout
#!/bin/sh
echo "post-checkout hook ran as $(basename "$PWD")" >> "$(git rev-parse --git-dir)/hook-ran.log"
$ git -C vendor-tool config get alias.st
!echo alias from the repository config ran
```

Clone it, then look for either in the clone. --no-local forces the normal transport even for a path on the same disk, which is what the manual prescribes for untrusted sources.

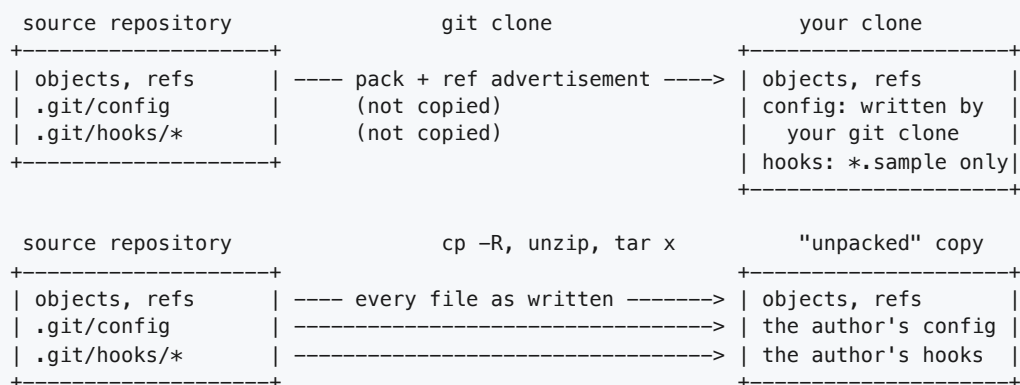
```
$ git clone -q --no-local vendor-tool cloned
$ ls cloned/.git/hooks | grep -v '\.sample$' | wc -l
0
$ git -C cloned config get alias.st
[exit status: 1]
$ git -C cloned switch -q -c try
$ ls cloned/.git | grep hook-ran || echo "no hook ran"
no hook ran
```

The clone has no active hook, no alias, and switching branches ran nothing. Now receive the same repository the other way, as a directory copy, which is what unpacking a .zip or .tar.gz that includes .git gives you:

```
# The same repository received as an archive: every file under .git arrives as written.
$ cp -R vendor-tool unpacked
$ cd unpacked
$ git st
alias from the repository config ran
$ git switch -q -c try
$ cat .git/hook-ran.log
post-checkout hook ran as unpacked
$ cd ..
```

git st ran the author's shell command, and git switch ran the author's hook. With a hostile archive those would have been any program, running as you, with your SSH agent and cloud credentials in reach.

Picture.



In production. An ML team receives a "reproduction package" for a paper as a tarball that contains a full repository. A shell prompt that shows the branch name runs git status as soon as someone changes into the

directory, and `git status` consults `core.fsmonitor` from the package's own `.git/config`. The safe procedure is the manual's: `git clone --no-local package clean`, and work in `clean`.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git clone <url></code>	created from the tip of the remote's default branch	created	created, points at the default branch	created	fresh <code>config</code> and sample hooks from your installation; <code>refs/remotes/origin/*</code> ; the pack	unchanged (read only)	unchanged

21B.3 The three client guards: `safe.directory`, `safe.bareRepository`, `protocol.file.allow`

Git added three standing defenses in 2022 ([safe configuration](#), [protocol configuration](#)). Each closes one way of getting you to run Git inside configuration that you did not write.

`safe.directory`: a repository owned by another user

In one sentence. Git refuses to read the configuration of a repository whose directory is owned by a different operating-system user, unless you have listed that directory as safe.

Precisely. Git discovers a repository by walking up from the current directory. On a shared machine another user can create `/tmp/.git` or `/scratch/.git`, and without this check any Git command you run below that directory would load their configuration and hooks. The check compares the owner of the repository directory with the user running Git. `safe.directory` lists exceptions; it is multi-valued, accepts `/path/*` for everything under a directory and `*` to switch the check off, and is honored only in protected configuration (system, global and command scope), so that a repository cannot declare itself safe (`git help config`, `safe.directory`).

See it. The sandbox cannot change file ownership without `sudo`, so this transcript uses `GIT_TEST_ASSUME_DIFFERENT_OWNER=1`, the variable Git's own test suite uses to simulate a foreign owner. The real message appears in containers and CI, where the checkout is often owned by another user ID.

```
# GIT_TEST_ASSUME_DIFFERENT_OWNER=1 makes Git treat the repository as owned by another user.
$ GIT_TEST_ASSUME_DIFFERENT_OWNER=1 git -C unpacked status -sb
fatal: detected dubious ownership in repository at '$LAB/ch21b/client-safety/unpacked'
To add an exception for this directory, call:
```

```
    git config --global --add safe.directory $LAB/ch21b/client-safety/unpacked
[exit status: 128]
```

```
$ git config set --global --append safe.directory "$PWD/unpacked"
$ GIT_TEST_ASSUME_DIFFERENT_OWNER=1 git -C unpacked status -sb
## try
[exit status: 0]
# The setting is honoured only in protected configuration. In the repository itself it is ignored:
$ git config unset --global safe.directory
$ git -C unpacked config set safe.directory '*'
$ GIT_TEST_ASSUME_DIFFERENT_OWNER=1 git -C unpacked status -sb 2>&1 | head -1
fatal: detected dubious ownership in repository at '$LAB/ch21b/client-safety/unpacked'
```

The last three lines are the important ones: writing `safe.directory = *` into the repository's own configuration changes nothing, because that file is exactly what Git declined to trust.

Outdated advice. Answers from 2022 tell you to run `git config --global --add safe.directory '*'` to make the message go away. That switches the protection off for every repository on the machine. List the

one directory, or fix the ownership.

`safe.bareRepository`: a bare repository hidden in a working tree

In one sentence. With `safe.bareRepository=explicit`, Git uses a bare repository only when you name it with `--git-dir` or `GIT_DIR`, never because you happened to be inside one.

Precisely. A bare repository is a directory with `HEAD`, `objects/`, `refs/` and `config`, and it does not need to be called `.git`. A project can therefore contain one as ordinary tracked files in a subdirectory. A clone does copy it, because to Git those are ordinary files. If you then change into that subdirectory and run any Git command, Git discovers the embedded bare repository and reads its `config`. The default in Git 2.x is `all`; `explicit` will be the default in Git 3.0 (`git help config`, `safe.bareRepository`).

```
$ git -C embedded.git log --oneline
418a3a1 Add tool
$ git config set --global safe.bareRepository explicit
$ git -C embedded.git log --oneline
fatal: cannot use bare repository '$LAB/ch21b/client-safety/embedded.git' (safe.bareRepository is
'explicit')
[exit status: 128]
$ git --git-dir=embedded.git log --oneline
418a3a1 Add tool
$ git config unset --global safe.bareRepository
```

If you do not work inside bare repositories by hand, set `explicit` globally now. The labs in this course inspect the bare "server" with `git -C server.git ...`, which is why the lab configuration leaves the default in place.

`protocol.file.allow`: clones that you did not ask for

In one sentence. Since 2022 the `file` transport defaults to the policy `user`: you may use it directly, and commands that start a clone on their own, such as submodule initialization, may not.

Precisely. `protocol.<name>.allow` is always, never or `user`. The safe network protocols default to `always`, `ext` to `never`, and everything else, `file` included, to `user`, which permits a transport only when `GIT_PROTOCOL_FROM_USER` is unset or 1 (`git help config`, `protocol.allow`). Git sets that variable to 0 when it clones a submodule. The risk it closes: a `.gitmodules` file that names a local path makes a recursive clone read from a directory on your disk that the attacker chose.

```
$ cd app
$ git submodule add ../vendor-tool vendor/tool
Cloning into '$LAB/ch21b/client-safety/app/vendor/tool'...
fatal: transport 'file' not allowed
fatal: clone of '$LAB/ch21b/client-safety/vendor-tool' into submodule path '$LAB/ch21b/client-
safety/app/vendor/tool' failed
[exit status: 128]
$ git config get protocol.file.allow
[exit status: 1]
$ git -c protocol.file.allow=always submodule add -q ../vendor-tool vendor/tool
$ git submodule status | cut -c1-9,42-
418a3a1f vendor/tool (heads/main)
```

The one-off `-c protocol.file.allow=always` is correct for a local experiment such as this one. Setting it globally removes the guard. Leave the default.

21B.4 Recursive clones, hooks, and repositories that look popular

The recurring vulnerability is the recursive clone. The statement "cloning is safe" has had exceptions, and they share a shape: a repository with submodules, cloned with `--recurse-submodules`, tricks Git into writing a file where a hook is expected and then running it.

CVE	Severity, fixed in	Mechanism in one line	Source
CVE-2024-32002	critical; 2.45.1 and backports, May 2024	on a case-insensitive filesystem with symbolic links, a submodule could write a hook into <code>.git/</code> that ran during the clone	advisory
CVE-2025-48384	high; 2.50.1 and backports, July 2025	a trailing carriage return in a submodule path checked content out to an unintended location where a hook could run	advisory
CVE-2025-48385	2.50.1 and backports	bundle URIs, which are not enabled by default	advisory
CVE-2024-52005	default changed in core Git 2.55	unfiltered terminal escape sequences sent by a remote	advisory

macOS meets the first row's conditions: its default filesystem is case-insensitive and supports symbolic links. Release 2.45.2 reverted part of the hardening of 2.45.1 because it had broken Git LFS ([release notes](#)). No core-Git advisory was published in 2026 up to 1 October; Git for Windows published several Windows-specific ones ([advisories](#)). Your 2.55.0 contains every published core fix, and so does Apple's 2.50.1. The advisory for CVE-2025-48384 names the standing workaround: do not recursively clone submodules of untrusted repositories.

Hooks. `githooks(5)` documents 28 hooks, six of which run on the receiving side of a push. Client-side hooks live in `$GIT_DIR/hooks` unless `core.hooksPath` redirects them, are not cloned, and `pre-commit` and `commit-msg` are skipped by `--no-verify` ([githooks](#)). Since Git 2.54 hooks can also be declared in configuration, including global and system configuration, so the question "why did a hook run?" is now answered with `git hook list --show-scope <event>` ([git-hook](#)). A client-side hook is therefore not a control: the person it is meant to stop can skip it (section 21B.12).

Fake popularity. For an engineer who clones research code weekly, the ecosystem is a larger risk than the client. Researchers documented a network of more than 3,000 GitHub accounts that distributed malware through repositories made to look popular ([Check Point](#)), hundreds of fake project repositories that stole about 5 BTC ([Kaspersky](#)), and about six million suspected fake stars ([arXiv](#)). Stars, forks and a polished README are not evidence of legitimacy.

A rule set that follows from sections 21B.2 to 21B.4:

1. Clone; do not unpack somebody else's `.git`. If you must, `git clone --no-local` it first.
2. Treat `git clone --recurse-submodules` of an untrusted repository as running its code. Clone without submodules, read `.gitmodules`, then decide.
3. Set `safe.bareRepository=explicit`. Leave `protocol.file.allow` alone. Never set `safe.directory=*` on a workstation.
4. Cloning is the safe part. `pip install -e .`, `make` and a notebook's first cell run the project's code. Read before you run, or run in a container without your credentials.

21B.5 Identity: author fields are assertions

In one sentence. The author and committer of a commit are two lines of text that the person running Git supplies, and neither Git nor a push checks them against anything.

Analogy. The return address on an envelope. The postal service delivers the letter whatever you write there. A signature is the wax seal: it proves which seal pressed the wax, and it still says nothing about what the letter asks you to do.

Precisely. `git commit` takes the author from `GIT_AUTHOR_NAME` and `GIT_AUTHOR_EMAIL`, or from `--author`, or from `user.name` and `user.email`, and the committer from the `GIT_COMMITTER_*` variables or the same configuration. Those strings become the `author` and `committer` headers of the commit object and are covered by the commit ID. The transport authenticates the *pusher* (Chapter 16). It does not compare the pusher with the names inside the commits being pushed, and it could not: pushing other people's commits is what a merge, a rebase and a cherry-pick do every day.

See it. In a billing service, the lab user makes a commit that claims Asha as both author and committer:

```
$ printf 'RATE = 0.0\n' > tax.py
$ GIT_AUTHOR_NAME='Asha Rao' GIT_AUTHOR_EMAIL=asha@example.com GIT_COMMITTER_NAME='Asha Rao'
  GIT_COMMITTER_EMAIL=asha@example.com git commit -q -am 'Set tax rate to zero'
$ git log --format='%h author=%an <%ae> committer=%cn <%ce> signature=%G?'
42f5693 author=Asha Rao <asha@example.com> committer=Asha Rao <asha@example.com> signature=N
be06446 author=Lab User <you@example.com> committer=Lab User <you@example.com> signature=N
```

`%G?` prints `N`: no signature. Nothing else distinguishes this commit from one Asha made. The object is unremarkable:

```
$ git cat-file -p HEAD
tree 11b14212895ed45f1584d7ddb52f93c7e9727b61
parent be06446cd1b687a100823496b6bedd9ea544b35c
author Asha Rao <asha@example.com> 1788755700 +0530
committer Asha Rao <asha@example.com> 1788755700 +0530

Set tax rate to zero
```

Chapter 14B, section 14B.18 continues this locally: the forged commit next to a signed one, and what `git verify-commit` and an allowed-signers file report. This chapter takes the platform side.

GitHub, not Git. GitHub attributes a commit to an account by matching the email address in the commit with the addresses on accounts ([why are my commits linked to the wrong user](#)). The avatar and the profile link beside a commit are therefore a lookup of a string that the committer chose. Anyone with push access can make a commit display as a colleague, or as any public figure whose email is known ([demonstration](#)).

Four separate questions hide behind "who made this commit", and they have four separate answers:

Question	Answered by	Layer	Can it be forged by someone with push access?
Who is displayed?	email matching	GitHub	yes
Who pushed it?	the authenticated credential, recorded in the Activity view and audit log	GitHub	no, but a stolen credential pushes as its owner
Who signed it?	a signature verified against a key	Git locally, GitHub for display	only with the private key
What is enforced?	a rule that rejects unsigned or unverified commits	GitHub	no

In production. A reviewer approves a pull request because "the last three commits are from the platform lead". A compromised contributor account authored them under her email. Displayed identity is not evidence; the audit log's pusher and a required signature are.

21B.6 Signatures on GitHub: the verification states

In one sentence. GitHub checks each commit's signature against keys registered on accounts and shows the result as a badge; without vigilant mode an unsigned commit gets no badge at all.

Precisely. GitHub verifies GPG, SSH and S/MIME signatures. A cryptographically verifiable signature is marked "Verified" or "Partially verified"; a signature that cannot be verified is "Unverified"; an unsigned commit shows no status by default ([about commit signature verification](#)). For SSH, the public key must be added to the account as a *signing* key, which is a separate registration from an authentication key even when it is the same key file:

```
gh ssh-key add ~/.ssh/id_ed25519_signing.pub --type signing --title "laptop signing key"
```

Vigilant mode changes what silence means. With "Flag unsigned commits as unverified" enabled in your account's SSH and GPG keys settings, the states become ([displaying verification statuses](#)):

State	Without vigilant mode	With vigilant mode enabled by the person the commit names
Verified	signed, and the signature verifies	signed and verified, and the committer is the only author who has enabled vigilant mode
Partially verified	not shown	signed and verified, but the commit has an author who is not the committer and who has enabled vigilant mode
Unverified	signed, but the signature could not be verified	also any <i>unsigned</i> commit attributed to that person
no badge	not signed	does not occur for that person's commits

Vigilant mode is the cheap defense against the forgery of section 21B.5: once Asha enables it, the commit that claims her name without her key is displayed as "Unverified" instead of looking like all her other unsigned commits. Enable it only after you sign on every machine you commit from, or your own work is flagged.

Persistent verification records. Since 10 December 2024, once GitHub has verified a signature the commit stays verified within its repository network: a record is stored with the commit, cannot be edited, and persists when the key is later rotated, revoked or expired, or the contributor leaves the organization. GitHub does not re-verify old commits when a key's state changes; the time of verification is exposed as `verified_at` in the REST API ([persistent verification](#), [changelog](#)). Rotating a key no longer turns years of history "Unverified". But revoking a *stolen* key does not un-verify what an attacker signed with it: you find those commits by date and by the audit log, not by the badge.

Commits that GitHub signs. Commits you make in the web interface are signed by GitHub with its own key, which you can check locally against `https://github.com/web-flow.gpg`; that includes the merge commit of "Create a merge commit" and the single commit of "Squash and merge" ([about commit signature verification](#)). That badge says GitHub created the commit on behalf of a signed-in user, not that the user's key was involved. Dependabot signs its commits by default, and the Copilot cloud agent has signed its commits since 3 April 2026 ([changelog](#)).

Why "Rebase and merge" yields unsigned commits. A rebase creates new commits with new parents and a new committer (Chapter 9). A signature covers the commit's content including its parents, so the original signatures cannot be carried over, and GitHub does not have your private key to make new ones. The documentation states the result: the commits are created without signature verification ([signature verification for rebase and merge](#)). A history rewrite strips signatures for the same reason (section 21B.16).

The require-signed-commits rule. Display is not enforcement. The ruleset rule "Require signed commits" means contributors and bots can push only commits that are signed and verified; under rulesets GitHub checks only the commits that are not reachable from other branches ([available rules](#)). The rule therefore makes "Rebase and merge" unusable on the protected branch, while squash and merge commits pass because GitHub signs them (Chapter 18).

Unverified. Sigstore's keyless `gitsign` signs with short-lived certificates tied to an OIDC identity, and its README says GitHub does not display those commits as verified ([gitsign](#)). The Phase 0 report could not re-verify this, or GitHub's support for SSH certificates in verification, against a 2026 primary source.

21B.7 What a signature does not prove

A verified signature proves one thing: the holder of a particular private key created this exact commit object. It does not prove that the change is correct, reviewed, or benign, or that the key holder is who you think, or that the person had not been turned or impersonated socially.

The xz-utils backdoor of 2024 is the case to remember. It was introduced by a contributor who had earned commit rights over more than two years of useful work, and the build script that activated it was present only in the release tarball, not in the Git repository ([timeline](#)). Every commit could have carried a perfect signature. Signing would have changed nothing, for two reasons that generalize:

- **The signer was authorized.** Signatures authenticate; they do not judge. Review, and limits on what one maintainer can ship alone, are the controls for a trusted insider.
- **The artifact was not the repository.** What users ran was a tarball. A signature on commits says nothing about a release archive, a wheel or a container image built somewhere else. That gap is closed by build provenance and artifact attestations, which belong to the Actions chapters.

Signing removes cheap impersonation and anchors the audit trail. It is one layer.

21B.8 Credentials: what a stolen one can reach

Chapter 16 explains how each credential type authenticates. The incident question is different: if it leaks, what can it reach, and for how long?

Credential	Reach if stolen	Lifetime	Source
Actions <code>GITHUB_TOKEN</code>	the workflow's repository, within the job's <code>permissions</code>	the job	credential types
GitHub App installation token	the installation's repositories and permissions	one hour	same
GitHub App private key	mints installation tokens for every installation of the app	never expires	GitGuardian
Fine-grained personal access token	one owner, optionally selected repositories, per-permission	configurable	credential types
Deploy key	one repository, read or read-write	long-lived	credential types
Personal SSH key	everything the account can reach over SSH	long-lived	credential types
Classic personal access token	every repository and organization its owner can reach, by scope	long-lived	credential types

Two automatic protections exist and both are narrow: GitHub revokes its own tokens when they are pushed to a public repository or gist, and removes tokens unused for a year ([token expiration and revocation](#)). The App private key is the row people forget. Of 4,802 leaked GitHub App private keys tested in September 2026, 474 still authenticated, 44 of them with organization admin access ([GitGuardian](#)). Short-lived installation tokens are only as safe as the long-lived key that mints them.

A hierarchy for automation. The Phase 0 report derives this order from the facts above; it is an inference, not a GitHub statement:

1. OIDC-issued cloud tokens, and the job-scoped `GITHUB_TOKEN`, wherever the work happens inside a workflow.
2. GitHub App installation tokens, with the App's private key in a secrets manager.
3. Fine-grained personal access tokens with an expiry and organization approval.
4. Deploy keys, for read access to a single repository.
5. Classic tokens and shared machine users: last, and with a written reason.

Real compromises abuse tokens, sessions and laptops, not Git.

Case	What happened	Lesson
Heroku and Travis CI OAuth tokens, April 2022 (GitHub Blog)	stolen OAuth tokens issued to two integrators were used to clone private repositories of dozens of organizations	a third party's token store is part of your attack surface; review authorized OAuth apps
CircleCI, December 2022 (incident report)	malware on an engineer's laptop stole an already authenticated session, bypassing two-factor authentication; all customers were told to rotate their secrets	two-factor authentication protects the login, not the session that follows it
GitHub, May 2026 (GitHub Blog)	a poisoned third-party VS Code extension on an employee device led to exfiltration of GitHub-internal repositories; GitHub rotated critical secrets, including the GitHub Enterprise Server signing key	editor extensions run with the developer's access; device security is repository security

For the third case, attribution to a named group and details of the extension come from vendor reports; GitHub's post names neither, and one vendor gives the detection date as 19 May where GitHub says 18 May.

Token forensics. After a suspected compromise the question is "what did this token do?". GitHub's audit log can be searched by token without storing the token itself: compute its SHA-256 and search for the hash ([identifying audit log events performed by an access token](#)).

```
# The token is read from a variable, never typed on the command line or pasted into chat.
printf '%s' "$LEAKED_TOKEN" | openssl dgst -sha256 -binary | base64
# then search the audit log for:   hashed_token:"<the value printed above>"
```

The documented form is `echo -n TOKEN | openssl dgst -sha256 -binary | base64`; the `printf` variant hashes the same bytes and keeps the token out of your shell history. Two limits decide whether this works on the day. A search by token hash returns no Git events, in the interface or through the REST API: clones, fetches and pushes made with the token have to be identified in an export of Git events data ([identifying events performed by an access token](#)). And the retention differs: the organization audit log covers 180 days, while Git events are an Enterprise Cloud feature that the audit log retains for seven days ([organization audit log](#), [enterprise audit log](#); Chapter 13, section 13.15). To answer "was the repository cloned with this token?" months later, streaming or export must be set up before the incident.

21B.9 Secrets in repositories: the scale

The numbers below are why this chapter exists, and each comes with its source.

- GitGuardian counted 28,649,024 new secrets on public GitHub in 2025, up 34%, the largest single-year jump it has recorded. 1,275,105 of them were tied to AI services, up 81%. 24,008 unique secrets sat in MCP configuration files. Commits co-authored by Claude Code leaked secrets at roughly twice the baseline rate. 64% of the secrets confirmed valid in 2022 were still valid when retested. Internal repositories were about six times more likely than public ones to contain a hardcoded secret ([State of Secrets Sprawl 2026](#)).

- GitHub detected more than one million leaked secrets on public repositories in the first eight weeks of 2024 ([GitHub Blog](#)).
- A scan of AI companies found that 65% of the Forbes AI 50 companies with a GitHub footprint had leaked verified secrets ([SecurityWeek](#)).

"Still valid years later" means that detection without revocation achieves nothing. "Internal repositories are worse" means that privacy is being used as a substitute for hygiene. And the AI-service growth is your field: an LLM key is created in a browser tab, pasted into a notebook or a `.env`, and committed by `git add -A` or by an agent that stages everything it touched.

21B.10 Why deleting, force-pushing and making it private do not remove a secret

In one sentence. A commit is a permanent snapshot, so a later commit that deletes a file leaves every earlier snapshot intact, and moving or hiding refs changes who can easily find the old snapshot, not whether it exists.

Analogy. Publishing a correction in tomorrow's newspaper does not recall yesterday's edition from the people who bought it. Withdrawing yesterday's edition from your own archive (a force push) does not recall it either. The analogy breaks for *unpushed* commits: an edition that never left the building can be pulped (section 21B.12).

Precisely. Three operations are commonly mistaken for removal:

What people do	What it changes	Where the secret still is
Commit a deletion	adds a new commit whose tree lacks the file	in every commit between the one that added it and the deletion; reachable from the branch, from tags, from other branches
Amend or reset, then force-push	moves a ref to different commits	in the old commits, which stay in the server's object database, in every clone and fork that fetched them, and on GitHub in cached views and through pull requests that reference them
Make the repository private, or delete it	changes who may read through the normal interface	in every clone and fork made while it was public; in the fork network

GitHub states the second row itself: after a rewrite and force push, the old commits can still be accessible in clones and forks, directly by commit ID in cached views, and through pull requests that reference them ([removing sensitive data](#)). It states the third row too: Git data pushed to any repository in a fork network may be accessed from any other repository in that network, even after a fork is deleted ([forks reference](#)). Researchers showed in 2024 that this is exploitable at scale, finding 40 valid API keys in commits from deleted forks of three commonly forked repositories ([Truffle Security](#)), and in 2025 that commits orphaned by force pushes can be enumerated from public event archives and scanned, which yielded thousands of secrets including a GitHub token with admin access to all repositories of the Istio project ([Truffle Security guest post](#)).

See it. The fixture for the rest of the chapter is `ragdesk`, a retrieval-augmented helpdesk service with a bare `server.git` and two teammates' clones. Four commits back, `git add -A` swept a local `.env` into the commit "Add staging settings":

```
$ cd ragdesk
$ git log --oneline --decorate
d4b8762 (HEAD -> main, origin/main) Document setup in README
b509fe3 Add request timeout
64b9b89 (tag: v0.2.0) Add retry with backoff
0805fd8 Add staging settings
987a49d (tag: v0.1.0) Add evaluation harness
6388058 Add LLM client
0c55276 Add answer prompt template
```

```
dfd59fd Add BM25 retriever
$ cat .env
LLM_API_KEY=DUMMY-KEY-not-a-real-secret-12345
LLM_BASE_URL=https://llm.example.com
```

The usual first reaction is to stop tracking the file (🟡 `git rm --cached` changes the index and leaves the working tree file alone), ignore it, and commit:

```
$ git rm --cached -q .env
$ printf '.env\n' > .gitignore
$ git add .gitignore && git commit -q -m 'Stop tracking .env and ignore it'
$ git ls-files
.gitignore
README.md
eval.py
llm_client.py
prompts/answer.txt
retriever.py
settings.yaml
$ git grep -n DUMMY-KEY
[exit status: 1]
# The tip is clean. The commit that added the file still has it:
$ git show HEAD~4:.env
LLM_API_KEY=DUMMY-KEY-not-a-real-secret-12345
LLM_BASE_URL=https://llm.example.com
```

`git grep` on the tip finds nothing and exits with status 1. `git show HEAD~4:.env` prints the key. After the push, the server agrees on both counts: the tip of `main` is clean, and the tag `v0.2.0`, which points into the affected range, still serves the file to anyone who fetches it.

```
$ git push -q origin main
$ git -C ../server.git grep -c DUMMY-KEY main
[exit status: 1]
$ git -C ../server.git grep -n 'DUMMY-KEY' v0.2.0 -- .env
v0.2.0:.env:1:LLM_API_KEY=DUMMY-KEY-not-a-real-secret-12345
```

Inside .git. The deletion commit `d42d1a1` has a tree without `.env`. Its parent `d4b8762` has a tree with it, and so do `b509fe3`, `64b9b89` and `0805fd8` on `main` and `7fae871` on Asha's branch. All of those trees name the same blob, `e523d04`, the content of `.env`. The blob exists once in the object database and is reachable from five commits. Nothing in a new commit can change an old one, because a commit's ID is the hash of its content (Chapter 3).

```
Observed behavior : the key is gone from the files, and a scanner still reports it
Git state          : tip tree has no .env; commits 0805fd8..d4b8762 have trees that contain blob
e523d04
Mechanism          : "git rm" changes the next snapshot; earlier snapshots are immutable objects
Root cause         : the secret was pushed, so it exists in every repository that fetched those
commits
Why Git does this  : history that could be edited in place could not be verified by its hash
Correct fix        : revoke the key at its issuer; then decide whether a history rewrite is warranted
(21B.14)
Prevention         : keep secrets out of the working tree's tracked paths; block them at commit and at
push (21B.12)
```

Root cause. "Private", "deleted" and "force-pushed" change discoverability, not accessibility. The exposure window starts at the first push and does not end at deletion.

21B.11 Finding a secret in history with built-in commands

You need three commands, and you need to know what each one cannot see.

`git log -S<string>`, the pickaxe, lists commits in which the *number of occurrences* of the string changed, which means the commits that added it and the commits that removed it. Without `--all` it walks only the current branch.

```
$ git log --oneline -S'DUMMY-KEY-not-a-real-secret' -- .env
d42d1a1 Stop tracking .env and ignore it
0805fd8 Add staging settings
$ git log --oneline --all -S'DUMMY-KEY-not-a-real-secret'
d42d1a1 Stop tracking .env and ignore it
0805fd8 Add staging settings
```

Two commits: the one that introduced the key and the one that deleted it. Add `-p` to see the lines, and `--diff-filter=A` to keep only commits that added a file, which is the introduction:

```
$ git log -p --format='commit %h%nAuthor: %an <%ae>%nDate: %ad%n%n    %s' -S'DUMMY-KEY-not-a-real-secret' --diff-filter=A
commit 0805fd8
Author: Lab User <you@example.com>
Date:   Mon Sep 7 10:10:00 2026 +0530

    Add staging settings

diff --git a/.env b/.env
new file mode 100644
index 0000000..e523d04
--- /dev/null
+++ b/.env
@@ -0,0 +1,2 @@
+LLM_API_KEY=DUMMY-KEY-not-a-real-secret-12345
+LLM_BASE_URL=https://llm.example.com
```

That one transcript answers four assessment questions: which commit, who, when, and which file.

`git log -G<regex>` lists commits whose diff has an added or removed line matching a regular expression. Use it when you know the shape of a secret and not its value, and with `--all` to cover every ref:

```
$ git log --oneline --all -G'DUMMY-(KEY|TOKEN)-[a-z-]+-[0-9]+'
d42d1a1 Stop tracking .env and ignore it
54093fe Add rerank debugging notebook
0805fd8 Add staging settings
$ git log --oneline --all --name-only --format='%h %s' -G'Bearer [A-Za-z0-9-]+'
54093fe Add rerank debugging notebook

notebooks/rerank-debug.ipynb
```

The regular expression found a second leak that nobody had reported: a bearer token captured in the output cell of a notebook on a branch that was pushed and never merged. Notebook outputs leak secrets that the author never typed (Chapter 28 covers stripping outputs).

`git grep <pattern> $(git rev-list --all)` searches the *content of every commit's tree*, not the diffs. It answers "in which snapshots is the secret present?", which is the exposure, where the pickaxe answers "where did it enter and leave?".

```
$ git grep -n 'DUMMY-KEY' $(git rev-list --all) | cut -c1-9,41-
d4b876277:.env:1:LLM_API_KEY=DUMMY-KEY-not-a-real-secret-12345
7fae87198:.env:1:LLM_API_KEY=DUMMY-KEY-not-a-real-secret-12345
b509fe363:.env:1:LLM_API_KEY=DUMMY-KEY-not-a-real-secret-12345
64b9b890c:.env:1:LLM_API_KEY=DUMMY-KEY-not-a-real-secret-12345
0805fd8e8:.env:1:LLM_API_KEY=DUMMY-KEY-not-a-real-secret-12345
```

```
# Which files, in how many commits:
$ git grep -l -E 'DUMMY-(KEY|TOKEN)' $(git rev-list --all) | cut -d: -f2 | sort | uniq -c
  5 .env
  1 notebooks/rerank-debug.ipynb
```

Five snapshots contain the key (the first column is the commit, shortened by `cut`). On a large repository the argument list becomes too long for one command; pipe instead: `git rev-list --all | xargs git grep -l <pattern>`.

Finally, turn the commit into scope: which branches and tags contain it?

```
$ first=$(git log --all --format=%H --diff-filter=A -- .env)
$ git log --oneline -1 $first
0805fd8 Add staging settings
$ git branch -a --contains $first
* main
  remotes/origin/feature/streaming
  remotes/origin/main
$ git tag --contains $first
v0.2.0
```

Command	Finds	Does not see
<code>git grep <p></code>	the pattern in the current working tree's tracked files	anything in history
<code>git log -S<s></code>	commits that added or removed the string on the current branch	other branches without <code>--all</code> ; a secret that was moved within a file
<code>git log --all -G<re></code>	commits on any ref whose diff touches a matching line	commits no ref reaches
<code>git grep <p> \$(git rev-list --all)</code>	every snapshot on any ref that contains the pattern	commits no ref reaches; binary and compressed files
the same with <code>--all --reflog</code>	also commits reachable only from reflogs, such as amended ones	unreachable objects without reflog entries; other people's clones

Lab 30.3 runs all five rows. Built-in commands need you to know the pattern; a dedicated scanner knows hundreds and can test candidates against the issuer. Use the built-ins during an incident to establish scope, and a scanner for coverage.

21B.12 Secret scanning and push protection, and what they do not cover

GitHub, not Git. Everything in this section up to the local analogue is platform behavior, described from the linked documentation. Nothing here was run against GitHub.

Secret scanning scans the entire Git history on all branches, plus issues, pull requests, discussions and wikis. It runs for free on public repositories; organization-owned private and internal repositories need GitHub Secret Protection, which costs \$19 per active committer per month and is sold on Team and Enterprise plans ([secret scanning](#), [price list](#)).

Push protection blocks a push that contains a recognized secret before it reaches the repository. It comes in two forms ([about push protection](#)):

	Push protection for users	Push protection for repositories
Default	on, since 29 February 2024	off

	Push protection for users	Push protection for repositories
Requires	nothing; GitHub.com only	GitHub Secret Protection (free on public repositories)
Protects	your pushes to public repositories	every push to that repository, by anyone
Bypass leaves an alert	no, unless repository protection is also on	yes

It covers command-line pushes, commits made in the web interface, file uploads, REST API requests and, on public repositories, interactions through the GitHub MCP server. By default anyone with write access can bypass a block by choosing a reason: "It's used in tests" and "It's a false positive" create a closed alert, "I'll fix it later" an open one; the bypass is written to the audit log and emailed to administrators. Delegated bypass, part of Secret Protection for organization-owned repositories, restricts who may bypass and puts other contributors' requests through a review that expires after seven days ([delegated bypass](#)).

What it does not cover. GitHub's default protection is narrower than most engineers assume:

- User push protection guards only pushes to public repositories. A private repository without Secret Protection has no push-time check at all.
- It blocks high-confidence provider patterns. Generic secrets such as passwords and connection strings are not blocked; AI-detected passwords are explicitly excluded from push protection ([changelog](#)).
- A block is a prompt, not a wall, unless delegated bypass is configured.
- Coverage differs by provider, in three independent columns:

Credential type	Provider notified for public leaks	Blocked by push protection	Validity check
OpenAI API key; Anthropic API key; Hugging Face user access token; xAI, Groq, OpenRouter, Replicate, Databricks tokens	Yes	Yes	Yes
Google API key	Yes	No	Yes
Google Gemini API key	No	No	User alert only
Mistral AI, Cohere, DeepSeek, Pinecone	No	Yes	Yes
Perplexity, LangSmith, Weights & Biases API keys	No	Yes	No
PyPI API token; npm access token; GitHub personal access tokens	Yes	Yes	Yes (not listed for PyPI)

Source: GitHub's supported-pattern data as read for the Phase 0 report on 1 October 2026 ([supported patterns](#)). The list changes; re-read it for the providers you use. Partner notification does not guarantee revocation. GitHub revokes its own leaked tokens; Anthropic documents automatic deactivation of keys found in public GitHub repositories ([Anthropic](#)); Hugging Face lets anyone invalidate a leaked token through a revocation endpoint ([Hugging Face](#)); but an xAI key flagged on 2 March 2025 stayed valid until 30 April 2025 ([Krebs on Security](#)).

Unverified. Whether OpenAI disables API keys it finds on the public internet could not be confirmed from a primary page for the Phase 0 report. How Hugging Face and OpenAI handle partner notifications from GitHub is not documented in the pages read.

The local analogue: a push-time check in plain Git

Git, not GitHub. What follows is a `pre-receive` hook on a bare repository that you control. It reproduces the *behavior* of a push-time check so that you can see it; it is not how GitHub implements push protection.

A `pre-receive` hook runs on the receiving side before any ref is updated and rejects the whole push by exiting non-zero. This one scans every commit that the push would introduce:

```
$ cat kit/pre-receive
#!/bin/sh
# Server-side guard: refuse a push if any commit it introduces contains a secret pattern.
pattern='DUMMY-(KEY|TOKEN)-[a-z-]+-[0-9]+'
while read old new ref; do
  case "$new" in *(!0)* ;; *) continue ;; esac          # a deletion: nothing to scan
  for c in $(git rev-list "$new" --not --all); do
    if git grep -q -E "$pattern" "$c"; then
      echo "push declined: $ref: commit $(git rev-parse --short "$c") contains a secret pattern"
      exit 1
    fi
  done
done
$ cp kit/pre-receive server.git/hooks/pre-receive
```

A commit with a `.env` is declined, and nothing reaches the server:

```
$ cd triage
$ printf 'LLM_API_KEY=DUMMY-KEY-not-a-real-secret-12345\n' > .env
$ git add -A && git commit -q -m 'Add local settings'
$ git push 2>&1
remote: push declined: refs/heads/main: commit 49384d9 contains a secret pattern
To ../server.git
! [remote rejected] main -> main (pre-receive hook declined)
error: failed to push some refs to '../server.git'
[exit status: 1]
```

Now the instructive part. The developer deletes the file in a second commit and pushes again:

```
$ git rm -q .env && git commit -q -m 'Remove .env'
$ git log --oneline origin/main..main
8546eb9 Remove .env
49384d9 Add local settings
$ git push 2>&1
remote: push declined: refs/heads/main: commit 49384d9 contains a secret pattern
To ../server.git
! [remote rejected] main -> main (pre-receive hook declined)
error: failed to push some refs to '../server.git'
[exit status: 1]
```

Still declined, and for the reason of section 21B.10: the push would deliver commit `49384d9`, whose snapshot contains the key, whatever a later commit does. GitHub's push protection behaves the same way and for the same reason: the secret has to be removed from the commit that introduced it. Because nothing has been pushed, that is cheap: replace the unpushed commits.

```
# Nothing was pushed, so the two commits can be replaced. This is the cheap moment.
$ git reset -q --soft origin/main
$ git status -s
$ printf '.env\n' > .gitignore && git add .gitignore && git commit -q -m 'Ignore .env'
$ git push 2>&1
To ../server.git
c3cf659..5584d4f main -> main
[exit status: 0]
$ git -C ../server.git log --oneline main
5584d4f Ignore .env
c3cf659 Add ticket router
```

Before the first push is the one moment at which rewriting history fully removes a secret. A push-time block exists to keep you there.

A client-side `pre-commit` hook catches the mistake one step earlier, and can be skipped by the person it is meant to stop:

```
$ cat ../kit/pre-commit
#!/bin/sh
# Client-side guard: refuse a commit whose staged additions match a secret pattern.
if git diff --cached -U0 | grep -E '^\\+.*DUMMY-(KEY|TOKEN)-[a-z-]+-[0-9]+' > /dev/null; then
    echo "pre-commit: a staged line matches a secret pattern; commit refused" >&2
    exit 1
fi
$ cp ../kit/pre-commit .git/hooks/pre-commit
$ printf 'token: DUMMY-TOKEN-not-a-real-secret-67890\\n' > debug.yaml
$ git add debug.yaml
$ git commit -m 'Add debug settings'
pre-commit: a staged line matches a secret pattern; commit refused
[exit status: 1]
```

```
$ git commit -q --no-verify -m 'Add debug settings'
[exit status: 0]
$ git log --oneline -1
4be6dc6 Add debug settings
# The client hook is advice. The server hook is enforcement:
$ git push 2>&1
remote: push declined: refs/heads/main: commit 4be6dc6 contains a secret pattern
To ../server.git
! [remote rejected] main -> main (pre-receive hook declined)
error: failed to push some refs to '../server.git'
[exit status: 1]
```

Local hooks are advice; a server-side check is enforcement. On GitHub the corresponding enforcement points are push protection, push rulesets and required checks (Chapter 18).

Scanners. As of 1 October 2026 ([TruffleHog](#), [gitleaks](#), [Betterleaks](#), [detect-secrets](#), [git-secrets](#)):

- TruffleHog (v3.97.9, AGPL-3.0) verifies candidates against provider APIs and can enumerate deleted and hidden commits.
- gitleaks (v8.30.1) declares itself feature-complete, with security patches only; its author moved to Betterleaks (v1.9.0, MIT), whose governance and detection-quality claims come from one news article and are not independently verified.
- detect-secrets' last release is 1.5.0 of May 2024; git-secrets has no tagged releases.

None is installed here, so none is demonstrated. Run your choice in two places: as a pre-commit hook for fast feedback, and in CI over the full history, where it cannot be skipped.

21B.13 Dependabot, code scanning, and a way to report vulnerabilities

These are the remaining layers of a repository's baseline. All are GitHub features, described from the documentation.

Dependabot is three features that share a name ([Dependabot alerts](#), [security updates](#), [version updates](#)):

Feature	What it does	Enabled by	Limits worth knowing
Alerts	tells you a dependency on the default branch has a known vulnerability	repository or organization settings; needs the dependency graph	only advisories reviewed by GitHub raise alerts; archived repositories are not scanned; for Actions, alerts exist only for actions referenced by semantic version, not by commit SHA

Feature	What it does	Enabled by	Limits worth knowing
Security updates	opens a pull request that raises a vulnerable dependency to the minimum patched version	settings; needs alerts	grouped per ecosystem if you enable grouping
Version updates	opens pull requests to keep dependencies current, vulnerable or not	committing <code>.github/dependabot.yml</code>	since 14 July 2026 a default cooldown of 3 days applies to version updates and not to security updates (changelog)

All three are free on every plan. Note the trade-off with Chapter 21A's advice to pin actions by SHA: pinned actions get no Dependabot *alerts*, so version updates must move the pins.

A configuration for a Python service with a Dockerfile and workflows. It is assembled from keys documented in the [options reference](#), which requires `version: 2`, `updates`, and for each entry `package-ecosystem`, `directory` or `directories`, and `schedule.interval`; for GitHub Actions the documented directory value is `/`. It was parse-checked locally and not run on GitHub.

```
$ cat .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "uv"
    directory: "/"
    schedule:
      interval: "weekly"
    groups:
      python-minor-and-patch:
        patterns:
          - "*"
        update-types:
          - "minor"
          - "patch"

  - package-ecosystem: "docker"
    directory: "/"
    schedule:
      interval: "weekly"

  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
    cooldown:
      default-days: 7
```

Two things older configurations get wrong: the `reviewers` option was removed on 8 August 2025 in favor of CODEOWNERS, and the comment commands such as `@dependabot merge` stopped working on 27 January 2026 ([reviewers](#), [comment commands](#)). Without explicit configuration at most five version-update pull requests stay open at once; security updates do not count toward that limit.

Code scanning analyzes code for vulnerabilities and coding errors with CodeQL or third-party tools that upload SARIF. It runs on GitHub Actions and consumes minutes; it is free on public repositories and needs GitHub Code Security (\$30 per active committer per month) on private ones. GitHub recommends default setup, which requires Actions to be enabled and the repository to be public or licensed ([code scanning](#), [default setup](#)). One documented surprise: if a repository with default setup sees no pushes and no pull requests for six months, the weekly scheduled scan is disabled. Code scanning does not find secrets or vulnerable dependencies; those are the features above.

A security policy and private vulnerability reporting. A `SECURITY.md` file states which versions you support and how to report a problem; an organization can provide a default through its `.github` repository ([security policy](#)).

Private vulnerability reporting is a separate feature that administrators of public repositories can enable; it adds a "Report a vulnerability" button on the Advisories page, and the report lands in a private draft advisory ([reporting privately](#)). The reason to do both is in section 21B.15: a researcher who found a government contractor's leaked credentials could not find a way to report them and went to the press.

21B.14 Responding to a leaked secret: six steps, in this order

The order matters more than the tools. The names of the phases follow common incident-response practice; the Phase 0 report did not check them against a cited standard, and legal notification duties are outside this course.

Step	What the sources prescribe	Evidence of what goes wrong otherwise
1. Contain	Revoke or rotate the credential first. That alone may be sufficient, and a history rewrite may not be warranted (GitHub , OWASP)	an xAI key valid about two months after the first alert; a contractor's AWS keys valid 48 hours after the repository was taken down (Krebs on Security); the Internet Archive breached a second time through tokens it had not rotated (BleepingComputer)
2. Assess	Identify the secret, its owner and what it can reach; check validity status and exposure labels; review GitHub audit logs and the provider's logs for use; include forks, deleted forks and force-pushed commits in scope (remediation guide)	an affected company declined to say whether logs showed third-party use of an exposed token (TechCrunch)
3. Eradicate	Remove the secret from current code; rewrite history only where warranted, with <code>git-filter-repo</code> 2.47 or later, then force-push, then a GitHub Support request. Support assists only where rotation cannot mitigate the risk (GitHub)	after a supply-chain compromise the scope is every credential reachable from the affected runtime (CISA)
4. Recover	Update dependent services with the new credential; if history was rewritten, have collaborators re-clone and re-enable force-push protection; close the alert as revoked and document	partial rotation produced a second breach (Internet Archive)
5. Communicate	Track internally; tell collaborators exactly what to do with their clones; keep a reachable disclosure contact such as <code>SECURITY.md</code>	a researcher could not find a way to report a contractor's leak and went to the press (Cybersecurity Dive); concealment turned Uber's 2016 breach into a regulatory case (FTC)
6. Prevent	Push protection; pre-commit and CI scanning; secrets out of code; short-lived credentials through OIDC; least privilege; staging named files and reviewing <code>git diff --cached</code> instead of <code>git add .</code> ; scheduled rotation	hardcoded long-lived keys recur in every case of section 21B.15

Why containment comes first. The repository is not where the damage happens. The damage happens at the issuer, where the key is accepted. Until revocation the key works for whoever copied it, and revocation is the only step that is complete: it works against clones, forks, caches and screenshots alike. GitHub's own guidance says the same: rotate immediately; removing the secret from history is time-consuming and often unnecessary once the credential is revoked ([secret scanning](#)).

What assessment has to produce. Five facts, written down: which secret and what it can reach; the first commit that contains it and when that commit was first pushed; which refs contain it (section 21B.11 gives the commands); who could read it (repository visibility, forks, collaborators, CI logs); and whether it was used, from the *provider's* logs. Git answers the second and third. Only the issuer answers the last.

When a history rewrite is warranted. Rewrite when the data stays harmful after rotation or cannot be rotated: personal data, customer records, proprietary model weights, a private key whose public half is pinned in devices,

a secret whose revocation takes weeks. Do not rewrite by reflex for an API key that was revoked within the hour: the rewrite costs every collaborator their clone, invalidates every recorded commit ID, strips signatures, and cannot recall copies that already exist. Record the decision and its reason.

21B.15 Case studies

Each row is from the Phase 0 report with its source. The root-cause column repeats one pattern: a long-lived, over-scoped credential written where it should never have been.

Case	What was exposed and for how long	Root cause	Lesson
Uber, 2016 (FTC)	a cloud access key in a private GitHub repository; intruders downloaded tens of millions of records in a month	plaintext key in source code; reused passwords; no multi-factor requirement on GitHub accounts	a private repository is only as private as its weakest member account
Toyota, 2022 (BleepingComputer)	a data-server access key public from December 2017 to September 2022; 296,019 customers potentially exposed	a subcontractor published internal code with a hardcoded key	keys that never expire turn one mistake into a five-year exposure
Samsung, 2022 (GitGuardian)	6,695 secrets inside stolen source code	thousands of credentials hardcoded in private code	assume source code will leak
Microsoft AI research, 2020 to 2023 (Wiz)	a storage URL in a public model repository granted full control of an account holding 38 TB	an over-scoped, long-lived sharing token committed to the repository	a data-sharing URL is a credential; share weights through read-only, expiring mechanisms
Hugging Face tokens, 2023 (TechTarget)	1,681 valid tokens giving access to 723 organizations, 655 with write permission	tokens committed to code	a model-hub write token is a supply-chain credential
Mercedes-Benz, 2024 (TechCrunch)	an employee token public for about four months gave unrestricted access to the company's GitHub Enterprise Server	broad, long-lived token in a public repository	one over-scoped token equals the whole code estate
The New York Times, 2024 (BleepingComputer)	a 273 GB archive of repositories leaked five months after a GitHub credential was exposed	exposed token with organization-wide read access	least-privilege, expiring tokens bound the damage
xAI, 2025 (Krebs on Security)	an LLM API key with access to at least 60 private and fine-tuned models, valid two months after the first alert	key hardcoded and pushed; alert sent only to an individual	an alert in one inbox is not an incident process
CISA contractor, 2025 to 2026 (Krebs on Security)	administrative cloud credentials and plaintext passwords in a public repository for six months	work material copied to a personal public repository with secret detection deliberately disabled	controls an individual can switch off need organizational backstops and a leak playbook

The report flags these details as partially confirmed, so quote them as the cited articles do: the Mercedes-Benz day-level timeline; the New York Times archive size (273 GB in the cited article, often rounded to 270 GB); the Internet Archive user count; Samsung's own confirmation; Toyota's original notice. The report found no verified 2026 incident caused specifically by a key in a public Jupyter notebook: notebooks are documented as a leak mechanism, not through a named breach.

For an LLM engineer: an LLM API key reaches more than a bill (the xAI key reached private and fine-tuned models); a link that shares weights or data is a credential with a scope and a lifetime; and a model-hub token with

write permission lets its holder replace what your users download.

21B.16 History rewriting as an operation

In one sentence. A whole-history rewrite replaces the first affected commit and every descendant with new commits that have new IDs, and the command is the smallest part of the work.

Analogy. Recalling a printed book to remove one page: you can reprint every copy in your warehouse, but each reader's copy stays as it was until that reader exchanges it, the page numbers after the removed page all change, every citation by page number now points at the wrong place, and one reader who lends an old copy to the library puts the page back on the shelf.

Precisely: what git-filter-repo does and requires. The recommended tool is [git-filter-repo](#), a separate program that is not part of Git. From its manual and from GitHub's procedure ([removing sensitive data](#)):

- It refuses to run outside a fresh clone unless forced with `--force`, because the rewrite is irreversible: by default it ends with an immediate pruning of reflogs and old objects.
- `--sensitive-data-removal` exists since version 2.47. It fetches all refs first and gathers the extra information needed to clean up other copies.
- It records its work in `.git/filter-repo/`: `commit-map` (old and new ID of every commit), `ref-map`, `changed-refs` and `first-changed-commits`.
- Commits get new IDs, so signatures on commits and tags cannot remain valid and are removed; signed tags become annotated tags.
- By default a full rewrite removes the `origin` remote, as a forcing function against pushing by reflex.

GitHub's documented sequence, shown without output because git-filter-repo is not installed here and nothing in this course contacts GitHub:

```
# 1. Install the tool (GitHub's page gives this command for macOS). You need version 2.47 or later.
brew install git-filter-repo

# 2. A fresh clone, never your working clone.
git clone https://github.com/YOUR-USERNAME/YOUR-REPOSITORY
cd YOUR-REPOSITORY

# 3a. Remove a file from every commit ...
git-filter-repo --sensitive-data-removal --invert-paths --path PATH-TO-YOUR-FILE-WITH-SENSITIVE-DATA

# 3b. ... or replace strings listed in a file (one expression per line; by default each is
#       literal text and is replaced by ***REMOVED***; "regex:" and "glob:" prefixes exist).
git-filter-repo --sensitive-data-removal --replace-text ../passwords.txt

# 4. How many pull requests are affected? Support will ask.
grep -c '^refs/pull/.*/head$' .git/filter-repo/changed-refs

# 5. Force-push every ref.
git push --force --mirror origin
```

Unverified. Whether git-filter-repo keeps the `origin` remote when `--sensitive-data-removal` is used was not tested for the Phase 0 report: GitHub's instructions push to `origin` afterwards, and the manual says a default full rewrite removes it. If the push fails for a missing remote, the manual's remedy is `git remote add origin <url>`.

Before pushing, the manual's verification is `git log --all --name-status -- <file>` and `git log -S"<string>" --all -p --`: both must print nothing.

What makes it an operation. Around those five commands:

Fact	Consequence
Other contributors must stop work during the cleanup	announce a freeze; work pushed during the rewrite is discarded or forces a restart
All refs, tags included, must be force-pushed	rules that block force pushes and tag updates must be switched off temporarily, and back on afterwards
Every descendant commit ID changes	review comments on open pull requests detach; diffs of closed ones break; every recorded ID is stale
Signatures are removed, also on commits that predate the removed data	a "require signed commits" rule now rejects the rewritten history unless bypassed
Orphaned LFS objects are not removed by the rewrite	they are purged separately
Pull request refs, forks and other clones keep the old history	sections 21B.18 and 21B.19

For an ML team the changed IDs have a specific cost: every experiment record, model card and deployment manifest that stored an old commit ID now points at a commit that no longer exists on the remote. The `commit-map` file belongs in the incident record so that old IDs can be translated.

Outdated advice. Older guides use `git filter-branch` or the BFG Repo-Cleaner. Git's own manual says of `git filter-branch` that its "use is not recommended" and points to `git-filter-repo` (`git help filter-branch`, section WARNING). GitHub's current page documents `git-filter-repo` only.

21B.17 The mechanics, seen locally

`git-filter-repo` is not installed in the lab, and the course installs nothing. The transcripts in this section and the next use `git filter-branch`, **only because it ships with Git**. It is not the recommended tool. It suffices to show four mechanics that are the same whichever tool rewrites: descendants get new IDs, tags must be rewritten too, old objects remain until pruned, and the server keeps them after the push.

Start from a fresh mirror clone, which has every ref of the server as a local ref, and note the commit that introduced the file:

```
$ git clone -q --mirror server.git cleanup.git
$ cd cleanup.git
$ git for-each-ref --format="%(objectname:short) %(objecttype) %(refname)"
7fae871 commit refs/heads/feature/streaming
d4b8762 commit refs/heads/main
b4a2514 tag refs/tags/v0.1.0
c38ee42 tag refs/tags/v0.2.0
$ git log --oneline main
d4b8762 Document setup in README
b509fe3 Add request timeout
64b9b89 Add retry with backoff
0805fd8 Add staging settings
987a49d Add evaluation harness
6388058 Add LLM client
0c55276 Add answer prompt template
dfd59fd Add BM25 retriever
$ first=$(git log --format=%h --diff-filter=A main -- .env); echo $first
0805fd8
```

Mistake first: rewriting the branches and forgetting the tags. 🚫 A history filter replaces commits on every ref it is given; run it only in a clone you can throw away.

```
# First attempt: rewrite the branches and forget the tags.
$ FILTER_BRANCH_SQUELCH_WARNING=1 git filter-branch --index-filter 'git rm --cached --ignore-unmatch
-q .env' -- --branches > ../filter-1.log 2>&1
$ grep -v 'seconds passed' ../filter-1.log
Ref 'refs/heads/feature/streaming' was rewritten
Ref 'refs/heads/main' was rewritten
$ git grep -l DUMMY-KEY $(git rev-list --branches) | wc -l
0
$ git grep -l DUMMY-KEY $(git rev-list --all) | cut -c1-9,41-
d4b876277:.env
7fae87198:.env
b509fe363:.env
64b9b890c:.env
0805fd8e8:.env
$ git tag --contains $first
v0.2.0
```

The branches are clean, and a scan over `--all` still finds five snapshots with the key. The tag `v0.2.0` points at the old commit `64b9b89`, and a tag keeps its commit and all of that commit's ancestors alive. Anyone who fetches the tag fetches the secret. (`--all` also includes the backup refs that `filter-branch` wrote under `refs/original/`.)

All refs, with tags following their commits.

```
# Second attempt: every ref, and --tag-name-filter cat so that tags follow their commits.
$ FILTER_BRANCH_SQUELCH_WARNING=1 git filter-branch -f --index-filter 'git rm --cached --ignore-
unmatch -q .env' --tag-name-filter cat -- --all > ../filter-2.log 2>&1
$ grep -v 'seconds passed' ../filter-2.log
WARNING: Ref 'refs/heads/feature/streaming' is unchanged
WARNING: Ref 'refs/heads/main' is unchanged
WARNING: Ref 'refs/tags/v0.1.0' is unchanged
Ref 'refs/tags/v0.2.0' was rewritten
v0.1.0 -> v0.1.0 (987a49db5aa45d6cb1e4b10024438f21b8e28444 ->
987a49db5aa45d6cb1e4b10024438f21b8e28444)
v0.2.0 -> v0.2.0 (64b9b890cf46c6a921380d722eaf0b33d790bb9a ->
66a99cccb3cd6ac483eccafadf8c4804770799ea)
```

The branches are reported "unchanged" because the first run already rewrote them. `v0.1.0` is unchanged because it points before the leak. `v0.2.0` now points at `66a99cc`.

Every descendant has a new ID; ancestors keep theirs.

```
# old ID, new ID, subject (newest first)
$ paste ../before.txt ../after.txt | while read o n; do printf '%s %s %s\n' $o $n "$(git log -1 --
format=%s $n)"; done
d4b8762 c8ce738 Document setup in README
b509fe3 51e2d95 Add request timeout
64b9b89 66a99cc Add retry with backoff
0805fd8 0ac4257 Add staging settings
987a49d 987a49d Add evaluation harness
6388058 6388058 Add LLM client
0c55276 0c55276 Add answer prompt template
dfd59fd dfd59fd Add BM25 retriever
```

`0805fd8` became `0ac4257` because its tree changed: `.env` is no longer in it. `64b9b89`, `b509fe3` and `d4b8762` changed for two reasons. A commit is a full snapshot, so `.env` was in the tree of every commit from the leak onward, and each of those trees is a different tree without it. And each commit records its parent's ID, and the parent changed. The second reason is enough by itself: a commit whose tree stayed identical would still get a new ID once its parent has one. What these three commits introduce, their diff against the parent, is the same as

before. The four commits below the leak are byte-for-byte the same objects. This table is what `git-filter-repo` writes to `commit-map`.

before		v0.2.0	
	dfd59fd--0c55276--6388058--987a49d--0805fd8--64b9b89--b509fe3--d4b8762		main
		(.env added)	
	v0.1.0	\ 7fae871	feature/streaming
after		v0.2.0	
	dfd59fd--0c55276--6388058--987a49d--0ac4257--66a99cc--51e2d95--c8ce738		main
		(no .env)	
	v0.1.0	\ ba9f0e0	feature/streaming
shared, unchanged: dfd59fd .. 987a49d replaced: everything from the first changed commit on			

The old objects are still there. A rewrite adds new objects and moves refs. It deletes nothing by itself.

```
$ git for-each-ref --format="%(objectname:short) %(refname)" refs/original
64b9b89 refs/original/refs/tags/v0.2.0
$ git cat-file -t $first
commit
$ git show $first:.env
LLM_API_KEY=DUMMY-KEY-not-a-real-secret-12345
LLM_BASE_URL=https://llm.example.com
```

Removing them takes three deliberate steps: delete the backup refs, expire the reflogs, prune.

```
$ git for-each-ref --format="delete %(refname)" refs/original | git update-ref --stdin
$ git reflog expire --expire=now --all
$ git gc -q --prune=now
$ git cat-file -t $first
fatal: Not a valid object name 0805fd8
[exit status: 128]
$ git grep -l DUMMY-KEY $(git rev-list --all) | wc -l
0
$ git fsck --no-progress
```

`git-filter-repo` performs this pruning for you at the end of its run, which is why it insists on a fresh clone.  `git reflog expire --expire=now --all` followed by `git gc --prune=now` destroys the local safety net of Chapter 13: after it, nothing that was only reachable from a reflog can be recovered. In a throwaway mirror clone that is the intention. In your working clone it also discards every stash.

The force push moves the server's refs, and only the refs.  `git push --force --mirror` overwrites every ref on the remote and deletes those the clone lacks (section 21B.23).

```
$ git push --force --mirror origin 2>&1
To $LAB/ch21b/rewrite-mechanics/server.git
+ 7fae871...ba9f0e0 feature/streaming -> feature/streaming (forced update)
+ d4b8762...c8ce738 main -> main (forced update)
+ c38ee42...17124a2 v0.2.0 -> v0.2.0 (forced update)
```

```
$ cd ..
$ git -C server.git log --oneline -1 main
c8ce738 Document setup in README
$ git -C server.git grep -l DUMMY-KEY $(git -C server.git rev-list --all) | wc -l
0
# No ref on the server reaches the old commits any more, and they are still there:
$ git -C server.git cat-file -t $first
commit
$ git -C server.git show $first:.env
LLM_API_KEY=DUMMY-KEY-not-a-real-secret-12345
```

```
LLM_BASE_URL=https://llm.example.com
$ git -C server.git fsck --no-progress --unreachable | grep commit
unreachable commit 0805fd8e82dfc4c6a50cb14514d431dbd43df4cd
unreachable commit b509fe3635defadb4fc29d14c01ca2953ca8cd27
unreachable commit d4b876277f85823455b617a02ea443e6e9afd070
unreachable commit 64b9b890cf46c6a921380d722eaf0b33d790bb9a
unreachable commit 7fae8719801ebfc91df813470f8a380ac13184fc
```

No ref on the server reaches the secret, the scan over all refs is clean, and `git show` on the server still prints the key from the unreachable commit. A bare repository of your own can be pruned with `git gc --prune=now`. On GitHub you cannot run that; section 21B.19 explains who can.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
history rewrite in the cleanup clone	rewritten to the new tip (none in a mirror clone)	rewritten	unchanged (still names the branch)	moved to the new tip	every affected branch and tag moved; backup refs or <code>filter-repo/</code> metadata written; old objects kept until pruned	unchanged	unchanged
<code>git reflog expire --expire=now --all</code> then <code>git gc --prune=now</code>	unchanged	unchanged	unchanged	unchanged	all reflog entries and all unreachable objects deleted	unchanged	unchanged
<code>git push --force --mirror origin</code>	unchanged	unchanged	unchanged	unchanged	unchanged	every ref set to the local value; refs missing locally are deleted ; old objects stay as unreachable	branch and tag refs move; pull request refs are refused; cached views and old objects stay until Support acts

`--mirror` deletes remote refs that the cleanup clone does not have. A branch that a colleague pushed after you cloned is removed by your push. That is the mechanical reason for the freeze.

21B.18 The stale clone that pushes the secret back

Asha cloned before the incident and has one local commit. She was not told to stop, or did not read the message. She does what she does every morning:

```
$ cd asha
$ git log --oneline -3
95f98b0 Add contains metric
d4b8762 Document setup in README
b509fe3 Add request timeout
$ git pull --no-rebase 2>&1
From ../server
+ d4b8762...c8ce738 main -> origin/main (forced update)
+ 7fae871...ba9f0e0 feature/streaming -> origin/feature/streaming (forced update)
Merge made by the 'ort' strategy.
$ git log --oneline --graph -6
* f7ed5a9 Merge branch 'main' of ../server
|\
```

```
| * c8ce738 Document setup in README
| * 51e2d95 Add request timeout
| * 66a99cc Add retry with backoff
| * 0ac4257 Add staging settings
* | 95f98b0 Add contains metric
$ git push 2>&1
To ../server.git
c8ce738..f7ed5a9 main -> main
```

The fetch reports forced updates. `git pull` then merges the new `origin/main` into her `main`, which still descends from the old history. The merge commit `f7ed5a9` has two parents: the clean tip `c8ce738` and her commit `95f98b0`, whose ancestors include `0805fd8`. Her push is a fast-forward from `c8ce738` to `f7ed5a9`, so the server accepts it without force, and no branch rule that forbids force pushes objects.

```
$ cd ..
$ git -C server.git log --oneline -3 main
f7ed5a9 Merge branch 'main' of ../server
95f98b0 Add contains metric
c8ce738 Document setup in README
$ git -C server.git grep -l DUMMY-KEY $(git -C server.git rev-list --all) | wc -l
6
$ git -C server.git log --oneline -S'DUMMY-KEY' main
0805fd8 Add staging settings
```

```

      0805fd8--64b9b89--b509fe3--d4b8762--95f98b0
      /      (old history, with .env)      \
...--987a49d                                f7ed5a9  main (on the server again)
      \      \                              /
      0ac4257--66a99cc--51e2d95--c8ce738-----
      (rewritten history)
```

```
Observed behavior : the day after the cleanup, the scanner reports the same secret on main
Git state         : main is a merge commit with the rewritten tip and a commit from the old history
as parents
Mechanism         : a pull in a stale clone merged old and new history; the push was a fast-forward
Root cause        : a clone that still held the old commits was allowed to push
Why Git does this : to Git the two histories are unrelated lines of work that someone chose to merge
Correct fix       : force-push the clean refs again; clean or replace the stale clone; replay only
her own commit
Prevention        : freeze and re-clone; rebase onto the new history, never merge; a server-side
check that
                    rejects the first changed commit or the secret pattern
```

GitHub's page lists "high risk of recontamination" first among the side effects of a rewrite and gives the rule: collaborators must rebase, not merge, branches created from the old history, because one merge commit can reintroduce some or all of it. The filter-repo manual says the easiest way to clean other clones is to delete and re-clone them. Where that is impossible, it prescribes, per clone:

```
git tag -l | xargs git tag -d      # tags are not updated by a normal fetch; delete, then refetch
git fetch --prune --tags
git rebase --onto origin/main <old-upstream-tip> <branch>      # replay only your own commits
git reflog expire --expire=now --all
git gc --prune=now
git cat-file -t <first-changed-commit>      # must fail
```

The base of that rebase matters. In a clone that has already fetched, `git rebase origin/main` treats every old commit that is missing from the new history as yours and tries to replay it, the commit that added the secret included:

```
$ cd asha2
$ git fetch -q 2>&1
$ git rebase origin/main 2>&1 | grep -E "^(CONFLICT|error|Could not)"
```

```

CONFLICT (add/add): Merge conflict in settings.yaml
error: could not apply 0805fd8... Add staging settings
Could not apply 0805fd8... # Add staging settings
$ git status -sb | head -1
## HEAD (no branch)
$ git rebase --abort
$ cd ..

```

When run for this chapter, `git pull --rebase` as a clone's *first* contact with the rewritten history replayed only the local commit, because the pull still knew the previous value of `origin/main` (labs/run_ch21b/stale-clone-rebase shows all three variants). That knowledge is gone after a plain fetch or a merge, so the instruction to colleagues is the explicit `--onto` form. Lab 31.1 performs the whole sequence in Asha's clone. The manual adds two warnings: colleagues must not run the same filter command themselves, because identical commands can still produce different commit IDs; and the expiry step drops their reflogs and stash entries.

Few hosting services can ban a specific commit from being pushed again; the filter-repo manual says so, and the Phase 0 notes found no documented built-in control of that kind on GitHub. Push protection (when the secret matches a supported pattern) and a push ruleset that blocks the file path are the nearest platform equivalents. The local analogue is the `pre-receive` hook of section 21B.12, which Lab 31.1 installs and tests against a second stale clone.

21B.19 The GitHub side of a rewrite: pull requests, forks, cached views, Support

GitHub, not Git. This section is described from GitHub's page on [removing sensitive data](#). Nothing here was run.

After your force push, four copies remain that you cannot reach with Git:

Where	Why it survives	Who can remove it
Unreachable commits in the repository	a force push moves refs; the objects stay, and are served by commit ID in cached views	GitHub Support, by running garbage collection and removing cached views
Pull request refs	<code>refs/pull/N/head</code> is read-only for you and keeps the old commits reachable	GitHub Support, by dereferencing or deleting the affected pull requests
Forks	each fork has its own refs to the old commits, and the fork network shares objects	each fork's owner; GitHub cannot provide their contact information
Clones	they are on other people's disks	their owners

The Support request must contain the repository owner and name, the number of affected pull requests, and the first changed commit or commits from `git-filter-repo`'s output; if the output contained the line about orphaned LFS objects, mention it and attach the named file. Support removes only sensitive data, and only in cases where it determines that rotating the affected credentials cannot mitigate the risk.

Unverified. GitHub publishes neither how long it retains unreachable commits without a Support-initiated garbage collection nor a turnaround time for Support. Researchers who mined force-pushed commits observed that they appear to be kept indefinitely; that is their observation, not a GitHub statement.

The same retention makes the recovery of a force-pushed branch possible in Chapter 13: a force push does not delete.

21B.20 Governance controls that limit blast radius

Prevention fails sometimes, so design for the day it does. These controls decide how much one leaked credential can do and whether you can reconstruct what it did. They are GitHub features from the Phase 0 report; the chapters named cover each in depth.

Control	What it limits	Source
Fine-grained or App tokens with expiry, and organization approval of tokens	the reach and lifetime of a stolen token	token policy
Mandatory two-factor authentication and SSO authorization	account takeover by password	Chapter 16
Rulesets on branches, tags and pushes; push rulesets can block file paths, extensions and sizes across the whole fork network	what a compromised contributor can change, and what can be pushed at all	Chapter 18
Code-owner review of <code>.github/workflows/</code> and of the CODEOWNERS file itself	silent changes to automation	code owners
Audit log, 180 days at organization level, exported or streamed if Git events must be available later (the enterprise log retains them for seven days)	your ability to answer "what did the token do?"	audit log
The preview ruleset rule that blocks merging a pull request which introduces an open secret alert (9 September 2026; needs Secret Protection)	secrets entering the default branch through review	changelog

These controls reduce blast radius; they do not prevent leaks. The report's observation across the cases is sobering: the worst outcomes involved single credentials with estate-wide read access, and detection came from outsiders in nearly every case. An organization that wants to know first needs its own monitoring of public GitHub, employees' and contractors' personal accounts included.

21B.21 What can go wrong

Symptom	Diagnosis	Fix	Prevention
<code>fatal:</code> <code>detected</code> <code>dubious</code> <code>ownership in</code> <code>repository</code>	the repository directory belongs to another user ID (containers, CI, shared volumes)	add that one directory to <code>safe.directory</code> in global configuration, or fix ownership	build images so the checkout is owned by the user that runs Git
A Git command ran a program you did not expect	<code>git hook list --show-scope <event>; git config list --show-origin</code> and look for aliases, <code>pagerefs</code> , <code>core.fsmonitor</code> , <code>core.hooksPath</code>	remove the setting; if the <code>.git</code> came from an archive, stop and <code>git clone --no-local</code> it	clone, never unpack; <code>safe.bareRepository=explicit</code>
A colleague's name is on a commit they did not write	<code>git log --format='%h %an %cn %G?'</code> ; on GitHub, the pusher in the Activity view or audit log	treat as an incident on the pushing account, not the named one	vigilant mode; require signed commits
A scanner reports a secret you deleted	<code>git log --all -S<secret>; git grep <secret> \$(git rev-list --all)</code>	revoke at the issuer; decide on a rewrite	push protection; pre-commit and CI scanning
Push blocked for a secret that "is not in the code any more"	<code>git log --oneline @{{u}}..</code> shows an unpushed commit that still contains it	rewrite the unpushed commits (<code>git reset --soft @{{u}}</code> and	review <code>git diff --cached</code> before committing

Symptom	Diagnosis	Fix	Prevention
		recommit, or an interactive rebase)	
After a rewrite, the scan is still positive	tags or other branches were not rewritten; backup refs under <code>refs/original/</code>	rewrite all refs; delete backup refs; scan again	git-filter-repo with <code>--sensitive-data-removal</code> in a fresh clone
The secret returned to <code>main</code> after the cleanup	<code>git log --merges -1</code> ; a merge whose parents span old and new history	force-push the clean refs again; clean the stale clone	freeze; re-clone; server-side check
The mirrored force push fails for <code>refs/pull/*</code>	the forge's pull request refs are locked (git-filter-repo manual)	expected; count them for the Support request	none

21B.22 When not to use it, and dangerous edge cases

Do not rewrite history

- for a credential that is revoked and whose provider logs show no use. Record the decision and stop.
- on a public repository in the belief that it un-publishes anything.
- before containment.
- in your working clone. The tool prunes reflogs and stashes.
- by having each colleague run the same command. Identical commands can produce different IDs.

Dangerous edge cases

- `git push --force --mirror` **deletes**. Every remote ref absent from the cleanup clone is removed. Take the mirror clone after the freeze, not before.
- **Tags are not updated by an ordinary fetch**. A clone that pulled after the rewrite still holds the old `v0.2.0`, and with it the old history. Delete tags and refetch.
- **A deleted fork is not a deleted copy**. Objects in a fork network remain reachable from the other repositories of the network.
- **Rotating one of several credentials**. After a compromise of a runtime (a CI job, a laptop) the scope is every credential that runtime could read, rotated together. Partial rotation is how the Internet Archive was breached twice.
- **The secret in a commit message, a branch name, or a pull request description**. File-path removal does not touch these. `--replace-text` rewrites file contents; text on GitHub (issues, pull request bodies, comments) is edited or deleted on the platform.
- `.gitignore` **after the fact**. Ignoring a tracked file changes nothing; the file stays tracked until `git rm --cached`.

21B.23 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git log -S</code> , <code>git log -G</code> , <code>git grep <p> \$(git rev-list --all)</code>	🟢 SAFE	nothing	not needed	not needed
<code>git clone --no-local <dir> <new></code>	🟢 SAFE	creates a new repository	not needed	delete the directory
<code>git config set --global</code>	🟡 CAUTION	global configuration, a file	<code>git config get --show-origins</code>	<code>git config unset --global</code>

Command	Label	What it changes	Preview	Recovery
<code>safe.bareRepository</code> <code>explicit</code>		that keeps no history; it changes how later commands treat bare repositories	<code>safe.bareRepository</code>	<code>safe.bareRepository</code>
<code>git config set --global --append safe.directory <dir></code>	⚠ CAUTION	makes Git trust that directory's configuration and hooks	read <code><dir>/.git/config</code> and <code><dir>/.git/hooks</code> first	<code>git config unset --global --value=<dir> safe.directory</code>
<code>git rm --cached <file></code>	⚠ CAUTION	removes the file from the index; the working tree copy stays	<code>git status</code>	<code>git restore --staged <file></code>
<code>git reset --soft @{u}</code>	⚠ CAUTION	moves the branch to the upstream tip; index and working tree keep your changes	<code>git log @{u}..</code>	<code>git reset --soft ORIG_HEAD</code>
<code>git rebase --onto <new> <old-base></code>	⚠ CAUTION	recreates your commits on the new history	<code>git log <old-base>..</code>	<code>git reset --hard ORIG_HEAD</code> , or the reflog
<code>git filter-repo ...</code> , <code>git filter-branch ...</code>	🔴 DANGEROUS	every affected commit and all descendants, on every rewritten ref	<code>git-filter-repo --dry-run</code> ; run in a fresh clone	the untouched server and other clones, until you push
<code>git reflog expire --expire=now --all</code> and <code>git gc --prune=now</code>	🔴 DANGEROUS	deletes all reflog entries and all unreachable objects	<code>git fsck --unreachable --no-reflogs</code> lists what would go	none locally
<code>git push --force --mirror origin</code>	🔴 DANGEROUS	sets every remote ref to the local value and deletes the rest	<code>git push --dry-run --force --mirror origin</code>	another clone that still has the old refs; on GitHub, the instruments of Chapter 13

For the three 🔴 commands, what the table does not say:

- **A history filter** can destroy signatures and, with a wrong path argument, files you meant to keep. It is appropriate for data that stays harmful after rotation. For an unpushed mistake on a private branch a rebase is the smaller tool.
- **Immediate expiry and prune** can destroy every commit, stash and staged blob that only a reflog, or nothing at all, was keeping. It is appropriate in a cleanup clone, and in a stale clone after its owner has saved their work.
- **A mirrored force push** can destroy branches and tags that exist only on the remote. It is appropriate once, after a freeze, at the end of a verified rewrite. Every other force push uses `--force-with-lease` (Chapter 12).

21B.24 Version notes

Version note. Older behavior: Git read the configuration of any repository it discovered, whoever owned it. Current behavior: it refuses when the owner differs, unless `safe.directory` lists the directory. Since: the

2022 security releases. Recommended: list single directories; never * on a workstation.

Version note. Older behavior: `safe.bareRepository` defaults to `all`. Current behavior: unchanged in 2.55; `explicit` becomes the default in Git 3.0. Since: the setting was added in 2022. Recommended: set `explicit` now if you do not work inside bare repositories.

Version note. Older behavior: history was rewritten with `git filter-branch` or the BFG Repo-Cleaner. Current behavior: Git's manual recommends against `filter-branch`; GitHub documents `git-filter-repo`, with `--sensitive-data-removal` in version 2.47 or later. Recommended: `git-filter-repo`.

Version note. Older behavior: a verified commit could become unverified when its key was revoked or expired. Current behavior: verification records persist. Since: 10 December 2024 on GitHub. Recommended: do not use the badge to find commits made with a stolen key.

The `git config get|set|unset|list` subcommands used in this chapter need Git 2.46 or later; older scripts write `git config --get` and `git config --add`.

21B.25 Practice

- Lab 30.3: scan a full history for planted dummy secrets with built-in commands, including the commit that only the reflog still reaches.
- Lab 30.1 and Lab 30.2: push protection on your practice repository, and a Dependabot configuration.
- Lab 24.3: the verification states on GitHub, with the forged commit of section 21B.5.
- Lab 31.1: the tabletop. Do it twice; the second time from the symptom alone.
- The operational checklists and runbooks are in the security guide.
- Chapter 30 turns the committed secret into a timed incident drill; Chapter 21A covers the same questions for GitHub Actions.

21B.26 Interview questions

1. A developer says: "I committed an API key, deleted it in the next commit and pushed. We are fine." Walk through exactly where the key still exists, and what you do first.
2. Explain why `git clone` of an untrusted repository is generally safe and why unpacking a tarball of the same repository is not. Name the files involved.
3. A commit on `main` shows your tech lead's avatar and she says she did not write it. What does GitHub's display prove, what evidence do you look at, and which controls would have prevented or flagged it?
4. What changes for a team when "Require signed commits" is enabled and the repository uses "Rebase and merge"? Explain the mechanism.
5. Your signing key was stolen and you revoked it. Which commits still show "Verified", and how do you find the attacker's?
6. Rank the credentials an automation could use to push to a repository by blast radius, and justify the order.
7. You rewrote history to remove a customer data file and force-pushed. List every place the file may still exist and who controls each.
8. A day after a history rewrite the secret is back on `main` and nobody force-pushed. Reconstruct what happened from the commit graph, and describe the fix for the server and for the clone.
9. When is a history rewrite the wrong response to a leaked secret? What does it cost?

10. Push protection is enabled for all your users. Name three kinds of leak it will not stop.

21B.27 Sources

Primary sources

- Git 2.55 manual pages: `git help git` (SECURITY), `git help config` (`safe.*`, `protocol.*`), `git help filter-branch` (WARNING), `git help hook`, `git help githooks`.
- [Git security advisories](#); [safe configuration](#); [protocol configuration](#); [githooks](#).
- GitHub Docs, fetched 2 October 2026: [removing sensitive data from a repository](#); [remediating a leaked secret](#); [about push protection](#); [Dependabot options reference](#). Through the Phase 0 report: [about secret scanning](#); [supported patterns](#); [commit signature verification](#); [credential types](#); [security features](#).
- [git-filter-repo manual](#) (fetched 2 October 2026).

Secondary sources

- [GitGuardian, State of Secrets Sprawl 2026](#); [GitGuardian on leaked GitHub App private keys](#).
- [Russ Cox, timeline of the xz open source attack](#).
- The research, incident reports and articles linked where they are used, in sections 21B.4, 21B.8, 21B.10, 21B.14 and 21B.15.

Videos (from the Phase 0 report, with its caveats)

- ["git-filter-repo for rewriting Git history"](#), Elijah Newren, Git Merge 2024, 22 min. A contributor talk on what the tool does and how it compares with filter-branch and BFG; not a step-by-step incident tutorial.
- [Git Merge 2022 workshop on SSH commit signing](#), listed by the roadmap for Module 24.
- ["Day-2: DevSecOps for Git and GitHub"](#), Abhishek Veeramalla, 46 min, 22 January 2026. Shows a branch ruleset, CODEOWNERS, gitleaks through pre-commit and in Actions, and Dependabot together; no signing, OIDC or action pinning.

Further reading

- Chapter 13: Recovery for reflogs, pruning and the GitHub-side recovery instruments; Chapter 14B for local signing; Chapter 16 for credential mechanics; Chapter 12 for force pushes.